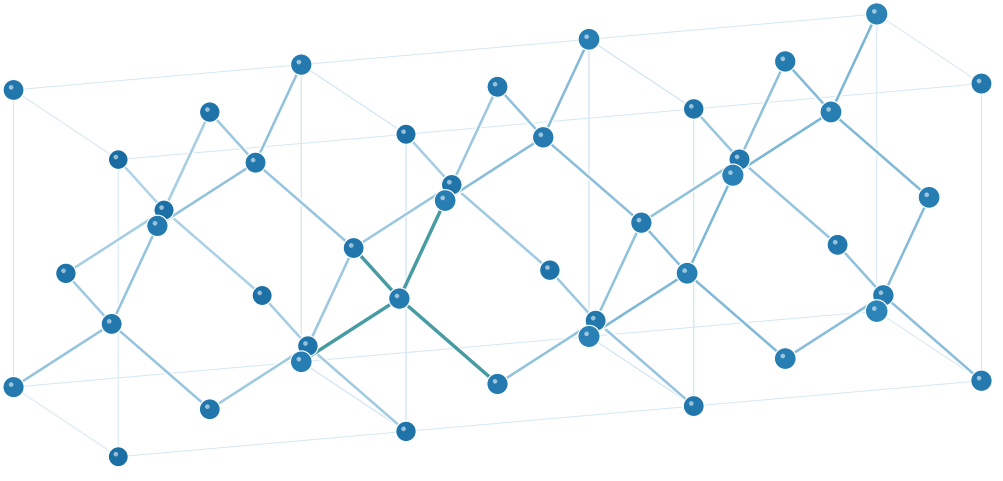
瀚海量子
Hanhai Quantum

# KSSOLV Toolbox

## User Manual



V0.5.0 / 2026

Hefei Hanhai Quantum Technology Co., Ltd.

# KSSOLV Toolbox User Manual

Standalone application, MATLAB® toolbox, and KSSOLV Toolbox Online

For version V0.5.0 October 2026

This manual covers installation, structure preparation, electronic structure calculations, and analysis of results in KSSOLV Toolbox. The tutorials begin with small molecules and crystals, then move on to surfaces, heterostructures, twisted structures, low-dimensional systems, and soft matter. The standalone application, MATLAB® toolbox, and KSSOLV Toolbox Online share the same modeling and calculation concepts. Their differences in installation, file management, and remote execution are explained where relevant.

Interface labels follow the English interface. Each calculation example includes its input conditions and reference results so that you can follow the procedure and check your work.

## Trademarks

Hanhai Quantum® and KSSOLV Toolbox are product brand names. MATLAB® is a registered trademark of The MathWorks, Inc.

## The cover

The cover shows the silicon structure supplied with the examples, extended over three conventional unit cells along [100]. Spheres represent silicon atoms, and lines connect nearest neighbors. Four teal bonds highlight the tetrahedral coordination of the diamond structure.

# About this manual

## Software version

This manual describes KSSOLV Toolbox V0.5.0. To check your software version, choose Home → Help → About KSSOLV.

## Where to start

If you are new to the software, start with Chapter 1 to choose how you will use it. Follow the installation or Online instructions to prepare your environment, then read Chapter 3 to learn your way around the workspace. The hydrogen molecule tutorial in Chapter 5 takes you through a complete calculation, from the initial structure to the final results, with checks at each stage.

When you are ready to build your own models, use Chapter 7 for basic structure editing and Chapter 8 for worked materials examples. Chapter 9 is a reference for individual modeling tools. The calculation parameters chapter explains common settings; the workflow, volumetric data, and results chapters describe how to organize further calculations and interpret their output. You do not need to read the whole manual before you begin.

The accompanying `examples/` directory contains structures, projects, scripts for reproducing calculations, and volumetric datasets. Start each exercise from the specified file and save your own copy of the project. When comparing numerical results, check the structure, pseudopotentials, and calculation settings as well as the numbers themselves.

## Software and support

- Software and releases: `https://github.com/USTC-Hu-Group/KSSOLV-Toolbox`
- Online: the official Hanhai Quantum® KSSOLV Toolbox Online service is forthcoming. Private deployments on computing clusters are also supported.

- Feedback: use the Issues page in the software repository, or open the relevant resources from the application's Help menu.

# Contents


Chapter 1 Getting started with KSSOLV Toolbox . . . . . 11
1.1 From structures to results . . . . . 11
1.2 Choose how to run KSSOLV Toolbox . . . . . 12
1.3 Understand the project structure . . . . . 14
1.4 Find your way through the manual . . . . . 15
1.5 Conventions used in this manual . . . . . 16
1.6 Start with a clear question . . . . . 16

Chapter 2 Installation and first launch . . . . . 17
2.1 Check the system requirements . . . . . 17
2.2 Before you install . . . . . 18
2.3 Install on Windows . . . . . 19
2.4 Install on macOS . . . . . 21
2.5 Install on Linux . . . . . 23
2.6 Install the toolbox in MATLAB® . . . . . 24
2.7 Your first launch . . . . . 24
2.8 What to check if something goes wrong . . . . . 27

Chapter 3 Using the workspace . . . . . 29
3.1 The main window . . . . . 29
3.2 Exercise: import and inspect silicon . . . . . 35
3.3 Save the project . . . . . 37
3.4 Choose what kind of file to save . . . . . 39
3.5 Use the Command Window . . . . . 40
3.6 Use the Quick Access Toolbar . . . . . 41
3.7 Find structures in Materials Project . . . . . 41
3.8 Before you continue . . . . . 43

Chapter 4 Using KSSOLV Toolbox Online . . . . . 46
4.1 Open your workspace . . . . . 46

4.2 Understand Online files . . . 48
4.3 Exercise: create a silicon project . . . 49
4.4 Bring a project back to your computer . . . 52
4.5 Check the current account . . . 54
4.6 Leave the workspace . . . 55
4.7 Manage the workspace connection . . . 57

Chapter 5 Calculate the electronic structure of a hydrogen molecule . 61
5.1 Prepare the hydrogen example . . . 61
5.2 Create a project and import the structure . . . 62
5.3 Build the SCF workflow . . . 65
5.4 Run the calculation and follow progress . . . 69
5.5 Read your first calculation result . . . 71
5.6 Complete the exercise . . . 73
5.7 Reproduce the example with a script . . . 73

Chapter 6 Choosing calculation parameters . . . 75
6.1 From atomic positions to electron density . . . 75
6.2 Choose a molecule or a crystal . . . 78
6.3 Functionals: approximate electron interactions . . . 80
6.4 Pseudopotentials: describe the ionic cores . . . 81
6.5 Plane-wave cutoff energy . . . 82
6.6 $k$ points and Brillouin-zone sampling . . . 84
6.7 Set electronic occupations . . . 87
6.8 Spin and magnetism . . . 88
6.9 Converge the SCF calculation . . . 88
6.10 Two accuracy requirements in geometry optimization . . . 90
6.11 Choose a hybrid exchange algorithm . . . 91
6.12 Distinguish ground and excited states . . . 91
6.13 Choose parameters for a new system . . . 92

Chapter 7 Editing atomic structures . . . 93
7.1 Understand the structure model . . . 93
7.2 Example: inspect a silicon cell . . . 96

7.3 Edit atoms precisely . . . 98
7.4 Apply and save structural changes . . . 100
7.5 Example: a silicon supercell with one vacancy . . . 101
7.6 Example: build benzene and pyridine from an aromatic ring . . . 105
7.7 Example: adjust an ethanol conformation . . . 109
7.8 Reuse modeling work . . . 114

Chapter 8 Building materials models . . . 115
8.1 Build a crystal from a space group . . . 115
8.2 Adjust the cell . . . 117
8.3 Screen defect configurations . . . 118
8.4 Example: build a Pt(111) surface . . . 119
8.5 Place adsorbates on a surface . . . 125
8.6 Treat surface boundaries . . . 129
8.7 Build a heterojunction . . . 130
8.8 Build a twisted bilayer . . . 133
8.9 Prepare an initial NEB path . . . 143
8.10 Example: build low-dimensional models . . . 146
8.11 Example: build polymer chains . . . 157
8.12 Pack molecular mixtures . . . 161

Chapter 9 Modeling tools in depth . . . 164
9.1 Edit structures precisely . . . 164
9.2 Work with periodic crystals . . . 174
9.3 Build soft-matter models . . . 184
9.4 Finish a surface model . . . 189
9.5 Reuse a modeling procedure . . . 190

Chapter 10 Building calculation workflows . . . 194
10.1 Edit a workflow . . . 194
10.2 Choose a task . . . 196
10.3 Add node ports . . . 197
10.4 Understand general node settings . . . 198
10.5 Allocate node resources . . . 198

10.6 Silicon: your first periodic calculation . . . 204
10.7 Silicon: bands along a high-symmetry path . . . 206
10.8 Silicon: calculate total density of states . . . 210
10.9 Hydrogen: find a lower-energy geometry . . . 211
10.10 Symmetry as an independent workflow . . . 214
10.11 View the Brillouin zone . . . 214
10.12 Plot band-energy surfaces . . . 218
10.13 Prepare an excited-state calculation . . . 219
10.14 Make workflows reproducible and comparable . . . 221

Chapter 11 Viewing volumetric data . . . 223
11.1 Understand the data . . . 223
11.2 Import volumetric data . . . 224
11.3 Isosurfaces: shapes at a fixed value . . . 226
11.4 Miller slices: inspect internal values . . . 232
11.5 Direct volume rendering and structure overlays . . . 236
11.6 Export volumetric results . . . 239
11.7 If the image is not what you expected . . . 241

Chapter 12 Interpreting calculation results . . . 243
12.1 Find results in the project . . . 243
12.2 Read a dataset . . . 244
12.3 Check electronic convergence . . . 246
12.4 Compare total energies . . . 247
12.5 Check the optimized structure . . . 249
12.6 Read a band structure . . . 250
12.7 Read the total density of states . . . 253
12.8 Check three-dimensional results . . . 256
12.9 Prepare atomic structure views . . . 257
12.10 Export atomic structure views . . . 260
12.11 Export calculation figures . . . 263
12.12 Write a complete account of the result . . . 266

Chapter 13 Configuring the application environment . . . 267

13.1 Adjust application settings . . . 267
13.2 Connect a chat service . . . 268
13.3 Set up Materials Project access . . . 274
13.4 Understand remote computing . . . 275
13.5 Create a cluster configuration . . . 277
13.6 Exercise: submit and retrieve a calculation . . . 279
13.7 Use the remote Command Window . . . 281
13.8 Find help . . . 282

Chapter 14 Troubleshooting . . . 284
14.1 The application will not start after installation . . . 284
14.2 A command appears to do nothing . . . 284
14.3 Online stays on the connection page . . . 285
14.4 An uploaded file does not appear as a project structure . . . 285
14.5 The displayed atom count is not what you expected . . . 285
14.6 A modeling command is missing or unavailable . . . 286
14.7 A preview disappears when the dialog closes . . . 286
14.8 A modeling command reports an error . . . 287
14.9 A workflow runs but produces no curve . . . 287
14.10 SCF reaches its iteration limit without converging . . . 287
14.11 Results differ from the tutorial . . . 288
14.12 Volumetric data is blank or has little color contrast . . . 289
14.13 A saved project cannot be found locally . . . 289
14.14 A remote job finishes but its results are not imported . . . 289
14.15 A chat question is interpreted as a command . . . 290
14.16 A new task does not produce the expected data . . . 290
14.17 Write a useful problem report . . . 290

Appendix A Quick reference . . . 292
A.1 Supported file formats . . . 292
A.2 Common units and conversions . . . 294
A.3 Keyboard and mouse reference . . . 295
A.4 Terminology . . . 297
A.5 Workflow task reference . . . 298

A.6 Accompanying example files . . . . . 299
A.7 References . . . . . 306

Subject index . . . . . .307

Chapter 1

# Getting started with KSSOLV Toolbox

KSSOLV Toolbox brings atomic structures, calculation steps, and results together in a single workspace. Start with a molecule or crystal, inspect its geometry, choose a calculation method, and use a workflow to obtain total energies, electronic energy levels, or other properties. This chapter explains how to use the manual and introduces the concepts you will need for your first calculation.

## 1.1 From structures to results

A materials calculation usually begins with a question: How does the total energy change with the lattice constant? What local environment does a defect create? How does a molecule's bond length change during geometry optimization? KSSOLV Toolbox provides tools for preparing structures and performing electronic structure calculations to investigate these questions. It describes the electronic system using plane waves and pseudopotentials, with KSSOLV solving the Kohn–Sham equations numerically.

You do not need to master all the theory before trying the software. Begin by understanding what system you are studying, which steps the software will perform, and what the resulting quantities can tell you. For a hydrogen molecule, the structure specifies the positions of two hydrogen atoms. A self-consistent field calculation finds the electronic ground state at those fixed positions. Geometry optimization then adjusts the atomic coordinates to find a lower-energy geometry. Understanding these distinctions matters more at first than learning every parameter.

### The basic calculation sequence

1. **Prepare the structure.** Import a file or create a structure with the modeling tools. Check the elements, atom count, lattice, and periodicity.
2. **Build a workflow.** Start from a template and select nodes for structure input, calculation settings, solving, and plotting.
3. **Set the calculation conditions.** Check the pseudopotentials, cutoff energy, $k$ points, convergence thresholds, and requested outputs.
4. **Run and check.** Read the output messages, iteration history, and completion status to determine whether the calculation finished successfully.
5. **Analyze and save.** Examine the numbers and plots, record the input conditions, and save the workflow and results in a project.

The same sequence applies to more complex tasks. Surface adsorption requires a surface model with vacuum; a band structure calculation requires a self-consistent electron density; and a remote calculation requires you to choose the execution environment before submitting the job. Later chapters explain these requirements through worked examples.

## 1.2 Choose how to run KSSOLV Toolbox

KSSOLV Toolbox is available as a standalone application, a MATLAB® toolbox, and KSSOLV Toolbox Online. They share the same core concepts of structures, projects, and workflows. Your choice mainly depends on your existing software environment, where you want to store files, and where calculations will run.

### Standalone application

The standalone application runs locally using MATLAB® Runtime. Projects, structures, and exported files are normally stored on your computer, and file dialogs use your operating system's desktop environment. Once Runtime

**Table 1.1** Requirements and typical uses for the three ways to run KSSOLV Toolbox.

| Edition | Requirements | Typical use |
|---|---|---|
| Standalone | No full MATLAB® installation or MATLAB® license is needed. The MATLAB® Runtime must match the application release. | Launch from your operating system, prepare structures, run calculations, and save projects locally. |
| MATLAB® toolbox | A supported MATLAB® installation with a valid license; MATLAB® R2025a or later is required. | Use the graphical workspace alongside MATLAB® variables, scripts, and other licensed products. |
| KSSOLV Toolbox Online | A browser; no client installation. The workspace and calculation session run on a server. | Work in a centrally configured environment, upload inputs, manage server-side projects, and download results. |

is installed, launch the application from the system application menu or a shortcut; you do not need to open MATLAB® first.

### MATLAB® toolbox

Install the toolbox in a supported MATLAB® environment to combine the graphical interface with the MATLAB® workspace and scripts. Structure import, modeling, workflow configuration, and result inspection use the same workspace described throughout this manual. Requirements involving MATLAB® variables, additional products, or remote jobs are noted in the relevant sections. The accompanying scripts provide reproducible inputs and reference results.

### KSSOLV Toolbox Online

KSSOLV Toolbox Online provides the workspace in a browser. The browser handles display and interaction, while a server session performs the calcu-

lations. The official Hanhai Quantum® service is forthcoming, and private cluster deployments are supported.

The Online working directory and your computer's download directory are separate locations. Uploading transfers local inputs to the server; saving a project writes the current work to server-side storage. To keep a local copy, download the project or export its results. Chapter 4 covers sign-in, file transfer, and session management.

## 1.3 Understand the project structure

A **project**, saved as a `.ks` file, holds the work for a task. It can contain several structures, several workflows, and their results. Think of it as a laboratory notebook that records both the systems under study and the way the calculations are organized.

A **structure** specifies the elements, atomic positions, and, for a periodic system, the unit cell. The 3D view helps you inspect its geometry. Sphere sizes, background colors, and display repetitions are visualization settings rather than electronic structure parameters.

A **workflow** defines how tasks fit together. Nodes represent tasks, and connections pass data between them. Connect a structure to an SCF calculation so that the solver can read it; connect the calculation output to a convergence plot to inspect the iteration history. A workflow can also be saved as a `.wf` file for reuse in another project.

**Data and figures** appear in the results branches of the Project Browser. Data can feed subsequent calculations, while figures help you inspect trends. An image alone is usually insufficient for further analysis, so retain the project and any necessary original data as well.

> ### Display repetitions and the calculation model
>
> Displaying a crystal as $2 \times 2 \times 2$ cells makes neighboring cells easier to see, but does not create a calculation model with eight times as many

atoms. To study a vacancy or dopant in an actual supercell, use Build Supercell. The modeling tutorial demonstrates this distinction.

## 1.4 Find your way through the manual

### Complete your first calculation

Read the installation, Online, and workspace chapters, then work through the hydrogen molecule calculation in Chapter 5. It covers importing a structure, opening a template, setting basic parameters, running the solver, and saving the results. The calculation fundamentals chapter will then help you connect the parameters you have used with their physical meaning.

### Prepare a materials model

If you are familiar with electronic structure calculations and want to build a model, begin with Chapter 7. The silicon, benzene, and ethanol examples introduce everyday operations on periodic structures and molecules. Chapter 8 extends these methods to surfaces, heterostructures, twisted bilayers, nanostructures, and soft matter. Chapter 9 explains individual tools and their parameters. After each structural change, check the atom count, unit cell, and local distances before passing the model to a calculation workflow.

### Organize and present results

Chapter 11 covers volumetric data, and Chapter 12 covers numerical and graphical results. Always interpret an output in the context of its inputs: Which structure does this total energy describe? Which path was used for this band structure? Which dataset produced this isosurface? These details give a figure a clear meaning and let you explain the calculation in a research report.

## 1.5 Conventions used in this manual

An interface path such as Home → Structure → Import structure from file... means to select the tab, button, and menu item in sequence. Labels follow the English application; `silicon.cif` denotes a filename. Keys appear as Enter, Esc, or Ctrl/Cmd+Z; Ctrl/Cmd means Ctrl on Windows/Linux and Command on macOS.

Copy the accompanying `examples` directory to your own working directory, such as `KSSOLV-Study`, before starting. Keep each exercise's input structures, projects, and exported figures in a separate subdirectory. Descriptive filenames such as `H2-SCF.ks` or `Si-vacancy.ks` make the contents easier to recognize later.

Tutorial parameters are chosen to demonstrate the procedure and explain the results. For your own research, test convergence against the physical quantity of interest. The relevant chapters explain how to vary parameters one at a time, compare results, and assess whether the accuracy is sufficient.

## 1.6 Start with a clear question

Start with a small, familiar system before creating complex structures or changing materials. Check the self-consistent calculation before adding band structure or density of states steps, and save a well-organized project before trying batch operations. You should be able to explain each step's purpose in a single sentence.

Keep brief notes as you work through the examples: the atom count, unit cell, main parameters, convergence status, and one or two observations are enough. These notes will help you connect models and results as your work progresses.

To find a specific task, use the workflow descriptions in Chapter 10 and the task reference in the appendix. They link the desired result to its required inputs and the relevant tutorial.

Chapter 2

# Installation and first launch

KSSOLV Toolbox can run as a standalone application, as a MATLAB® toolbox, or in a browser through KSSOLV Toolbox Online. Table 1.1 compares their requirements. This chapter covers standalone installation on Windows, macOS, and Linux, followed by MATLAB® toolbox installation and the first launch.

KSSOLV Toolbox Online needs no client installation. Once you have a browser and the service address, follow Chapter 4 to open a workspace. For a local application or MATLAB® toolbox, complete the relevant installation steps below, check the interface, import a small structure, and save a project.

## 2.1 Check the system requirements

The standalone application uses a matching MATLAB® Runtime. It does not require a full MATLAB® installation or a MATLAB® license. Runtime executes the application; it does not provide the MATLAB® desktop programming environment.

The MATLAB® toolbox is intended for users with a supported, licensed MATLAB® installation. It requires MATLAB® R2025a or later; see "Install the toolbox in MATLAB®" below. If you already have MATLAB®, choose the edition according to whether you need workspace variables, scripts, or other MATLAB® products.

### Understand the Runtime options

The standalone application needs the MATLAB® Runtime specified for its release. Packages generally provide it in one of three ways: a web installer downloads Runtime when needed, an offline installer includes it, and an application-only installer uses a compatible Runtime already on the com-

puter. The application version identifies KSSOLV Toolbox; the Runtime release identifies its execution environment. These are separate version numbers.

The installer checks for the required Runtime. Even if full MATLAB® is installed, follow the detection result to select a matching Runtime or install one.

> **Choose the right download**
>
> Check the operating system, processor architecture, and package description. Apple silicon and Intel Macs use different application architectures, and Windows and Linux packages are not interchangeable. Follow the filenames and instructions on the release page.

## 2.2 Before you install

Download the installer from the KSSOLV Toolbox release page and put it somewhere easy to find, such as your Downloads folder. Check the release notes for the version, operating system, and processor architecture before choosing a package.

Save your projects and close any older KSSOLV Toolbox instance before installing. Upgrades normally retain projects and personal settings. Keep an internet connection available for a web installation. For an offline package, keep all supplied files in the same directory.

Choose a local disk with enough free space. The application, Runtime, and your projects can be in separate directories: the installer manages the first two, while projects are best kept in a familiar working folder. For example, create a “Materials calculations” folder with subfolders for individual materials or projects. This makes your work easier to find than leaving everything in Downloads.

## 2.3 Install on Windows

### Run the installer

A Windows standalone distribution is normally a versioned EXE installer with `KSSOLV_Toolbox` in its name. Some offline distributions also contain `.bin` volume files with the same base name. Keep those files together and run the EXE; do not open the volumes separately.

1. Double-click the installer. If User Account Control appears, check the publisher information before allowing it to continue.
2. Choose an installation language and read the instructions. Select English to match this edition of the manual.
3. Read and accept the toolbox and Runtime license terms as prompted, then continue to the application location and Runtime settings.
4. Choose the application directory. Keep the default location or select another local disk.
5. Review the Runtime detection result. Reuse a compatible installation when offered, or choose the Runtime installation method supplied by the package.
6. Start installation and wait for file copying and final configuration to finish. Close the installer after it displays the completion page.

The application and Runtime can use separate locations, for example:

- Application: `E:/Apps/KSSOLV Toolbox`.
- Runtime: `E:/MathWorks/MATLAB Runtime`.

The Runtime path must use ASCII characters; letters, digits, and spaces are supported. Project names can describe your research in whichever language you prefer.

### Wait for installation to finish

The installer configures the runtime environment near the end of installation. This can take longer when Runtime is being installed for the first time. Wait until the installer reports completion.

If configuration does not finish, follow the retry instructions and check free disk space, directory permissions, and the Runtime source.

The default installation is for all users and normally needs administrator authorization. Per-user installation and reuse of an existing Runtime depend on the options provided by the package. A normal installation does not require changing the system PATH or copying libraries into a MATLAB® directory.

### Launch and update

After installation, start KSSOLV Toolbox from the Start menu or the shortcut created by the installer. A startup card shows the product name and loading status. When the main window opens, allow the project panels and ribbon to finish loading.

To update, run the new installer and follow its instructions to replace the older application. The installer handles the registered previous version; shared Runtime installations, personal settings, and projects are normally retained. Keep personal projects outside the application installation directory so that program files and research data remain separate during upgrades.

> #### Check offline package files
>
> If the installer cannot find an additional file, check that the EXE and all matching volume files are together. Also check whether the browser added a duplicate-number suffix to a filename. Confirm that every download is complete, then run the installer again.

## 2.4 Install on macOS

### Choose the package for your Mac

Open “About This Mac” from the Apple menu to check your processor. Apple silicon packages are for Macs with Apple chips. Confirm the architecture before choosing a download.

macOS uses the native Installer to install the PKG. The application normally goes into `/Applications`. The installer detects the required Runtime and, depending on the package, reuses, downloads, or installs it. These steps take place in the system installation wizard.

The introduction page lists the application version, processor architecture, and MATLAB® Runtime requirements. Check them against your Mac before proceeding. Also check whether the package contains only the application or includes Runtime; the introduction page for your package is the authoritative description.

1. Open the downloaded distribution and run its PKG installer.
2. Read the introduction and check the application version, processor architecture, and Runtime release.
3. Read and accept the license terms as prompted, then continue to installation type and location.
4. Select KSSOLV Toolbox and check the application location. The Applications folder is usually appropriate.
5. Review the Runtime settings and choose an existing installation or let the installer prepare one, as described below.
6. Begin installation and authenticate as an administrator when macOS requests it.
7. Wait for the installer to report success, then open KSSOLV Toolbox from Applications.

Figure 2.1 shows the license page. Read the terms in the window or use Print... or Save... to keep a copy. Click Continue, then confirm your choice in the agreement dialog. You must agree to proceed with installation. Installation is complete only when the wizard displays its success page.

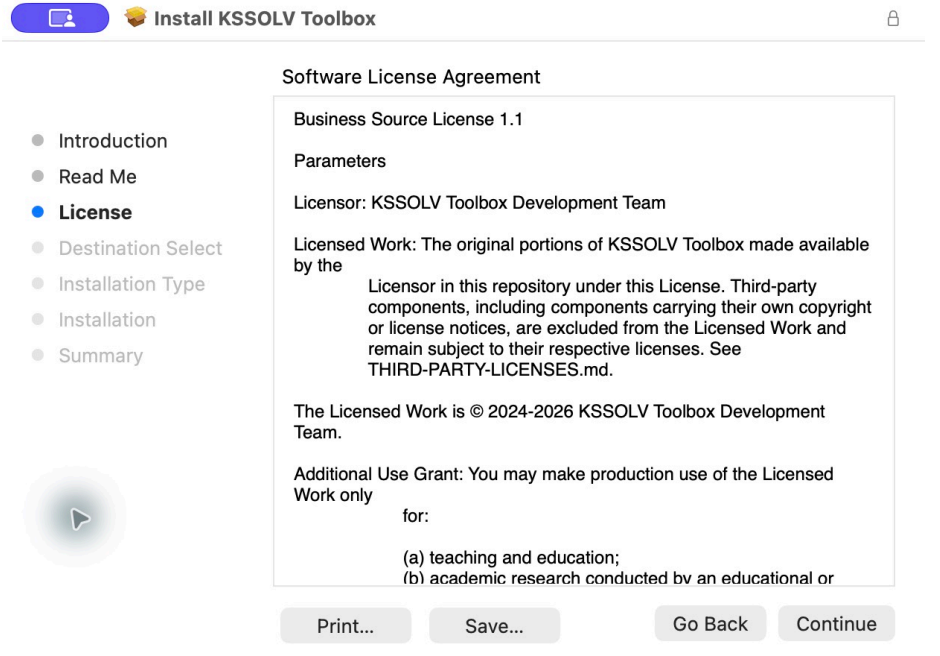



**Figure 2.1** The license page in the macOS installation wizard. Read the terms in the window, or print or save a copy.

### Use an existing Runtime

An application-only package requires a compatible Runtime to be installed already. If automatic detection does not find it, select "Existing MATLAB® Runtime" on the installation type page. Under Location, choose Other and select the release folder requested by the installer. Select the Runtime root directory rather than a library subdirectory within it.

A web installer downloads missing Runtime components from MathWorks; an offline installer uses the supplied files. An existing Runtime can

be reused when its release, architecture, and required components match. The installer prepares Runtime before replacing the older toolbox application.

Final configuration takes place in the background and does not necessarily open the Toolbox main window. Wait for the system installer to report success, then launch the application yourself. A startup card on first launch is normal. Later launches still load the runtime environment, but do not repeat installation.

### Upgrade while retaining projects

Save your work and close the application before running a new installer. There is no need to move shared Runtime installations, user settings, or projects simply because you are replacing the Toolbox. Keep `.ks` files in your Documents folder or a research directory, rather than inside the application bundle.

If macOS asks whether to open an application downloaded from the internet, review the displayed source information before confirming.

## 2.5 Install on Linux

Use the Linux standalone package in a graphical desktop session. If the downloaded installer is already executable, open it from the file manager. Otherwise, open a terminal in the directory containing it.

1. Change to the installer directory in a terminal.
2. Make the file executable: `chmod +x KSSOLV_Toolbox.install`.
3. Run `./KSSOLV_Toolbox.install` to open the graphical installation wizard.
4. Choose the application and Runtime locations, then wait for installation to finish.
5. Launch the application using the entry point or command shown on the completion page.

The filename above illustrates the usual form; use the actual name of your download. Keep the Runtime path and launch command shown at the end, especially if your desktop environment does not receive an application menu entry automatically.

The Linux standalone application requires a graphical desktop session. To use KSSOLV Toolbox Online, open the Online service in your browser instead.

## 2.6 Install the toolbox in MATLAB®

If you have full MATLAB®, use `KSSOLV_Toolbox.mltbx`. Confirm that MATLAB® is R2025a or later, then follow these steps.

1. Download `KSSOLV_Toolbox.mltbx` from the project release page.
2. Open the file in MATLAB® and complete the Add-On installation dialog.
3. Start KSSOLV Toolbox from MATLAB® or the Add-On Manager.
4. Wait for the main window before beginning a project exercise.

If another version is installed, use MATLAB®'s Add-On Manager to inspect and manage it. After installation or an upgrade, check that your MATLAB® and toolbox versions are compatible before continuing with existing projects.

This manual focuses on the graphical interface. Structure import, modeling, and workflow procedures generally apply to all three editions. Operations involving local files, server-side storage, the MATLAB® workspace, or additional MATLAB® products are identified where they occur.

## 2.7 Your first launch

The standalone application first displays a startup card. In Figure 2.2, the card shows the product version and loading status while the workspace is being prepared. Loading Runtime may take longer on the first launch or

after an update. Leave the startup window open and wait for the main window.




**Figure 2.2** The KSSOLV Toolbox V0.5.0 standalone startup card, showing the product version and loading status.

Once the startup card closes, the main window shows the Project Browser, ribbon, and central document area. You can then check the version and try a project.

### Check the interface and version

Locate the Home tab at the top and the Project Browser on the left. Choose Home → Help → About KSSOLV to check the product version. This manual describes V0.5.0. Select the interface language under Home → Settings → General; the change takes effect the next time the application starts.

Figure 2.3 shows an empty project with the main panels expanded. The ribbon provides structure import, workflow creation, and execution controls. The Project Browser lists the project's contents, and the Information Browser below it shows properties of the selected item. The central docu-

ment area initially displays a welcome message and opens a 3D view when you import a structure. The Config Editor on the right contains parameters, and the Simulation Browser shows run output. The Command Window at the bottom accepts interactive commands; the graphical tutorials do not require you to enter commands there.

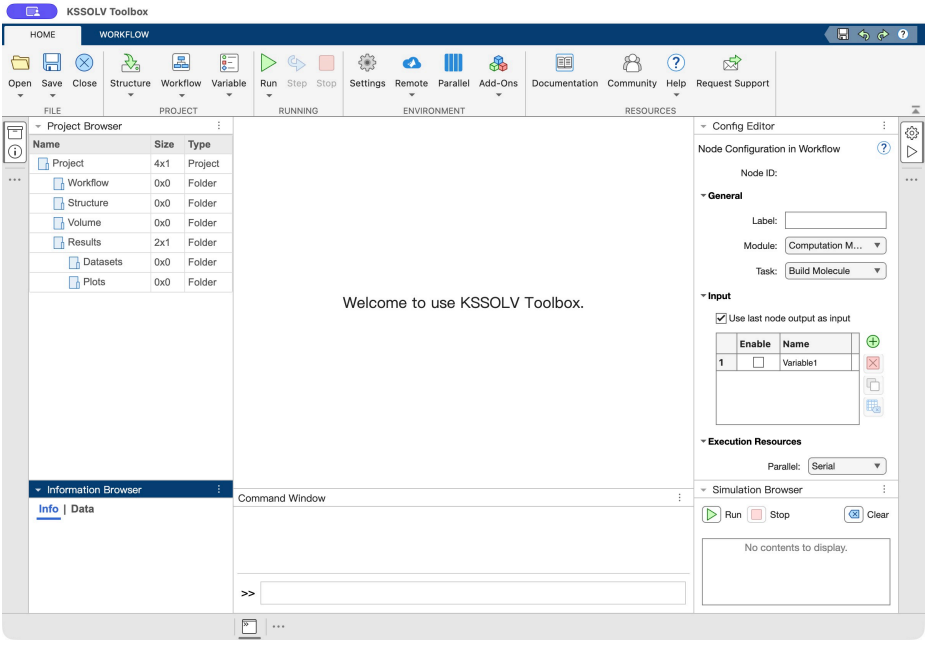



**Figure 2.3** The complete desktop workspace. The project, information, configuration, simulation, and command panels are expanded; no structure or workflow document is open yet.

Some panels are collapsed on first launch. Use the button groups at the sides to expand them, or choose a panel from the ellipsis button's Open Panels list. The first exercise mainly uses structure import and project saving. Chapter 3 explains each workspace area in detail.

### Create a project you can reopen

This exercise checks file handling and the interface without running a calculation. Use the supplied `examples/silicon.cif`, or your own small CIF or

XYZ file. The supplied silicon structure also provides a direct starting point for the workspace exercise in Chapter 3.

1. Choose Home → Structure → Import structure from file... and select the structure file.
2. Check that its name appears in the Project Browser and that atoms appear in the central 3D view.
3. Drag to rotate the view and use the mouse wheel to zoom. Check that the complete structure is visible.
4. Click Save and save `first-project.ks` in your working directory.
5. Once saving is complete, close the current project and reopen the saved file through Open Project....
6. Check the Project Browser and structure document to confirm that the imported structure is still present.

You have now checked structure import, 3D display, and project saving. Chapter 5 takes you through your first electronic structure calculation.

## 2.8 What to check if something goes wrong

If the application does not start, check that the package matches your processor architecture and that Runtime meets the installer's requirements. When reporting a problem, include the Toolbox version, operating system, installation location, and the complete message shown in the window.

If the main window opens but no 3D structure appears, retry with the small structure used in this chapter and look for a specific graphics message. Online also requires a browser with working WebGL support. If importing a file adds no project item, check the file's format and contents.

If you cannot find a saved file, check where it was saved. The standalone application and MATLAB® toolbox use local directories; KSSOLV Toolbox

Online uses the server's working directory. Chapter 4 covers the server-side Files panel and upload and download exercises.

### Organize files for the next exercises

Use a consistent project directory. Keep original structures separate from working projects and retain their source information and filenames. Save project copies after structure preparation, parameter checks, and result analysis so that you can return to an earlier stage. When moving work between computers, transfer the complete `.ks` project where possible; a structure file alone usually does not contain workflows or calculation results.

Chapter 3

# Using the workspace

The workspace is where you inspect structures, organize calculations, and retain results. This chapter introduces the main window, then takes you through importing, inspecting, naming, and saving a silicon project. These habits will carry through the later tutorials.

## 3.1 The main window

The workspace brings structures, calculation steps, and results into one window. Start by recognizing the empty layout, then open the panels needed for an exercise. This helps distinguish what the project contains from what you are editing at the moment. Use the ribbon to select commands. The central document area displays structures, workflows, and figures, and the side panels hold project items, object information, parameters, and output.

### Your first view of the workspace

An empty project initially shows a welcome message with no structure or workflow document open. Figure 3.1 shows this layout: the Project Browser lists categories on the left, the Information Browser below it is collapsed to its title bar, and the right-side panel group is closed. Import a structure or create a workflow to open its document in the central area.

Locate Structure and Workflow on the Home tab, then inspect the buttons at the sides. Project and information buttons control one panel group on the left; configuration and simulation buttons control another on the right. A group button collapses or expands that side's region. The small triangle in an individual panel title expands its contents. If a panel is missing, open the sidebar ellipsis and select it by name from Open Panels.

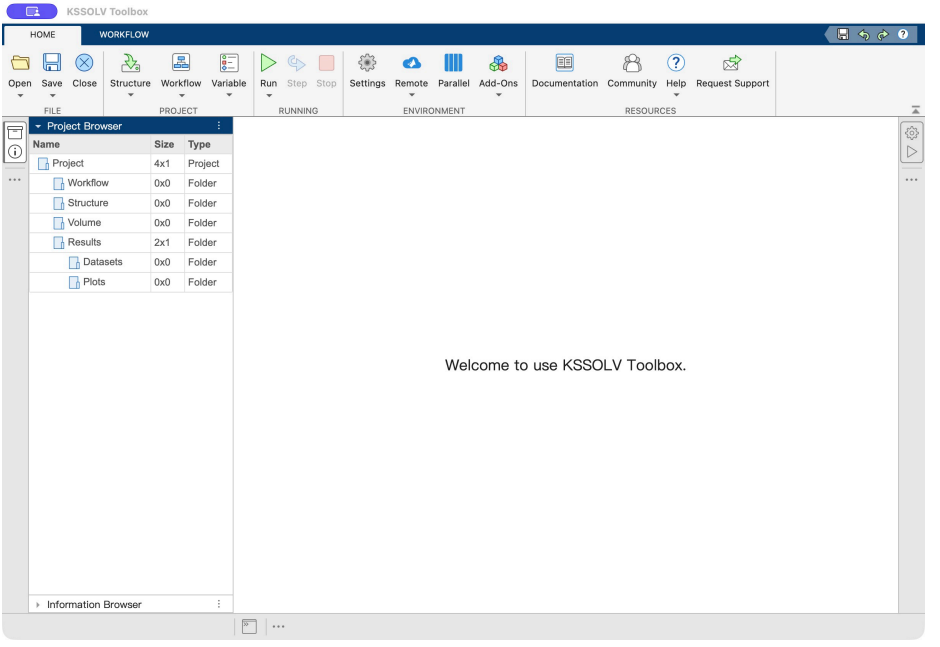



**Figure 3.1** The initial desktop workspace. No document is open in the center. The Project Browser lists project categories; the Information Browser is collapsed to its title bar, and the right-side panel group is closed.

## Open the panels needed for an exercise

Your first calculation involves moving between the project, structure, workflow, and parameters. Figure 2.3 shows an empty project with the main panels expanded: project and information on the left, configuration and simulation on the right, the Command Window below, and the welcome message in the center. Some panels are collapsed on first launch.

Expand the two side groups and their individual panels to learn their locations. Importing a structure or creating a workflow adds items to the tree and document tabs in the center. Double-click a project item to open its document, then select a tab to make it active. A structure tab shows a 3D model; a workflow tab shows nodes and connections. Check the active workflow's name before configuring nodes or running a calculation.

**Table 3.1** Main workspace areas and what to look for first.

| Area | Location | First check |
|---|---|---|
| Ribbon | Top of the window | Find structure import, workflow, save, and run actions on Home. |
| Quick Access Toolbar | Upper-right corner | Identify Save, Undo, Redo, and Help. |
| Project and information | Left-side group | Recognize the categories, then select an imported structure to view its information. |
| Configuration and simulation | Right-side group | Locate parameters and output. After creating a workflow, select a node to configure it. |
| Document area | Center | An empty project shows the welcome message. Use tabs to switch between open structures and workflows. |
| Command Window | Expanded lower region | Use it for interactive commands when needed; it can stay open or collapsed during graphical tutorials. |
| Dividers and status bar | Panel boundaries and window bottom | Drag dividers to adjust space and watch the status text during operations. |

Adjust the layout to suit the task. Collapse unused side panels to inspect a 3D structure, or widen the Config Editor when entering parameters. Drag panel boundaries to change widths or vertical proportions; structures and workflows remain in the project. The following sections explain the areas in detail, and the silicon exercise puts them to use.

### The ribbon

Home groups common file, project, run, environment, and resource actions. Open reads an existing project, and Save writes the current work to a project file. Structure creates or imports structures; Workflow provides creation,

template, import, and export actions. Use Run to execute a calculation. Settings and help are also available from Home.

The Workflow tab provides node and canvas operations for the central workflow document. Activate the intended workflow before adding, connecting, or configuring nodes. Selecting the ribbon tab alone does not identify which workflow will run; check the central document tab too.

When a 3D structure document is active, Crystal Modeling or Molecule Modeling appears on the ribbon. These contextual tabs provide tools for the current document and may change when you switch document types. If a modeling command is missing, return to the structure tab before considering another import.

### Project Browser

The Project Browser presents project contents as a tree. Its main branches are Workflow, Structure, Volume, and Results. Results contains Datasets and Plots. These correspond to workflows, atomic structures, volumetric data, numerical data, and figures.

Single-click an item to select it, or double-click to open its document. A project can retain many items at once. Closing a central document tab closes that view; it does not delete the item. Double-click the item again to reopen it. A removal action, by contrast, changes the project contents.

Name is the display name, Type identifies the object type, and Size describes item organization or data dimensions. Do not interpret Size directly as the atom count. Read structure information or the site count in the 3D view to find the independent atoms in a crystal. Displayed spheres may include neighboring-cell images and need not equal the model's site count.

### Information Browser

The Information Browser follows the Project Browser selection. Select a structure to inspect its properties, or a workflow, dataset, or figure to inspect that object. Use it to confirm which item you are working with, especially when names are similar.

Some items allow their display names to be edited. Check the updated name in both the tree and document tab. Names describing the material and principal change, such as `Si-primitive`, `Si-2x2x2`, or `Si-vacancy`, are more informative than `Structure1`. Cell details and atom counts remain available in the structure information.

### Config Editor

The Config Editor changes workflow-node parameters. Select a node on the canvas, then choose Workflow → Configure Node or use the node's gear button. Scroll down to its Options. General fields such as the node name, inputs, and outputs are separate from task parameters such as cutoff energy and iteration limits. If only general fields are visible, invoke Configure Node before scrolling. After entering values, switch to another node and back to check that the entries were committed.

### Read run output

The Simulation Browser displays calculation output, including SCF iterations, messages, and errors. After a run, inspect its final messages and the new items under Results. Chapter 5 demonstrates how to judge completion using a small calculation.

### Arrange the panels

The document area can hold structure, workflow, and figure tabs simultaneously. Selecting a tab changes the active document. If tabs are hidden, move the pointer to the top of the document area to reveal document titles and tabs. Check the active workflow name before running. Clear names help avoid running the wrong one when several similar tasks are open.

The sidebars control panel groups. Project and information icons belong to the left group; configuration and simulation icons belong to the right. Collapsing the left group hides both the Project Browser and Information Browser; expanding it makes both available again. The right group similarly allows configuration and output to be viewed together. Figure 3.1 shows

the right group collapsed, while Figure 2.3 shows the manually expanded layout.

Individual panels can also be collapsed. The initial Information Browser shows only a title bar at the lower left; click its triangle to reveal the contents. Collapsing a panel affects just that panel, whereas collapsing the side group hides the entire region.

The sidebar ellipsis opens Open Panels. Select Project Browser, Information Browser, Config Editor, Simulation Browser, or Command Window by name if you cannot find it. Figure 3.2 shows the Online list, which also includes Files for server-side storage. The other five panels are available in the standalone application too.

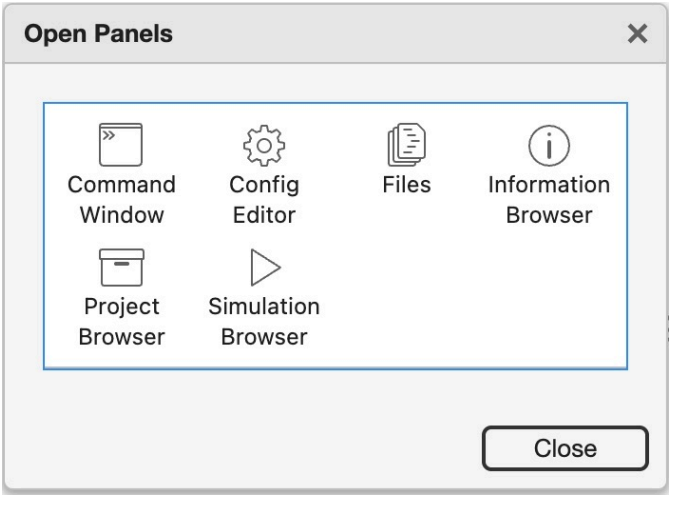



**Figure 3.2** The panel list. Open an individual panel by name; sidebar buttons collapse or expand panel groups. Files manages server-side storage.

Collapse unused parameter and output panels while inspecting a model, then reopen configuration when needed. Drag the dividers between panels and documents to change widths, or the divider within a group to change vertical proportions. Choose widths appropriate to your screen.

## 3.2 Exercise: import and inspect silicon

The supplied `silicon.cif` describes a conventional silicon cell with sides of approximately 5.44370 Å and 8 explicitly listed sites. It uses P1 to list every position, making the atom count easy to check. The lattice constant is an input to this example, not a result calculated during import.

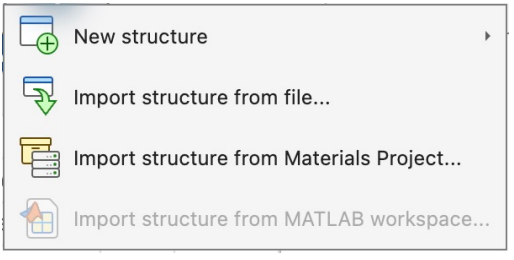



**Figure 3.3** The Structure menu on Home. This exercise uses Import structure from file.... Materials Project is another source. Importing from the MATLAB® workspace requires the MATLAB® toolbox edition.

1. Open Structure on Home, then choose Import structure from file....
2. Select `silicon.cif` in the supplied example directory and open it.
3. Wait for `silicon` to appear under Structure and for its 3D document to open.
4. Drag to rotate and use the mouse wheel to zoom. Fit the whole cell in the view; there is no need to adjust every display setting immediately.
5. Check that the element is Si and there are 8 independent sites. Any extra spheres may be boundary or neighboring-cell images.
6. Select a site to inspect its atom information, then clear the selection using empty space or the appropriate selection action to view the complete structure again.

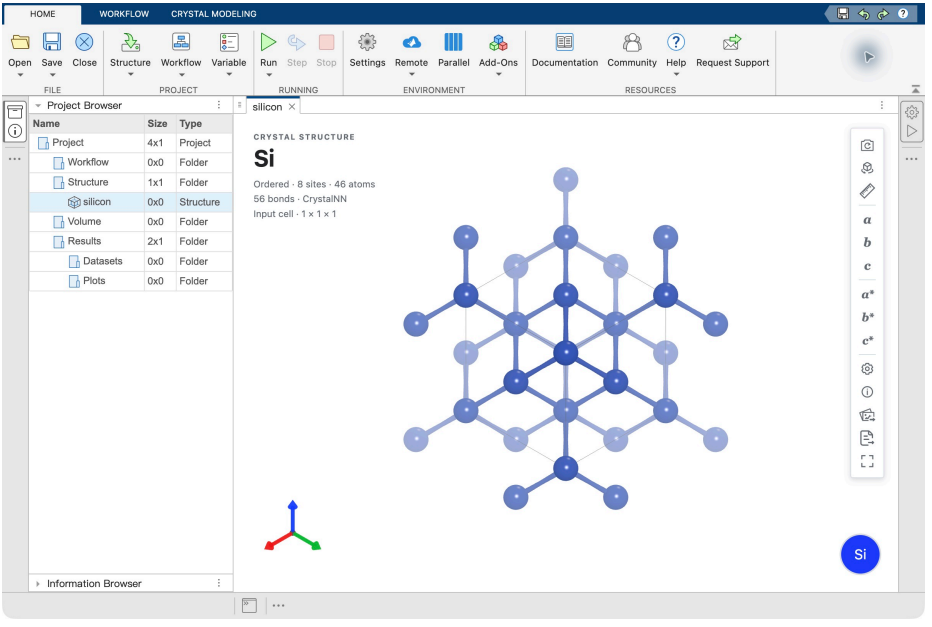



**Figure 3.4** The imported silicon structure. The summary reads 8 sites · 46 atoms: 8 independent input sites and 46 displayed spheres including periodic boundary images.

### Import several files at once

The desktop file picker allows multiple structure files to be selected. Use Import structure from file... and open the selected inputs together when comparing models. Each successfully read file becomes a separate project structure; the atoms are not automatically combined into one model.

Open each added item under Structure to inspect it. If an individual file fails to parse, address its diagnostic separately; the other imported structures remain in the project. In Online, upload the files first, then import them individually through Files. A successful upload does not itself import a structure.

### Read site information

The upper-left summary describes the structure and current display. Here, 8 sites refers to the eight silicon positions in the file; 46 atoms includes the pe-

riodic images drawn near the boundaries. Input cell · 1 × 1 × 1 indicates the input-cell display. Additional boundary spheres clarify connections across cells without turning the model into 46 independent atoms.

Select a silicon–silicon bond to inspect its sites and geometric length. Figure 3.5 shows Sites 2–7, a length of 2.35719 Å, and image indices 0, 0, 0. Site indices identify atoms in the model; image indices identify the cell translation used by that connection. Check both when inspecting other bonds.

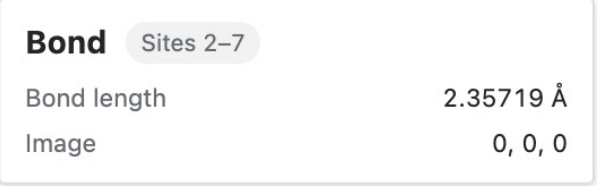



**Figure 3.5** The inspection card for a selected silicon–silicon bond. Its length of 2.35719 Å is a geometric property of the input structure.

Clear the selection afterward to restore the complete view. The aim is to understand how file contents, model sites, and displayed images relate. Use the same checks after building a supercell or optimizing a structure: read the structure information first, then inspect relevant atoms and connections.

For a CIF from another source, check the cell, occupancies, elements, and atom count after import. Files may use primitive cells, conventional cells, or partial occupancies. An identical chemical formula does not guarantee an identical structure. Understand the file's representation before converting or expanding it.

## 3.3 Save the project

### The first save

1. Click Save on Home.
2. In the standalone edition, choose your working directory and enter `Si-introduction.ks`. In Online, choose a server-side directory as described in Chapter 4.

3. After saving, check the window title and project filename.
4. Make a small, recognizable change, such as renaming the structure, and observe the asterisk in the title.
5. Save again and confirm completion.

An asterisk indicates unsaved changes. Saving retains workflows and results as well as pending structure edits. Modeling has distinct preview, apply, and save stages: preview checks a proposed change, Apply accepts it, and Save writes the current work into the project file. Chapter 7 demonstrates them through editing exercises.

### Keep copies for different studies

Use Save As... to branch a project for different conditions. For example, save the original silicon project, then save a copy as `Si-supercell.ks` before building a supercell. This retains a clear starting point. Renaming a project does not change the model; inspect the structure to confirm that the intended operation was performed.

For cutoff comparisons, you can keep several clearly named workflows in one project. Separate projects may be easier to organize if results are numerous or substantially different. Either approach is valid as long as results can be traced to their structures and parameters.

### Reopen and check

Close the project and use Open to read the saved `.ks` file. Expand Structure, double-click silicon, and check that its view reopens. This confirms that you saved a usable project rather than merely an image of the screen.

In Online, opening a saved server-side project and opening a downloaded local copy use different routes. Select the existing `.ks` in Files for the former; upload the local copy first for the latter. The contents may be identical, but check the storage location.

### Close the current project

Home → Close closes the entire current project, not just one central structure tab. Unsaved project or structure changes trigger a prompt to save, discard, or cancel. Choose Cancel to stay in the project. If saving, wait for completion before the empty project opens. Closing a document tab instead retains its tree item for reopening.

### Remove trial items

Remove workflows, structures, or figures you no longer need from the project. Save a separate project copy first if you want to retain a milestone.

1. Select the specific trial item in the Project Browser, rather than Project or a category folder.
2. Keep focus in the tree and press Backspace to remove it. A workflow can also be removed by selecting its project item and choosing Delete on the Workflow tab.
3. If its document has unsaved changes, respond to the close prompt. If you cancel, return to the tree and check the selection.
4. Check that the remaining structures and workflows still correspond, then save. Removing a project item does not delete its original source file from disk.

## 3.4 Choose what kind of file to save

`.ks` stores the complete project; `.wf` stores a workflow; CIF, POSCAR, and XYZ mainly store structures. Choose according to what a collaborator needs. A project is usually best for continuing a calculation. A workflow template is useful for reusing task organization with another material. Export a structure file when only geometry is needed.

Each structure format retains different information. CIF is suited to cells and crystal positions. XYZ directly represents elements and Cartesian co-

ordinates, but ordinary XYZ does not fully specify periodic cells or explicit bonds. When saving important molecular connectivity or a complex periodic model, check what the chosen format supports and inspect the result after reimporting it.

## 3.5 Use the Command Window

The Command Window is an additional entry point for interactive commands. Standalone and Online sessions support the functions and built-in operations included in their distribution. The supplied reproducibility scripts run in full MATLAB®. Normal import, modeling, and workflow tutorials do not require this window. Before using a command, check that it addresses the intended workspace object, then verify the result in the tree or view.

Documentation and Help on Home provide further reading; Help → About KSSOLV shows version information. If behavior differs from this manual, first check the version and active document type. Chapter 14 explains how to prepare a problem report and collect logs.

### Run a simple command

Open Command Window from Open Panels and place the cursor in the input area after >>. Press Enter to submit a command; its output appears below. This exercise uses a simple array without changing project structures.

```
a = [1 2 3];
sum(a)
```

Submit the lines separately; the second should return 6. Use `whos` to inspect variables in the command workspace. Up and Down recall previous commands, Tab selects completion suggestions, and Esc closes suggestions. `clc` clears the displayed history without deleting variables. Copy any commands and output you need into your notes before clearing it.

Command variables and project items are stored differently. Creating `a` does not add a structure to the project, and saving a project does not save every temporary command variable. Check the corresponding tree items when retaining structures, workflows, and results. For repeatable command sequences, prepare a script in full MATLAB®.

## 3.6 Use the Quick Access Toolbar

The toolbar at the top provides Save, Undo, Redo, and Help. Quick Save saves the same project as Home → Save; it does not export the current figure. With several documents open, check which is active: Undo and Redo address structure-modeling history in a structure document and graph-editing history in a workflow document.

1. Make a recognizable edit in the 3D structure and observe the change.
2. Click Undo to return to the previous state, then Redo to restore the edit.
3. Once the model is correct, click Save and check the save indicator.

After creating the first workflow in Chapter 5, move a canvas node and try Undo and Redo to compare these two editing contexts.

Saving files, running calculations, and undoing edits are separate operations. Undo does not restore external files or reverse a completed scientific calculation. Retain separate results and explicit parameter records when comparing calculations.

## 3.7 Find structures in Materials Project

A database search provides a starting structure when you do not yet have a local file. Save your Materials Project API key in Settings as described in Chapter 13, then choose Home → Structure → Import structure from Materials Project.... Query conditions appear at the top, the periodic table in the middle, and results with pagination below. After import, create a calculation workflow for the selected structure.

### Choose a search mode

**Table 3.2** The three Materials Project search modes.

| Mode | Use |
|---|---|
| Only these elements | Results contain every selected element and no others. Select only Si to search elemental silicon. |
| At least these elements | Results contain the selected elements and may include others. Select Fe and O, for example, to search materials containing both. |
| Formula / MP ID | Enter a formula or known material identifier to locate a particular record. |

The first two modes use periodic-table selections, with the element text updating above. Switch to Formula / MP ID for a formula search. Clear elements removes the current selection before you choose another combination.

### Exercise: find a silicon structure

1. Choose Only these elements, clear the existing selection, and select Si in the periodic table.
2. Click Search and wait for the query to finish. If there are no records, check the mode and selection before checking access settings.
3. Inspect Material ID, formula, crystal system, space group, and atom count. One formula can have several structures; do not simply choose the first row.
4. Choose 10, 15, 20, or 30 entries per page. Use the first/last and previous/next page buttons to browse. Click a column heading to sort the current results when needed.
5. Select mp-149: Cubic, Fd-3m (227), 2 atoms. Selecting the row enables Import selected; click it and wait for retrieval to finish.

6. Close the search dialog, return to the Project Browser, and find the new Structure item. Open its 3D document, check the elements, cell, and independent site count, then save.

Database results change over time. Identify the material by Material ID and structural information. mp-149 imports a two-site primitive cell, representing the same crystal as the eight-site conventional silicon cell. For detailed comparisons with the supplied reference calculations, use the specified `silicon.cif` or `silicon_primitive.cif` to keep inputs identical.

The table also reports volume, density, band gap, formation energy, energy above hull, and stability. Units appear in the column headings: gaps are in eV, while formation and hull energies are in eV/atom. These are database values; cite the relevant record when using them. An asterisk marks a record observed experimentally. Record the Material ID to document the structure's source.

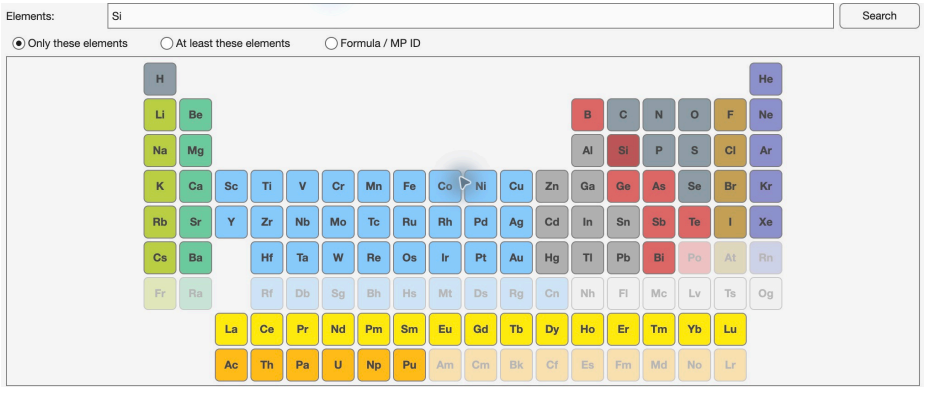



**Figure 3.6** Si selected in the periodic table with Only these elements to search elemental silicon. The search mode and elements appear above the results.

## 3.8 Before you continue

You should now be able to locate the imported silicon structure, open and close documents, expand and collapse side panels, and save a `.ks` project. Retain this silicon project and have the supplied hydrogen example ready

| Material ID | Formula | Crystal system | Space group | Sites |
|---|---|---|---|---|
| ★ mp-27 | Si | Cubic | Fm-3m (225) | 1 |
| ★ mp-34 | Si | Hexagonal | P6/mmm (191) | 1 |
| ★ mp-92 | Si | Tetragonal | I4_1/amd (141) | 2 |
| mp-109 | Si | Orthorhombic | Fmmm (69) | 4 |
| ★ mp-149 | Si | Cubic | Fd-3m (227) | 2 |
| ★ mp-165 | Si | Hexagonal | P6_3/mmc (194) | 4 |
| ★ mp-168 | Si | Cubic | Ia-3 (206) | 8 |
| ★ mp-10649 | Si | Hexagonal | P6_3/mmc (194) | 2 |
| ★ mp-16220 | Si | Cubic | Fd-3m (227) | 34 |

**Figure 3.7** Identity columns in the silicon search results. The selected mp-149 record is Cubic, Fd-3m (227), with two atoms. Other silicon polymorphs are also listed.

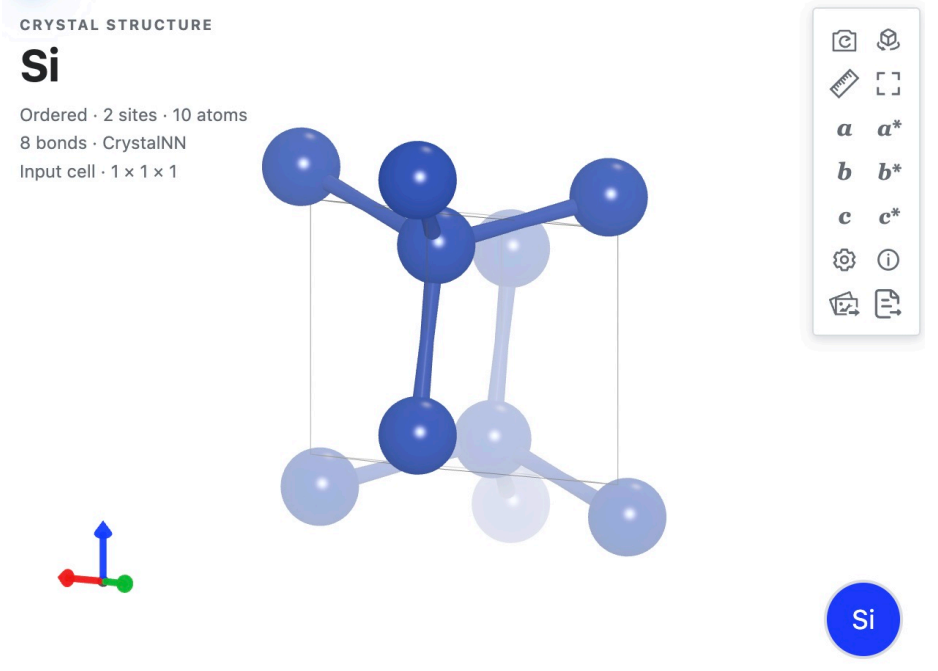



**Figure 3.8** The silicon primitive-cell view opened after importing mp-149. The summary reports two independent sites; translucent images illustrate connections across periodic boundaries.

before your first calculation. Later chapters return to silicon; use the structure file specified in each chapter when comparing numerical results.

Chapter 4

# Using KSSOLV Toolbox Online

KSSOLV Toolbox Online brings the toolbox workspace to your browser without a local client installation. You edit structures, build workflows, and analyze results in the main window, while the server performs the calculations. The official Hanhai Quantum® Online service is forthcoming, and private cluster deployments are supported. Sign in through the entry page provided by your deployment to open your allocated workspace.

Files and sessions are the main differences from the desktop application. Upload local inputs before using them, save projects in the server-side Files panel, and download them to obtain copies on your computer. Closing a page, returning to the entry page, and signing out also have different effects. This chapter connects these operations in a complete silicon-project exercise.

## 4.1 Open your workspace

### What you need

Have the service address, a working account, and one of the small supplied structure files ready. Registration may require an invitation code; follow your deployment's registration instructions. Your browser must support WebGL to display atomic structures in 3D.

Use a reasonably large browser window. In a small window, both the central view and parameter editor become narrow, and long forms may require scrolling. On a laptop, collapse side panels you are not using to leave more room for the ribbon and active document.

Use one active browser tab per workspace. If the workspace is open in another tab, return to that tab or close the old connection before reconnecting.

### Register an account

The service entry page initially displays the sign-in form. To create an account, switch to registration and complete the fields in order. Usernames must start with a letter and contain 3–32 lowercase letters, digits, or hyphens. Passwords must contain 12–128 characters and be entered again for confirmation.

Registration also requires a recovery question, recovery answer, and invitation code. Choose a question and answer you can remember without disclosing your password. Complete the form, register the account, and return to sign-in after registration succeeds.

### Sign in and wait for startup

1. Enter your account details on the service entry page.
2. Choose the action to sign in and open the workspace.
3. Wait while the entry page reports workspace preparation. The first startup may take longer.
4. Enter the toolbox when the workspace is ready. If the entry page displays an action to open the workspace, select it.
5. Check that the ribbon is visible, that the Project Browser and Files panels can be opened on the left, and that the central document area responds.

The portal may report states such as starting and ready. Keep the page open as the state changes; repeatedly clicking the startup action is unnecessary. If your organization has reached its concurrent-workspace limit, follow the portal’s instructions and try again later. The deployment administrator determines whether capacity can be increased.

## 4.2 Understand Online files

### The project tree and file tree

Use the left-side buttons to switch between the Project Browser and Files panel. They occupy the same area, with one shown at a time. Return to the Project Browser to work with structures, or switch to Files to inspect uploaded files.

The Project Browser organizes the contents of the current project: structures, workflows, calculation data, and figures. The Files panel shows files and folders in server-side storage. A single `.ks` file can contain many project items, so the two trees show different things.

Think of a project as a notebook holding your structures and results, and the server-side folder as the shelf where notebooks are stored. Importing adds material to the notebook; saving writes the notebook to a `.ks` file. Uploading a CIF only places the source material on the shelf; it does not yet add it to the current project.

**Table 4.1** What common file operations do.

| Operation | Result |
|---|---|
| Upload | Copies a local file into the current server-side directory. |
| Import | Reads a file and adds a structure, workflow, or other object to the project. |
| Save | Writes the current project to a server-side `.ks` file. |
| Download | Saves a copy of the selected server-side file on your computer. |
| Open project | Reads a `.ks` file as the current project. |

### Everyday operations in Files

The tree is sorted by name, with folders before files. Expand a folder to see its contents. The header arrow switches between ascending and descending order. Open the ⋮ menu or right-click an item for file actions.

When a folder is selected, its path appears at the bottom of the panel. Selecting a file also shows its size. Upload, New folder, and the first project save use the current directory, so check that path before proceeding.

Refresh rereads the directory and normally preserves expanded folders and the selection. If a new file is not visible after another operation, refresh and check that the correct folder is expanded. Organizing files by study or exercise is easier than repeatedly searching a long directory listing.

Figure 4.2 shows this exercise after saving, with Files on the left. Use the Project Browser's side button to return to structures and workflows inside the project.

## 4.3 Exercise: create a silicon project

### Create an exercise directory

Use the supplied `examples/silicon.cif`. The same file operations apply to another small structure. Start with a separate directory so that source structures, working projects, and downloaded copies are easy to distinguish.

1. In Files, select the root or your chosen working directory.
2. Choose ⋮ → New folder and enter `Si-study`.
3. Check that the folder appears, then select and expand it.
4. Check the path at the bottom of the panel to confirm that `Si-study` is the current destination.

### Upload the structure

1. Choose Upload in the `Si-study` directory.
2. Select the local silicon CIF in the browser or system file picker.
3. Confirm your choice and wait for the transfer to finish.
4. Check the filename and size in the tree; use Refresh if necessary.

Browsers supporting a native file picker open the system dialog directly. Other browsers may first show a selection button within the page. In both cases, choose a file from your own computer. Keep the page and transfer window open until uploading finishes.

The upload is complete when the file appears in the server-side directory. Import it next to add the structure to the project.

Figure 4.2 shows the workspace after upload and import.

### Import the file into the project

1. Double-click the uploaded CIF in Files, or select it and choose Import.
2. Check that a structure item has been added to the Project Browser.
3. Check that atoms and the unit cell appear in the central document area.
4. Drag to rotate the 3D view and use the mouse wheel to zoom until the whole cell is visible.

The server-side CIF remains the source file. The imported project structure is the object used for modeling and calculation. Building a supercell, deleting atoms, or editing that object does not automatically rewrite the original CIF. Save a project to retain the work as a whole, or export a structure when you need a separate structure file.

The file tree treats `.ks` and `.wf` differently: double-clicking `.ks` opens a project, while double-clicking `.wf` imports a workflow. Other recognized structure files are imported as structures. Double-clicking a folder expands it; it is not imported as a structure.

### Save the project for the first time

1. Check that `Si-study` is selected in Files.
2. Click Save, or press Ctrl+S; use Command+S on macOS.
3. Enter `Si-ground-state.ks` in the in-page dialog and confirm.
4. Check that the project file appears in the exercise directory.

5. Check the title and save status to confirm that your changes are saved.




**Figure 4.1** The server-side project save dialog. The filename is Si-ground-state.ks, and the selected `Online/Si-study` directory is shown in parentheses.

Enter a filename, not a full server path. The dialog uses the selected directory and appends `.ks` if you omit it. Subsequent Save operations update this project file.

Once a project has a save location, selecting another folder in Files does not change where Save writes it. Use Save As... to rename it, keep a milestone copy, or move to another directory. The Files menu's Save project to Files also asks for a name and can save the current project into the selected directory.

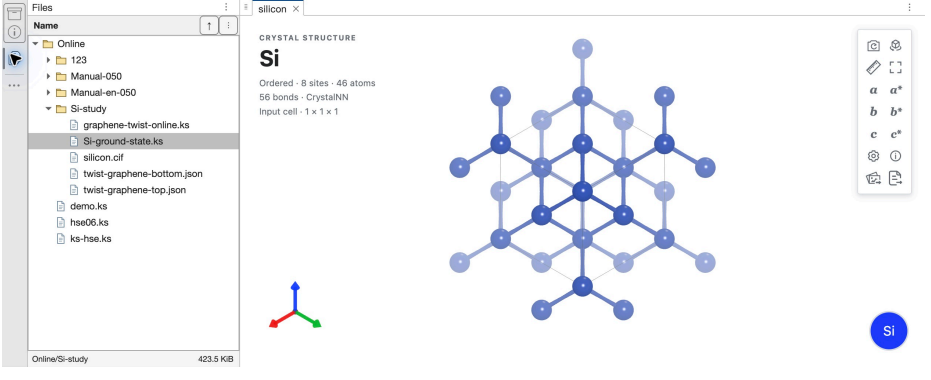



**Figure 4.2** The relationship between files and the model. Si-ground-state.ks is selected in Si-study on the left. The central view contains 8 independent sites and 46 displayed atoms including periodic images.

In Figure 4.2, the model contains 8 independent sites, while the 46 displayed atoms include periodic images. The project saved in this section contains the silicon structure. Chapter 10 covers its ground-state calculation.

### Save a comparison copy

Materials modeling often involves comparing supercells, defects, or parameters. Save a milestone project before a major change to make those comparisons easier to follow. Here, practice Save As without changing the model.

1. Select the destination directory in Files.
2. Open the save menu and choose Save As....
3. Enter a new name, such as `Si-comparison.ks`.
4. Check that the new file appears and the original remains.

After Save As, further edits apply to the new project file. Names can identify the material, operation, and stage, such as a silicon primitive cell, silicon supercell, or optimized geometry. You need not pack every calculation parameter into a filename; the project retains the detailed settings.

## 4.4 Bring a project back to your computer

### Download a server-side file

Server-side saving lets you resume work after signing in again. Local downloads provide backups or copies for sharing. Use both where appropriate. A download contains the file's currently saved contents, so save before downloading.

1. Select `Si-ground-state.ks` or your comparison project.
2. Choose Download from the context menu or ⋮.
3. Wait for file preparation. Chrome may open a local save dialog immediately; choose a folder, check the project name, and click Save.

4. If no system dialog opens, use the in-page Save as… action or Browser download to use the browser's native download mechanism.
5. Wait for the browser's download list to report completion. If the transfer window has a Finish button, click it only after confirming that the local file was saved.
6. Check the project filename and size in your local file manager.




**Figure 4.3** The local macOS save dialog for the supplied silicon project. Choose a directory, check the project name, and save. The browser may add a number if a file with the same name exists. System dialog labels follow the operating system language.

Your browser may save directly into Downloads or ask for a location each time. Check its download list if you are unsure where a file went. Ready to download in an in-page dialog means that preparation is complete, but you must still finish local saving through Save as…, Browser download, or the download link. The preparation dialog may close after local saving. Confirm success from the browser's completion status and the actual local file.

If preparation fails, the window may offer Retry download; use it to prepare the file again. Canceling a download does not undo a completed server-side save. Select the same file later to try again.

### Transfer limits

An individual upload or download can be at most 128 MiB. One upload batch can contain at most 100 files totaling no more than 128 MiB. These are browser transfer limits, not the account's total storage allocation.

Projects can grow substantially as calculation results accumulate. Check the size in Files before downloading. For files above the transfer limit, use the large-file transfer mechanism provided by your deployment.

The structure project saved and downloaded here is supplied as `examples/Si-ground-state.ks`. Use it to check the silicon structure and practice the Online file round trip.

## 4.5 Check the current account

An Online workspace connected to the account service displays the account name at the top. Open its menu and choose My Account to view the account settings page, or open that page through Home → Settings. It shows the account name, connection status, and the inactivity limit set by the deployment.

1. Check that the displayed name is your own account name, then open My Account.
2. Check the connection status. A green dot and connected message identify the current session. The inactivity limit is how long a session is kept without interaction.
3. If the connection needs restoring, use Reconnect, then check the project and Files after returning to the workspace.
4. Save your project before finishing. Use Sign out when you want to leave the account.

If your deployment does not provide an account settings page, connect through its entry page.

## 4.6 Leave the workspace

### Return to the entry page temporarily

The account menu's exit action returns to the entry page while retaining the workspace until its inactivity limit. Keep the current browser tab open and use the portal's reconnect action to return to the original interface. This is useful for checking the portal before continuing your work.

Returning to the portal is different from signing out. It does not immediately release computing resources or automatically save the project to disk. Save important changes first.

After completing the hydrogen calculation in Chapter 5, practice returning and reconnecting with its saved project:

1. Save the current project and check that the calculation has finished.
2. Open the account menu and choose the action to exit KSSOLV Toolbox. Read the workspace-retention notice and confirm to return to the portal.
3. Check that the portal reports that the workspace is still running, then reconnect.
4. In the restored workspace, check the original project. Its SCF workflow should still contain Configuration, SCF, and Plot nodes; Results should retain the Run SCF dataset and EnergyConvergence plot. Open the plot to continue inspecting it.

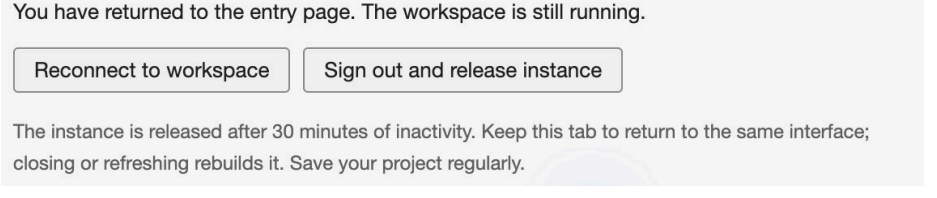



**Figure 4.4** Workspace status and the reconnect action after returning to the portal. Reconnect returns to the original project; signing out and releasing the instance ends the workspace.

### Close or refresh the page

Closing the page does not sign you out. Return to the same service entry page while the original workspace is still alive to reconnect and rebuild the browser interface. Projects, open structures, and workflows can be restored within that running instance, but temporary dialogs, undo history, and the complete layout are not guaranteed to remain unchanged.

Refreshing rebuilds the browser interface. After a brief network interruption, wait for recovery or reconnect through the service entry page. For a temporary departure, return to the portal through the account menu and reconnect when ready.

### Sign out and release the workspace

When you finish, particularly on a shared computer, save the project and use Sign out in the account menu, or the portal's sign-out-and-release action. Confirmation ends the access session and releases the running instance.

Your next sign-in uses the same account's saved files and settings. In-memory variables, unsaved edits, and running calculations cannot be relied on to survive sign-out. Save before leaving and download a file if you need a local copy.

### Understand inactivity release

The platform sets the inactivity limit. Check account settings or portal notices for the current value. Deployments may use different limits, and a warning appears as release approaches. Keyboard, mouse, touch, and scroll interactions count as activity. Merely leaving the tab in the foreground or allowing status updates does not extend the limit.

A running calculation does not itself imply user interaction. Before scheduling a long task, check your platform's policy for inactivity release and long calculations. Follow your cluster's instructions for independent batch queues. Chapter 13 covers remote submission from the full MATLAB® toolbox.

> **Resume after signing in again**
>
> Find the exercise directory in Files and double-click the saved `.ks` project to continue editing structures, configuring workflows, or viewing results.

## 4.7 Manage the workspace connection

The resource status at the bottom shows CPU and memory usage for the current environment. In deployments allocating resources per account, it describes the current workspace's allocation. A plain container deployment may instead show the host environment. Use the status labels and deployment documentation to identify the scope of the reported resources.

Workflow resource options depend on the node's task, solver method, and deployment capabilities. Select a node and inspect its resource panel; Section 10.5 explains the choices and checks. The Home tab's Parallel button does not replace node resource settings. Process-pool and GPU availability must also be checked against the server environment.

Once comfortable with files, continue with the hydrogen calculation in Chapter 5. Upload the supplied `h2.cif` into your server-side directory and import it into the project. Create the SCF workflow and enter that chapter's structure and solver settings.

Figure 4.5 shows the hydrogen tutorial's structure input. Select the node, choose Workflow → Configure Node, and scroll to Options. The node labeled Configuration performs the Build Molecule task. Choose Molecule, structure h2, PBE, default pseudopotentials, 20 Ha, 0 K, and Nspin=1. Set the SCF method and iteration limit as in Chapter 5.

The reference calculation converges in 7 iterations; Chapter 5 gives the log and numerical results. After completion, open the Project Browser and expand Results. Run SCF under Datasets holds the run data; EnergyConvergence under Plots opens the convergence plot.

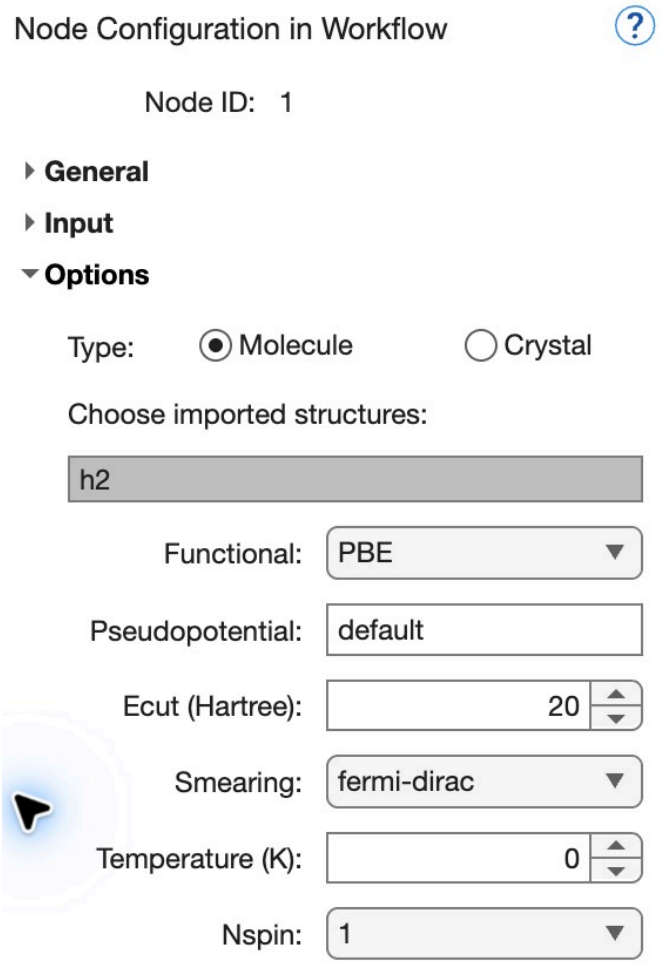



**Figure 4.5** Structure-input options for the hydrogen workflow. Check the selected h2 structure and calculation settings in the Config Editor on the right.

After checking the log and curve, save again, close the project, and double-click its saved `.ks` file in Files to reopen it. Figure 4.6 shows the reopened project's result groups, with datasets and plots stored separately. Double-click EnergyConvergence again to inspect the seven-iteration history. Finally, download the project for a local copy.

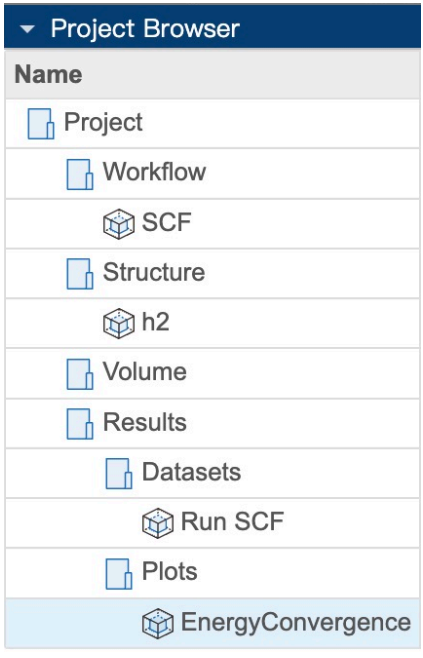



**Figure 4.6** Results in the hydrogen project. Results contains the Run SCF dataset and EnergyConvergence plot. Double-click the plot entry to view the iteration history.

This completes the sequence from structure preparation through server-side calculation, saving, and reopening.

If connection fails, return to the entry page and check whether the workspace is still starting. Use its retry action first when offered. For a persistent problem, the portal may provide a restart action. Restarting retains saved files and settings but ends the in-memory session; confirm that this is appropriate before proceeding.

If you forget your password, use the recovery action on the sign-in page to retrieve your recovery question, then enter the answer and a new password. Resetting ends existing sessions. Sign in again after success. Contact the administrator if no question was set or you cannot remember the answer.

If the interface reports Persistent Files is unavailable in this deployment., upload local files to open them and download through the browser when saving. Confirm that you have a local copy before leaving.

Chapter 5

# Calculate the electronic structure of a hydrogen molecule

A hydrogen molecule has only two atoms, yet introduces the essentials of an electronic structure calculation: where to place the atoms, how the electrons are distributed, how to recognize completion, and which geometry the final numbers describe. Starting from a prepared file, this chapter runs a self-consistent field calculation and examines the energy and error histories. The same procedure can then be applied to larger molecules and crystals.

The example uses plane waves and pseudopotentials. You do not need their mathematical derivation for a first run. Learn the structures, nodes, and results in the interface first, then read Chapter 6 to understand the parameters in that practical context.

## 5.1 Prepare the hydrogen example

Two hydrogen atoms are placed in a cubic simulation box. Their positions remain fixed while the electronic ground state is calculated. This is a single-point energy calculation: "single point" means one geometry, not one numerical iteration. The density is updated repeatedly until successive solutions agree sufficiently well, a procedure called self-consistent field calculation, or SCF.

At the end, your project should contain a hydrogen structure, a reusable workflow, and an energy/error plot. Save it for the later geometry-optimization tutorial. For a first run, use the current workspace's computing environment; Online performs the same steps in its server session.

`examples/h2.cif` stores this box and these atomic positions. Using it avoids manual coordinate entry and gives desktop and Online sessions the

**Table 5.1** Calculation conditions for the hydrogen tutorial.

| Item | Setting |
|---|---|
| Structure type | Molecule |
| Simulation box | Cube with sides of 10 Bohr, approximately 5.29177 Å |
| Hydrogen positions | Fractional coordinates $(0.5, 0.5, 0.425)$ and $(0.5, 0.5, 0.575)$ |
| Initial bond length | 1.5 Bohr, approximately 0.793766 Å |
| Exchange-correlation functional | PBE |
| Pseudopotential | default, built-in PBE-1.3 ONCV pseudopotentials |
| Plane-wave cutoff | 20 Ha |
| Spin components | Nspin = 1 |
| Temperature | 0 K |
| SCF method | davidson_qe; maximum 100 iterations |
| Forces | Calculate force selected |

same starting structure. Here, the CIF lattice describes the simulation box; you must still select Molecule in the Build Molecule node. File format and calculation-object type are separate choices.

> **Why start with a small molecule?**
>
> A small system makes every step easy to inspect and parameter changes easy to compare. The specified box and cutoff let you complete the exercise and compare results. Test their convergence again when using another molecule.

## 5.2 Create a project and import the structure

### Use a separate project

A project holds the structure, workflow, and results together. Give it a clear name so that you can recognize its purpose when returning later. For example, call this one "Hydrogen introduction" and save it as `h2-first.ks`. The

application starts with an empty project. If another project is open, save it and close it to return to an empty one.

1. If another project is open, save it and choose Home → Close.
2. Check that the Project Browser shows categories for structures, workflows, and results.
3. Keep the empty project open and proceed with import.
4. After importing, use Save to choose a project name and location.

Desktop editions normally use a system file dialog for local files. Online file selection and downloads pass through the browser: supply the tutorial file to the session, then select it in the application. In either environment, the imported structure must appear in the Project Browser, not merely in the browser's download list.

### Import hydrogen

1. On Home, choose Import structure from file... from Structure.
2. Select the supplied `h2.cif`.
3. Select the imported structure in the Project Browser, expand the Information Browser below, and check the elements and atom count. If the panel is missing, select it from the ellipsis button's Open Panels list.
4. Open the structure's 3D view and rotate it until both hydrogen atoms are clearly visible.

You should see two hydrogen atoms. The box boundary may be visible or hidden according to display settings. Sphere sizes and bond styles affect only the picture, not atomic coordinates. Use fit-to-view or zoom if the structure is off-center or outside the view.

### Check the geometry

Before calculating, measure the distance between the atoms in the viewer. The input bond length is approximately $0.793766$ Å. The displayed precision may differ; $0.794$ Å agrees with this input. The underlying numerical coordinate unit is Bohr, giving a bond length of $1.5$ Bohr.

You can also select a hydrogen atom to inspect its coordinate card. Figure 5.1 selects site 2, with fractional coordinates $(0.500, 0.500, 0.575)$ and displayed Cartesian coordinates $(2.646, 2.646, 3.043)$ Å. Fractional coordinates locate the atom relative to the box; Cartesian coordinates give its position in Å. Both atoms lie near the box center with space around them. A CIF-imported document may show Crystal Modeling; the solver must nevertheless use Molecule in the structure configuration node for this example.




**Figure 5.1** The coordinate card for site 2 of the initial hydrogen molecule, showing Cartesian coordinates, fractional coordinates, and occupancy. Use it to check the selected site's geometry.

Electronic structure depends on atomic positions. For this exercise, confirm two hydrogen atoms, a bond length of approximately $0.794$ Å, and a cubic box of approximately $5.292$ Å per side.

An XYZ file can reproduce the same bond length without reproducing the same box. When a molecular file has no lattice, the software constructs a box from the atomic extent and vacuum spacing. Use an identical simulation

box, as well as bond length, for a detailed numerical comparison with this chapter.

## 5.3 Build the SCF workflow

### Start with the SCF template

A workflow divides the calculation into nodes, connected in processing order. This example needs three tasks: pass the imported structure to the solver, calculate the electronic structure, and display convergence. A template establishes these connections before you adjust individual settings.

1. Open the Workflow import menu and choose Import workflow from template.
2. Select the SCF template.
3. Open the new workflow document and inspect its nodes and connections.
4. Check that structure configuration, SCF, and energy plotting are connected in that order. The structure node may be labeled Configuration or Build Molecule; both refer to the same task. The next nodes are SCF and Plot.

With a blank workflow, add and connect these three nodes yourself. Display labels can be edited, but task types must remain correct. Renaming a node to "Calculate hydrogen," for example, does not change its SCF task type. Keep template names for your first run so that they match the tutorial.

Use Figure 5.2 to check the sequence: the first node supplies the structure, the second solves its electronic structure, and the last plots the calculation history. Configure them in that order.

### Configure structure input

Select Configuration, then choose Workflow → Configure Node or click its gear. This opens the right-side panel if collapsed and opens or refreshes

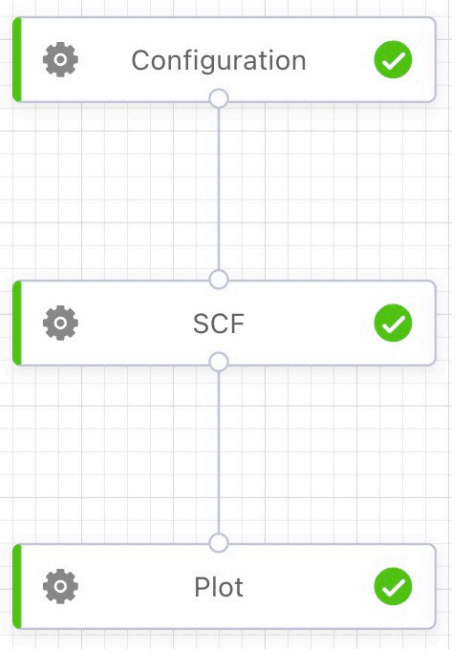



**Figure 5.2** The hydrogen SCF workflow. Structure configuration, electronic solving, and plotting run from top to bottom. Configuration is the Build Molecule task, and Plot produces the convergence curve.

the task settings in the Config Editor. Scroll to Options to find Type and the other parameters. If only general properties such as ID, label, and module are visible, invoke Configure Node first. This node selects the structure and defines the physical and numerical model. Check each input before running.

A narrow panel may truncate parameter labels or dropdown values. Open the ellipsis at the upper right of the Config Editor title bar and choose Maximize to give it more space. Use Restore in the same menu afterward to return to the side-by-side layout. General, Inputs, Options, and Advanced Options expand independently. Collapse General and Inputs while entering the tutorial settings if needed.

1. Set Type to Molecule.
2. Under Choose imported structures, select only this hydrogen structure.

3. Set Functional to PBE and Pseudopotential to default.
4. Enter 20 in Ecut (Hartree).
5. Keep Smearing at fermi-dirac, Temperature (K) at 0, and Nspin at 1.
6. In Advanced Options, keep Extra bands at 0.

With Molecule selected, automatic crystal $k$-point settings are not part of this exercise. The molecule-in-a-box calculation does not need a crystal sampling grid.

Pay particular attention to Ecut (Hartree). Entering 20 means 20 Ha, approximately 544 eV, not 20 eV. The interface gives the unit in the label. Convert units before comparing parameters with papers or other software.

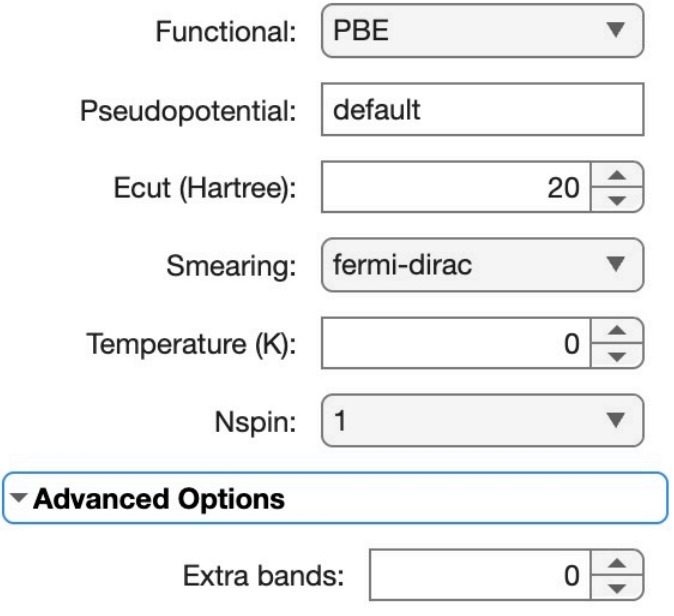



**Figure 5.3** Regular and advanced structure-node options. Check PBE, default pseudopotentials, 20 Ha, 0 K, Nspin=1, and Extra bands=0 here. Figure 4.5 shows the Molecule and h2 selections.

### Configure SCF

Select SCF, choose Workflow → Configure Node, and scroll to Options. DiagMethod selects the numerical method for solving electronic states at each iteration; Max SCF steps limits density updates. Use the default davidson_qe and 100 iterations here.

The DiagMethod dropdown offers several methods. Choose davidson_qe; if the closed field truncates it, maximize the panel to check the complete name.




**Figure 5.4** The SCF algorithm list, with davidson_qe selected for this example.

Select Calculate force. Atomic forces indicate directions in which moving atoms would locally lower the energy for the current electronic state. This chapter holds positions fixed; calculating forces lets you compare the starting and optimized structures later. An ordinary single-point energy calculation can leave this option cleared when forces are not needed.

Keep the advanced ExxMethod at default. It mainly concerns hybrid-functional calculations and needs no adjustment for this PBE example.

Use Table 5.2 for a final check. Maximize the Config Editor to read long method names, then restore the layout.

**Table 5.2** SCF settings for the hydrogen tutorial.

| Option | Setting |
|---|---|
| DiagMethod | davidson_qe, the electronic-state solver at each iteration |
| Max SCF steps | 100, the maximum density updates |
| Calculate force | Selected to calculate forces in this single-point run |
| ExxMethod | default, retaining the advanced default |

### Check the plotting node

The energy-curve task, labeled Plot here, draws the SCF history. It does not solve the electronic structure again or alter the preceding result. Check that it follows SCF. If you add it after an SCF run, rerun this complete small workflow to obtain a plot tied to that run.

Save before running. The saved workflow records the conditions directly when you later compare cutoffs, structures, or functionals.

## 5.4 Run the calculation and follow progress

### Activate the intended workflow

Several workflow documents can be open at once. Select this workflow's document tab before choosing Run, so that a different workflow is not executed accidentally.

Expand the right panel group and the Simulation Browser title bar. Reopen it by name from Open Panels if closed. If configuration still fills the workspace, use Restore in its title menu to view the canvas and output together. The browser displays messages as the calculation starts. SCF output normally lists error, total energy, and time by iteration. A slowly decreasing error at one step need not indicate a mistake. Judge the full history and final convergence status.

1. Activate the hydrogen SCF workflow document.
2. Choose Home → Run, or use Run in the Simulation Browser.
3. Find the message marking the start of the hydrogen calculation.
4. Wait for convergence, then inspect the plot that opens automatically.
5. Return to the canvas and check that all required nodes finished.

The local Stop button resets the run-button state but does not interrupt an ongoing numerical solve. Synchronous Online calculations also offer no stop operation. Wait for this example to finish. Read the log for progress; closing a browser tab neither saves the project nor ends the server calculation.

### Measure elapsed time

The dropdown next to Run offers Run and time. Activate this SCF workflow and choose it. When a local run finishes, the window bottom reports its elapsed time. This includes execution and associated preparation for the whole workflow, not a separate performance profile for each node.

Compare timings only with the same model, settings, and execution environment, and state whether a run was the first one. Initializing numerical functions or creating a figure can add startup time. For a remote submission, local submission time is not remote calculation time; read the remote job records. Record timing separately in your notes if useful. Scientific validity still rests on energy, error, and convergence.

### Read the three main log values

Begin with iteration number, relative density error, and total energy. The iteration number counts updates, density error measures the difference between successive input and output densities, and total energy estimates the system's energy at the current density.

Energy generally stabilizes as density becomes self-consistent, but need not decrease monotonically. Mixing historical densities can cause a tempo-

rary increase during an otherwise normal iteration sequence. Read the final convergence message rather than judging completion from two nearly equal energy entries.

```
SCF iter   7:
Diag time =         1.08574e-01
Rel Rho Err    =          6.236e-09
Convergence is reached!
Total Energy    = -1.1652136532206e+00
Etot          = -1.1652136532206e+00
```

**Figure 5.5** Part of the hydrogen SCF log. Iteration 7 reports Convergence is reached!, with total energy $-1.1652136532206$ Ha and relative density error approximately $6.24 \times 10^{-9}$.

## 5.5 Read your first calculation result

### Understand the convergence plot

The plot combines two histories from the same SCF run. Iteration number is horizontal, total energy in Ha uses the left axis, and error uses the right. The logarithmic error axis means that equal vertical distances can correspond to one or more orders of magnitude.

Read the energy from left to right, then check whether the error decreases overall. In this small hydrogen molecule, energy is nearly stable after the first few iterations while the density error still needs to fall. A visually flat curve and a satisfied convergence criterion are different things.

### Compare with the reference run

The accompanying script converges in 7 iterations, with total energy approximately $-1.1652136531$ Ha and final relative density error approximately $8.85 \times 10^{-9}$. Use these values to compare your result.

The run in Figure 5.5 also converges in 7 iterations, at approximately $-1.1652136532$ Ha and $6.24 \times 10^{-9}$ relative density error. Both meet the example's convergence criterion. Figure 5.6 plots the history saved by the script.

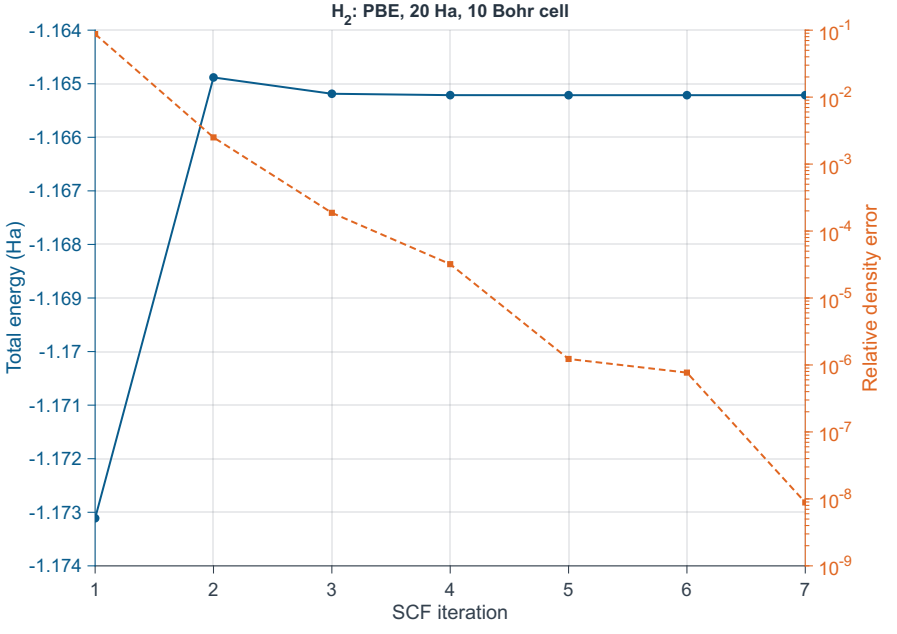



**Figure 5.6** Total energy and relative density error for hydrogen SCF. Read the blue line against the left axis and the orange dashed line against the logarithmic right axis. Data are from the supplied reference run.

Initial electronic states, software versions, and numerical environments may change iteration counts or the final digits in a graphical run. First match the structure, box, functional, pseudopotentials, and cutoff, then check convergence. Timing also depends on hardware and session load. Use the numerical results and convergence state to assess agreement.

### What this total energy means

This energy belongs to hydrogen at this geometry, in this box, with this functional and pseudopotentials. It is not directly an experimental binding energy. A dissociation-energy calculation also needs consistently calculated atomic reference states, including appropriate spin treatment.

### Save results and the project

Once added under Results, the plot can be reopened from the Project Browser. A figure presents the calculation record; the workflow records how it was obtained. Keep both. Save the project after the run as well as before it.

Use a clear figure name such as "Hydrogen: PBE, 20 Ha, initial geometry." Distinct names prevent confusion with later optimized-geometry results. Include the main conditions in the name and retain complete parameters in the workflow rather than creating an excessively long filename.

## 5.6 Complete the exercise

Reopen the saved project and check that the structure, workflow structure selection, and plot are restored.

For an extension, import `h2.cif` again and name it `H2-longer`. Use the geometry-editing procedure in Chapter 7 to set a bond length of 0.85 Å. Return to Build Molecule, select this structure under Choose imported structures, and run another single-point calculation without changing cutoff, functional, or box. Save both results and compare total energy and force along the bond. Chapter 10 continues with geometry optimization; Chapter 12 explains result interpretation.

## 5.7 Reproduce the example with a script

This section is for full MATLAB® users. Standalone and Online users can continue with the graphical procedures without running the script. The interface is useful for learning and inspecting structures; scripts make repeated identical settings convenient. `examples/h2_scf_and_relax.m` contains the complete input. The excerpt below is its single-point calculation and assumes an initialized KSSOLV numerical environment.

```
pseudopotential.PpType('default');
molecule = example.structure.H2();
molecule.ecut = 20;
molecule.funct = 'PBE';
scfOptions = options.SCF('force', true);
scfOptions.enableGPU = false;
scfOptions.enableParallelPool = false;
[molecule, H, X, info] = scf(molecule, scfOptions);
```

In the returned values, `info.Etot` is the total energy, `info.Etotvec` and `info.SCFerrvec` hold the energy and error histories, and `info.converge` gives convergence status. Use them for tables or plots. The full script also optimizes positions and saves data. Run only the single-point section above to reproduce this chapter. The interface and script use the same numerical model; retain the explicitly specified tutorial parameters.

Chapter 6

# Choosing calculation parameters

After completing a calculation, you will usually want to know why its parameters were chosen, which ones to change for a new system, and how to tell whether the accuracy is sufficient. This chapter introduces the essentials of electronic structure calculations through the settings in the interface. Keep the hydrogen molecule project from Chapter 5 open so that you can relate each parameter to the physical problem it describes.

A complete calculation specification has three parts: geometry, the electronic model, and numerical accuracy. Geometry defines the elements, positions, and lattice. The electronic model specifies assumptions such as the functional, pseudopotentials, and spin treatment. Numerical accuracy depends on the cutoff energy, $k$-point sampling, and iterative convergence criteria. All three matter. A numerically converged calculation still gives an answer within the chosen model, and a suitable model still needs adequate numerical accuracy to produce stable results.

## 6.1 From atomic positions to electron density

### What density functional theory describes

For fixed atomic positions, electrons are not stationary particles arranged around the atoms. Their motion and interactions determine the system's energy, chemical bonding, and many material properties. Tracking the full many-electron wavefunction directly is difficult. Density functional theory instead formulates the ground-state problem in terms of the electron density in space.

Electron density describes the spatial distribution of electrons. A region of higher density generally contains more electrons, but the density is unrelated to the sizes of the atomic spheres in the viewer. A ball-and-stick

model depicts geometry; electron density is a calculated quantity. The two visualizations serve different purposes.

KSSOLV solves the Kohn–Sham equations for the electronic states. Their effective potential depends on the electron density, which in turn is obtained from those states. A single solution step is therefore insufficient. The solver updates the density until the density used to construct the equations agrees with the density obtained from their solution. This is self-consistency.

For the nonmagnetic PBE pseudopotential tutorials in this manual, the equations can be written as

$$\begin{aligned} \widehat{H}[\rho]\psi_{n\mathbf{k}} &= \varepsilon_{n\mathbf{k}}\psi_{n\mathbf{k}}, \\ \widehat{H}[\rho] &= -\frac{1}{2}\nabla^2 + V_{\text{loc}}^{\text{ps}} + \widehat{V}_{\text{NL}}^{\text{ps}} + V_{\text{H}}[\rho] + V_{\text{xc}}[\rho]. \end{aligned} \tag{6.1}$$

Atomic units are used here: lengths are in Bohr and energies in Ha. The kinetic term describes spatial variation of the electronic states; the local and nonlocal pseudopotentials describe the ionic cores. The Hartree and exchange-correlation potentials are updated with the density. The Pseudopotential and Functional settings select the corresponding parts of the model. HSE06 also includes nonlocal exchange and cannot be described solely by the local exchange-correlation potential shown above.

For the nonmagnetic setting Nspin=1, the weighted states give the electron density:

$$\begin{aligned} \rho(\mathbf{r}) &= 2\sum_{\mathbf{k}} w_{\mathbf{k}} \sum_{n} f_{n\mathbf{k}} |\psi_{n\mathbf{k}}(\mathbf{r})|^2, \\ N_{\text{e}} &= \int_{\Omega} \rho(\mathbf{r})\, \mathrm{d}^3 r. \end{aligned} \tag{6.2}$$

Each wavefunction is normalized within one calculation cell. The factor of 2 accounts for nonmagnetic spin degeneracy: a fully occupied spatial state has $f = 1$ and contains two electrons. Do not also set $f$ to 2. The molecular tutorial uses a single $\Gamma$ point with weight 1. In a pseudopotential calculation, $N_{\text{e}}$ is the number of explicitly treated valence electrons. The density is an electron number per unit volume; it differs in sign from a charge density that includes the negative electron charge.

**Table 6.1** Common symbols in electronic structure equations.

| Symbol | Meaning and relation to the settings |
|---|---|
| $\psi_{n\mathbf{k}}$, $\varepsilon_{n\mathbf{k}}$ | Electronic state $n$ and its energy; $\mathbf{k}$ labels the crystal wavevector |
| $\rho$, $\Omega$ | Valence electron density and calculation-cell volume; in atomic units, Bohr$^{-3}$ and Bohr$^{3}$, respectively |
| $f_{n\mathbf{k}}$ | Occupation weight, determined by the electron count, Smearing, and Temperature |
| $w_{\mathbf{k}}$ | $k$-point integration weight; the weights sum to 1 within a spin channel |

Increasing Extra bands includes more unoccupied states in the solution, but does not change the electron count required by Eq. (6.2). Spin-polarized and noncollinear modes require the appropriate channel treatment; the factor of 2 used here cannot simply be carried over.

### Why fixed atoms still require iterations

An SCF calculation repeatedly improves an electron distribution. It starts from an initial density and constructs an effective potential, solves for electronic states, forms a new density from the states and their occupations, and mixes new and previous information to obtain the next starting point.

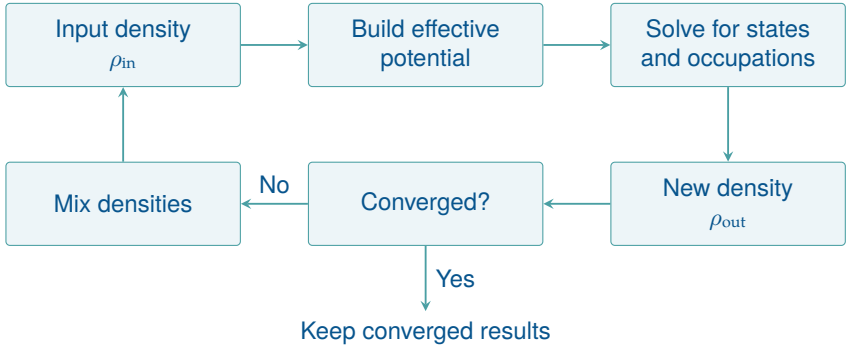



**Figure 6.1** The electronic self-consistency cycle (schematic). With atomic positions fixed, update the density until the input and output satisfy the convergence criterion.

The atomic positions can remain fixed throughout this process. The tenth SCF iteration does not mean that the atoms have moved ten times. Geometry optimization adds an outer loop: converge the electronic problem for one geometry, move the atoms according to the forces, and calculate the electronic structure again for the new geometry. Distinguishing electronic iterations from geometry steps is essential when reading a log.

### SCF and non-self-consistent calculations

SCF establishes an electron density consistent with the structure and electronic model. Band structures and densities of states require electronic energies at additional or different $k$ points. These can be obtained using the converged density, without updating the effective potential self-consistently for each new sampling set. Such calculations are called non-self-consistent field calculations, or NSCF.

In a workflow, SCF first supplies the converged density. NSCF specifies the solver options for the subsequent calculation. Band Calculation or DOS then performs the calculation on its own path or grid. A band path and a DOS grid serve different sampling purposes; do not give them the same list merely because both contain $k$ points.

## 6.2 Choose a molecule or a crystal

### What the calculation type means

The Molecule and Crystal choices in Build Molecule determine how the calculation object is organized. For the isolated-molecule tutorial, choose Molecule and provide a suitable simulation box. For a periodic material, choose Crystal and specify its lattice and $k$-point sampling.

A molecular calculation still needs a box: the plane-wave expansion requires a finite spatial region. Use enough vacuum to reduce interactions between the molecule and its periodic images. Charged, strongly polar, or spatially extended systems may need different box sizes from compact neutral molecules.

File formats do not determine the calculation type on their own. CIF usually describes a crystal, but can also store a molecular example's simulation box and atomic coordinates. XYZ usually stores molecular coordinates; the software can construct a box after import. Inspect the actual geometry and explicitly select the calculation type in the node.

### Lattice vectors and atomic coordinates

A periodic structure consists of three lattice vectors and atomic positions within the cell. Vector lengths and angles define the cell shape. Atomic positions can be expressed in Cartesian coordinates or as fractional coordinates relative to the three lattice vectors.

The fractional position $(0.25, 0.25, 0.25)$ is the sum of one quarter of each lattice vector. It does not generally mean that each Cartesian coordinate is one quarter of a lattice constant. That simple correspondence holds only for the appropriate orthogonal cell. Check the coordinate type carefully for hexagonal, monoclinic, and other nonorthogonal cells.

Let $\mathbf{a}_i$ denote the real-space lattice vectors and $f_i$ the fractional coordinates. Positions and reciprocal vectors satisfy

$$\mathbf{r} = \sum_{i=1}^{3} f_i \mathbf{a}_i, \qquad \mathbf{k} = \sum_{j=1}^{3} \kappa_j \mathbf{b}_j, \qquad \mathbf{a}_i \cdot \mathbf{b}_j = 2\pi\delta_{ij}. \tag{6.3}$$

Here $\delta_{ij}$ is 1 when $i = j$ and 0 otherwise. This reciprocal-lattice convention includes $2\pi$. The $\kappa_j$ are dimensionless coordinates in the reciprocal basis, whereas $\mathbf{k}$ has units of inverse length. Thus $(0.5, 0, 0)$ on a path means half of the first reciprocal vector, not half a real-space lattice length.

A crystal may be represented by a primitive cell, conventional cell, or supercell. These can describe the same infinite periodic structure while containing different atom counts and having different Brillouin-zone sizes. A larger supercell contains more electronic states per cell. The same integer $k$-point grid therefore no longer gives the same sampling density.

### Keep track of units

Structure editing and common file formats generally use Å, numerical geometry interfaces use Bohr, and electronic energies may be reported in Ha, Ry, or eV. The software performs the required conversions between import and solving, but you must still check units when reading external data or organizing results.

**Table 6.2** Common quantities and units in this manual.

| Quantity | Units and where to find them |
|---|---|
| Length | Å in the viewer, CIF, and most XYZ files; numerical interfaces may use Bohr |
| Cutoff energy | Hartree (Ha) for Ecut in Build Molecule |
| Total energy | Ha on the left convergence-plot axis; logs may also report Ry |
| Band and DOS energy | eV; tutorial plots use the Fermi energy as zero |
| Electronic temperature | K in Temperature |
| Atomic forces | Atomic units in numerical records; convert when comparing external sources |

Useful conversions are $1\,\mathrm{Ha} = 2\,\mathrm{Ry} \approx 27.2114\,\mathrm{eV}$ and $1\,\mathrm{Bohr} \approx 0.529177\,\text{Å}$. These approximations are adequate for checking inputs. For batch numerical processing, use the program's conversion constants rather than mixing differently rounded values across tables.

## 6.3 Functionals: approximate electron interactions

### Common functionals

Functional offers PBE, PZ, and HSE06. They approximate exchange and correlation differently. PZ is a local-density approximation, PBE is a generalized-gradient approximation, and HSE06 is a screened hybrid functional. These choices affect equilibrium geometries, relative energies, and electronic levels.

PBE is a convenient starting point for learning workflows and for initial materials calculations. The basic tutorials consistently use PBE so that

you can focus on the structure and calculation sequence. When changing functionals, consider the suitability of the pseudopotentials, your research objective, and the conditions used in the literature, as well as the computational cost.

HSE06 includes a fraction of exact exchange and is usually more expensive than a semilocal functional. It also requires appropriate exchange-algorithm settings and follow-up calculation conditions. Begin with PBE SCF, band structure, and DOS calculations, then learn the hybrid-functional settings using the same small system.

### Compare functionals consistently

First decide whether you want to compare electronic models at the same geometry or compare results at each model's own equilibrium geometry. The former holds the structure fixed and changes only the electronic model. The latter allows each model to find a different minimum-energy structure. These comparisons answer different questions.

For example, comparing the total energies from two functionals for a hydrogen molecule at one fixed bond length does not tell you which functional predicts a bond length closer to experiment. Optimize the geometry with each functional if bond length is your target. Use the same geometry if you want to compare electronic levels at a fixed structure. State this choice explicitly when describing the calculations.

## 6.4 Pseudopotentials: describe the ionic cores

A pseudopotential combines the effects of the nucleus and tightly bound core electrons into an effective description, leaving the calculation to treat mainly valence electrons. This makes plane-wave calculations practical for many elements. Pseudopotentials contain modeling choices of their own and are part of the calculation specification.

The default choice under Pseudopotential uses the collection supplied with the software. The hydrogen and silicon tutorials use the built-in PBE-

1.3 ONCV files. Check the run messages for the filenames actually loaded and confirm that a pseudopotential was found for every element.

Use pseudopotentials appropriate to the functional. Changing Functional does not imply that the software selects a new pseudopotential collection automatically or that all combinations are equally suitable. For a new materials study, record the source, version, and valence-electron treatment, and keep them consistent across calculations you intend to compare.

### Why cutoff requirements vary

Some pseudopotentials produce smoother valence wavefunctions that need fewer plane waves; others require a higher cutoff. An adequate cutoff depends on both the system and its pseudopotentials, rather than simply on atomic numbers or atom counts.

For a multielement structure, test the complete system. The element with the most demanding plane-wave requirement often sets the cutoff needed for the whole calculation. Perform separate cutoff convergence tests for oxides, transition-metal systems, and hydrogen-containing materials.

## 6.5 Plane-wave cutoff energy

### What Ecut controls

The plane-wave method expands electronic wavefunctions as sums of spatial waves. Shorter wavelengths represent finer spatial variation and have higher kinetic energies. The cutoff determines which plane waves are retained and therefore how finely the electronic states can be represented.

A crystal electronic state can be expanded as

$$\begin{aligned} \psi_{n\mathbf{k}}(\mathbf{r}) &= \frac{1}{\sqrt{\Omega}} \sum_{\mathbf{G}} c_{n\mathbf{k}}(\mathbf{G}) \exp[\mathrm{i}(\mathbf{k}+\mathbf{G})\cdot\mathbf{r}]\,, \\ T_{\mathbf{k}+\mathbf{G}} &= \frac{|\mathbf{k}+\mathbf{G}|^2}{2}. \end{aligned} \tag{6.4}$$

The vector $\mathbf{G}$ is an integer linear combination of reciprocal-lattice vectors, and the coefficients $c$ are to be determined. With lengths in Bohr and

wavevectors in inverse Bohr, the kinetic energy is in Ha. At $\Gamma$, $\mathbf{k} = 0$, and the base plane-wave set satisfies $|\mathbf{G}|^2/2 \leq E_{\text{cut}}$. Raising the cutoff adds more distant reciprocal-space components with shorter wavelengths.

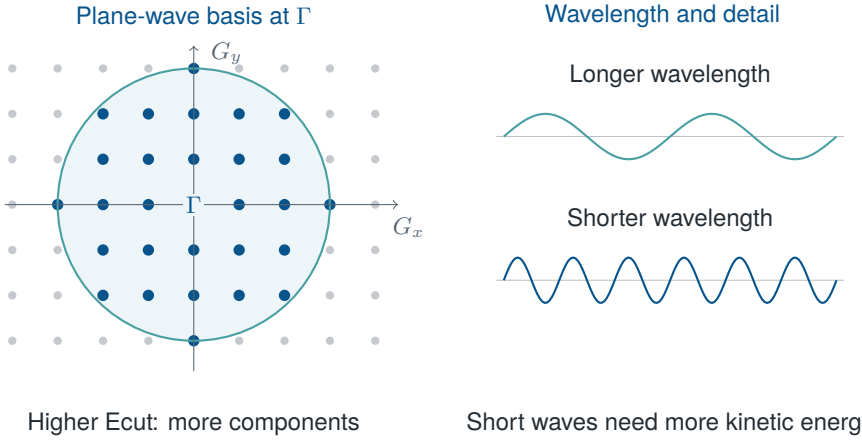



**Figure 6.2** Plane-wave cutoff in two dimensions. Blue points are reciprocal components retained inside the cutoff circle; calculations use a three-dimensional set. The right panel relates wavelength to spatial variation.

At nonzero $k$, the kinetic energy depends on $\mathbf{k} + \mathbf{G}$. Crystal calculations can share a basis selected by the base $\mathbf{G}$ vectors. The circle in the diagram therefore illustrates the $\Gamma$-point case; it does not imply that an identical kinetic-energy sphere is selected afresh at every $k$ point. Check Ecut by convergence testing in practice.

Increasing Ecut (Hartree) generally increases the plane-wave count, memory use, and time per iteration. It improves the basis representation; it does not automatically increase $k$-point sampling or change the SCF convergence criteria. Accuracy parameters interact, but they cannot replace one another.

### Test cutoff convergence

Hold the structure, box, functional, pseudopotentials, and $k$ points fixed, then increase Ecut in stages. For a small example, you might try 15, 20, 25, and 30 Ha and record the converged energy and target properties. This

sequence illustrates the method; choose an appropriate range for your own system.

Assess changes between successive settings rather than whether a total energy looks sufficiently negative. For relative energies, record a per-atom energy difference or the difference between two structures. For geometry optimization, also check the forces and optimized dimensions.

1. Create another workflow from the same template, or save the existing workflow as a `.wf` file and import a copy. Keep the structure unchanged.
2. Change only Ecut and converge each SCF calculation to the same criterion.
3. Tabulate the cutoff and the property of interest.
4. Choose a setting at which that property is stable within your required accuracy, then use it for production calculations.

Do not include unconverged SCF results in this comparison. Their differences may reflect unfinished iterations rather than a basis-set effect. A convergence test usually saves more work overall than choosing very large parameters without checking them.

## 6.6 $k$ points and Brillouin-zone sampling

### Why a crystal needs $k$ points

Electronic states in a periodic structure are labeled by wavevector $k$. Many quantities contributing to the total energy and density require sums or integrals over the Brillouin zone. A calculation approximates them using finitely many $k$ points. Autokpts specifies a regular grid: the first three integers give the grid dimensions and the last three give its shifts.

For a quantity $Q(\mathbf{k})$, a Brillouin-zone average can be approximated by a weighted sum:

$$\langle Q \rangle_{\mathrm{BZ}} \approx \sum_{\mathbf{k}} w_{\mathbf{k}} Q(\mathbf{k}), \qquad \sum_{\mathbf{k}} w_{\mathbf{k}} = 1. \tag{6.5}$$

A complete regular grid can use equal weights. Increasing the grid density improves this zone sampling. Adding points along a path only refines the curve on that path and cannot replace an integration grid covering the Brillouin zone.

For example, `[2 2 2 0 0 0]` is the starting grid in the silicon tutorial. It is small enough for learning; it does not establish convergence for every silicon property. Compare denser grids to check whether the total energy, forces, or band edges are stable.

An elongated or flat cell need not use the same integer in every direction. A longer real-space period gives a shorter reciprocal-space interval, which usually changes the required number of samples. Choose a grid that suits the cell shape and target accuracy, rather than making the three numbers equal by default.

### Match the sampling to the task

The SCF grid builds the electron density and effective potential. A band path follows selected symmetry directions to show how energy varies with $k$. A DOS grid covers the Brillouin zone to count states near each energy.

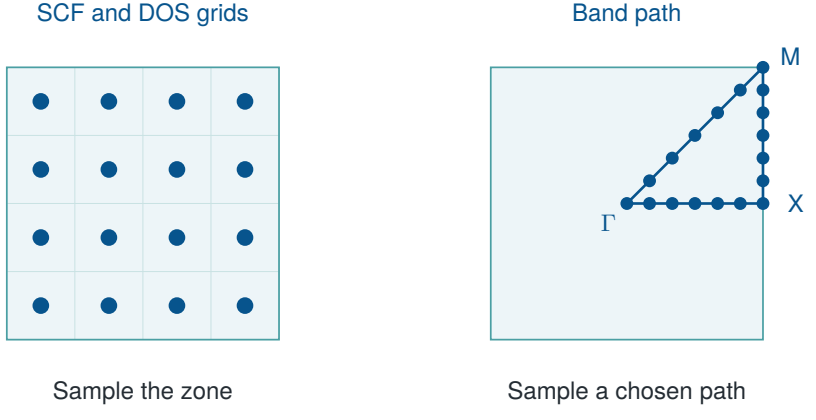



**Figure 6.3** Comparison of a $k$-point grid and a band path (2D schematic). The grid samples a region for integration or state counting; the path follows selected directions. A square region and a triangular path illustrate the two approaches.

All three sampling types can appear in a project. A periodic workflow might demonstrate SCF on a $2 \times 2 \times 2$ grid, then calculate bands along a recommended high-symmetry path or a total DOS on a $4 \times 4 \times 4$ grid. Each follow-up task inherits the self-consistent density from its own workflow and uses its own sampling scheme. The introductory silicon DOS workflow in this manual uses an eight-atom cell; the band workflow uses a standardized two-atom primitive cell. Each performs its own SCF calculation. Chapter 12 also explains how to redraw DOS data stored in the two-atom project. Always check which cell a dataset belongs to.

A smooth band curve does not establish accurate Brillouin-zone integration. Conversely, a dense DOS grid does not smooth a band path containing only its endpoints. Decide what the current node is meant to calculate before choosing its sampling parameters.

### Understand high-symmetry paths

Symmetry analysis recommends high-symmetry points and paths from the lattice and atomic arrangement. Labels such as Γ, X, and L identify reciprocal-space positions. The same label does not necessarily have the same coordinates in different crystal systems.

Changing the cell representation, atomic arrangement, or symmetry tolerance can change the recommended path. When comparing band plots, check the cells and paths as well as the functionals and energy references. Similar axis labels alone do not justify subtracting the curves point by point.

This manual uses Symmetry & K-Path to generate paths and Segment Points in Band Calculation to set the number of samples per segment, including endpoints. Start with fewer points to inspect the overall structure. Increase the count for smoother plots or detailed curvature comparisons.

## 6.7 Set electronic occupations

### Occupations and the Fermi energy

Once the electronic states have been found, the calculation must assign electrons to them. The Fermi energy depends on the electron count and occupation scheme and is a common reference for band and DOS plots. Metals have partially occupied states near the Fermi energy; insulators and semiconductors may have a gap between occupied and unoccupied states.

A calculated Fermi energy is not an absolute vacuum level that can be compared directly across arbitrary systems. Band plots in this manual shift it to zero to show nearby states clearly. Work functions, band alignment, and absolute-level comparisons require an additional, consistent energy reference.

### Smearing and electronic temperature

Smearing offers fermi-dirac, cold, gaussian, and mp. Smearing smooths occupations in energy and often improves integration and convergence for metals or systems with nearly degenerate states. Temperature (K) supplies the associated electronic temperature parameter.

Electronic temperature does not heat the atoms or imply that atomic vibrations at that temperature are included. A static SCF calculation with 300 K electronic occupations can still hold every atom fixed. Choose a method suited to your physical question when studying finite-temperature structural properties.

Changing smearing or temperature changes occupations and the associated energy quantities. When comparing results, identify whether you are using total energy, free energy, or another processed quantity, and keep the definition consistent. For a first tutorial, use the specified conditions before investigating smearing separately.

### Extra bands

The electron count sets the minimum number of occupied states, but band and DOS plots often need unoccupied states above the Fermi energy as well. Extra bands increases the number of unoccupied bands solved for. It does not add electrons or directly change atomic valence states.

If a plot contains occupied states but lacks the conduction-band range you need, check Extra bands rather than simply expanding the vertical axis. An axis can display existing data; it cannot create states that were never calculated. The silicon tutorial adds 4 bands as a starting point for low-energy unoccupied states. Higher energy ranges require more.

## 6.8 Spin and magnetism

Nspin offers settings such as 1, 2, and 4 for different spin treatments. The nonmagnetic examples use 1. For unpaired electrons, magnetic order, or spin–orbit coupling, choose a mode appropriate to the physical problem and check its advanced settings.

Spin is not merely a performance switch. A hydrogen atom and a hydrogen molecule have different occupation patterns; the spin treatment of the atomic reference directly affects a molecular dissociation energy. Transition-metal systems may have several self-consistent solutions, with the final state depending on the starting magnetism and geometry.

The basic examples do not use magnetism, noncollinearity, or spin–orbit coupling. Learn the nonmagnetic workflow first, then create a separate project for a well-defined magnetic question. This makes structural, sampling, and spin effects easier to distinguish.

## 6.9 Converge the SCF calculation

### Diagonalization methods

DiagMethod selects the numerical method used to solve for electronic states. Methods such as davidson_qe and lobpcg apply different iterative strategies

to the same electronic structure problem. Once sufficiently converged, they should agree physically; their main differences are efficiency and convergence behavior.

Compare algorithms using identical inputs and target accuracy, and assess agreement from their converged results. The SCF examples use the default method, and subsequent NSCF examples retain the default solver settings.

### Iteration limits and convergence thresholds

Max SCF steps limits the number of electronic iterations. Raising it from $100$ to $200$ gives the solver more iterations; it does not double the accuracy. Convergence is determined by an error measure and its threshold. Reaching the iteration limit without meeting that threshold means that the calculation still needs attention.

The basic SCF panel provides the method, iteration limit, force options, and related settings. Density thresholds and mixing parameters are more advanced numerical settings available through `options.SCF` in the scripting interface. The basic tutorials retain the default density threshold of $10^{-8}$. To adjust density convergence and mixing, use that interface in full MATLAB®.

Let the difference between the output and input densities of an electronic iteration be

$$\Delta\rho = \rho_{\mathrm{out}} - \rho_{\mathrm{in}}, \qquad e_{\mathrm{SCF}} < \varepsilon_{\mathrm{SCF}}. \tag{6.6}$$

The first expression identifies the densities being compared. The second states that the chosen error measure must fall below its threshold. Default density mixing forms a weighted reciprocal-space measure of $\Delta\rho$ to obtain $e_{\mathrm{SCF}}$ and compares it with $10^{-8}$. Although the log calls the quantity `RelRhoErr`, it is neither a percentage nor simply the norm of the density difference divided by the input-density norm. Check the metric against its corresponding threshold and read the final run status.

### What density mixing does

The latest output density is not always the best starting point for the next iteration. Density mixing combines current and historical information to reduce oscillations and improve convergence. Responses differ between systems; metals, large cells, and magnetic materials may require more careful settings.

If the error repeatedly alternates between two levels, check the structure and occupations before adjusting mixing and sampling. More iterations may help a steadily decreasing error finish converging, but will not necessarily cure persistent oscillations. Keep the full log rather than just the final error message when diagnosing such behavior.

## 6.10 Two accuracy requirements in geometry optimization

Geometry optimization calculates the electronic structure at each trial geometry before using the forces to update atomic positions. Electronic convergence controls the reliability of each energy and force evaluation. The geometry threshold determines when the atomic positions are considered sufficiently relaxed.

The force on atom $I$ is the negative gradient of the total energy with respect to its position:

$$\mathbf{F}_I = -\nabla_{\mathbf{R}_I} E, \qquad F_{I\alpha} = -\frac{\partial E}{\partial R_{I\alpha}} \quad (\alpha = x, y, z). \tag{6.7}$$

Forces are in Ha/Bohr in the calculation's atomic units. A force points locally downhill in energy. A small energy change does not by itself imply that every atomic force is small. Optimization records may report the maximum atomic force, root-mean-square force, or full gradient norm. Check column names and definitions before comparing them.

RelaxTolerance controls the stopping tolerance, while MaxRelaxSteps limits the number of geometry steps. The hydrogen optimization example explicitly sets them to $10^{-4}$ and $100$. Graphical and scripting interfaces may have different defaults; record the values actually used.

Check that the geometry optimization and its final electronic calculation both converged, and that the final forces meet your research requirements. A bond length that no longer changes visibly is not enough to establish a relaxed geometry.

## 6.11 Choose a hybrid exchange algorithm

Selecting HSE06 under Functional in Build Molecule enables hybrid-functional exchange. The SCF node's advanced ExxMethod option selects the algorithm used for that part of the calculation. Available choices are default, normal, ace, qrcp, and kmeans. The functional defines the approximation; the exchange algorithm organizes its evaluation. These settings are not interchangeable.

Begin with default. To compare algorithms, save a separate project for a small system with an established ground-state calculation, change only ExxMethod, and compare total energy, convergence, and elapsed time. Keep the functional, model dimensions, and sampling unchanged. For a semilocal functional such as PBE, leave default selected.

## 6.12 Distinguish ground and excited states

SCF orbital eigenvalues, differences between orbital energies, and excitation energies of a many-electron system are different quantities. A converged ground-state calculation answers the ground-state problem. Studying excitations also requires a response method and a calculation space built from occupied and unoccupied orbitals.

In a TDDFT task, nvbands and ncbands specify the numbers of occupied and unoccupied orbitals included in the response calculation; nroots is the requested number of excited states. None of these is an atom count or a $k$-point count. First obtain enough extra bands in Build Molecule, then select the unoccupied orbitals for TDDFT. With one occupied and four unoccupied orbitals, there are at most four such orbital pairs; that space cannot supply ten excitation roots.

Use TDA enables the Tamm–Dancoff approximation. Use IPA and Use RPA allow comparisons between response approximations. Start from a clearly specified set of options, then vary one approximation at a time. Section 10.13 in Chapter 10 covers input preparation, applicability, and example parameters. Absorption peaks require the corresponding response calculation.

## 6.13 Choose parameters for a new system

Define the objective first: relative energies of adsorption sites, the effect of strain on bands, or a molecular equilibrium geometry, for example. Establish a small calculation that runs, check the structure, elements, and electron count, then improve accuracy one parameter at a time against the property of interest. Keep a baseline result so that every comparison has a clear reference.

Save a short specification with the project: structure source and preparation, cell, functional, pseudopotentials, cutoff, $k$ points, occupations, spin, and both electronic and geometry convergence criteria. Include the conditions that affect your conclusions in a report, and retain complete workflows and inputs with the project so that the calculation can be understood and reproduced later.

Chapter 7

# Editing atomic structures

A calculation-ready structure needs more than the right elements and atomic positions: it must also represent periodicity, connectivity, and the unit cell correctly. This chapter begins with a silicon cell, introduces selection, editing, and saving, and then combines the tools in worked examples involving a supercell, a vacancy, aromatic molecules, and ethanol.

## 7.1 Understand the structure model

### Crystals and molecules

A crystal model describes a periodic material through a unit cell and the atoms within it. Its three lattice vectors, **a**, **b**, and **c**, define the cell's size and shape. They need not be perpendicular. In an oblique cell, moving along a lattice direction is different from moving horizontally or vertically on the screen.

A molecular model is usually a finite collection of atoms with explicit chemical bonds. For ethanol, for example, the model must specify which atoms are connected and whether each connection is a single or double bond. Two atoms that look close together on screen do not necessarily have the connectivity needed for molecular editing. Check atomic positions and chemical bonds as two aspects of the same model.

When a 3D structure document is active, the ribbon displays Crystal Modeling or Molecule Modeling according to the object type. Crystal tools cover cells, symmetry, defects, and surfaces; molecular tools cover sketching, chemistry, fragments, and geometry. Availability also depends on the selection. Setting a distance requires two atoms, and setting an angle requires three; a single selected atom is insufficient for either command.

To begin, open the Project Browser from the sidebar, expand Structure, and double-click the structure. Then select its modeling tab in the ribbon. If the project tree is hidden, choose Project Browser from the sidebar's Open Panels gallery. The sidebar controls working panels; the ribbon provides the structure commands used here.

### Cartesian and fractional coordinates

Cartesian coordinates specify positions directly in space, usually in angstroms (Å) in the modeling interface. Fractional coordinates give proportions along the three lattice vectors. For $(u, v, w)$, the position is $u\mathbf{a} + v\mathbf{b} + w\mathbf{c}$. These components are dimensionless: they specify how far to go along each lattice vector, rather than distances along global Cartesian axes.

To read the lattice input table, arrange the vectors as matrix rows:

$$A = \begin{bmatrix} a_x & a_y & a_z \\ b_x & b_y & b_z \\ c_x & c_y & c_z \end{bmatrix}, \qquad \mathbf{f} = (u, v, w), \qquad \mathbf{r} = (x, y, z).$$

The rows are **a**, **b**, and **c**; the columns within each row are global $x, y, z$ components. With the row-vector convention used in this manual,

$$\mathbf{r} = \mathbf{f}A, \qquad \mathbf{f} = \mathbf{r}A^{-1}. \tag{7.1}$$

Here $A^{-1}$ is the inverse of the complete lattice matrix, not the reciprocal of each entry. The cell must have nonzero volume for a Cartesian position to determine unique fractional coordinates.

For a simple example, take $\mathbf{a} = (4, 0, 0)$, $\mathbf{b} = (1, 3, 0)$, and $\mathbf{c} = (0, 0, 6)$, all in Å. The fractional position $(0.5, 0.5, 0)$ gives

$$\mathbf{r} = 0.5(4, 0, 0) + 0.5(1, 3, 0) = (2.5, 1.5, 0)\,\text{Å}.$$

Half of **b** includes $0.5$ Å along global $x$ and $1.5$ Å along global $y$. The second fractional component of a skewed cell is therefore not its Cartesian $y$ coordinate. Figure 7.1 shows the addition as two joined arrows.

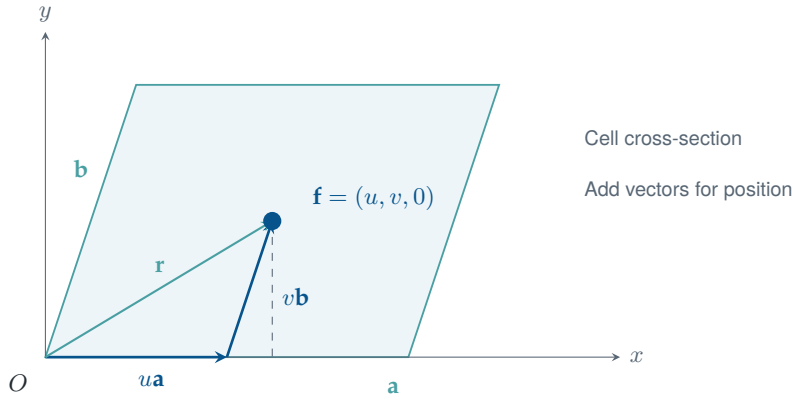



**Figure 7.1** Fractional and Cartesian coordinates (2D schematic). Adding $u\mathbf{a}$ and $v\mathbf{b}$ gives $\mathbf{r}$. In a skewed cell, $\mathbf{b}$ has both global $x$ and $y$ components.

In a cubic cell of side $5$ Å, $(0.5, 0, 0)$ instead gives a displacement of $2.5$ Å along $\mathbf{a}$. The lattice differs, not the conversion rule. Holding $\mathbf{f}$ fixed while changing $A$ generally moves atoms with the cell. To keep their spatial positions fixed, retain $\mathbf{r}$ and recalculate $\mathbf{f}$ using the new lattice. Section 9.2 in Chapter 9 explains these options.

| Symbol | Units | Meaning in the parameters |
|---|---|---|
| $A$ | Å | Lattice matrix with rows $\mathbf{a}, \mathbf{b}, \mathbf{c}$. |
| $\mathbf{f}$ | Dimensionless | Fractional coordinates along the lattice vectors. |
| $\mathbf{r}, \mathbf{p}$ | Å | Cartesian position and a rotation anchor or point on a mirror plane. |
| $\mathbf{n}, \hat{\mathbf{n}}$ | Direction | A nonzero direction vector and its unit vector; direction alone is not a length in Å. |
| $\theta$ | Degrees | Dialog rotation angle, positive by the right-hand rule. |
| $S$ | Integers | Supercell transformation matrix defining the new lattice. |
| $N, V$ | Count, Å$^3$ | Independent model-site count and cell volume. |
| $\varepsilon_a, \varepsilon_b, \varepsilon_c$ | Dimensionless | Relative extension along the three lattice vectors. |

**Table 7.1** Symbols used in modeling formulas. Coordinates and anchors have length units; direction vectors specify orientation. Their roles in dialogs differ.

After selecting an atom, use the information card in the 3D view to check its position. A displayed periodic image may lie in a neighboring cell; its im-

age offset identifies that translation. Before entering three numerical components, check the coordinate type. A displacement of 0.5 Å is not the same as half a lattice vector.

> **Atom indices**
>
> Atom indices in modeling dialogs start at 1. They identify sites in the current document and are different from atomic numbers. Index 6 might identify the sixth silicon atom; it does not mean carbon. Check the indices again after changing the atom count or reordering the model.

## 7.2 Example: inspect a silicon cell

This example uses `Si.poscar` from the accompanying examples. It contains 8 silicon atoms in a cubic cell with a side length of approximately 5.46873 Å. That lattice constant is the starting parameter for this modeling exercise.

The silicon calculation in Chapter 10 uses a different file, `silicon.cif`, with a side length of approximately 5.44370 Å. Both contain 8 silicon sites, but their lattice constants differ. Use `Si.poscar` when checking supercell scaling here, and start from the specified file when comparing energies in the calculation chapters.

Open `Si-introduction.ks` saved in Chapter 3, then import `Si.poscar`. The project tree will contain both the original `silicon` and the newly imported `Si`. Perform the following operations in the `Si` document. Save the modeling work under a new project name to retain the original exercise.

1. Open `Si-introduction.ks`, import `Si.poscar`, and open the new `Si` structure's 3D document. Wait for the atoms, cell, and information area to appear.
2. Drag with the left mouse button to rotate and use the wheel to zoom. Inspect the whole cell first, then zoom in until individual atoms are easy to distinguish.

3. Click an atom and inspect its site index, element, and coordinates. Select another atom and observe how the information changes.
4. Use the viewer toolbar's axis controls to view the structure along different lattice directions. In a nonorthogonal cell, a lattice axis and its corresponding reciprocal axis can point in different directions; this becomes particularly useful when inspecting inclined surfaces.
5. Press Space to recenter and fit the view. Rotating the view changes only the viewing direction, not the atomic positions.

Figure 7.2 shows the information card for a periodic image of Site 7. Subtracting the Image offset from the displayed fractional coordinates gives the coordinates of the site in the model:

$$(0.750, 0.250, 1.250) - (0, 0, 1) = (0.750, 0.250, 0.250).$$

The offset is a translation by one cell along $\mathbf{c}$. The Cartesian coordinates locate the displayed image. A fractional coordinate outside the range 0 to 1 does not add an independent site to the model. The viewer summary reads 8 sites and 41 atoms; the latter includes displayed images.

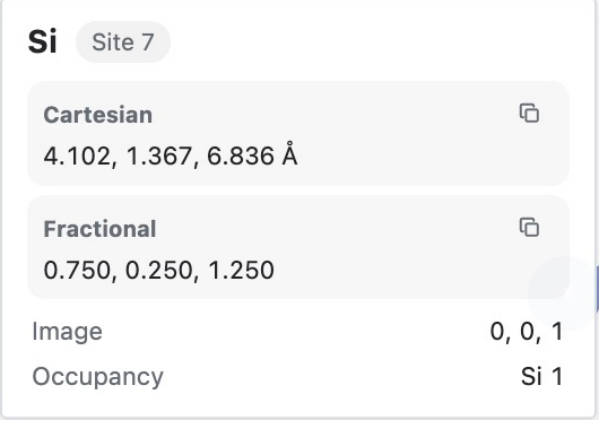



**Figure 7.2** Information card for a periodic image of Site 7. Read the fractional coordinates together with the image offset: this is still the model's seventh site. Si 1 under Occupancy means full silicon occupancy.

### Display more cells

One cell may not show enough of the crystal's extended connectivity. Open the viewer settings from the 3D view's own toolbar, find Repeat cell, and set the $a$, $b$, and $c$ repetitions to 2, 2, and 1. More periodic images appear, making the spatial arrangement easier to inspect. Use the reset control afterward to return to $1 \times 1 \times 1$.

This setting changes only the displayed repetition range. It does not turn the eight-atom project structure into a larger calculation model. To model a local defect or reduce interactions between its periodic images, use Build Supercell, described below. Both operations can put more atoms on screen, but they change different things.

### Choose a representation

Ball-and-stick views make connectivity easy to see; larger spheres help show the space occupied by atoms. Viewer settings provide element colors, radius modes, atom scaling, and visibility controls for bonds, cell edges, coordination polyhedra, and other objects. When preparing a teaching figure or report, decide what the figure should explain before choosing a representation. Showing every auxiliary object at once can obscure the local feature of interest.

Crystal bonds are often inferred from neighboring atoms. Changing the bonding strategy can change the displayed bonds without moving any atoms. For a molecule with explicit topology, prefer connectivity from the source file. For a crystal, interpret displayed bonds in the context of the coordination environment.

## 7.3 Edit atoms precisely

### Make the right selection

Click to select an atom. Hold Shift or Ctrl while clicking to add or remove atoms from the selection. Double-click to select a connected structure.

For larger models, press B and drag a box, or press L and drag a lasso. Cmd/Ctrl+A selects all; Esc exits the current tool or clears the selection.

The selection is not necessarily every visible sphere within an outline. Periodic display may show several images of the same underlying site. Before editing, check the selection count and site indices, especially when displaying more than one cell. Save frequently used groups as selection sets and recall them through Selection Sets.

### Why sites and atoms differ

The viewer's sites count is the number of independent model sites. Its atoms count includes images used to display boundaries and periodic connections. Check supercell scaling, element counts, and defects against the site count. An eight-site silicon cell may show more than eight spheres; after building a $2 \times 2 \times 2$ supercell, check for 64 sites rather than expecting exactly 64 visible spheres.

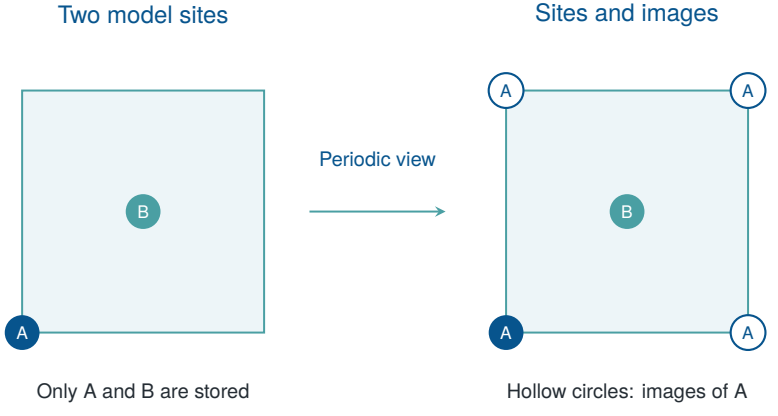



**Figure 7.3** Independent sites and boundary images (2D schematic). Site A can appear on several periodic boundaries. Drawing more spheres does not add independent model sites.

For geometry operations where order matters, clear the selection and select atoms individually. Two atoms define a distance; three define an angle

with the second atom at its vertex; four define a dihedral in sequence. A different order can define a different geometric quantity.

### Drag interactively or enter numerical values

After selecting atoms, press G and drag to translate them, or R and drag to rotate them. Press X, Y, or Z during a transform to constrain its direction. Dragging is useful for arranging an approximate geometry. For an exact displacement, angle, or bond length, finish with the relevant numerical command.

Translate Atoms takes a displacement vector, whereas Move Atoms takes new coordinates. To move an atom from $(1, 0, 0)$ to $(2, 0, 0)$, either translate it by $(1, 0, 0)$ or set its new coordinates to $(2, 0, 0)$. Crystal commands also offer the relevant fractional or Cartesian representation; check that choice before entering values.

Rotate Atoms requires an axis, angle, and anchor. Angles are in degrees, and the anchor is a point through which the rotation axis passes. To rotate a fragment about its attachment site, place the anchor near that site rather than leaving it at a distant origin. The preview will then show the intended motion more clearly.

## 7.4 Apply and save structural changes

Most parameter dialogs that modify a structure offer Preview, Apply, and Cancel. Preview shows the geometric effect of the parameters. Apply commits the change to the current document's editing state. Saving writes that state into the project. After changing parameters, preview again. Cancel closes the dialog and restores the original view.

1. Open the modeling command and check the selection and parameter units.
2. Enter the parameters and click Preview. Rotate or zoom the view to inspect the changed region.

3. If the result is suitable, click Apply. Otherwise change the parameters and preview again, or click Cancel to restore the original view.
4. Check the atom count, cell, or geometric values after applying. Use Cmd/Ctrl+Z to undo and Shift+Cmd/Ctrl+Z to redo.
5. Save the project after a meaningful stage of work and give the model a clear name.

Reset in History restores the structure from when the document was opened or last saved. It differs from the camera recenter control. Saving establishes the latest saved model as the editing baseline and clears the current undo history. Keep two clearly named project structures for a lasting comparison rather than relying solely on undo.

Analysis commands do not necessarily create a structure preview, and their output varies. Diagnose Molecule checks valence, close contacts, and bond lengths without changing the structure; the graphical interface reports Molecule diagnostics completed. on completion. Interpret the result alongside composition, connectivity, and measured geometry. Section 7.7 gives a checking procedure; MATLAB® toolbox users can also inspect diagnostic data with the accompanying script.

## 7.5 Example: a silicon supercell with one vacancy

### Build the supercell

A supercell repeats a base cell along selected directions and includes all replicated atoms in a new model. Diagonal repetition $(n_x, n_y, n_z)$ multiplies both atom count and volume by $n_x n_y n_z$. Here an eight-site silicon cell becomes a $2 \times 2 \times 2$ supercell containing 64 independent sites. The input and repeat factors determine that count, providing a direct check.

1. Return to the `Si` document imported from `Si.poscar`. Restore display repetitions to $1 \times 1 \times 1$ so that displayed images are not confused with model atoms.

2. On Crystal Modeling, open Supercell & Lattice in the Crystal & Cell section and choose Build Supercell.
3. Check the default `2 2 2` under Supercell scaling and click Preview. Inspect the enlarged cell boundaries in all three directions. A message below the controls confirms that the preview is displayed.
4. Click Apply if the preview is correct.
5. Check for 64 silicon sites and lattice vectors twice their original lengths. The illustrated summary reads 64 sites and 166 atoms; use the first number to check replication. Save a separate project now if you want to retain a defect-free supercell before creating a vacancy.

A general supercell is defined by an integer matrix $S$. Each row specifies one new lattice vector as a combination of the original three. For example, the first row $(2, 1, 0)$ means $\mathbf{a}' = 2\mathbf{a} + \mathbf{b}$. With row-vector lattices,

$$A' = SA, \qquad V' = |\det S|V, \qquad N' = |\det S|N. \tag{7.2}$$

The entries of $S$ must be integers and its determinant nonzero. The magnitude $|\det S|$ is both the volume ratio and the scale factor for the independent site count. $N$ always counts model sites, not extra boundary-image spheres. For three repeat counts, $S = \mathrm{diag}(n_a, n_b, n_c)$ reduces Eq. (7.2) to their product. The silicon example uses $S = 2I$, so $|\det S| = 8$ and 8 sites become 64.

For a nondiagonal algebraic check,

$$S = \begin{bmatrix} 2 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \qquad \begin{cases} \mathbf{a}' = 2\mathbf{a} + \mathbf{b}, \\ \mathbf{b}' = \mathbf{b}, \quad \mathbf{c}' = \mathbf{c}. \end{cases}$$

The determinant is 2. An eight-site input should produce 16 sites and twice the volume. The new outline can be skewed, while the local periodic arrangement is copied from the original. This differs from strain: choosing a larger repeat unit does not double every interatomic bond length.

Supercell scaling accepts either three repeat integers or a full $3 \times 3$ matrix. Write out the new basis vectors, check the determinant, preview the outline, and verify the applied site count to distinguish reorientation from replication. Input `1 1 1` gives $I$ and does not enlarge the model. The tool still wraps equivalent positions into the unit cell, which can help collect representatives lying outside it.

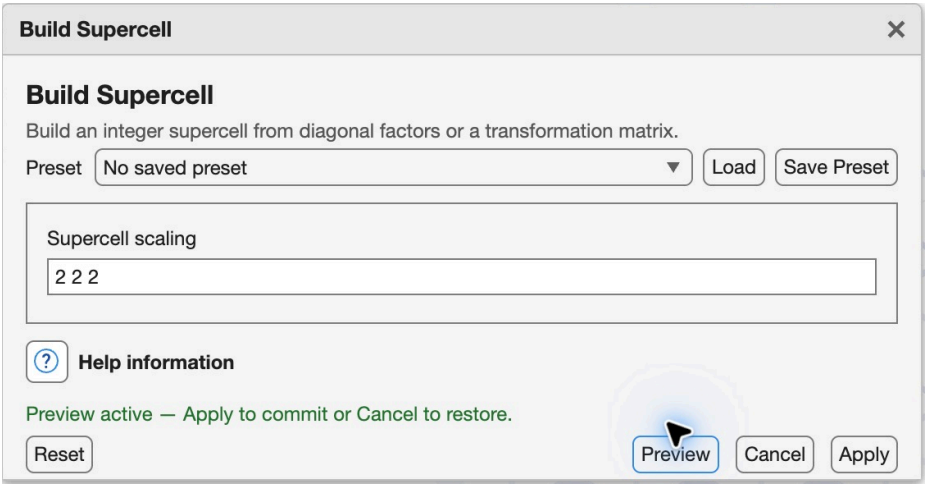



**Figure 7.4** Supercell parameters and preview status. `2 2 2` repeats the cell twice in each direction.

### Create and check a vacancy

A vacancy removes an atom from a lattice site. Expanding the cell first can reduce excessive interactions between neighboring periodic images of the defect. Determine the required supercell size through convergence tests for the subsequent calculation. Once the model is prepared, an appropriate calculation can be used to obtain the defect formation energy.

1. Confirm that the active document is the 64-site `Si` supercell just created. This example removes site 1.
2. On Crystal Modeling, open Defects & Alloys in the Crystal Materials section, then choose Create Point Defects → Vacancy. In the dialog, check

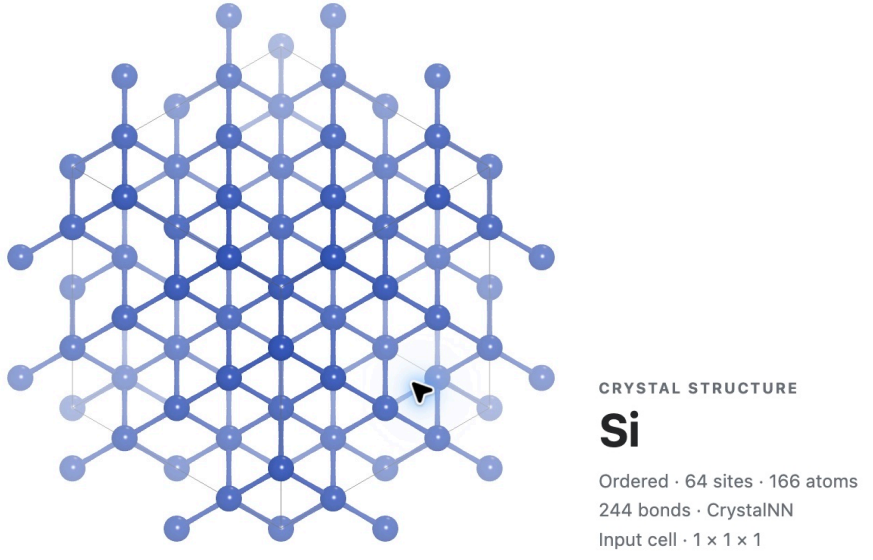



**Figure 7.5** The applied silicon supercell, with the full model and a separate summary detail. The model contains 64 independent sites; the displayed count of 166 atoms includes boundary images. Input cell $1 \times 1 \times 1$ is the display range for the new supercell.

that Defect type is Vacancy and enter `1` under Site indices (MATLAB 1-based).

3. Check the type and index, then click Apply. Rotate the view to inspect the connections around the removed atom.

4. Check that the site count falls from 64 to 63 while the supercell remains unchanged. The example reads 63 sites and 162 atoms; removing one independent site need not remove only one displayed image.

5. Use Save As to save `Si-vacancy-tutorial.ks`, retaining `Si-introduction.ks`. Check that the unsaved-change asterisk disappears from the title.

The supplied `Si-vacancy-tutorial.ks` contains the original `silicon` reference and the vacancy-edited `Si` structure. For a defect-free comparison, rebuild the 64-site model from `Si.poscar` and save a separate project before creating the vacancy.




**Figure 7.6** Silicon vacancy parameters: Vacancy at site 1. After applying, check that the independent site count decreases from 64 to 63.

Symmetry handling in crystal editing affects equivalent positions. For a local defect, distinguish a change to one site from a change to an entire equivalent-site set. Create Point Defects provides vacancy, substitutional, interstitial, and antisite operations. Check the independent site and element counts after applying.

## 7.6 Example: build benzene and pyridine from an aromatic ring

### Build the benzene skeleton

Molecular sketching can place atoms individually in the 3D view or build a skeleton quickly with rings and fragments. Press S to enter sketch mode. Click empty space to place an isolated atom, drag from an existing atom to add a connected atom, or drag to another existing atom to create a connection. The sketch toolbar selects elements, bond orders, and rings.

Here we first use the parameter dialog to create a regular six-membered ring, making the roles of aromaticity and hydrogen addition clear.

1. Choose Home → Structure → New structure → Blank molecular structure, activate the new document, and open Sketch Ring on Molecule Modeling.
2. Set ring size to 6, element to C, and side length to 1.4 Å. Enable the aromatic ring option. Enter 0, 0, 0 in the three center fields and 0, 0, 1 in the three normal fields.
3. Leave attachment to other atoms disabled, then preview and apply. This creates the carbon skeleton; hydrogens still need to be added.
4. Press Cmd/Ctrl+A to select the six ring atoms and run Add Hydrogens. Check for the benzene composition $C_6H_6$.
5. Run Diagnose Molecule and close its completion message. Rotate the view, inspect the ring, each carbon's hydrogen, and local distances, then save the model as `Benzene`.

An aromatic ring has different connectivity information from a ring of ordinary single bonds. Arranging six carbons in a hexagon, or making their displayed bonds look alike, does not replace aromatic bonds and atomic chemistry settings. Check composition, bond order, and aromatic flags together.

Inspect the entire molecule from the side after adding hydrogens. A planar ring skeleton does not guarantee that automatically placed hydrogens lie in the ring plane. The supplied `benzene.json` preserves the initial hydrogenated model, while `benzene-optimized.json` contains the structure after generic molecular-model optimization. The latter is still a modeling reference. Use native JSON to exchange these aromatic molecules while preserving chemical properties and aromatic bonds, and check connectivity after reimporting.

### Replace one carbon with nitrogen

Save the benzene model, then choose Home → Structure → Import structure from file... and import `benzene.json` again. Use the new document for

pyridine so that the original benzene remains available for comparison. To make hydrogen addition easy to check, remove hydrogens before replacing a ring carbon.

1. Select all atoms in the newly imported structure and run Remove Hydrogens. Select one ring carbon and open Atom Chemistry.
2. Change its element to N, set formal charge to 0 and hybridization to `sp2`, and retain the aromatic atom option.

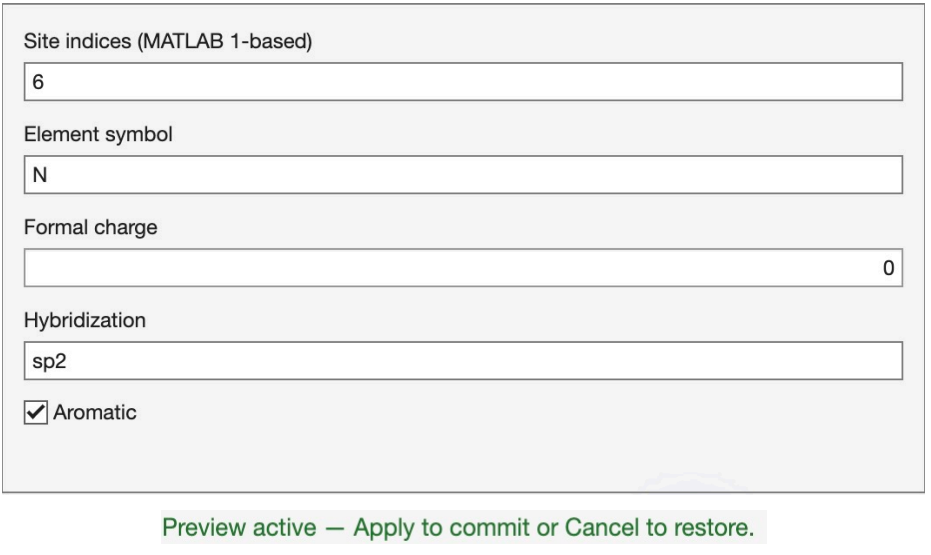



**Figure 7.7** Replacing ring atom 6 with nitrogen: element N, formal charge 0, hybridization sp2, and Aromatic enabled. The preview message appears below; click Apply to commit.

3. Apply, select all ring atoms, and run Add Hydrogens. Check for $C_5H_5N$ and inspect connectivity around nitrogen.
4. Run Diagnose Molecule, close the completion message, and compare benzene and pyridine: composition, ring connectivity, and hydrogens near nitrogen. Import the supplied `pyridine.json` to compare with a complete neutral pyridine model.

Element, formal charge, and hybridization hints together affect how connectivity is interpreted. Keeping every original hydrogen after changing an element can produce an unintended valence. Editing the skeleton first and adding hydrogens afterward is usually easier to check.

Neutral pyridine has 42 electrons; this example uses a singlet. Adding hydrogens to an odd-electron skeleton can retain the skeleton's spin multiplicity, causing a charge–spin mismatch when the file is read again. The supplied `pyridine.json` has charge 0 and multiplicity 1. Use it as a starting point for further calculations.

Figure 7.7 shows the parameters used to change ring atom 6 to nitrogen. Figure 7.8 shows the completed six-membered ring and five hydrogens.

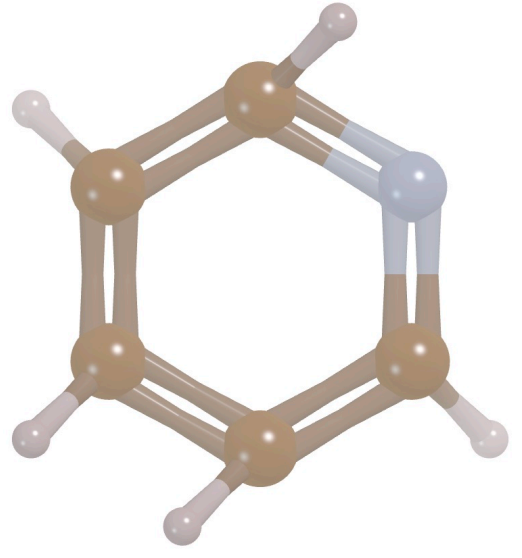


**Figure 7.8** Initial pyridine after element replacement and hydrogen addition to the six ring atoms: five carbons, one nitrogen, five hydrogens, 11 atoms in total, and 11 connections. For further calculations, use the neutral singlet structure in `pyridine.json`.

## 7.7 Example: adjust an ethanol conformation

### Build an initial molecule with clear connectivity

Ethanol has a C–C–O heavy-atom skeleton. Choose Home → Structure → New structure → Blank molecular structure, open the document, sketch the three atoms in sequence, and add hydrogens. The atom and bond commands provide an alternative. For readily checked input, this example uses the following coordinates in Å.

**Table 7.2** Input for the ethanol skeleton. The collinear geometry is only a starting point for editing.

| Atom and index | Position and connection |
|---|---|
| 1: C | $(0, 0, 0)$; the first isolated atom. |
| 2: C | $(1.52, 0, 0)$; connected to atom 1 by a single bond. |
| 3: O | $(2.95, 0, 0)$; connected to atom 2 by a single bond. |

With Use force-field ideal bond length enabled, sketching uses an ideal bond length along the connection direction. Disable it to reproduce the table's coordinates exactly. Add hydrogens after building the skeleton and check for $C_2H_6O$. This geometry has not yet been cleaned up; its collinear skeleton is not an equilibrium ethanol structure.

### Set geometric quantities

To read a distance, choose Measurements → Atom-to-atom distance in the viewer toolbar and click atoms 1 and 2 in order. To change it, open Set Distance in the ribbon, enter `1 2` as the atom indices, and set the target to $1.52$ Å. Then open Set Angle, enter `1 2 3`, and set $109.5°$. For the distance, angle, and dihedral operations here, choose the subtree movement scope so connected atoms move together.

The atoms scope moves local atoms only; subtree moves the connected atoms on one side; fragments adjusts a connected fragment as a whole. Changing an angle by moving a heavy atom without its attached hydrogens

can distort neighboring geometry. Inspect the whole molecule in the preview rather than only the target angle.

1. Set the required distances and angle in the C–C–O skeleton, measuring again after each application.
2. Adjust the view and select four connected atoms in order, such as C–C–O–H. Check that the last atom is a hydrogen bonded to O.
3. In Set Dihedral, enter a convenient comparison angle, such as $60°$, and preview the terminal fragment's rotation.
4. Try another dihedral, inspect it, and undo to return to the earlier conformation. Use this comparison to understand conformational changes, rather than interpreting a chosen angle as the minimum-energy conformation.

### Check an angle in the viewer

Read edited geometric values in the 3D viewer's measurement card. Geometry Editor → Measure Geometry in the ribbon is a separate command. Its graphical completion message reads Geometry measurement completed. and does not list values in the dialog. Use the viewer procedure below to check an angle.

Figures 7.9 and 7.10 show angle setting and measurement. To make atom selection easier, this section imports a copy of `ethanol-optimized.json` and changes one angle. Its starting point differs from the earlier construction using a collinear skeleton.

1. Import `ethanol-optimized.json` and open the copy's molecular document, retaining the original as a reference.
2. In Set Angle, enter atoms `1 2 3`, target $109.5°$, and movement scope `subtree`. Preview, then apply. This sets a geometric value directly; molecular optimization may follow later.

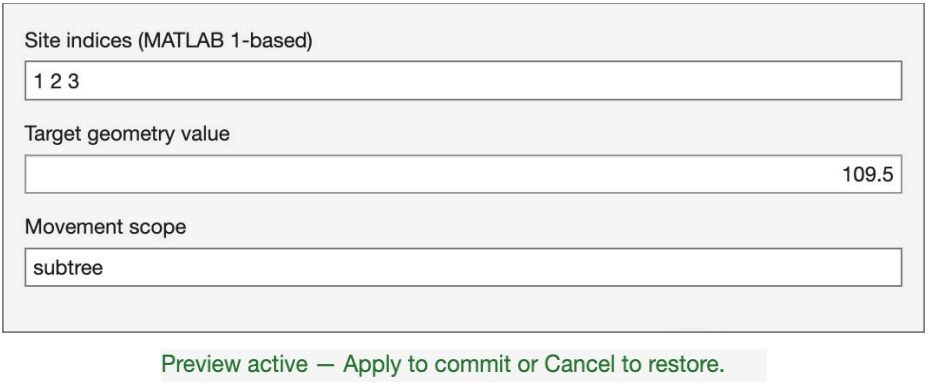



**Figure 7.9** Ethanol angle settings: atom sequence `1 2 3`, target $109.5°$, and movement scope `subtree`. The preview message appears below; click Apply to commit the edit.

3. Close the parameter dialog. On the 3D view's right toolbar, open the ruler-shaped Measurements button and choose Bond angle. Select the atoms afresh after entering measurement mode.

4. Click C atom 1, C atom 2, and O atom 3 in order; the second atom is the vertex. In the illustration, C1 is at the top, C2 is in the middle, and oxygen is red. After the third click, the left measurement card shows the angle and atom path, and two line segments and an arc appear on the model.

5. Check for $109.500°$ and the path #1 C, #2 C, #3 O. Record the value and atom order, close the card, and save the edited copy.

This reading confirms that editing produced the specified angle. Ethanol's equilibrium angle must be determined by optimization. For a before-and-after comparison, measure the two separately saved models using the same atom order and record their actual readings.

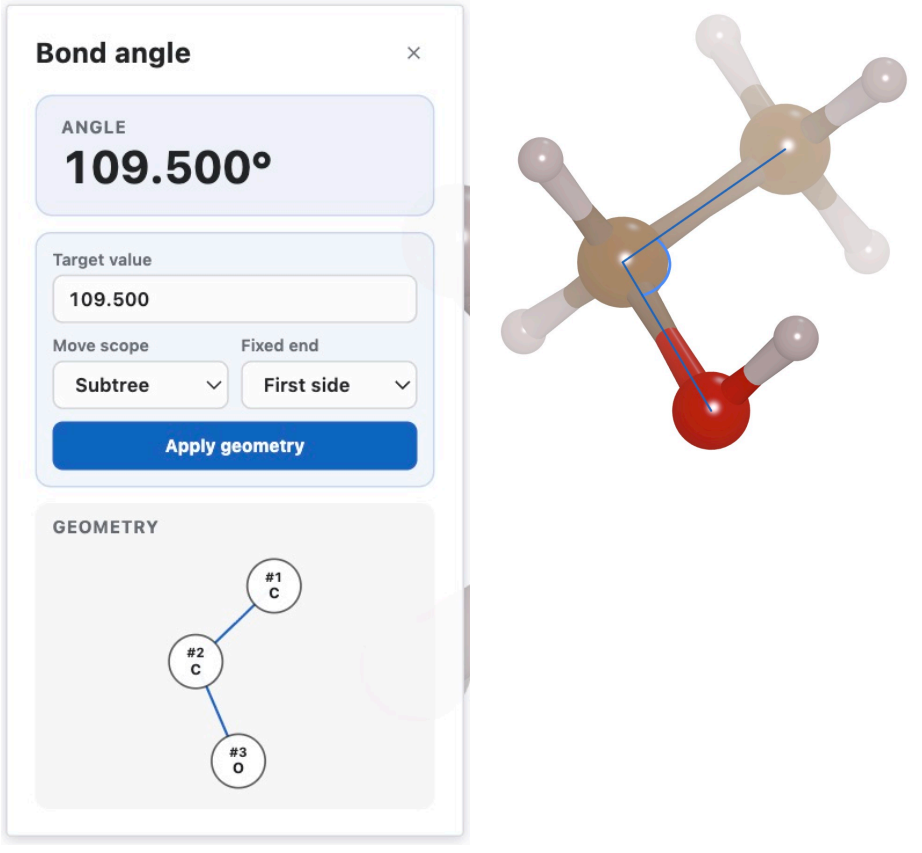



**Figure 7.10** Checking the edited angle with Bond angle. Left: $109.500^\circ$ along #1 C, #2 C, and #3 O. Right: the three heavy atoms and angle marker. The exercise starts from a copy of `ethanol-optimized.json` and sets the target to $109.5^\circ$; this is a prescribed exercise angle.

### Clean up and optimize the geometry

Clean Geometry applies rules to improve obvious local geometry problems. Optimize Geometry further adjusts the structure using a generic molecular model. These tools are useful for initial preparation, especially after manual sketching or connectivity changes. Optimization parameters include an iteration limit, force tolerance, and optional fixed-atom indices.

Start with the defaults of 200 iterations and a 0.001 force tolerance. Preview and apply, compare the structural changes, and remeasure relevant bond lengths, angles, and close contacts. The accompanying script records the optimization method and convergence history in detail. For accurate equilibrium geometries, conformational energies, or reaction studies, continue with a suitable method from the calculation chapters.

The supplied `ethanol-edited.json` preserves the editing result. Reimporting gives a C–C distance of 1.52 Å, a C–C–O angle of 109.5°, and a C–C–O–H dihedral of 60° with atom 9 as the terminal hydrogen. `ethanol-optimized.json` has instead been cleaned and optimized with the generic molecular model, reaching a force threshold of 0.001 kJ/(mol Å) in 47 iterations. Optimization is free to change those geometric quantities; values set during editing are not automatically retained as constraints.

### Check molecular composition and geometry

Use the following sequence in the graphical interface, recording before-and-after readings together.

1. Compare the formula and atom count in the structure summary with the intended molecule: $C_6H_6$ for benzene, $C_5H_5N$ for pyridine, and $C_2H_6O$ for ethanol.
2. Rotate the model and inspect each modified heavy atom and its attached hydrogens. Assess connectivity using the assigned elements, formal charges, bond orders, and aromaticity. To correct it, select the atom, explicitly adjust Atom Chemistry or Set Bond Order, then add hydrogens according to the connectivity.
3. For a suspiciously short contact, choose Measurements → Atom-to-atom distance and select two atoms in order. For an angle, choose Bond angle and select three. Compare actual readings before and after editing rather than judging proximity from sphere sizes alone.

4. Save the model and checking notes. Export native JSON for complete explicit bond and atom-property records. Run the accompanying checking script in the MATLAB® toolbox environment when individual diagnostic data are needed.

Diagnose Molecule checks the entire molecule, regardless of a local selection. Compare edited models using composition, connectivity, and measurement records.

## 7.8 Reuse modeling work

Once a molecule is built, you need not start from individual atoms every time. Select an attachment site, open Fragment Sketcher from its dropdown on Molecule Modeling, choose the fragment and connection mode, and use Attach Fragment. Check the fragment name and attachment head first. User fragments store reusable local structures in a library; repeat units serve polymer construction. They have different purposes.

Presets in parameter dialogs store frequently used settings, such as a supercell transformation or a combination of slab thickness and vacuum. Give them meaningful names, load them later, and adjust the values for the current material. Selection sets instead store groups of sites for repeated selection.

Use Record and Save Recipe for a longer sequence of operations. For research records and collaboration, keeping the original structure, operation parameters, and final files is easier to audit than retaining only a 3D screenshot. A figure explains the structure; a model file preserves its atoms and cell. Together they form a complete modeling result.

Chapter 8

# Building materials models

Crystal defects, molecules on surfaces, and polymer mixtures can all be built step by step from a small set of well-defined structural inputs. This chapter explains how to choose those inputs, interpret construction parameters, and check the resulting models. Each example aims to produce a suitable initial structure with a clear provenance for subsequent calculations.

## 8.1 Build a crystal from a space group

### Understand asymmetric sites

Many atomic positions in a crystal are related by symmetry. A space-group construction therefore needs only a lattice, space group, and asymmetric sites, rather than every atom individually. The tool expands their equivalent positions. Understanding that the input is asymmetric sites and the output is a complete working cell is central to using the crystal builder.

Specify the space group by symbol or number. Lattice matrix is an editable $3 \times 3$ table: rows a, b, and c are the lattice vectors; columns x, y, and z are their Cartesian components in Å. Click and edit individual cells. Each asymmetric site also requires an element, fractional coordinates, and occupancy. Occupancy 1 means full occupancy by that element. Partial occupancy describes an average structure; calculations requiring explicit atomic arrangements generally need an ordered approximation or alloy model.

### Example: face-centered cubic copper

Build a conventional face-centered cubic Cu cell with $a = 3.61$ Å. This is an example input; replace it with appropriate experimental or calculated data for your work.

1. Open and activate a crystal document from the Project Browser. On Crystal Modeling, open Symmetry Tools in Crystal & Cell, then choose Build from Space Group.
2. Enter `Fm-3m` under Space-group symbol or number. In Lattice matrix, set the a/x, b/y, and c/z diagonal cells to 3.61 and the other six cells to 0.
3. Enter Cu under Element symbol and `0 0 0` in the Asymmetric fractional coordinates text field. Set Asymmetric-site occupancies to 1 and initially retain the expanded-position overlap tolerance of 0.1 Å.
4. Preview and inspect the expansion. A conventional face-centered cubic cell should contain 4 Cu sites, rather than only the one input position.
5. Click Apply. In the result window, inspect asymmetric and equivalent site information, check elements, occupancies, and positions, then save as `Cu-fcc`.

Only one asymmetric site is needed; the space group generates the other positions. If expanded sites collide, first check whether equivalent positions have been entered more than once, rather than immediately reducing the collision tolerance. Supplying a site already generated by the space group as another independent input can duplicate the same atom.

The supplied `Cu-fcc.cif` is the four-site cell generated with these settings. `Cu-vacancy.cif` is one result of the later vacancy enumeration and contains three sites. The accompanying script regenerates both structures.

### Choose a cell representation

A primitive cell uses fewer atoms to represent translational periodicity. A conventional cell often makes symmetry and crystal axes clearer. Different shapes and atom counts for the same material do not imply a composition change. Compare formula, volume, and site count together rather than judging only the displayed cell size.

Symmetry, Primitive Cell, and Conventional Cell use positional and angular tolerances. Their defaults are 0.01 Å and 5°. These determine whether

slightly displaced positions are still treated as symmetry equivalent. A large tolerance may interpret a real distortion as numerical noise, so assess the result in light of the model's origin.

Make P1 explicitly retains the current atoms without additional declared space-group constraints, which is useful for local editing. Atomic positions and their geometric symmetry are preserved; subsequent edits no longer depend on the original space-group declaration.

## 8.2 Adjust the cell

### Choose the right cell operation

Edit Lattice lets you choose whether to preserve Cartesian atomic positions. Redefine Lattice preserves them while changing the lattice that encloses them. Both dialogs edit a table with lattice vectors in rows a, b, c and Cartesian components in columns x, y, z. Strain Structure instead applies a homogeneous deformation to the entire structure. Decide whether your goal is to change the boundary description or to scale internal distances with the lattice.

For example, entering `0.01` under Strain applies engineering strain 0.01 in all three directions, multiplying lengths by 1.01 rather than adding 0.01 Å. To stretch only the first direction, enter `0.01 0 0` in the same field. Preview, inspect lattice lengths, angles, and local distances, then save.

Orthogonalize Cell searches integer supercells subject to length, angle, and atom-count limits. It can prepare an approximately orthogonal box for later processing, but not every lattice has an exact orthogonal representation within a finite search range. If no result is found, consider increasing the maximum length or atom count, or relaxing the angular tolerance where appropriate. Larger supercells also increase subsequent calculation costs.

## 8.3 Screen defect configurations

Creating the same defect at symmetry-related sites can produce equivalent models. Enumerate Point Defects combines such duplicates and reports their degeneracy: the number of positions belonging to the same class. Degeneracy does not rank defect energies.

1. Open the earlier `Cu-fcc` model and choose Enumerate Point Defects.
2. Set the defect type to `vacancy`, using the Cu cell's sites as candidates. Retain the default positional and angular tolerances initially.
3. Click Apply and inspect the candidate table. The four sites in ideal conventional face-centered cubic Cu are equivalent, so there should be one vacancy candidate with degeneracy 4.
4. Check its remaining atom count, composition, and contacts. Open the added defect structure from the Project Browser and save the project, retaining both pristine copper and the defect result.

A substitution requires the replacement element; an interstitial requires the new atom's coordinates and coordinate type; an antisite exchanges elements between sites. Similar names describe different structural changes. Check element counts first, then positions, to catch input mistakes more easily than by inspecting the image alone.

Generate SQS Model converts partial occupancies or mixed compositions into explicit arrangements in a finite cell. The tool enumerates or performs a time-limited search against pair-correlation targets, with configurable supercell scaling and search budget. The budget affects candidate quality and elapsed time. Retain the model together with its composition, cell size, and search settings for comparison.

## 8.4 Example: build a Pt(111) surface

### Understand a slab model

A surface model usually consists of a finite crystal slab and vacuum. It repeats periodically in-plane, while vacuum separates neighboring slabs along the normal. Miller indices $(hkl)$ specify the cut plane. Slab thickness, termination, and vacuum together define the resulting model.

### Obtain the normal from Miller indices

A crystal plane's normal is determined by reciprocal vectors. Using the convention in Chapter 6,

$$\mathbf{G}_{hkl} = h\mathbf{b}_1 + k\mathbf{b}_2 + l\mathbf{b}_3, \qquad \hat{\mathbf{n}} = \frac{\mathbf{G}_{hkl}}{\|\mathbf{G}_{hkl}\|}. \tag{8.1}$$

The reciprocal vectors $\mathbf{b}_1, \mathbf{b}_2, \mathbf{b}_3$ have units $\text{Å}^{-1}$; $h, k, l$ are dimensionless integers and $\hat{\mathbf{n}}$ is a unit normal. In a cubic cell, the $(111)$ normal is parallel to $\mathbf{a} + \mathbf{b} + \mathbf{c}$. Here the lattice axes align with global axes, so it is also parallel to global $(1, 1, 1)$. A skewed cell generally lacks this correspondence: do not use Miller indices directly as a Cartesian adsorbate-orientation vector.

For a constructed slab with in-plane vectors $\mathbf{a}, \mathbf{b}$, check its actual normal and cell height using

$$\hat{\mathbf{n}}_{\text{slab}} = \frac{\mathbf{a} \times \mathbf{b}}{\|\mathbf{a} \times \mathbf{b}\|}, \qquad H = |\mathbf{c} \cdot \hat{\mathbf{n}}_{\text{slab}}| \tag{8.2}$$

The height $H$ is the repeat distance between periodic slabs along the normal, and need not equal $\|\mathbf{c}\|$. First choose periodic images that form one continuous slab, then project the atomic positions onto the normal to measure the distance between the outermost layers. Record this distance separately from the requested minimum thickness and the thickness of the complete structural repeats used for construction.

**Table 8.1** Lengths used in surface construction.

| Quantity | Interpretation and checks |
|---|---|
| Minimum slab thickness | A criterion for selecting complete repeat units, not a direct layer count. |
| Atomic-layer span | Normal distance between outermost layers; select one continuous slab first. |
| Cell height $H$ | Measured along the surface normal, Eq. (8.2); includes the slab and periodic gap. |
| Added vacuum $\Delta$ | Length added along the third lattice vector by Add Vacuum, not the final total vacuum. |

For nonorthogonal input, also check the output normal, layer count, and periodic separation. Orientation handling can affect thickness conversion and candidate reduction; the minimum input thickness alone does not establish the result. The platinum example starts from a cubic cell and checks the generated slab.

A minimum slab thickness is not an exact number of atomic layers. The builder selects complete structural repeats to preserve stacking, and different planes and terminations can give different atom counts. After entering 8 Å, inspect the layer count in the generated model.

First build bulk Pt with Build from Space Group. Enter `Fm-3m`. Set the a/x, b/y, and c/z cells of Lattice matrix to 3.92 and the other six cells to 0, giving a cubic lattice with side length 3.92 Å. Enter Pt under Element symbol, `0 0 0` under Asymmetric fractional coordinates, and 1 under Asymmetric-site occupancies. Preview and inspect the bulk cell before generating its (111) surface.

1. Activate bulk Pt. On Crystal Modeling, open Surface Modeling in Surfaces & Interfaces, then choose Build Slab.

2. Enter the table's values and preview. Inspect the in-plane arrangement nearly along the normal, then turn to the side to inspect thickness and vacuum.

**Table 8.2** Parameters for the Pt(111) surface exercise.

| Parameter | Setting and meaning |
|---|---|
| Miller indices (h k l) | Enter 1, 1, 1 in the three fields to select (111). |
| Minimum slab thickness | 8 Å; check the generated layers and geometric thickness. |
| Minimum vacuum | 10 Å for this construction exercise. |
| Center slab | Enabled, placing the slab centrally along the cell normal. |
| Primitive surface cell | Enabled, using the corresponding reduced surface repeat. |
| Termination index | 1; compare terminations separately if several are available. |

3. Set the termination index to 1. After applying, inspect termination information in the result window, considering surface elements, stacking, and your research objective.

4. Check the atoms against the cell boundary, record the actual atom and layer counts, and save as `Pt111-slab`. To try another termination, return to bulk Pt, reopen the command, and change the termination index. The candidate table is for inspection only.

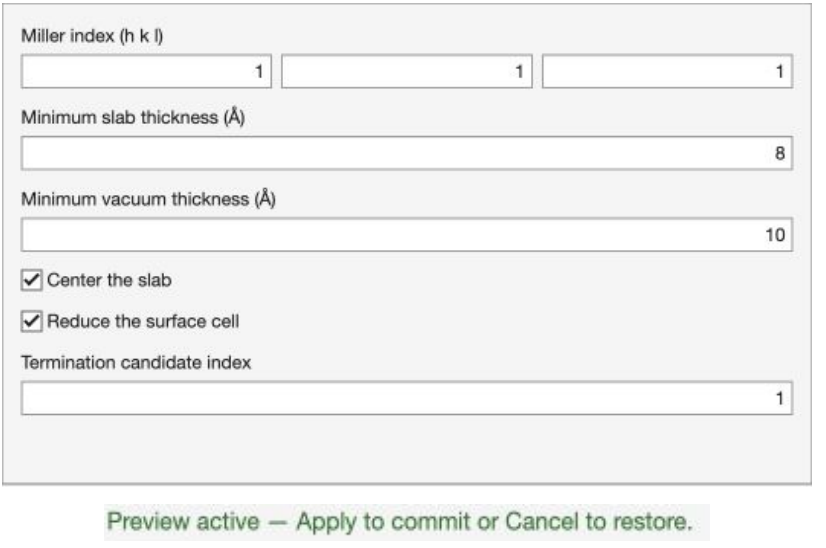



**Figure 8.1** Pt(111) construction after importing `Pt-fcc.cif`: Miller indices 1, 1, 1; minimum slab thickness 8 Å; minimum vacuum 10 Å; centering and primitive surface cell enabled; termination candidate 1. The preview message appears below.

The supplied `Pt-fcc.cif` and `Pt111-slab.cif` contain the bulk and slab models. These parameters give 4 independent sites in 4 geometric layers. Along the normal, the cell's periodic height is approximately 20.3689 Å, the outermost atomic planes span approximately 6.7896 Å, and the separation between outermost planes of neighboring periodic slabs is approximately 13.5793 Å. The thickness used for construction is based on complete crystal repeats and differs from the distance between the outermost atomic planes. Project distances along the normal; the third vector's length in an oblique cell is not directly the vacuum thickness.

### Fix the bottom layers

Surface relaxations often hold some bottom layers fixed to approximate the constraint from deeper bulk material. After slab construction, the tool can identify geometric layers. Open Fix Atoms and set the bottom-layer count to 1 to select the lowest layer. To use an explicit selection instead, set the

bottom-layer count to 0 as directed by the interface and select the intended sites.

Fixed flags are stored in the model. Enable the required constraints in the relevant calculation task when submitting it. Thicker slabs may need several fixed layers. Begin with one and inspect its extent to understand the setting.

### Add vacuum

Add Vacuum increases space along the third lattice vector; the input is an additional thickness. If that vector is inclined to the surface normal, the added normal projection is smaller than the entered length. Here the exercise uses a cell with its third vector along the normal. Adding $5$ Å adds to the existing model rather than setting total vacuum to $5$ Å. Check the cell and slab from the side after applying. Adsorbates or solvent can occupy some previously empty space, so inspect the complete model again after adding them.

Adding vacuum changes the geometry according to

$$\mathbf{c}' = \frac{\|\mathbf{c}\| + \Delta}{\|\mathbf{c}\|}\,\mathbf{c}, \qquad \Delta H = \Delta \left| \frac{\mathbf{c}}{\|\mathbf{c}\|} \cdot \hat{\mathbf{n}}_{\text{slab}} \right|. \tag{8.3}$$

Both $\Delta$ and $\Delta H$ are in Å. The former is the requested added length; the latter is the increase in cell height measured perpendicular to the slab. The dot-product factor lies between $0$ and $1$. With $\mathbf{c}$ along global $z$ and the surface normal along $(1, 1, 1)$, adding $2$ Å increases this perpendicular height by only approximately $2/\sqrt{3} = 1.155$ Å. In Figure 8.2, $\Delta\mathbf{c} = \mathbf{c}' - \mathbf{c}$ has length $\Delta$, and $\Delta H$ is the length of its projection onto the surface normal. Aligning $\mathbf{c}$ with the normal first makes the added length equal to the increase in the gap between periodic slabs.

1. Import `Pt111-normal-c.json` afresh, retaining an unmodified slab. Its third vector already follows the surface normal. Open Add Vacuum under Surface Modeling on Crystal Modeling.

2. Set additional thickness to 5 Å. Preview and compare the slab with the cell boundary from the side, confirming that the extra space lies outside the slab.

3. Apply and recheck Pt site count and internal geometry. Adding vacuum should neither add Pt atoms nor increase the slab's internal layer spacing.

4. Save as `Pt111-extra-vacuum`, separately from the original slab. Restart later adsorption exercises from their specified original files.

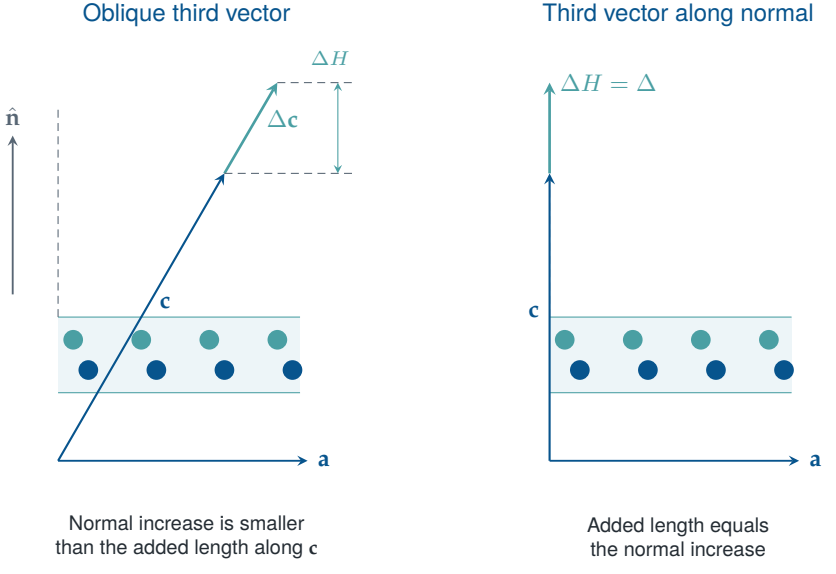



**Figure 8.2** Directions when adding vacuum (section schematic). Relative positions within the slab remain fixed as the third lattice vector extends along its original direction and the slab is recentered. Only a vector normal to the surface converts the full added length into normal separation.

## 8.5 Place adsorbates on a surface

### Choose an adsorption site

Adsorption modeling must specify where to place a molecule, which atom anchors it, and its orientation. An ontop site lies above a surface atom, a bridge site between two atoms, and a hollow site within a region enclosed by several surface atoms. The anchor atom determines which molecular atom is placed at the specified height; an orientation atom helps define its direction.

Use Find Adsorption Sites to relate candidate positions to the surface. The default height is 2 Å. Sites are geometric references for initial configurations, not evidence of stable adsorption.

### Prepare the adsorption cell

Import an unmodified `Pt111-slab.cif`; do not use the version with extra vacuum from the previous section. Its surface normal follows global $(1, 1, 1)$, while the original $c$ vector follows global $z$. The current parameter command adds anchor height along $c$. Aligning $c$ with the normal first makes the entered height agree with the vertical distance in the side view.

Open Redefine Lattice, retain rows $a$ and $b$, and set the $x$, $y$, and $z$ components of row $c$ to 11.76, 11.76, and 11.76 in Å, preserving Cartesian positions. This uses $c_{\text{new}} = c+6a+6b$ without building a supercell, so there are still 4 Pt sites. The new $c$ is perpendicular to the surface; the in-plane angle between $a$ and $b$ remains $60°$. Alternatively, import the prepared `Pt111-normal-c.json`.

Inspect the slab and cell boundary from the side. Vector $c$ should point outward normal to the surface, with internal layer positions unchanged. For another surface, determine its normal from the in-plane vectors rather than copying these components.

### Example: place an O–H fragment interactively

Interactive placement lets you see how a molecule approaches the surface while adjusting the complete fragment's height, orientation, and contacts.

1. Open the prepared `Pt111-normal-c`. Press O to enter adsorbate placement and choose O–H. The fragment toolbar can also use project molecules or user fragments.
2. Drag outward from a surface atom to establish the relationship between the surface site and fragment anchor. Adjust the view so the drag direction is clear.
3. Drag with the left button to change the fragment's orientation, Alt+left-drag to change anchor distance, and right-drag to rotate about the anchoring direction.
4. Check that O–H connectivity is retained and that the fragment is not too close to nearby surface atoms. Press Enter or Apply when the preview is suitable.
5. Save a separate adsorption model. Undo and place it again if another approach is needed.

### Example: place O–H outward using parameters

Parameter-based placement makes configurations easy to record and reproduce. The supplied `oh-fragment.mol` contains O and H joined by one single bond of length 0.96 Å. Import it into the project, then import and activate a fresh `Pt111-normal-c.json` to start from a clean four-site host.

1. Open Place Adsorbate and select the imported O–H molecule under Project adsorbate.
2. Choose ontop, candidate index 1, and Anchor height (Å) of 2.
3. Set anchor atom to 1 (O) and orientation atom to 2 (H). Enter 1, 1, 1 in the direction fields so the O-to-H direction follows this surface normal. Retain the minimum allowed distance of 0.5 Å.
4. Preview from the side: O should lie above the selected Pt atom, with H farther from the surface. Then click Apply.

5. This input has negative fractional coordinates. To view host and adsorbate within one unit cell, open Build Supercell, enter 1 in all three scaling components, and apply. This wraps equivalent sites into the unit cell without enlarging the model. The lattice and 6 independent sites remain unchanged.

6. Rename the result `Pt111-OH-outward` and save. Display the cell boundary and inspect the O–H direction from the side. Boundary images may show the same fragment several times; check composition against the 6 independent sites in the summary.

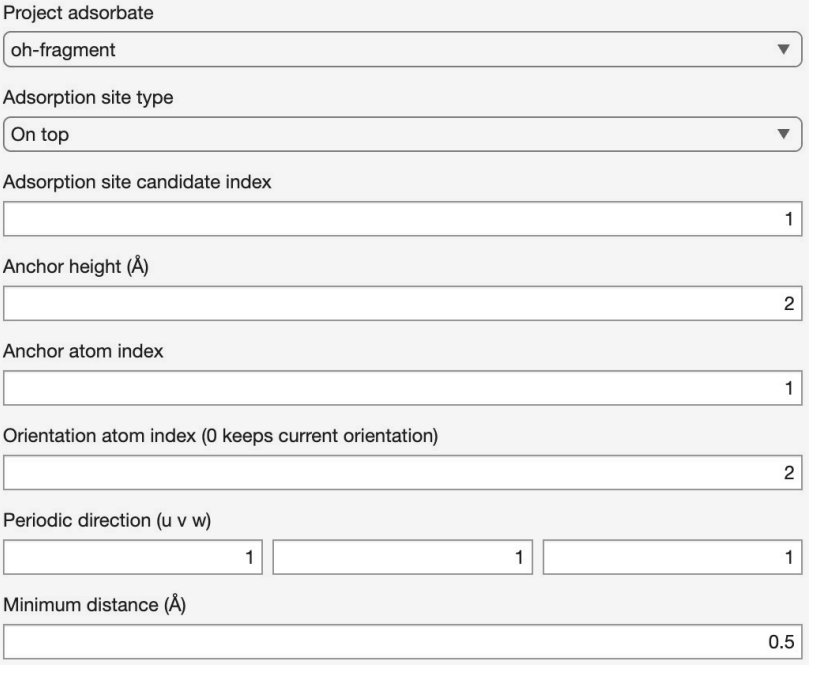



**Figure 8.3** Outward O–H placement: project fragment `oh-fragment`, ontop candidate 1, anchor height $2$ Å, anchor atom 1, orientation atom 2, periodic-direction indices 1, 1, 1, and minimum separation $0.5$ Å. The host $c$ vector is first aligned with the surface normal.

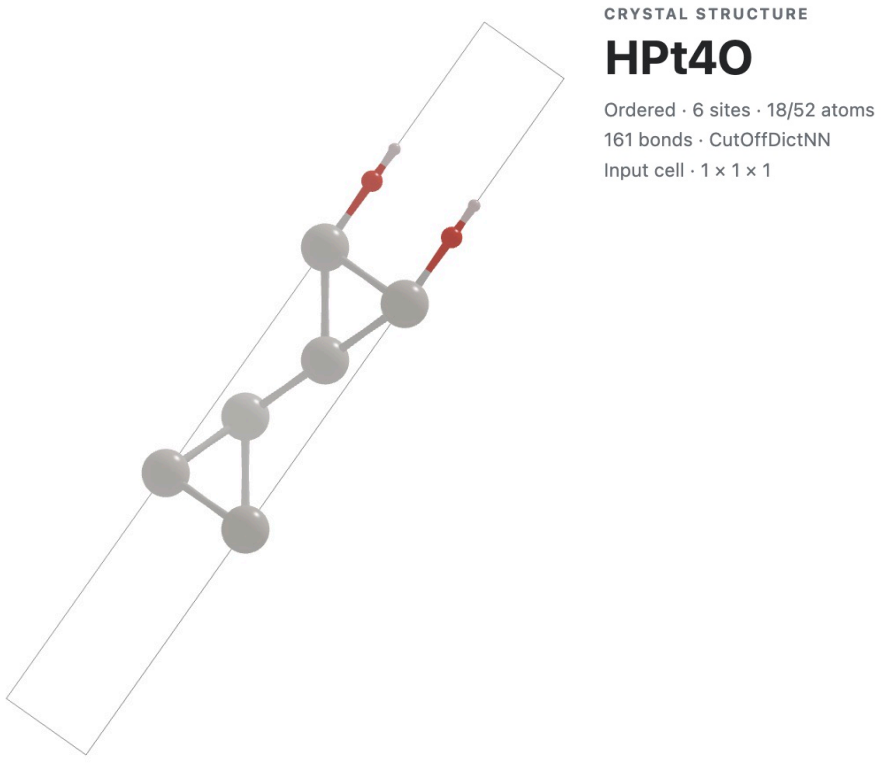



**Figure 8.4** Side view after applying adsorption placement and wrapping with a $1 \times 1 \times 1$ unit supercell. The $Pt_4OH$ model has 6 independent sites. The two displayed O–H groups are periodic images of one fragment. This is the initial placed geometry.

The reference is $Pt_4OH$ with 6 sites. O is $2.00$ Å above the top plane and H is $2.96$ Å above it; the O–H length remains $0.96$ Å. Use `Pt111-OH-outward.cif` to compare cell and positions; the explicit single bond is retained in the input MOL file. Setting orientation atom to 0 preserves the fragment's original orientation, so equal anchor heights need not produce equal orientations.

The supplied `Pt111-OH-Online-050.json` contains the wrapped six-site model with the same periodic geometry, ready for viewing and further editing. Adsorption energy requires subsequent calculations.

### Compare adsorption configurations in a batch

Adsorption Locator enumerates sites, heights, and rotations and retains candidates by score. Import a fresh `Pt111-normal-c.json`, activate the clean four-site host, and select the same O–H molecule. Enter `ontop bridge hollow` under Site types and `1.8 2.1 2.4` under Candidate heights (Å) to compare 1.8, 2.1, and 2.4 Å. Set anchor atom to 1 and orientation atom to 0. Use rotation step 30°, minimum contact distance 0.6 Å, at most 200 retained candidates, and seed 0. Keep the default geometric contact score.

These parameters produce 144 candidates. Each retains the O–H bond length and places its anchor at the specified height along the surface normal. `Pt111-OH-candidate-1.json` is the highest-ranked geometric result: hollow candidate 2, height 2.4 Å, and rotation 60°. Compare it with the earlier ontop model, then inspect the anchors and rotations of other candidates.

These values illustrate the search space. For a larger molecule, begin with fewer height and rotation combinations, confirm the anchor settings, then broaden the search. Select a candidate, Preview and compare it with others, then Apply the required configuration. Retain clearly named separate models for configurations that will be relaxed independently.

The default geometric contact score measures geometric suitability. The optional UFF van der Waals interaction energy estimates interactions in a rigid model. Both screen starting structures; neither replaces an adsorption energy after relaxation. Record the configuration's origin and the subsequent calculation method when reporting results.

## 8.6 Treat surface boundaries

### Passivate an unsaturated surface

Passivate Surface adds atoms to undercoordinated surface sites. Settings include passivating element, bond length, upper/lower surface selection, and detection depth. Defaults are H, 1.1 Å, both surfaces, and 1.5 Å. Choose bond lengths and coordination appropriate to the desired chemistry. A tar-

get coordination of 0 lets the tool infer it; specify it explicitly when the surface chemistry is known.

### Add a solvent layer

Add Solvent Layer packs solvent molecules from the project above the surface. Prepare and check one solvent molecule first, then set molecule count, layer thickness, surface gap, and remaining vacuum. A readily inspected exercise might use 20 molecules, thickness 10 Å, gap 2 Å, and remaining vacuum 5 Å. Contact tolerance and seed affect the packing. Inspect its extent from the side and local packing from above.

## 8.7 Build a heterojunction

### Understand the shared cell

A heterojunction brings two different materials into contact. For a layered interface, prepare each material as a slab and stack them along the normal. Their original lattices generally differ. A periodic model needs a common in-plane cell and a decision about acceptable deformation. That cell defines both interface periodicity and the atom count of a calculation.

Stack Heterostructure builds such vertically stacked interfaces. It compares in-plane supercells and rotations, then assembles the parts at the chosen separation. Graphene/h-BN is a small, easily inspected example. For thin-film interfaces from bulk materials, first use Build Slab for each desired face and check layers, terminations, and composition before matching. Equal material names do not imply equal surface orientations: choose the planes before assembling the interface.

In-plane strain measures how much the original lattices change to share a period. Interlayer spacing is the initial separation, and in-plane translation changes local registry. Each plays a different role. After constructing the geometry, compare separations and stacking arrangements using consistent calculations. Interfacial charge transfer and band alignment are properties of the subsequent electronic structure calculation.

### Insert a structure by translation

Insert Structure places another project structure into the active model using a Cartesian translation and minimum interatomic distance. Use it when the relative positions are already known. After insertion, check the cell, composition, and close contacts across boundaries.

### Find an in-plane lattice match

Stack Heterostructure searches in-plane lattice matches. Choose the top structure and specify layer spacing, total vacuum, maximum in-plane strain, repetition range, and rotation range. The default spacing of 3.35 Å is only an initial value, not a universal spacing for all materials. Maximum strain 0.03 means 3% and is dimensionless. Candidates can be sorted by strain, area, or atom count.

First assess whether the strain is appropriate, then whether cell area and atom count are affordable. The lowest-strain candidate need not be the smallest. After selecting a match, check in-plane orientation, layer separation, and vacuum, and save different stackings separately.

### Exercise: compare insertion with layer matching

The supplied `graphene-layer.json` contains two C sites, an in-plane lattice constant of 2.46 Å, and box height 15 Å. It is periodic in-plane and nonperiodic along the normal. Import it twice and name the structures `bottom` and `top` to practice on a same-material bilayer.

For direct insertion, activate the bottom layer, open Insert Structure, and choose the other project structure as the top layer. Enter 0, 0, 3.35 in the three Cartesian translation (Å) fields and retain minimum distance 0.5 Å. The result should contain 4 sites, with layers 3.35 Å apart and the original box height of 15 Å. `graphene-inserted.json` preserves this result.

Start again from the original bottom layer and open Stack Heterostructure, using `top`. Set spacing to 3.35 Å, total vacuum to 15 Å, maximum strain

to 0.03, maximum in-plane repetition to 2, and maximum rotation search angle to 0°. Sort by area.

Set Interface candidate index to 1, preview and apply, then inspect the candidate table. The reference lists 10 candidates. Candidate 1 has zero strain and an estimated 4 sites. Applying it gives a bilayer with box height 18.35 Å and layer spacing 3.35 Å, saved as `graphene-stacked.json`. Direct insertion preserves the original box, whereas matching rebuilds the normal dimension from the slab and total vacuum.

The list may include combinations above the requested strain limit; being listed does not mean that they can be applied. Candidates 2 and 3 here have approximately 52.49% strain and should be excluded. Candidates 4, 5, and 6 have nearly zero strain but require 8 sites. Check constraints before choosing a suitable area or size rather than relying on the atomic picture alone.

### Example: match graphene and h-BN

Different materials usually have slightly different in-plane dimensions. Here graphene is stacked with hexagonal boron nitride (h-BN) to inspect the matching strain. `hBN-layer.json` is an ideal monolayer with one B and one N site and in-plane lattice constant 2.50 Å, compared with graphene's 2.46 Å. Both inputs have normal box height 15 Å.

1. Import `graphene-layer.json` and `hBN-layer.json` into one project. Activate the newly imported `graphene-layer`, open Stack Heterostructure, and choose `hBN-layer` under Top-layer structure.
2. Set spacing to 3.35 Å, total vacuum to 15 Å, and maximum in-plane strain to 0.03.
3. Set maximum in-plane repetition to 1 and maximum rotation search angle to 0°. Sort by strain. This narrow initial search makes the lattice matching easy to understand.

4. Set Interface candidate index to 1, preview and apply, then inspect the spacing from the side. The result table has one candidate: in-plane strain 0.016, or 1.6%, with an estimated 4 sites.

The candidate table is for inspection and has no button for applying a different configuration within the table. To compare another candidate, undo to the original bottom layer or import it again, reopen the command, and enter the required index. Do not keep adding layers to an assembled bilayer.

The result, `graphene-hBN-stacked.json`, contains $C_2BN$. The h-BN in-plane lattice is matched to graphene; layer spacing is 3.35 Å and normal box height 18.35 Å. Unlike the same-material bilayer, this pair needs finite in-plane deformation to share a cell. Broaden the repetition and rotation ranges later to compare larger matches, saving each configuration intended for calculation. The entered spacing is a modeling starting value; relaxation determines the resulting distance.

## 8.8 Build a twisted bilayer

### From a relative rotation to a common period

Place two graphene monolayers on top of each other and rotate one about the surface normal. Their local registry then varies across the plane: in some regions atoms nearly coincide vertically, while elsewhere an atom in one layer lies near the center of a hexagon in the other. Viewed over a larger area, these local arrangements form a repeating moiré pattern.

A periodic calculation also needs a common cell. Simply rotating the upper layer within its original small cell generally leaves the two layers incompatible across the boundaries. Twist Moiré Superlattice selects an integer supercell for each layer so that both fit a shared in-plane period. The target angle guides the search, while a particular set of integer relations defines the resulting model.

The first example is a graphene bilayer twisted by approximately 21.7868°. Its relatively short common period gives only 28 carbon sites, making the layers and cell easy to inspect. Complete this example before

trying smaller angles to understand how angle, period, and model size are related.

### Prepare the two layers

The tool uses hexagonal in-plane primitive cells: the two in-plane vectors have equal lengths and enclose $60°$ or $120°$. Each input graphene monolayer here contains two C sites, has an in-plane lattice constant of $2.46$ Å, and a box height of $15$ Å. It is periodic in both in-plane directions and nonperiodic along the normal.

The supplied `twist-graphene-bottom.json` and `twist-graphene-top.json` come from the same monolayer input, with the same geometry as `graphene-layer.json` used earlier. Separate filenames identify their roles in the project; they do not mean that a relative rotation has already been applied.

1. Import both JSON files into one project and name the structures `twist-bottom` and `twist-top`.
2. Open the bottom layer's 3D document and check that it has two model sites. More than two atoms may be displayed to show periodic connections.
3. Inspect the lattice: both in-plane lengths should be $2.46$ Å and their angle approximately $120°$. Check that the top layer has the same lattice and composition.
4. Activate `twist-bottom`. On Crystal Modeling, open Interface Modeling in Surfaces & Interfaces, then choose Twist Moiré Superlattice.
5. Select `twist-top` under Top-layer structure. Choose an existing project structure rather than entering a disk path.

Begin with these two original monolayers. Keep any bilayer created by insertion or stacking separately; it is no longer a single input layer for this exercise.

Enter a small-model parameter set

**Table 8.3** Parameters for the 28-site twisted graphene bilayer.

| Parameter | Input and meaning |
|---|---|
| Top-layer structure | Select `twist-top`. |
| Angle (degrees) | Enter `21.786789` as the target relative angle. |
| Interlayer gap (Å) | Enter 3.35 for the initial separation between layers. |
| Total vacuum thickness (Å) | Enter 15 for the total empty height outside the slab. |
| Maximum in-plane strain | Enter 0.03, or 3%. |
| Maximum atom count | Enter 200 as the upper limit on total model sites. |
| Maximum coincidence index | Enter 6 to bound the integer matching search. |

Angle (degrees) is a numeric field; enter the value without a degree symbol. Interlayer gap (Å) and Total vacuum thickness (Å) are also numerical lengths. Maximum strain is a fraction: 0.03 means 3%; entering 3 would specify a different requirement.

The atom limit does not request exactly 200 atoms. Likewise, Maximum coincidence index limits the search over integer coincidence relations; it does not simply repeat a direction six times. Within the search bounds, the tool considers angle error, strain, and model size and selects an integer match. These limits therefore differ from the three repeat factors in Build Supercell.

This command selects one match internally. Unlike Stack Heterostructure, it has no candidate ranking or candidate-index field. Check the actual size and relative orientation after construction. To compare other matches, retain the original layers and generate and save separate results for each parameter set.

1. Enter the table's values and click Preview. The two-site monolayer should become a larger periodic structure containing two layers.

2. Look nearly along the surface normal and check that the hexagons in the two layers are rotated relative to each other. Inspect the side view to confirm that there are still only two layers, with no unintended atoms between them.

3. Check for 28 model sites. The expanded in-plane cell may display more atoms because of periodic images.

4. Apply, name the result `graphene-twist-21p786789`, and save the project. Retain both monolayers so later comparisons can start from the same inputs.

### Interpret the generated cell

The supplied `graphene-twist-21p786789.json` is the reference structure generated with these parameters; a CIF with the same base name is provided for geometry exchange. The model contains 28 C sites, 14 in each layer. Each layer's in-plane area is seven times that of the original two-site monolayer.

Both in-plane vectors are approximately $6.508548$ Å long, giving an area of approximately $36.6859\,\text{Å}^2$. The new cell angle is $60°$, whereas the input primitive cell uses $120°$. These are different lattice-basis representations; the angle change alone does not establish that the crystal has been distorted. Check lengths, area, site count, and periodic connectivity together.

For these graphene inputs, the period length can also be checked as $\sqrt{7} \times 2.46$ Å, or approximately $0.651$ nm. This is the common cell's in-plane repeat length, not the interlayer separation or a C–C bond length. Each layer retains a shortest nearest-neighbor distance of approximately $1.420282$ Å, matching the input monolayer.

The side view shows a layer separation of $3.35$ Å and a normal cell height of $18.35$ Å. Each layer is planar here, so the cell height is the layer separation plus the total vacuum. Total vacuum of $15$ Å means all empty height outside the slab, not $15$ Å on each side.

The reference model's achieved angle is $21.786789298°$, approximately $2.98 \times 10^{-7}$ degrees from the target, with zero in-plane strain. The relative

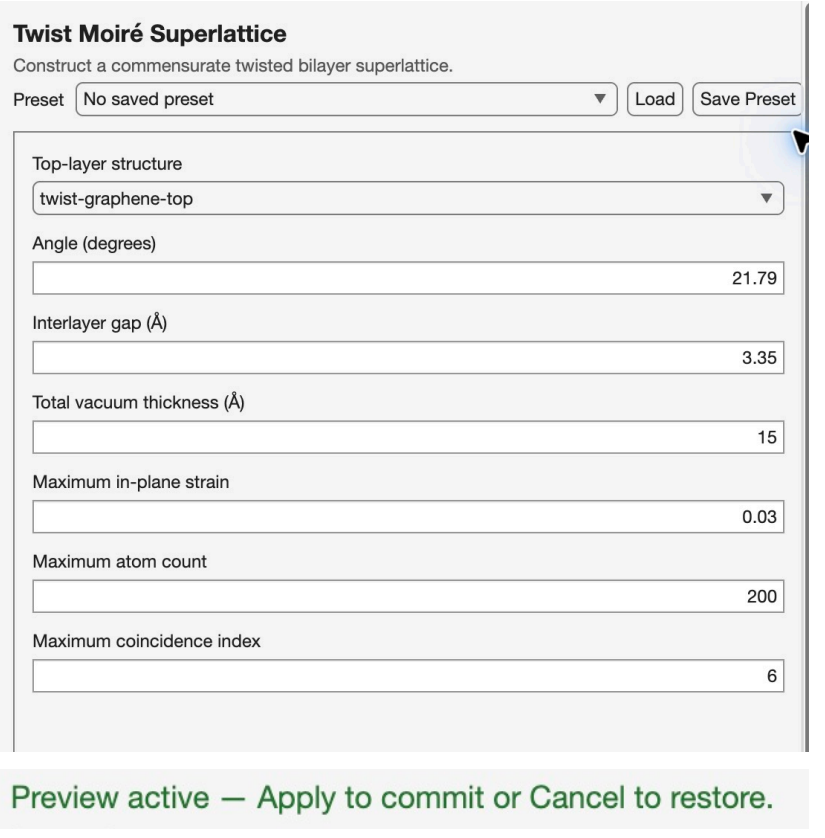



**Figure 8.5** Twist parameters and preview. The entered target is 21.786789 degrees; after the field loses focus, it displays 21.79 at the available precision. The top layer retains its imported name, `twist-graphene-top`, corresponding to `twist-top` in the instructions. The preview contains 28 sites.

angle can also be obtained from nearest-neighbor bond directions in the two layers.

Exact matching information is stored in the `moire` section under `properties` in the native JSON. Open the JSON in a text editor and search for `achieved_angle_degrees`. The fields `target_angle_degrees`, `angle_error_degrees`, and `in_plane_strain` record the target, angle error, and in-plane strain. For your own structures, choose Export file → Structure file, then export JSON. Do not

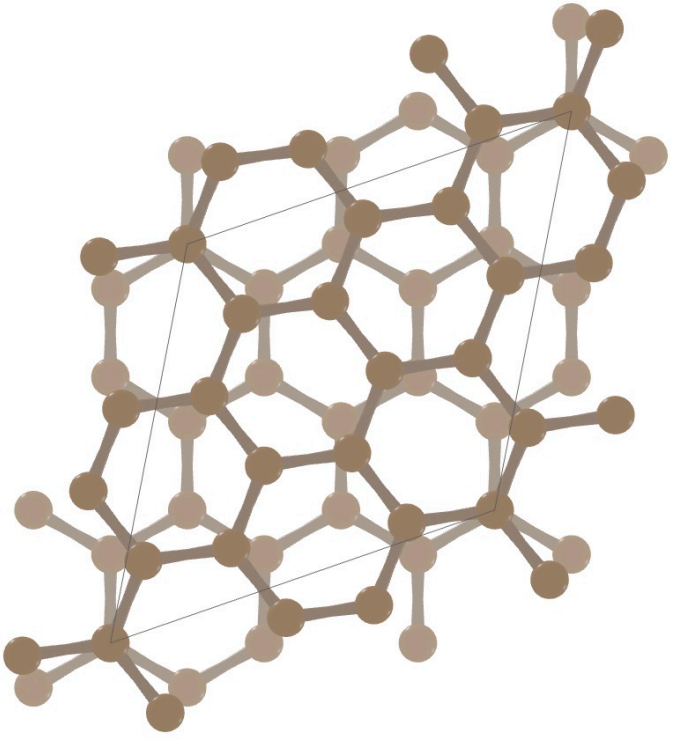


**Figure 8.6** View along the normal of the twisted graphene bilayer. The hexagons are rotated relative to one another. The 28 model sites appear as 68 displayed atoms at the current periodic display range.

choose Scene JSON: it records the viewing scene rather than the structural matching information.

### Why the achieved angle may differ from the target

Target angles range from $0°$ to $60°$. Using hexagonal in-plane symmetry, the tool represents achieved matching angles between $0°$ and $30°$. Apply that convention when comparing larger targets. The example stays within the smaller range so that target and achieved values can be compared directly.

A common period requires particular integer relations. An arbitrary decimal angle need not have a reasonably sized periodic cell that matches it ex-

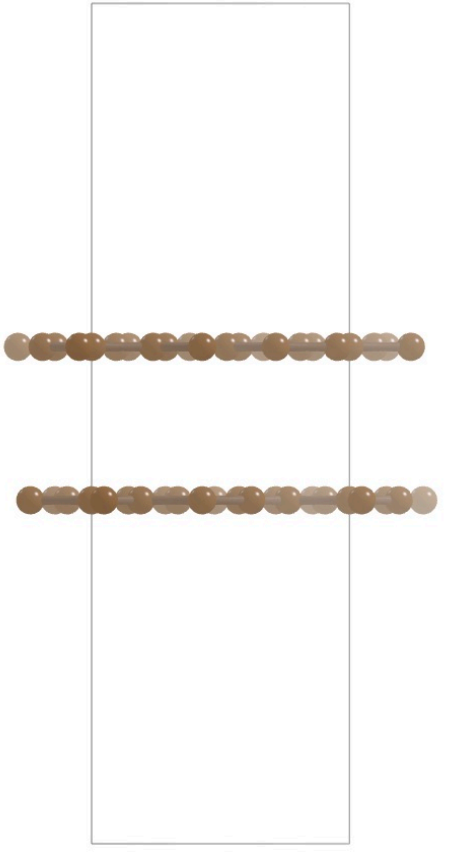


**Figure 8.7** Side view of the same model. The two atomic planes and the cell boundaries distinguish the interlayer gap from the external vacuum.

actly. Changing the target to $22^\circ$ while retaining the other settings still gives 28 sites and an achieved angle of approximately $21.786789298^\circ$, now with an error of approximately $0.213211^\circ$.

The two constructions have the same final geometry but different target inputs. The supplied `graphene-twist-target22.json` preserves the second result for comparison. Report the achieved angle rather than using only the target entered in the parameter dialog.

**Table 8.4** Effects of target angle and size limits for these inputs.

| Input change | Actual result |
|---|---|
| Target $21.786789°$, limit 200 | 28 sites; achieved angle approximately $21.786789298°$. |
| Target $22°$, limit 200 | The same 28-site geometry and achieved angle; error approximately $0.213211°$. |
| Target $21.786789°$, limit 24 | A four-site match close to $0°$, missing the original target. |

The third case shows why successful construction and an acceptable angle match are separate checks. Once the 28-site match is excluded by the limit of 24, the tool selects from the remaining search space. If the resulting size differs markedly from your expectation, check the parameters and achieved angle together before increasing the search bounds.

Smaller twist angles generally require longer common periods and more atoms. To compare angles, first hold layer spacing and strain requirements fixed, then gradually increase the allowed size and maximum coincidence index. This reveals whether the broader search improves the angle match or merely produces a larger structure. The approximately $21.8°$ example uses small integers and is a useful starting point for learning the procedure and checking periodicity.

### Choose an assembly tool for your purpose

The Interface Modeling menu groups several assembly methods. Decide which geometric relationships are already known and which ones the tool should find.

In the Stack Heterostructure candidate table, area, strain, atom count, and rotation each answer a different question. Area and atom count indicate calculation size; strain measures the change needed to match the original lattices; rotation defines their relative orientation. A low-strain candidate can be large, while the smallest-area candidate may require substantial deformation. Compare these quantities before inspecting the atomic arrangement.

**Table 8.5** Choosing insertion, lattice matching, or twist construction.

| Tool | Suitable starting situation |
|---|---|
| Insert Structure | The two parts already have defined positions; specify a translation and retain the current host cell. |
| Stack Heterostructure | Compare common in-plane cells using allowed strain, repetition, and rotation ranges. |
| Twist Moiré Superlattice | Start from hexagonal monolayers and a target angle, then find a common period from integer lattice relations. |

The twist command instead uses hexagonal integer relations to select a common period, then assembles the layers. It does not ask for a candidate index in the same dialog. To compare targets of $21.786789°$ and $22°$, build each result from the two original monolayers and record them separately. Do not apply the command again to an already assembled bilayer.

Rotating the camera only changes the viewpoint; rotating selected atoms changes their positions. Neither replaces periodic matching of the shared cell. Alongside layer orientation, display the cell boundaries and inspect repetitions along both in-plane directions to ensure that the local pattern and full periodic cell describe the same model.

These tools assemble layered interfaces along the normal. An in-plane lateral heterojunction requires separate preparation of the junction's atomic arrangement and boundaries.

### Build a twisted graphene/h-BN heterostructure

Twisting also applies to layers of different materials. The earlier zero-angle graphene/h-BN match illustrates how different in-plane lattice constants are accommodated. A relative rotation can be introduced as well. Here graphene is the bottom layer with lattice constant $2.46$ Å, and h-BN is the top layer with $2.50$ Å.

1. Import `twist-graphene-bottom.json` and `hBN-layer.json` afresh, naming them `graphene-bottom` and `hBN-top`. Do not use the previously twisted bilayer as the bottom layer.
2. Activate the new graphene monolayer. In Twist Moiré Superlattice, choose `hBN-top` under Top-layer structure.
3. Retain the table's target angle, layer separation, total vacuum, 200-site limit, and maximum coincidence index of 6. Keep maximum in-plane strain at 0.03.
4. Preview and check for $C_{14}B_7N_7$ with 28 total sites. Each layer has 14 sites, but their elements differ.
5. Apply and save as `graphene-hBN-twist-21p786789`, then compare with the zero-angle `graphene-hBN-stacked.json`.

The achieved angle is again approximately $21.786789298°$. The principal in-plane strain is 0.016, or 1.6%. The h-BN dimensions are matched to the common graphene lattice, while separation remains $3.35$ Å and normal cell height remains $18.35$ Å. Strain changes dimensions; twist changes relative orientation. They describe different modifications.

Both construction commands produce geometric models. To study interfacial charge transfer, band alignment, or junction properties, build the appropriate calculations on the saved structures and distinguish the input layer separation from the distance obtained after calculation.

### Save twisted models

A project retains the input layers, generated structures, and later workflows. JSON and CIF are supplied for structure exchange. JSON preserves in-plane/normal periodicity flags, layer indices, and twist-matching metadata. CIF is used to exchange the cell and atomic positions. These CIF files are imported as periodic in all three directions, and moiré metadata must be retained separately. Prefer JSON for further 2D modeling comparisons

here. When using CIF in another calculation, check normal boundary conditions and vacuum for the target method.

1. Give each result a name identifying the material, achieved angle, and main change, such as "graphene bilayer, achieved angle 21.7868 degrees," rather than a generic `result`.
2. Record the input lattices, target and achieved angles, strain, layer separation, total vacuum, site count, and in-plane period. Use a separate comparison-table column for each quantity.
3. Reopen the saved project and check both layers' composition and the cell. Keep the parameter record with an exported structure so its geometry can be interpreted in other software.
4. To regenerate the examples, start MATLAB® with `-nojvm`, configure the toolbox path, and run `twist_examples.m` from the supplied examples directory. To check saved files only, run `verify_twist_examples.m`.

The supplied structures are starting models for relaxation and electronic structure calculations. When comparing twist effects, use consistent functionals, pseudopotentials, cutoffs, and suitable sampling, and save those settings with the structures.

## 8.9 Prepare an initial NEB path

Interpolate NEB Path uses initial and final project structures. They must have identical atom counts, element order, and lattices so that atoms correspond one to one. By default, it generates 5 intermediate images and uses IDPP to improve interpolation distances. Inspect every image for motion and close contacts. These structures initialize a later path calculation; the reaction barrier requires energy calculations.

### How intermediate images are distributed

Requesting $M$ intermediate images produces $M + 2$ structures including both endpoints, indexed by $j = 0, \ldots, M + 1$. For a nonperiodic model, or endpoints represented by continuous periodic images, linear interpolation gives

$$\alpha_j = \frac{j}{M+1}, \qquad \mathbf{r}_i^{(j)} = (1-\alpha_j)\mathbf{r}_i^{(0)} + \alpha_j \mathbf{r}_i^{(M+1)}. \tag{8.4}$$

Index $i$ identifies the same atom at both endpoints; $\alpha_j$ is dimensionless and positions are in Å. For periodic structures, endpoint displacements must respect the applicable periodic boundaries. Coordinates in different periodic images can otherwise suggest an artificially long migration distance for the same atom.

IDPP further brings pair distances in each image toward interpolated targets:

$$\begin{aligned} d_{ik}^{*,(j)} &= (1-\alpha_j) d_{ik}^{(0)} + \alpha_j d_{ik}^{(M+1)}, \\ E_{\mathrm{IDPP}}^{(j)} &= \sum_{i<k} \frac{\left(d_{ik}^{(j)} - d_{ik}^{*,(j)}\right)^2}{\left(d_{ik}^{(j)}\right)^4}. \end{aligned} \tag{8.5}$$

Here $i, k$ are atom indices and $j$ is an image index. The current pair distance is $d_{ik}^{(j)}$ and its target is $d_{ik}^{*,(j)}$. The denominator strongly penalizes short separations, helping IDPP resolve collisions in a linear path. This expression describes one image's pair objective; path optimization also couples neighboring images.

$E_{\mathrm{IDPP}}$ is an auxiliary geometric objective, not a reaction barrier. A barrier requires subsequent energy calculations. The dialog's IDPP Force tolerance applies to this auxiliary optimization and is separate from electronic-structure force thresholds. For $M = 5$, the path parameters are $0, 1/6, 2/6, \ldots, 1$, corresponding to example structures 00 through 06.

### Exercise: inspect five intermediate images

The supplied `neb-exchange/initial.json` and `final.json` provide a geometric example with clear atom correspondence. Two differently labeled H atoms exchange positions in a nonperiodic cubic box of side 10 Å. The ini-

tial positions are $(2, 5, 5)$ and $(8, 5, 5)$ Å; the final structure swaps them. This tests how interpolation avoids overlap, rather than modeling a hydrogen molecule reaction.

1. Import both endpoint files, naming them `initial` and `final`. Check atom order and labels, then activate the initial structure.
2. On Crystal Modeling, open Interface Modeling in Surfaces & Interfaces, then choose Interpolate NEB Path. Select the project structure `final` under Endpoint structure.
3. Set Intermediate image count to 5, retain Maximum optimization steps of 200 and Force tolerance of 0.1 for IDPP. Preview and apply, then open `NEB image 00` through `NEB image 06` from the Project Browser. The seven structures include both endpoints and five intermediate images.
4. Inspect the labels and distance in sequence. Endpoints should retain their original coordinates, while intermediate images avoid the position where the atoms would overlap. Saving the project retains both input endpoints and all seven results. Export individual structures when needed.

Straight-line interpolation makes the atoms coincide in the middle image. The reference IDPP path reaches its threshold in 82 iterations. The minimum pair separation along the path is approximately $3.5755$ Å, and the middle image has approximately $3.7380$ Å. Files `image-00.json` through `image-06.json` preserve every image. IDPP objective values and numerical forces assess the initial path; they are not electronic structure energies or atomic forces.

Figure 8.8 compares these coordinates with linear interpolation. Lines on the left connect the generated images; the right panel gives their pair distances. Following the image numbers checks both fixed endpoints and improved intermediate contacts.

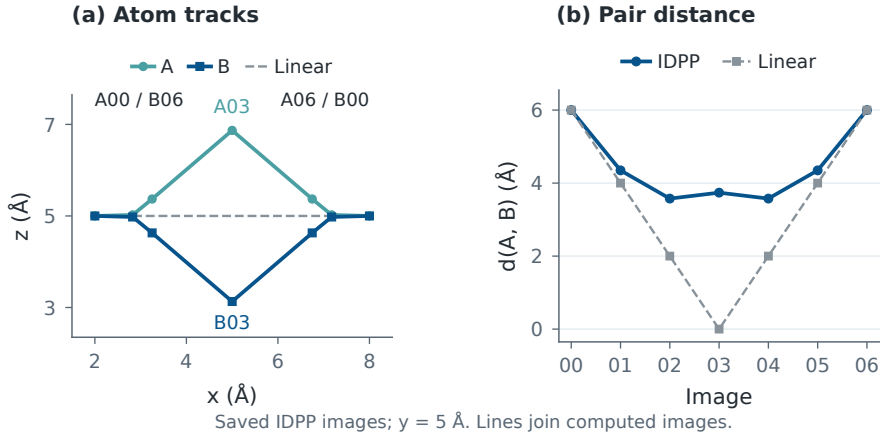



**Figure 8.8** Reference IDPP path for two exchanging atoms. Left: $x$–$z$ trajectories at fixed $y = 5$ Å; labels such as A00/B06 identify endpoint correspondence. Right: pair separations for images 00–06. Gray dashed lines show linear interpolation; 03 is the middle image.

## 8.10 Example: build low-dimensional models

Cutting a sheet into a ribbon or rolling it into a tube changes its boundaries and periodicity. A ribbon has edges across its width and repeats only along its length. A nanotube repeats along its axis, with vacuum in the other directions. A wire cut from a bulk crystal likewise has one periodic direction, whereas a finite cluster is nonperiodic in all three.

This section first builds a 24-site graphene ribbon and a 32-site nanotube from a small primitive cell, then uses face-centered cubic copper to build a wire, finite cluster, and spherical void. Native JSON and CIF files accompany every result for comparison as you work.

### Prepare a 60-degree graphene primitive cell

The supplied `nano-graphene-60.json` has two C sites, equal in-plane vector lengths of $2.46$ Å, an angle of $60°$, and normal box height $15$ Å. It is periodic in-plane and nonperiodic along the normal. It represents the same graphene sheet as `graphene-layer.json`, using a different lattice basis.

Starting from the earlier 120° cell, retain **a** and replace the second vector by **a**+**b**. The new vector still has length 2.46 Å, but forms 60° with **a**. This integer basis change has determinant 1, so the model still has two independent sites. Both ribbon and tube examples use this 60° input.

1. Import `nano-graphene-60.json`, name it `nano-graphene`, and open its 3D document. Check for two independent sites, equal in-plane lengths, and 60°.
2. To perform the basis change yourself, separately import and activate `graphene-layer.json`. Open Build Supercell under Crystal Modeling → Supercell & Lattice.
3. Enter `1 0 0; 1 1 0; 0 0 1` under Supercell scaling. This is a three-row integer matrix, with semicolons separating rows; it differs from editing individual cells in the lattice editor.
4. Preview and check that the site count remains 2, volume is unchanged, and the in-plane angle becomes 60°. Apply and save under a basis-specific name without overwriting the 120° monolayer.
5. Retain separate inputs for the ribbon and nanotube. Start each construction from a monolayer; rolling an already cut ribbon would be a different procedure.

The two cell boundaries look different, but describe the same periodic material. Check the input and basis before comparing site counts and dimensions. Widths or chiral indices alone do not fully specify the construction.

### Cut an armchair graphene nanoribbon

Ribbon width and axial length are entered as repeat-unit counts; vacuum is in Å. Armchair and Zigzag describe edge arrangements, not camera directions. Here choose armchair, inspect the paired carbon arrangement along the edges, then check connectivity along the periodic length.

**Table 8.6** Parameters for the 24-site armchair ribbon.

| Field | Input |
|---|---|
| Width repetitions | 3, a dimensionless integer. |
| Axial repetitions | 2, a dimensionless integer. |
| Edge type | Armchair. |
| Vacuum (Å) | 5; the tool adds this much on each side in both nonperiodic directions. |
| Maximum atom count | 500, an upper bound rather than a requested final count. |

1. Activate the prepared `nano-graphene` monolayer. Open Cut Nanoribbon from Nanostructures on Crystal Modeling.
2. Enter the table's values. Width 3 is not 3 Å, and axial count 2 is not 2 Å.
3. Click Preview and show the cell boundary. Inspect both edges along the sheet normal, then check from the side that the model is still flat.
4. Check for 24 independent C sites. Inspect the periodic connections along the ribbon; the network near the two cell ends is not two disconnected molecules.
5. Apply, name the result `graphene-armchair-w3-l2`, and save. Compare with the supplied JSON of the same name and retain the original monolayer separately.

The result is periodic along global $y$, with native flags `false true false`. Cell lengths are approximately $16.15$, $8.521690$, and $10.00$ Å. The axial period is $8.521690$ Å. The atoms span $6.15$ Å across the width, plus $5$ Å vacuum on each side, giving $16.15$ Å. The flat sheet has no normal thickness, so its normal box length is $10$ Å.

For the 24 unpassivated C atoms, a neighbor cutoff of $1.65$ Å gives 16 atoms with three neighbors and 8 edge atoms with two. The shortest periodic distance is approximately $1.420282$ Å. This cutoff checks this carbon model; it is not a universal bonding distance. For hydrogen-passivated edges, save the unpassivated structure first, add hydrogen according to the intended edge coordination, and check composition.

### Roll graphene into a small nanotube

First identify which in-plane vector becomes the circumference. The geometry described in the help is

$$\mathbf{C}_h = n\mathbf{a}_1 + m\mathbf{a}_2, \qquad R = \frac{\|\mathbf{C}_h\|}{2\pi}. \tag{8.6}$$

Here $\mathbf{a}_1, \mathbf{a}_2$ are the input in-plane vectors in Å; $n, m$ are dimensionless chiral indices; $\mathbf{C}_h$ is the chiral vector; and $R$ is the initial radius. The circumference is the magnitude of that vector combination. Vectors of equal length enclosing $60°$ give $\|\mathbf{C}_h\| = a\sqrt{n^2 + nm + m^2}$. For another basis, use Eq. (8.6) directly. This section retains the prepared $60°$ basis: the same $(n, m)$ in different bases need not select the same rolling direction.

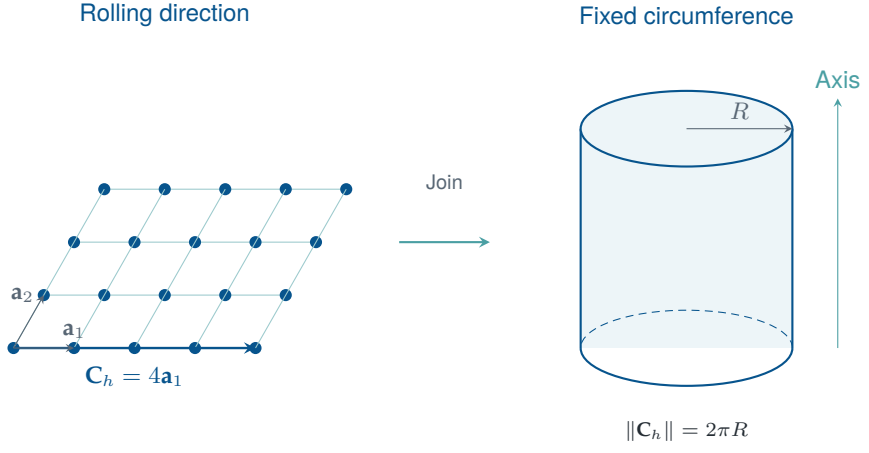



**Figure 8.9** Chiral vector and tube radius (schematic). The left panel shows lattice translation points and the $(4, 0)$ direction that rolls into one circumference. The right panel relates the resulting circumference to its radius.

The chiral indices $(n, m)$ specify the circumferential lattice direction; axial repetitions set the number of translational periods along the tube. The small-radius $(4, 0)$ example makes rolling, the seam, and periodicity easy to inspect. A relaxed equilibrium radius requires geometry optimization.

1. Open a fresh monolayer from `nano-graphene-60.json` and activate it. Do not use the ribbon just created.

2. Choose Roll Nanotube under Nanostructures. Enter 4 and 0 in the two Chiral indices (n m) fields.

3. Set Axial repetitions to 2, Vacuum (Å) to 6, and Maximum atom count to 500.

4. Preview the circular cross-section along the axis, then inspect the tube from the side. Rotate the camera to examine the seam and check periodic connections at the axial ends.

5. Check for 32 C sites. Apply and save as `graphene-tube-4-0-12`, retaining both monolayer and nanotube.

The example is periodic along global $z$, with flags `false false true`. Its rolling radius is

$$R = \frac{4 \times 2.46}{2\pi}\ \text{Å} \approx 1.566085\ \text{Å}.$$

Both transverse cell lengths are approximately $15.132169$ Å, the diameter plus $6$ Å vacuum on each side. Axial length is approximately $8.521690$ Å. This transverse vacuum differs from total vacuum in the twisted-bilayer dialog; use each command's definition when checking box dimensions.

A periodic neighbor check on the saved coordinates finds three neighbors within $1.65$ Å for every site, with a shortest distance of approximately $1.393202$ Å. Rolling a flat sheet onto a small circumference changes straight-line pair distances. Do not require every rolled C–C distance to remain exactly $1.420282$ Å. Check radius, seam, neighbors, and periodicity together, then relax the geometry separately.

### Inspect the tube end and side

Import `graphene-tube-4-0-12.json` directly to practice viewing before comparing other chiral indices. Initially retain Input cell and 1 × 1 × 1 so that the view represents one saved model period.

1. Select the JSON through Home → Structure → Import structure from file.... In Online, upload and then import it. Open the document and check for C and 32 sites.

2. Click c in the viewer toolbar, whose tooltip is View along c axis. The tube axis is parallel to $c$, so this looks through its mouth. Inspect the carbon ring and central hollow region.

3. Click a or b for a side view. Keep the same zoom and inspect the axial extent and curved hexagonal network. If spheres obscure bonds, reduce Atom scale under Viewer settings → Style.

4. Show Unit cell under Visibility and compare the tube with its transverse vacuum box. Hide Boundary image atoms to inspect one period's atoms, then show them again to check end connections.

5. Under Scientific scene → Repeat cell, change only c to 2 and inspect the axial join. Use Reset to 1 × 1 × 1 afterward, then Reset camera for an overall view.

6. Export end and side views, recording that they show the same 32-site structure, with $(4, 0)$ chirality, two axial repeats, and the original JSON filename.

The hole in the axial view confirms a tubular arrangement. The side and repeated views help establish its axial periodicity. Display repetition does not create a larger calculation supercell, and extra boundary spheres are scene images; obtain the independent site count from the structure. Together, these views communicate diameter, length, and periodicity more clearly than one perspective view.

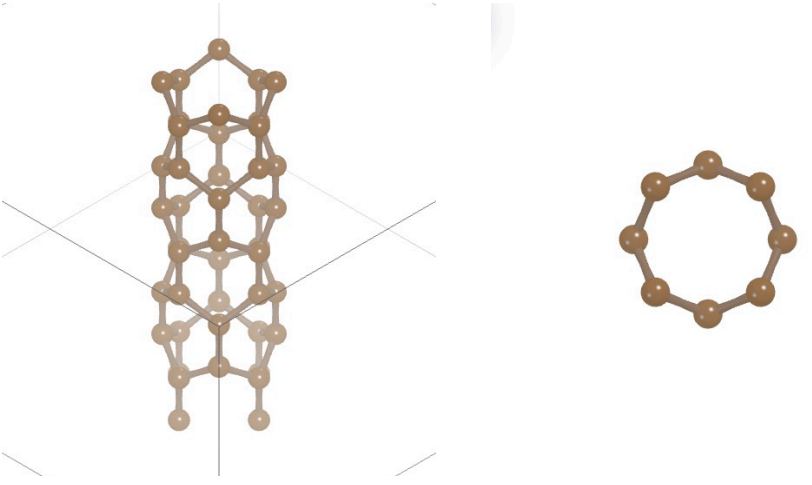


**Figure 8.10** Two views of the same 32-site carbon nanotube. Left: inspect the curved network and axial connections from the side. Right: look down the tube into its hollow center. Periodic boundary images are displayed; the independent site count remains 32.

### Cut a nanowire from copper

Use `nano-Cu-fcc-input.json`, derived from the same four-site face-centered cubic cell as `Cu-fcc.cif`, with lattice constant $3.61\,\text{Å}$. A nanowire retains translation along one lattice direction and selects a cross-section by radius. Atoms occupy discrete positions rather than forming a continuous medium: the outermost span of a spherical or cylindrical cut need not equal twice the entered radius.

Direction indices $[u\,v\,w]$ define the wire axis through the real-space lattice:

$$\mathbf{t} = u\mathbf{a} + v\mathbf{b} + w\mathbf{c}. \tag{8.7}$$

The dimensionless integers $u, v, w$ give $\mathbf{t}$ in the same length units as the lattice vectors. Construction uses an axis parallel to $\mathbf{t}$ and retains atoms within the specified transverse radius. Read the actual axial period from the output cell.

For cubic cells, a direction and the normal to a plane with the same indices are parallel; this is not generally true for oblique cells. Check whether

**Table 8.7** Direction indices and plane indices describe different geometry.

| Input | Geometry and use |
|---|---|
| Direction $[u\,v\,w]$ | A combination of real-space **a**, **b**, **c**, used for directions such as a wire axis. |
| Plane $(hkl)$ | A normal from reciprocal vectors, Eq. (8.1), used for surface cuts. |

the dialog asks for a direction or a plane, then inspect the preview axially and from the side.

1. Import and activate the copper input, then open Cut Nanowire under Nanostructures.
2. Enter 0, 0, 1 under Periodic direction (u v w). Set radius to 3 Å, axial repetitions to 2, vacuum to 4 Å, and maximum atom count to 2000.
3. Inspect the preview cross-section along the axis and check that atoms lie within the selected cylinder. Check axial connections from the side and confirm 24 Cu sites.
4. Apply and save as `Cu-wire-001-r3-12`. Restart from bulk copper when changing radius or direction, rather than cutting an already finite cross-section as if it were bulk.

The wire is periodic along $z$, with flags `false false true` and axial cell length 7.22 Å. Each transverse atomic span is 5.415 Å. Adding 4 Å on each side gives transverse box lengths of 13.415 Å. The shortest periodic neighbor distance is approximately 2.552655 Å, matching bulk face-centered cubic Cu.

The atom limit also bounds the parent supercell needed for construction. This example uses 2000. If a smaller limit fails, inspect the required parent size in the message before adjusting it.

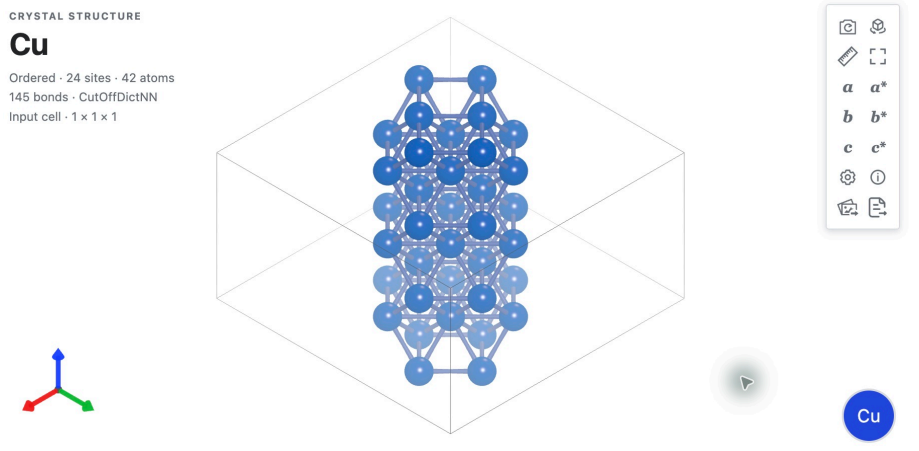



**Figure 8.11** The supplied 24-site copper nanowire in the viewer. Periodic boundary atoms show axial connectivity; transverse empty space is vacuum.

### Inspect the wire's cross-section and axis

Open `Cu-wire-001-r3-l2.json` to inspect the completed 24-site wire. Like the nanotube, it is periodic along $c$. The nanotube is made by rolling a sheet into a hollow cylinder; the copper wire retains crystal atoms inside a cylinder, including atoms within the cross-section.

1. Import the JSON, open the document, and retain Input cell. Check for Cu and 24 independent sites.
2. Click c to view down the axis. Discrete lattice positions may give a stepped outline rather than a smooth circle.
3. Click a or b for a side view and show Unit cell. The 7.22 Å axial length is the repeat period; the 13.415 Å transverse lengths accommodate the finite cross-section and vacuum.
4. Clear unrelated selections and rotate slightly to show both end and side. Retain the cell boundary and orientation markers, and check that the view does not crop any atoms.

5. If useful, set Repeat cell's c to 2 to inspect connections, then restore one period. Save the project and structure. Record [001], input radius 3 Å, and 24 sites in exported figure captions.

The input radius selects lattice positions; the retained discrete positions determine the actual outer span. The transverse span of 5.415 Å is smaller than diameter 6 Å, consistent with the cut. Continue with suitable workflows for relaxation and electronic properties.

### Cut a finite copper cluster

The Quantum Dot mode of Quantum Dot / Nano Void retains atoms inside a sphere and makes the result nonperiodic in all three directions. This 13-site Cu cluster illustrates finite boundaries versus periodic bulk, and can be saved for subsequent electronic structure calculations.

1. Reactivate the four-site copper input and open Quantum Dot / Nano Void under Nanostructures.
2. Choose Quantum Dot, radius 3 Å, vacuum 4 Å, and maximum atom count 2000.
3. Preview and rotate the model. Check that no direction continues as an infinite atomic chain and that there are 13 Cu sites.
4. Apply, name it `Cu-dot-r3`, and save the project and structure.

All three cell lengths are approximately 11.61 Å, while atoms span 3.61 Å in each direction. All periodicity flags are false. The shortest distance is approximately 2.552655 Å. Using a 2.8 Å neighbor cutoff gives the central atom 12 neighbors and each of the 12 surface atoms 5. This initial cluster is cut directly from the face-centered cubic arrangement; its stable geometry requires optimization.

### Cut a spherical void in periodic copper

Void mode retains the host outside the sphere and preserves the periodic box, the opposite of quantum-dot mode. Build a sufficiently large host first so that a small cell is not mostly or entirely removed.

1. Import `Cu-void-host-2x2x2.json`, or use Build Supercell on four-site Cu to make a $2 \times 2 \times 2$ host. Check for 32 Cu sites and cell lengths $7.22$ Å.
2. Activate this host, open Quantum Dot / Nano Void, choose Nano Void, and set radius to $2.7$ Å. No external vacuum is needed around this periodic host.
3. Preview with cell boundaries shown. The void center is fixed at host fractional position $(0.5, 0.5, 0.5)$. Check that central atoms are removed while the outer periodic host remains.
4. Check that 13 atoms are removed, leaving 19 Cu sites. The cubic lattice remains $7.22$ Å and periodic in all directions.
5. Apply and save as `Cu-void-r2p7`, retaining the complete 32-site host for later radius changes or defect comparisons.

The shortest periodic distance from the void center to a remaining atom is $3.61$ Å, larger than the input radius $2.7$ Å. This reflects discrete available lattice positions, not a failed radius setting. The host's shortest interatomic distance remains approximately $2.552655$ Å. Recording both the deletion-sphere radius and the closest remaining center-to-atom distance is clearer than a perspective view alone.

### Check periodicity and save the files

Each native JSON retains its lattice, coordinates, periodicity flags, and construction metadata. CIF is used to exchange cell and atomic geometry, but these CIFs are imported as periodic in all directions. Use JSON to retain the low-dimensional periodicity in further modeling.

**Table 8.8** Site counts and native periodicity of the supplied nanostructures.

| Result | Model and key checks |
|---|---|
| Graphene ribbon | 24C; periodic only in $y$; axial length $8.521690$ Å. |
| $(4, 0)$ nanotube | 32C; periodic only in $z$; radius $1.566085$ Å. |
| Copper wire | 24Cu; periodic only in $z$; axial length $7.22$ Å. |
| Copper cluster | 13Cu; nonperiodic in all directions; cell lengths $11.61$ Å. |
| Copper with a void | 19Cu; periodic in all directions; 13 removed from a 32-site host. |

1. Save the project, giving inputs and derived results distinct names.
2. For separate model files, choose JSON through Export file → Structure file. Save CIF separately for geometry exchange. Do not choose the scene-oriented Scene JSON.
3. Reopen saved structures and check site count, cell, and periodic directions, then neighbors at edges, seams, or spherical cut boundaries.
4. To regenerate these models, start MATLAB® with `-nojvm`, configure the toolbox path, and run `nano_examples.m` from the examples directory. `verify_nano_examples.m` checks existing structures.

These unpassivated starting models can be passivated, relaxed, and used in electronic structure calculations according to the material and boundaries. Retain both construction and calculation parameters, and confirm that the workflow references the intended structure.

## 8.11 Example: build polymer chains

### Understand repeat units

Polymer construction begins with repeat units. Built-in units include PE, PP, PS, PEO, and PPO. A user unit additionally needs head and tail atoms and atoms to remove when connecting. These definitions determine how adjacent units join, so check monomer topology before saving a repeat unit.

A homopolymer uses one unit type, a block copolymer groups types into consecutive blocks, and a random copolymer generates a sequence from

composition and random rules. Sequence and spatial conformation are separate parameters. The same PE/PP sequence can start extended, zigzag, or as a random coil. A seed makes the same random construction conditions reproducible.

### A PEO–PPO–PEO triblock

1. Activate a molecular document. On Molecule Modeling, open Polymer Builder in Soft Matter, then choose Build Block Copolymer.
2. Enter `PEO PPO PEO` under Block repeat units and `3 4 3` under Block lengths, separating entries with spaces. Set complete block-superunit repetitions to 1 and chain count to 1.
3. Set seed to 8, choose Extended under Initial conformation, and leave head/tail end groups at `none`.
4. Inspect the estimated atom count, preview, and apply. Check for 10 repeat units: 6 PEO and 4 PPO.
5. Inspect the overall sequence, then zoom in on both block boundaries. Save under a sequence-specific name such as `PEO3-PPO4-PEO3`.

The supplied `PEO3-PPO4-PEO3.json` has composition $C_{24}H_{50}O_{10}$, 84 atoms, and 83 bonds. Ten units form one chain with nine inter-unit connections. Although end groups are `none`, hydrogens still complete unsatisfied terminal valences; include them when checking the formula.

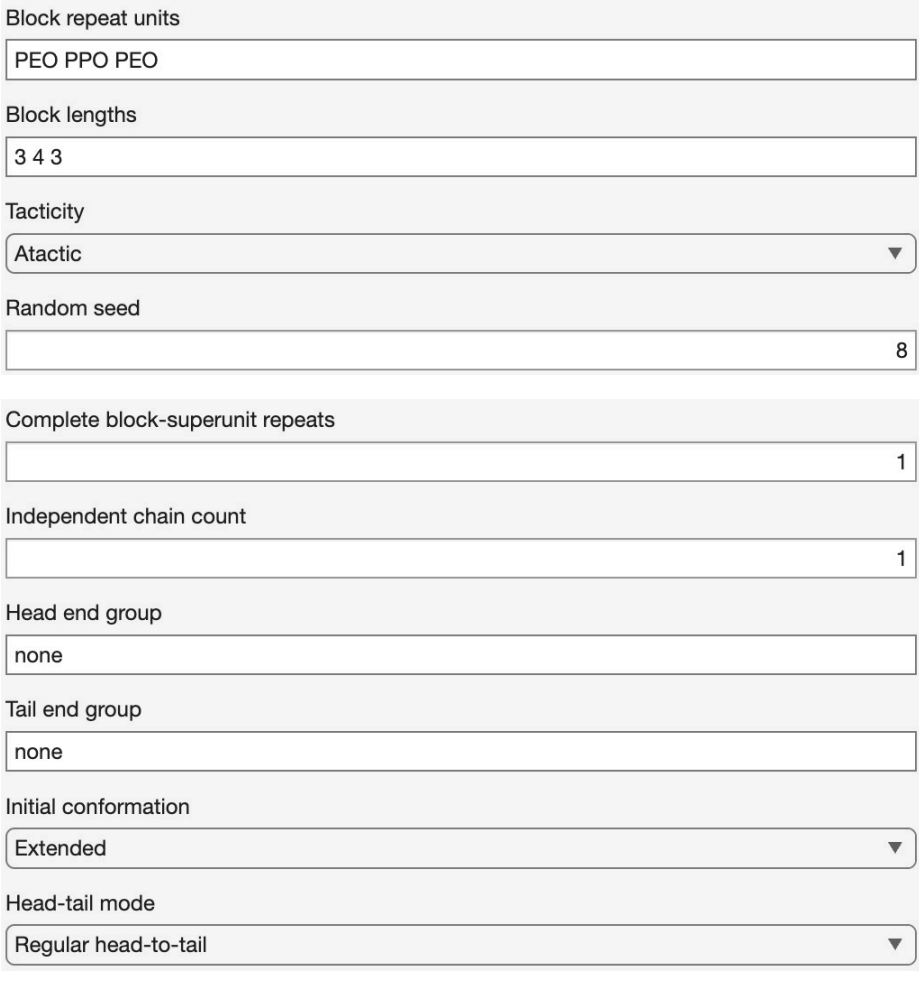



**Figure 8.12** Parameters for a PEO–PPO–PEO block chain. Above: `PEO PPO PEO`, `3 4 3`, and seed 8. Below: one repeat, one chain, both end groups `none`, an extended starting conformation, and regular head-to-tail connections.

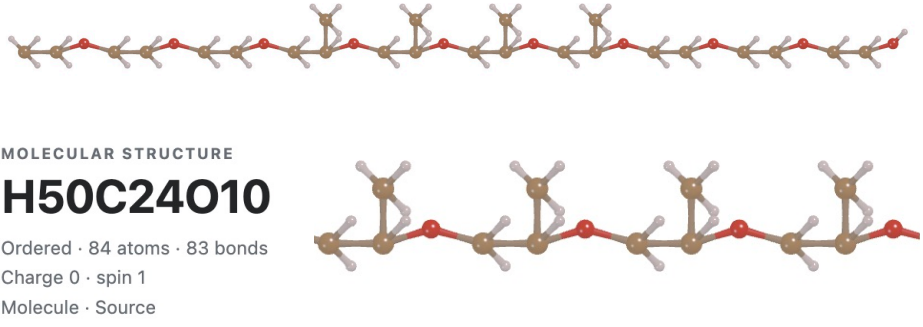



**Figure 8.13** The resulting initial block chain. Above: the complete chain. Below left: $C_{24}H_{50}O_{10}$, 84 atoms, and 83 connections. Below right: the four methyl branches in the central PPO block.

### A random chain with exact composition

Create another molecular document and choose Build Random Copolymer. Use PE and PP, mole fractions 0.3 and 0.7, 20 total repeats, exact composition enabled, and seed 91. The target is 6 PE and 14 PP units, with their sequence determined by the generation rules and seed.

Reactivity ratios control the terminal model for the random sequence. Retain `1 1`, choose Extended initial conformation and Syndiotactic tacticity. The result should be $C_{54}H_{110}$ with 164 atoms and 163 bonds, supplied as `PE6-PP14-seed91.json`. Its key difference from the block chain is unit order, not its filename or overall outline.

Tacticity offers Syndiotactic, Isotactic, and Atactic. The tool records the chosen flag. With the same seed, the syndiotactic and isotactic choices for this parameter set generate identical coordinates. This example checks exact composition, sequence, and connectivity. For tacticity studies, also verify the actual spatial arrangement of side groups.

Branched polymers specify a backbone unit, side-chain fragment, and attachment interval. Basic dendrimers specify generation, core functionality, and branching factor. Atom count grows rapidly with generation. Check estimated size before construction, verify connectivity in a small model, then scale up.

## 8.12 Pack molecular mixtures

### Density and molecule counts

Packing places molecules in a region while satisfying geometric constraints such as minimum contact distances. It is not a chemical reaction or molecular-dynamics equilibration. Target density is generally entered in $g/cm^3$ and minimum contact distance in Å. Higher density leaves less space and makes packing more difficult.

Construct Amorphous Cell uses the current molecule as its component, with a molecule count, target density, contact distance, and seed. Begin with fewer molecules, check molecular integrity and box dimensions, then increase the count. Different components need not produce equal box sizes for the same molecule count, because molecular mass also enters the density relation.

Pack Mixture uses the current molecule and a second project molecule. Specify composition by count, mole fraction, or mass fraction. Choose Exact counts under Composition mode and enter `10 10` under Component counts for ten molecules of each component. Fraction modes also require a total molecule count. Mole and mass fractions generally give different counts for components of different molecular masses; check the actual counts in the result report.

### Exercise: pack ten water and ten ethanol molecules

Import `water-packing.json` and `ethanol-optimized.json`. The former retains water's two O–H bonds; the latter is the optimized ethanol from Chapter 7. Activate water, open Pack Mixture, and choose ethanol under Second project molecule.

Choose Exact counts, enter `10 10`, and set density to $0.1\ g/cm^3$, tolerance to $1.2$ Å, and seed to 23. Retain 100 molecules per batch and leave density ramp stages blank. The low initial density makes molecular integrity easy to inspect. The equilibrium density of the liquid mixture is determined during subsequent equilibration.

The result, `water10-ethanol10-packed.json`, has 20 molecules, 120 atoms, and composition $C_{20}H_{80}O_{20}$. Its cubic side length is approximately $21.9954$ Å; total mass divided by volume gives $0.1\ \mathrm{g/cm^3}$. The shortest intermolecular distance, including periodic images, is approximately $4.0990$ Å, while intramolecular distances remain unchanged. Contact requirements apply between molecules, not to water's internal O–H bonds.

Retain both component inputs when saving the result. The packed structure records molecule and component indices but not a complete global explicit bond table. Further topology work therefore still needs the original component connectivity. The accompanying `interface_packing_examples.m` provides component inputs, candidate structures, and contact checks.

### Choose the packing region

Build Confined Layer adds a confinement axis and allowed region, concentrating molecules in part of the box. Check whether bounds are fractional or dimensional and match them to the selected axis. Pack into Existing Box retains the host and adds project molecules to its cell. Pack around Nanoparticle additionally specifies a gap around the host to avoid placing molecules directly against its surface.

For packing failures, adjust the geometry: lower the initial density or molecule count, enlarge the region, reconsider the contact distance, or try another seed. Optional density ramps and batch packing help control larger jobs. Check composition, cross-boundary contacts, and unresolved issues in the report before minimization, thermal treatment, or equilibration suited to your study.

> **Save reproducible models**
>
> For polymer and packed models, retain the repeat units or components, counts and density, seed, conformation, contact requirements, and generated structure together. A seed alone does not identify the

model's origin. Saving these conditions with the project makes rebuilding and comparison much easier.

Chapter 9

# Modeling tools in depth

The preceding chapters introduced common modeling procedures through worked examples. This chapter organizes tools by their purpose, providing a reference for parameters and checks while you edit. Start with the task at hand: selection, cell manipulation, soft-matter construction, or recording a modeling procedure.

Modeling tabs depend on the active document. Molecular transformations and measurements are mainly under Atomic Editor and Geometry Editor on Molecule Modeling. Local molecular components of a crystal use Component Transform and Component Geometry on Crystal Modeling. Use Lattice Editor or Supercell & Lattice to change the whole crystal. If a command is disabled, check document type and selection count first. Move Atoms, for example, requires one independent site; Set Angle requires three atoms in order.

## 9.1 Edit structures precisely

### Save and recall a group of atoms

A selection set names a group of model sites, making it useful for repeatedly working on bottom surface layers, adsorbates, or a chain segment. Recall the set to select those sites again.

1. Select atoms in the 3D view. A click selects one atom; Shift- or Ctrl-click adjusts a multiple selection. B enters box selection and L enters lasso selection. Double-click an atom to select a connected molecule, or choose Select Connected Component from its context menu. Connectivity follows the bonds used by the current display model, so a crystal network may select a large connected region.

2. To select all atoms of one element, open the context menu of an atom of that element and choose Select All … Atoms. Check the selected count before substituting or deleting.

3. Open Selection Sets on the modeling ribbon, click Save Current Selection, and enter a descriptive name such as "fixed bottom layer."

4. Click empty space or press Esc to clear the selection, then recall the saved name from Selection Sets. Check that the same sites are highlighted.

5. Save the project. Export native structure JSON as well to retain the selection metadata with the model. Exchange formats such as CIF and XYZ do not preserve all selection metadata.

Two numbers after a set's name show how many sites can still be found and how many were originally saved. Deleting a member reduces the first number, and recalling the set warns about missing sites. The remaining selection is no longer the original complete group. Saving again under the same name updates that set. The menu provides save and recall actions.

For periodic structures, check sites as well. A wider display range may show several images of a site, but selection sets still refer to model sites and do not gain members just because more spheres are visible.

### Correct elements and positions

Add Atom adds a site directly. Crystals offer fractional or Cartesian coordinates; molecules use Cartesian coordinates in Å. Molecular Sketch Atom additionally specifies a connection target and bond order. Use sketching or bond addition to create connectivity rather than merely placing atoms close together.

1. Open Add Atom under Atomic Editor. Enter the element, check coordinate type, then enter the three components. Preview to avoid overlapping an existing atom, then apply.

2. To change existing elements, select the atoms and open Substitute Atoms. Enter the new element. Substitution retains positions; it does not search for a suitable geometry for the replacement.

3. For an exact new position, open Move Atoms and enter the target coordinates. In crystals, choose coordinate type first so fractional values are not mistaken for Å.

4. To move a group together, use Translate Atoms. Cartesian $(0, 0, 1)$ translates the group $1$ Å along global $z$; fractional $(0, 0, 1)$ in a crystal translates it by one **c** vector.

5. Before deleting, check the independent site count, open Delete Atoms, and verify indices. After applying, inspect composition and remaining connections, then save a separate structure or project.

Local crystal commands may offer Preserve symmetry or Break symmetry. Preserving symmetry processes related sites according to equivalence. Choose the behavior appropriate to a local dopant or defect. Inspect every change in the preview, not just the atom initially clicked.

### Clean duplicate sites and reorder atoms

Merge Atoms handles crystal sites very close together, comparing periodic distances against a tolerance in Å. It does not reduce the number of repeated boundary spheres. The default is $0.01$ Å; start with a small tolerance when checking imported models.

1. Save an original copy and open Merge Atoms under Atomic Editor.

2. Choose the merge mode. Delete duplicates retains one composition record; Sum occupancies combines species occupancies within a cluster; Average properties handles compatible numerical site properties. Each mode relocates a cluster to its center using neighboring periodic images, so coordinates are not necessarily unchanged.

3. Preview and check site count, occupancy, and local positions. If two genuinely distinct neighbors are merged, cancel and reduce the tolerance.

4. Apply and save under a new name, keeping the input for comparison.

Sort Atoms orders sites by element; Reverse order reverses that ordering. This organizes files and indices without changing spatial positions. Recheck previously recorded atom numbers after sorting. Selection sets are better for tracking a group over time.

Perturb Atoms and Perturb Structure apply random displacements using minimum and maximum lengths in Åand a seed. A small exercise could use minimum 0, maximum $0.05$ Å, and seed 7. Both commands affect the whole model, including unselected atoms. Inspect the overall preview and keep an original copy.

### Change molecular orientation

Centering, rotation, and reflection change geometric placement; they do not optimize energy. Adsorbates in a crystal can use Component Transform, provided the complete component is selected first.

### Move a group to a target center

Centering applies the same displacement to every selected atom. For $N$ selected atoms, use Cartesian positions $\mathbf{r}_i$ for molecules and fractional positions $\mathbf{f}_i$ for crystals:

$$\begin{aligned} \Delta\mathbf{r} &= \mathbf{r}_{\text{target}} - \frac{1}{N}\sum_{i=1}^{N}\mathbf{r}_i, \quad \mathbf{r}_i' = \mathbf{r}_i + \Delta\mathbf{r}, \\ \Delta\mathbf{f} &= \mathbf{f}_{\text{target}} - \frac{1}{N}\sum_{i=1}^{N}\mathbf{f}_i, \quad \mathbf{f}_i' = \mathbf{f}_i + \Delta\mathbf{f}. \end{aligned} \tag{9.1}$$

The molecular expression uses Åfor positions and displacements; the crystal expression is entirely dimensionless. Typical targets are $(0, 0, 0)$ for molecules and $(0.5, 0.5, 0.5)$ for crystals. Every atom receives equal weight,

so centering is not mass-weighted and does not change distances within the selection.

For two molecular atoms with $x$ coordinates 1 and 3 Å, moving their mean to the origin gives a common displacement of $-2$ Å. Their new coordinates are $-1$ and 1 Åand their separation remains 2 Å. For crystal fractional components 0.1 and 0.3 with target 0.5, add 0.3 to obtain 0.4 and 0.6. The other components follow the same rule.

Near a periodic boundary, identify the images first. The ordinary mean of 0.99 and 0.01 is 0.5, although the sites may form a compact group across the boundary. To center the group as a continuous structure, first choose periodic images that keep its atoms together. Coordinates may be wrapped afterward, so inspect periodic connectivity rather than only coordinate signs.

### Rotate about a specified anchor

Subtract the anchor from each position, rotate the relative vector, and add the anchor back. The anchor stays fixed and distances from it are preserved. In row-vector notation,

$$\mathbf{r}' = \mathbf{p} + (\mathbf{r} - \mathbf{p})R(\hat{\mathbf{n}}, \theta)^{\mathsf{T}}. \tag{9.2}$$

Here $\mathbf{p}$ is the anchor in Å, $\hat{\mathbf{n}}$ is the unit vector along the rotation axis, and $\theta$ is the angle; $\mathsf{T}$ denotes transpose. Dialog angles are in degrees. Positive rotation follows the right-hand rule: point your right thumb along the positive axis and curl your fingers in the positive rotation direction. The vector specifies orientation only; `0 0 1` and `0 0 2` define the same positive axis.

For rotation about global $z$,

$$R_z(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

With $\mathbf{p} = (1, 1, 0)$ Å and $\mathbf{r} = (2, 1, 0)$ Å, a positive $90°$ rotation about $z$ gives $\mathbf{r}' = (1, 2, 0)$ Å. This checks the rotation direction; the atom remains 1 Å from its anchor. Crystal positions may be wrapped to equivalent images, so inspect boundary images in the preview as well.

### Reflect across a plane

A mirror plane needs a point $\mathbf{p}$ and a nonzero Cartesian normal $\mathbf{n}$. Normalize the normal, then subtract twice the signed projection:

$$\begin{aligned} \hat{\mathbf{n}} &= \frac{\mathbf{n}}{\|\mathbf{n}\|}, \\ \mathbf{r}' &= \mathbf{r} - 2\big[(\mathbf{r} - \mathbf{p}) \cdot \hat{\mathbf{n}}\big]\hat{\mathbf{n}}. \end{aligned} \tag{9.3}$$

The dot product $(\mathbf{r} - \mathbf{p}) \cdot \hat{\mathbf{n}}$ is the signed distance in Å. Reflection reverses its sign without changing its magnitude; atoms on the plane stay fixed. The normal `1 0 0` and a point at $(1, 0, 0)$ Å define the plane $x = 1$ Å. Reflecting $(1.5, 2, 0)$ Å gives $(0.5, 2, 0)$ Å, each 0.5 Å from the plane.

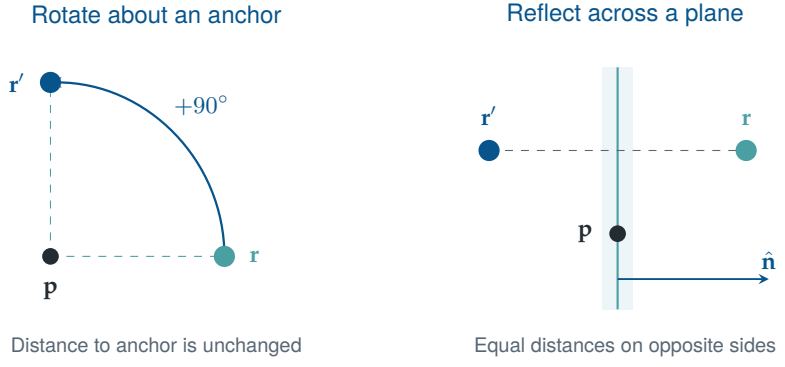



**Figure 9.1** Anchor rotation and plane reflection (section schematic). Left: rotation about $\mathbf{p}$. Right: reflection to the opposite side of a plane, preserving the magnitude of the normal distance.

Centering, rotation, and reflection are geometric operations. For a molecule, first select the part that should move together, then follow the procedure below. Rotate the camera instead if you only want to change the viewing direction.

1. Select the atoms and open Center Atoms. A molecular target center is in Å, commonly $(0, 0, 0)$; a crystal target is fractional, commonly $(0.5, 0.5, 0.5)$. The center is the arithmetic mean of the selected positions, not their center of mass.

2. For a specified rotation, open Rotate Atoms. The angle is in degrees, the axis is a global Cartesian direction, and the anchor is in Å. Axis $(0, 0, 1)$, anchor $(0, 0, 0)$, and $90°$ rotate about the $z$ axis through the origin.

3. For a reflection, open Mirror Atoms and enter a plane normal and a point in the plane. Normal $(1, 0, 0)$ and point $(0, 0, 0)$ define the $yz$ plane. The normal gives a direction, not a length in Å.

4. Preview and compare internal distances, connectivity, and nearby host atoms. A rigid transform of a complete molecule preserves its internal geometry; selecting only part can distort it or break connections.

5. Apply and save separately. To change only the viewing direction, leave the modeling tool and rotate the camera instead.

Align Geometry aligns a long molecule's principal axis or the fitted plane normal of approximately coplanar atoms with a target direction. Choose principal_axis or plane_normal, then enter a nonzero vector. Highly symmetric molecules may have nonunique principal axes; inspect the result from several directions.

### Edit explicit chemical bonds

An explicit bond records the two connected atoms and their bond order. Displayed lines may instead be inferred from distance-based neighbors. Use the molecular connectivity record when changing topology. Orders 1, 2, and 3 represent single, double, and triple bonds; 1.5 represents the aromatic connections used here.

1. Open or import the molecule and save a copy. Select exactly the two atoms forming the bond, in order, and check their indices.

2. If they are not connected, open Add Bond on Molecule Modeling, specify the bond order, and apply.

3. Use Set Bond Order to change an existing bond or Delete Bond to remove it. Deleting a bond retains both atoms; deleting an atom changes the atom count. Choose the operation that matches your purpose.

4. In Atom Chemistry, check the affected elements, formal charges, hybridization, and aromatic flags. Leave unchanged selected when not replacing an element. Formal charge is an atom property, not the molecular net charge divided among all atoms.

5. If existing hydrogens no longer suit the edited connectivity, select the heavy atoms, Remove Hydrogens, then Add Hydrogens. Check composition, heavy-atom connections, hydrogen count, and suspicious short distances. Clean local geometry if needed.

6. Save the project and native structure JSON to preserve bond orders and atom properties. The aromatic examples use JSON. Visible bond lines after exchange through another format do not establish that all aromatic information survived.

Diagnose Molecule performs rule-based checks and reports Molecule diagnostics completed.. Follow Section 7.7 in Chapter 7: inspect composition and connectivity, then measure local geometry. MATLAB® toolbox users can inspect individual diagnostic records in the returned data.

For direct sketching, press S. The panel sets element, bond order, formal charge, and hybridization hints. Drag from an existing atom to add a bonded atom, drag to another existing atom to attempt a bond, or click empty space to place an atom. Select the host attachment site before extending an existing molecule. Move the draft if a collision is reported. Press Esc to leave the tool so that subsequent clicks select atoms.

### Measure local geometry in periodic structures

Use Measurements in the 3D viewer, choose a measurement, and select atoms in the prompted order. The ribbon's Measure Geometry instead reports Geometry measurement completed. without listing readings in its

message. The viewer also provides cell parameters, bond statistics, point-to-plane distance, interplanar angle, coordination numbers, and nearest neighbors. Choose the tool according to the quantity required.

| Task | Selection and reading |
|---|---|
| Distance | Select two atoms in order; lengths are in Å. |
| Bond angle | Select three atoms, with the second at the vertex; angles are in degrees. |
| Dihedral | Select four atoms in order; the middle two define the axis. Note the sign. |
| Point to plane | Select the point, then three noncollinear points defining the plane. |
| Between planes | Select three points in the first plane, then three in the second. |
| Coordination/neighbors | Select the center atom and inspect counts or distances for the current neighbor criteria. |
| Cell parameters | Read the current cell from the menu without selecting atoms. |
| Bond statistics | Select two atoms to identify an element pair; read bond count, mean, minimum, and maximum distances found by the current neighbor algorithm. |

**Table 9.1** Choose a geometry measurement to match the question.

In periodic structures, the ribbon's Measure Geometry calculates local geometry using neighboring periodic images. Interactive viewer measurements instead connect the displayed images actually clicked, so choosing a different boundary image can change the reading. Nearest-neighbor lists include site indices and image offsets; read them together. Coordination counts also depend on neighbor criteria and are not universal fixed chemical constants.

To change a measured quantity, enter a target in its editing control or use Set Distance, Set Angle, or Set Dihedral. In molecules, atom moves only the target atom, while subtree and fragment use connectivity to identify movable parts. A ring that cannot be separated by cutting one bond restricts the movable scope. In periodic crystals, these precise geometry commands move only target sites; they do not automatically treat an adsorbate as a

molecular subtree. To translate an entire adsorbate, select all its sites and use Component Transform.

### Save and attach reusable fragments

Fragment Sketcher provides a searchable library of local structures. Attachment ports define connections to a host. Some use one attachment site, others require several, and noncovalent ports do not provide the covalent attachment action used here.

1. Select host attachment sites and open Fragment Sketcher from Molecule Modeling. Filter with the search box, select an entry, and inspect its 3D preview, atom count, source, and tags.
2. Choose a port under Connection mode, read its instructions, and check the host selection count. If Attach Fragment is disabled, check whether the selected port supports covalent attachment.
3. Click Attach Fragment and inspect the new geometry and host connection. Built-in entries already define their attachment heads; do not infer them from sphere sizes in the preview.
4. Check the attachment, hydrogen count, and local distances, run Diagnose Molecule, and save. To try another connection, undo and choose another port rather than editing repeatedly on an incorrect attachment.

To add your own fragment, make the active molecular document contain only the complete structure to reuse. Check connectivity and atom order, open Save User Fragment, and name it. This saves the current molecule, not only highlighted spheres. Explicitly enable overwrite to replace an existing name, or use a new name.

Import library and Export library in Fragment Sketcher exchange fragment-library JSON. Exporting a library for another computer differs from exporting the active structure. After import, search for the entry and check its source, ports, and preview. Save attached models separately.

## 9.2 Work with periodic crystals

### Change the cell while preserving the intended geometry

Lattice-matrix rows are $\mathbf{a}, \mathbf{b}, \mathbf{c}$; columns are global $x, y, z$ components in Å. Edit Lattice uses an editable table, filled cell by cell rather than a text field accepting all nine values.

Let $A$ and $A'$ be the original and edited lattices. Preserve Cartesian positions determines how atoms adapt to the new cell. Equation (7.1) gives

$$\begin{aligned} &\text{Keep fractional:} \quad \mathbf{f}' = \mathbf{f}, \quad \mathbf{r}' = \mathbf{f}A', \\ &\text{Keep Cartesian:} \quad \mathbf{r}' = \mathbf{r}, \quad \mathbf{f}' = \mathbf{r}(A')^{-1}. \end{aligned} \tag{9.4}$$

For example, increase a cubic cell side from 4 to 8 Å. An atom initially at fractional position $(0.5, 0, 0)$ has Cartesian position $(2, 0, 0)$ Å. Without preserving Cartesian coordinates, it remains halfway along the cell and moves to $(4, 0, 0)$ Å. With the option enabled, it stays at $(2, 0, 0)$ Å and its fractional position becomes $(0.25, 0, 0)$. Both choices can be useful; decide which geometric relationship to retain before editing the table.

1. Save the original and open Edit Lattice under Lattice Editor. Record lengths, angles, and representative Cartesian atomic positions first.
2. Enter the new matrix cell by cell. Preserve fractional coordinates for atoms to deform with the lattice. Choose Preserve Cartesian positions to change the boundary while retaining current atomic positions, as is often appropriate when enlarging a box around an isolated structure.
3. Preview and check the atomic motion, particularly cross-boundary neighbors. Apply, recheck cell volume and positions, and save separately.

Redefine Lattice directly assigns a new matrix while preserving Cartesian positions. It does not replicate atoms according to the volume ratio. Use Build Supercell when the atom count should actually increase.

| Operation | Preserves | Main checks |
|---|---|---|
| Edit lattice, keep fractional | **f** and site count | Cell lengths, angles, and changed interatomic distances. |
| Edit lattice, keep Cartesian | **r** and site count | New fractional positions and periodic boundary contacts. |
| Build supercell | Original periodic arrangement | Volume and independent site count multiplied by $\lvert \det S \rvert$. |
| Rotate lattice | Internal geometry and site count | Lattice and atoms rotating together. |
| Swap axes | Cartesian geometry and site count | Basis order and matching fractional components. |
| Mirror lattice | Fractional coordinates and site count | Reversed vector and cell handedness. |
| Apply strain | Fractional coordinates and site count | Vector lengths and affected distances. |

**Table 9.2** Choose a lattice operation according to the quantities to preserve. Internal geometry means interatomic distances and angles; after changing periodic boundaries, inspect neighboring images in the new cell as well.

### Reverse one lattice vector

Mirror Lattice accepts vector index 1, 2, or 3 and reverses that vector, leaving the other rows unchanged. Let $D_i$ be diagonal, with entry $-1$ at position $i$ and 1 at the other two positions. Then

$$A' = D_i A, \qquad \mathbf{f}' = \mathbf{f}. \tag{9.5}$$

Selecting axis 1 gives $\mathbf{a}' = -\mathbf{a}$. Position $(u, v, w)$ changes from $u\mathbf{a} + v\mathbf{b} + w\mathbf{c}$ to $-u\mathbf{a} + v\mathbf{b} + w\mathbf{c}$. Since $\det D_i = -1$, the volume magnitude stays the same while basis handedness changes.

This differs from Mirror Atoms. In a general skewed cell, reversing **a** does not simultaneously reflect **b** and **c** in one Cartesian plane, so some interatomic distances may change. To reflect an entire molecule across a spatial plane, use the preceding plane-reflection formula. To reverse a lattice basis vector, use this operation and check the resulting atomic positions.

Rotate Lattice takes an angle and global axis and rotates the lattice and its atoms together: $A' = AR^{\mathsf{T}}$, with fractional coordinates unchanged. Swap Axes reorders both lattice rows and fractional components. Entering 2, 1, 3 swaps **a** with **b** and $u$ with $v$, leaving Cartesian positions unchanged. Perform one operation at a time and check Table 9.2 before saving.

### Apply strain along lattice directions

Apply Strain and Strain Structure use dimensionless relative extensions. To emphasize their directions, write the inputs as $\varepsilon_a, \varepsilon_b, \varepsilon_c$. They scale the lattice rows:

$$A' = \begin{bmatrix} 1+\varepsilon_a & 0 & 0 \\ 0 & 1+\varepsilon_b & 0 \\ 0 & 0 & 1+\varepsilon_c \end{bmatrix} A, \qquad \mathbf{f}' = \mathbf{f}. \tag{9.6}$$

The dialog still names the components $x, y, z$, but this tool applies them to lattice vectors 1, 2, and 3. For a skewed cell they are not independent global Cartesian extensions. A scalar 0.01 multiplies every row by 1.01; `0.01 0 0` multiplies only **a** by 1.01. The value 0.01 means 1% extension, not an added $0.01$ Å or a final length of 1.01 Å.

Using the skewed-cell example from Chapter 7, take $\mathbf{a} = (4, 0, 0)$, $\mathbf{b} = (1, 3, 0)$, and $\mathbf{c} = (0, 0, 6)$ in Å. After `0.01 0 0`, only **a** becomes $(4.04, 0, 0)$ Å; the $x$ component of **b** remains 1 Å. Fractional position $(0.5, 0.5, 0)$ stays fixed, while its Cartesian position changes from $(2.5, 1.5, 0)$ to $(2.52, 1.5, 0)$ Å. Thus this operation does not multiply every atom's global $x$ coordinate by 1.01.

Save the undeformed model first, then enter the strain, preview, and apply. Check vector lengths and representative atomic positions using a small strain like this before saving a separate result. These tools scale the three lattice vectors and do not accept independent shear components. If the deformed structure needs relaxation, configure that as a separate calculation.

### Build an orthogonal supercell

Some later operations need angles close to $90°$. Orthogonalize Cell searches compatible integer supercells rather than arbitrarily stretching an oblique

cell into a right angle. Check atom count and volume together because the model may grow.

1. Start from the original small cell and open Orthogonalize Cell under Supercell & Lattice.
2. Enter minimum and maximum lengths in Å, with maximum greater than minimum. Set an atom-count limit and angular tolerance in degrees. The initial length range depends on the current lattice; defaults are 5000 atoms and $0.1°$ from a right angle.
3. Preview using an affordable atom limit. Check each angle's deviation from $90°$, actual lengths, and site count. A rectangle-like camera projection alone is insufficient.
4. If no candidate is found, increase maximum length, raise the atom limit where practical, or relax the angular tolerance, recording each set of conditions. Cancel unsuitable or failed previews to return to the original model.
5. Apply and save with a distinct filename, recording its independent site count. Calculation inputs should reference this new model.

### Analyze symmetry and choose a standard cell

Symmetry identifies the space group from positions and lattice without modifying the structure. Primitive Cell and Conventional Cell generate standardized representations, potentially changing coordinates and site count. Make P1 removes symmetry constraints for local editing; it is not primitive-cell reduction.

1. Choose Symmetry under Symmetry Tools, starting with positional tolerance $0.01$ Å and angular tolerance $5°$. Record the space-group symbol, Hall symbol, and international number.
2. For inputs with small numerical deviations, compare progressively finer or coarser positional tolerances. They define approximate equivalence;

a large tolerance can conceal real distortions. Choose with the structure's origin in mind.

3. Generate primitive and conventional representations separately from the same original, saving each. Compare formula, volume, site count, and element ratios to confirm that they represent the same periodic material.

4. Open Wigner-Seitz Cell to inspect the lattice geometrically. Its polyhedron is defined by the lattice; it neither cuts atoms into a finite cluster nor changes the current sites. Save the original crystal when a structure file is needed.

Before local defect editing, practice P1 conversion on the saved space-group-built `Cu-fcc`. Choose Make P1 under Symmetry Tools; no additional numerical parameters are needed. Check that the four Cu sites and coordinates remain unchanged and save as `Cu-P1`. Continue local edits in that copy and retain the original symmetry model. P1 removes extra space-group constraints; it does not automatically disorder a regular arrangement.

### Specify which atoms may move

Fix Atoms writes selective-movement flags for subsequent relaxation constraints. A stationary-looking atom in the 3D view does not establish that a constraint exists.

1. Select all intended fixed sites, open Fix Atoms under Atomic Editor, and check indices.

2. If a slab has geometric-layer information, a bottom-layer count is also available. Enter the number of lowest layers to fix, or use 0 to honor your explicit site list strictly.

3. Apply, save, and export native structure JSON. Reopen it and check site count and fixed-site indices. For a direct read-only check in a text editor, inspect `selective_dynamics`: three false values fix all directions; three true values permit motion in all directions.

The command fixes selected sites in all three directions and makes the others movable. To fix two groups simultaneously, combine them into one complete selection before applying. Selection sets help avoid omissions during repeated selection.

In molecular geometry optimization, enter constraints under Fixed atom indices; for example, `1 2` fixes those two atoms before previewing. Crystal fixed-site flags record constraints and do not replace the calculation task's own constraint settings. The hydrogen optimization in Chapter 10 has no fixed atoms. For constrained calculations, confirm the input required by the chosen method and compare fixed coordinates before and after optimization.

### Create point defects other than vacancies

Defects & Alloys → Create Point Defects also offers substitutions, interstitials, and antisites. Build each from a saved host copy to avoid unintentionally combining several defects.

1. For a substitution, choose the substitution type and specify a site and replacement element. The site count should remain unchanged, while one element count increases and another decreases.
2. For an interstitial, specify its element, choose fractional or Cartesian coordinates, and enter its position. Preview nearest-neighbor contacts; applying should add one site. Choose the position according to lattice voids and nearest-neighbor distances.
3. For an antisite, enter two site indices. The operation exchanges species while retaining positions and overall composition. Exchanging two identical elements creates no meaningful chemical antisite.
4. Name each result separately and record its host, supercell size, and defect position. Recheck indices after changing cell representation or atom order.

Enumerate Point Defects searches vacancy or substitution candidates using an allowed site range, replacement element, reference charge, and symmetry tolerances. Degeneracy counts equivalent candidates. NetCharge relates to the reference charge entered, and shortest contacts provide geometric checks. None is a defect formation energy or a calculated electronic charge state. Enumeration automatically adds structures to the Project Browser and opens their documents. The result table is for inspection, not model switching. Open candidates from the tree, check composition and site count, and save. Use consistent host and numerical conditions for later comparisons.

### Generate an alloy with explicit occupancies

An SQS, or special quasirandom structure, assigns elements to finite-cell sites so that statistics such as neighboring species pairs approximate random mixing. It turns mixed occupancies into one fully occupied species per site for further calculation, without implying a chemically ordered phase. The command reads elements and occupancy fractions from the input; it does not choose an alloy composition from a pure-element crystal automatically.

The four-site Cu–Ni example below provides the input, scaling, search budget, and result checks. For another composition, choose a cell that accommodates integer species counts; an equal binary mixture needs equal integer counts, for example. Retain both the mixed-occupancy input and the generated arrangement.

### Example: a four-site Cu–Ni SQS

The supplied `sqs-cu-ni-input.json` provides a ready-to-use input: a face-centered cubic cell with side length $3.61$ Å and four independent sites. Each site has Cu occupancy $0.5$ and Ni occupancy $0.5$, giving an overall composition equivalent to $Cu_2Ni_2$. Mixed occupancy describes a statistical composition; it does not place two physical atoms at one position. An SQS converts those fractions into explicit species assignments, with one element fully occupying each site.

The small cell and example lattice constant make the conversion from mixed occupancy to explicit arrangements easy to inspect. For alloy-property studies, use larger models as needed and test convergence of the correlation functions and lattice parameters.

| Item | Settings and checks |
|---|---|
| Input | `sqs-cu-ni-input.json`; a CIF with the same base name is also supplied. |
| Lattice | $a = b = c = 3.61$ Å; all angles are $90°$. |
| Independent sites | 4; fractional positions $(0, 0, 0)$, $(0, \frac{1}{2}, \frac{1}{2})$, $(\frac{1}{2}, 0, \frac{1}{2})$, and $(\frac{1}{2}, \frac{1}{2}, 0)$. |
| Occupancies | Cu 0.5 and Ni 0.5 at every site, summing to 1. |
| Supercell scaling | Enter `1` to retain the four-site cell. |
| Search budget | Enter 1 second; this small example uses exact enumeration. |
| Expected result | Two Cu and two Ni sites, four sites in total, each with occupancy 1. |

**Table 9.3** The four-site Cu–Ni SQS exercise.

1. Import `sqs-cu-ni-input.json` and open its 3D document. In Online, upload the file first, then import it from server-side storage. Name the document `CuNi-mixed` and save the starting project so that you can return to the mixed-occupancy input.

2. Check the independent site count, cell, and atom information. There should be four sites, each listing both Cu and Ni at 0.5. In the input JSON, every `sites` entry has two elements under `species`, each with its own `occu`. This is more reliable than inferring composition from display colors alone.

3. Keep the crystal document active. Open Generate SQS Model from Crystal Modeling → Defects & Alloys. Enter `1` under Supercell scaling and 1 under Search budget (seconds).

4. Click Preview. Check that the four sites have explicit assignments to the two elements and that the cell and site count have not grown, then click Apply. The lattice and positions remain unchanged; only the species assignments change.

5. Check for $Cu_2Ni_2$, four sites, and one fully occupied element at every site. The nearest-neighbor distance should be approximately $3.61/\sqrt{2} = 2.552655$ Å. Cell symmetry can produce equivalent species arrangements; assess composition, occupancy, and periodic geometry rather than requiring one visual ordering.

6. Export native JSON using the structure-export procedure described earlier, naming it `sqs-cu-ni-ordered.json` to retain the search record. Here `ordered` means that every site has a definite element, not that a phase with long-range chemical order has been generated.

7. Name the result `CuNi-SQS-4` and save the project. Also export CIF for cell and explicit-occupancy exchange. In Online, download the exports and keep the input JSON, result JSON, and project copy. Use CIF for structure exchange and native JSON for the SQS search record.

### Inspect the search record

Open the exported JSON in a text editor and inspect `properties.sqs` without editing it. `search_mode` identifies the search method, `configuration_count` counts candidate arrangements, and `objective_function` measures correlation matching.

The reference uses `exact-enumeration`, with `configuration_count` equal to 6 and `objective_function` approximately 0.1666667. The six arrangements choose two of four sites for Cu and assign Ni to the rest. All six distinct site pairs have the same shortest periodic distance. One pair is Cu–Cu and one Ni–Ni, so this finite cell cannot reproduce the pair proportions of independent random occupancy exactly. The objective is therefore nonzero. This dimensionless value measures deviation from random-occupancy target correlations; it is not an adsorption energy, total energy, or defect formation energy.

The time budget controls candidate searching. The full operation also includes exact enumeration, comparison of equivalent structures, and result assembly, so total elapsed time can exceed the entered value. Consider both the site count and candidate count when enlarging the model.

### Scaling and mixed occupancy

A single integer here specifies a cell volume multiplier, not that integer applied independently in all three directions. To give explicit repeats, enter a triplet such as `2 2 2` in the same field; it expands this example to 32 sites. Record the actual cell, composition, and search conditions before comparing models. Mixed occupancy is a modeling input; electronic structure calculations should use the generated, checked explicit assignments. A calculation object may warn that fractional occupancy cannot be represented directly. A resulting single-element display does not mean an SQS has been generated.

The corresponding results are supplied as `sqs-cu-ni-ordered.json` and `sqs-cu-ni-ordered.cif`. Run `sqs_example.m` in the MATLAB® toolbox environment started with `-nojvm` to regenerate inputs and results. Once explicit occupancies are established, proceed with an electronic structure workflow.

## Choose a low-dimensional construction

Section 8.10 demonstrates five constructions using graphene and copper. Choose according to the boundaries to retain, using that section's example parameters.

Wire, ribbon, and tube repeats are not total dimensions in Å. Compare actual cells, site counts, and periodic directions. Before passivation, save the unpassivated input, then check added species and boundary coordination.

| Desired model | Tool and distinction |
|---|---|
| Finite-width sheet | Cut Nanoribbon; width and axial repeats define a ribbon periodic along its length. |
| Rolled sheet | Roll Nanotube; chirality and axial repeats define a tube periodic along its axis. |
| Crystal with finite cross-section | Cut Nanowire; lattice direction and radius retain axial periodicity. |
| Finite cluster | Quantum Dot in Quantum Dot / Nano Void; retain atoms inside a sphere. |
| Spherical hole in periodic bulk | Nano Void in the same dialog; remove atoms inside a sphere and retain the outer periodic host. |

**Table 9.4** Choose a nanostructure tool by its boundaries.

## 9.3 Build soft-matter models

### Build a branched polymer

A branched polymer starts from a backbone and attaches fragments at intervals counted in repeat units. It illustrates backbone/side-chain topology. Backbone names come from the repeat-unit library, while side-chain names come from the fragment library; check them separately.

1. Import `user-repeat-ethane-input.json`, activate its molecular document, and open Build Branched Polymer through Molecule Modeling → Soft Matter → Polymer Builder. Retain the input: construction replaces the active molecule. Start small, for example backbone `PE`, 8 repeats, side chain `Methyl`, interval 3, and seed 1.

2. Inspect the estimated size and set an appropriate atom limit. The branch interval counts repeat units, not Cartesian length or individual backbone atoms.

3. Preview and check attachments near the intended units, both chain ends, and connectivity. To change branch density, return to the original

short input and change the interval rather than rebuilding repeatedly on an already branched result.

4. Apply, inspect composition, side-chain attachments, and local distances, then run Diagnose Molecule and save native structure data. Pack molecules later if a material at a specified density is needed.

Use Build Homopolymer for an unbranched reference, specifying the repeat unit, repeat count, independent-chain count, end groups, conformation, and seed. More than one independent chain does not create intermolecular covalent bonds. Extended, zigzag, and random-coil options define initial geometries. Check intrachain contacts and whether further cleanup is needed before calculation.

### Build a basic dendrimer skeleton

Build Basic Dendrimer expands a skeleton by generation and branching, illustrating hierarchical topology. Core functionality sets the first-generation branch count; the branching factor sets how many new branches each subsequent branch produces. Skeleton element and bond length define the initial geometry, which can be developed further with the functional groups and chemistry of the actual material.

1. Import and activate `user-repeat-ethane-input.json`, retaining a copy. Open Build Basic Dendrimer through Molecule Modeling → Soft Matter → Polymer Builder. Start with generation 2, core functionality 3, branching factor 2, C, skeleton bond length 1.54 Å, and seed 1.

2. Inspect the estimated size and set an atom limit. Branch count grows quickly with generation; check a small model first.

3. In the preview, locate the center and trace first- and second-generation connections. Check for branches passing through others and inspect shortest contacts.

4. Apply, check composition, branch connectivity, and local contacts, run Diagnose Molecule, and save the project and native structure. Clean and relax the starting skeleton as needed.

### Save a custom repeat unit

A repeat unit needs two distinct attachment ends in addition to atoms and bonds. Its head connects to the preceding unit, its tail to the following one, and leaving atoms are removed before connection. Here the two carbons of ethane define a simple skeleton that is joined into a dimer to illustrate these fields.

The supplied `user-repeat-ethane-input.json` contains two C and six H atoms: 8 atoms and 7 single bonds. C atoms are 1 and 2; H atoms are 3–8. The C–C distance is 1.52 Å. We remove all six hydrogens in the stored repeat unit and let chain construction add the required hydrogens. This is a geometric unit definition, not a representation of ethane polymerization.

| Field | Input |
|---|---|
| User repeat-unit name | `MANUAL_CC`. |
| Head and tail atoms | 1 and 2, respectively. |
| Leaving atom indices | `3 4 5 6 7 8`. |
| Homopolymer repeat unit | The saved name `MANUAL_CC`. |
| Repeats and chains | 2 and 1, respectively. |
| Conformation and seed | Extended, seed 7. |
| Head/tail end groups | Both `none`; remaining unsatisfied valences are filled with H. |
| Atom limit | 1000, sufficient for this model. |

**Table 9.5** Build a dimer from a custom carbon skeleton.

1. Import `user-repeat-ethane-input.json` and open its molecular document. Confirm that atoms 1 and 2 are C and 3–8 are H. Retain the input file and do not reorder it before saving the unit.

2. Open Save User Repeat Unit through Molecule Modeling → Soft Matter → Polymer Builder. Enter the table's name, head, tail, and leaving

indices. The ends must be distinct existing atoms and cannot also be leaving atoms.

3. Leave overwrite disabled on first save. If the name exists, confirm it is an earlier example record before overwriting, or choose another name and use it consistently. Saving the unit does not change the source molecule: ethane still has 8 atoms.

4. In the same document, open Build Homopolymer, enter the saved name, and set repeats, chains, conformation, end groups, and seed from the table. Retain default head-to-tail connectivity. Preview, apply, and inspect the complete chain.

5. Check for 4 C, 10 H, 14 total atoms, and 13 single bonds, including one new C–C connection between units. Rotate and inspect the backbone and terminal hydrogens, measure suspicious contacts, and run Diagnose Molecule. `user-repeat-CC-dimer.json` is the supplied reference.

6. Name the result `MANUAL-CC-dimer`, save the project, and export native structure JSON. Retain input ethane, unit parameters, and dimer as three separate stages. Exchanging only the dimer does not register the custom unit in another computer's library.

Open the input or output JSON in a text editor to inspect elements, coordinates, and connections. The supplied `user_repeat_unit_example.m` repeats the procedure and writes a separate example library, `user-repeat-unit-library.json`, without overwriting the personal library. The accompanying scripts check composition, connectivity, and diagnostics. For longer chains, increase repetitions from the same unit and inspect contacts throughout the resulting chain.

### Pack one molecule type into a confined region

Construct Amorphous Cell packs copies of the current molecule into a periodic box according to count and density; Build Confined Layer further restricts the usable region. Density is in $g/cm^3$ and minimum contact distance

in Å. Packing creates initial positions; physical equilibration is a separate step.

1. Open a molecule whose connectivity and geometry have been checked. Save the original component, then choose single-component packing or a confined layer under Amorphous Builder.
2. Enter molecule count, density, contact distance, seed, and atom limit. Start with few molecules and a low initial density to check components and boundaries.
3. For a confined layer, select axis 1, 2, or 3 and the lower/upper bounds. For example, `0.2 0.8` means 20% to 80% of that box length. When both values lie between 0 and 1 they are fractional, not 0.2 to 0.8 Å.
4. Inspect the preview from a side view that shows the extent along the confinement axis. Check that molecules lie in the permitted region and that excluded empty space is not being treated as available volume.
5. Apply, check molecule count, total atoms, box lengths, and periodic contacts, then save the result and input settings separately.

Density ramp stages (optional) accepts positive, nondecreasing densities ending at the target. Blank uses only the target density. Leave it blank for the confined-layer exercise to avoid conflicting with regional restrictions. Molecules per packing batch controls batching without changing the total count. For two components, use the earlier Pack Mixture tutorial. Mole- and mass-fraction modes also need a total count; check the converted integer counts.

### Pack molecules into an existing box

Pack into Existing Box retains the current cell and host atoms and adds another project molecule. It does not generate a new cubic box from a target density. Pack around Nanoparticle also reserves a gap around the host.

1. Import and name host and component in one project. Activate an orthogonal-cell host and choose the required tool under Host Packing. Prepare a suitable box first if the host is oblique.

2. Choose the component under Molecule to pack. Enter count, contact distance, seed, batch size, and atom limit. There is no density-based box resizing here, so make sure the host box has enough space.

3. For nanoparticle packing, specify a gap in Å. The exclusion region is a sphere centered on the host's mean position, enclosing its outermost atoms plus the gap. It does not follow an arbitrary corrugated surface atom by atom.

4. Preview from several directions and check that the host is retained, molecules are appropriately placed, and periodic contacts are acceptable. If space is insufficient, enlarge the empty box around the original host or reduce the count and restart.

5. Check added molecule counts and composition. Retain host, component inputs, and packed result separately, and save the project and native structure.

## 9.4 Finish a surface model

### Passivate the surface

The adsorption exercise in Chapter 8 covers sites and O–H placement. Here we terminate remaining slab boundaries. Start from a saved slab copy so that passivation and later solvent treatment have separate records.

1. If undercoordinated surface sites need capping atoms, open Passivate Surface. Choose element, bond length, upper/lower/both surfaces, and detection depth.

2. A target coordination of 0 lets the tool infer it; otherwise enter the known target. The minimum contact distance excludes overly close positions.

Preview both sides and check that existing atoms have not been incorrectly classified as surface atoms.

3. Apply, compare added element counts and boundary coordination, and save a separate passivated result before deciding whether to add solvent.

### Add solvent above a slab

Add Solvent Layer needs a complete solvent molecule already in the project. Keep the slab document active and select the solvent molecule under Solvent structure, then set molecule count, layer thickness, surface gap, remaining vacuum, contact distance, seed, and timeout in seconds. Thickness, gap, vacuum, and contacts are in Å; the molecule count is an integer. Start with a few molecules to determine which side receives them, then check internal solvent geometry, layer contacts, and remaining vacuum before increasing the count. After generating the initial layer, configure temperature and equilibration according to the calculation method.

## 9.5 Reuse a modeling procedure

### Save frequently used parameters

A parameter preset stores the conditions entered in a command. Reload and adjust them for the current material or model; save generated structures separately.

1. Open a frequently used command and check its parameters. Preview if useful; a parameter set may also be saved before applying.

2. Click Save Preset at the top of the dialog and give it a descriptive name such as “surface exercise thickness.” Check that it appears in Preset.

3. Close and reopen the same command, select the name, and click Load. Check every value for the current model, especially site indices, project structure references, and lengths.

4. Adjust as needed, preview, apply, and save the generated model. Loading a preset does not itself apply a structural change.

Reset at the bottom of a parameter dialog restores that dialog's initial values and clears its temporary preview. Reset in ribbon History restores the current structure draft's baseline. They affect different states. Expand Help information for parameter explanations and collapse it to continue editing.

### Record a traceable modeling procedure

A Recipe records applied commands, parameters, seeds, and checksums of structures before and after changes. It documents how a result was obtained; retain the starting and final structures as well.

1. Open and save a clearly named starting model. Activate its 3D document and click Record in History.
2. Perform the operations, previewing and checking before applying. Canceled operations are not recorded.
3. Click Save Recipe when finished. Recording stops, a time-based name is generated, and the status gives its save location.
4. Open History → Presets & Templates, click Refresh, and find the new entry of type Recipe. Save the result and project and archive the starting input with them.

Recipes cover supported applied modeling commands and parameters in the current structure document. View changes, selection-set management, and analysis inspection are not recorded. Save the final model separately after applying a candidate from an analysis result window. Presets & Templates provides entry inspection and template creation. MATLAB® toolbox users can replay a Recipe through the modeling API using the matching starting structure; graphical users can retain it as an operation record.

### Create a modeling project template

Presets & Templates lists command presets, Recipes, and modeling project templates by type, name, and details. A modeling project template records a Recipe name and input/output formats. It differs from a calculation workflow template saved as `.wf`.

1. Open Presets & Templates under History and click Refresh to confirm the intended Recipe exists.
2. Click New Project Template and enter a name, Recipe name, and input/output formats. Default `auto` detects input format; `same` retains it for output. These are format choices, not paths.
3. Save and check the type, name, and input-to-output relationship in Details. For an invalid entry, inspect its details and verify the referenced records before use.
4. Retain the template with its Recipe. Save actual models separately: creating a template does not open a new project or execute modeling automatically.

### Inspect existing batch jobs

Modeling Jobs lists saved batch modeling jobs for structure files, distinct from SCF runs in electronic structure workflows. Columns include ID, status, completed count, requested count, successes, resumability, and errors. A finished batch may still contain failed files.

1. Open History → Modeling Jobs, click Refresh, and identify the batch by ID.
2. Compare requested, completed, and successful counts, then read errors. Successes count files for which output was generated normally.
3. Select a resumable entry, click Resume, wait for a status update, and refresh. If the button is disabled, check whether the entry is marked resumable.

4. Use Cancel Job to request stopping a resumable job you no longer intend to continue, then refresh to confirm status. Canceling does not delete written results; inspect outputs according to the job record.

5. Check result structures and reports in the original output directory. Keep inputs for failures and retain inputs, outputs, and the job record together.

This window inspects and resumes existing jobs. MATLAB® toolbox users can create file batches through the modeling API; graphical users inspect existing jobs as described above.

### Recover unsaved structure edits

Undo and redo operate on the current structure draft. History's Reset restores its baseline, which updates on saving. Save separately named results for lasting model comparisons.

If reopening offers draft recovery, confirm the associated structure, recover it, then inspect atom count, composition, cell, and the last edited region. Save under a new name; recovery does not recreate the full earlier undo history. Before declining recovery and clearing a draft, confirm that the saved file is the version you need.

Chapter 10

# Building calculation workflows

A workflow organizes structure preparation, electronic structure solving, and result display into a readable, reusable graph. It can handle a simple SCF run or combine symmetry analysis, band structure, density of states, and geometry optimization. This chapter introduces workflow editing, then develops common calculations for silicon and hydrogen.

If you completed Chapter 5, continue with the silicon examples. `examples/silicon-tutorial.ks` contains SCF and DOS workflows for an eight-atom silicon cell; `examples/silicon-band-primitive.ks` contains band and symmetry workflows for a two-atom primitive cell; and `examples/h2-tutorial.ks` contains hydrogen SCF and geometry optimization. Reference results are saved in these projects so that you can inspect them before rerunning.

## 10.1 Edit a workflow

### How nodes organize tasks

Each canvas node represents a task. Select it and choose Workflow → Configure Node, or click its gear, to expand the Config Editor and open or refresh the task-specific Options. Scroll down to inspect parameters. "Configure a node" below refers to this procedure. Modules group functions by processing stage; the task determines the actual operation. The label is simply the name shown on the canvas and can be changed for clarity.

For example, a node labeled "Build silicon model" can perform Build Molecule, while "Electronic self-consistency" can perform SCF. Renaming does not change the task type. Choose the correct task first, then rename it if useful.

Connections define processing order. Downstream tasks need upstream data: a band calculation needs self-consistent density and a path; a con-

vergence plot needs an iteration history. Do not create cycles. Geometry optimization already manages repeated solves internally and does not need a canvas connection back to itself.

### Choose a starting workflow

Import workflow from template offers SCF, Energy Band, DOS, Relaxation, and Symmetry. Templates demonstrate common task combinations. After opening one, select the calculation structure and check parameters before running. With several structures in a project, explicitly select the intended object in the structure-input node.

Start blank for a customized workflow. Add nodes, adjust their input and output ports, connect them, and assign tasks. Zoom, fit-to-view, Undo, and Redo help organize the canvas. A clear layout makes dependencies easier to inspect.

1. Create a blank workflow and add its first node.
2. Select the node and choose Workflow → Configure Node, then set its module and task.
3. Add the next node and connect it in processing order.
4. Configure structures, physical parameters, and numerical settings in turn.
5. Name and save the workflow, then validate it with a small calculation.

### Projects and workflow files

A `.ks` project retains structures, workflows, results, and other contents. A `.wf` file stores one graph and its task settings for reuse across projects. When sharing a `.wf`, also supply the structure and explain how to select it as input.

Save the project for complete day-to-day work. Export a workflow when you want to reuse a calculation method on another system, then adjust the structure and accuracy settings for that system.

## 10.2 Choose a task

Templates provide common combinations; the task list provides the operations available to individual nodes. Module first chooses the processing stage, then Task selects its function. Each node performs one task, chosen in the Config Editor.

**Table 10.1** Workflow tasks and their roles.

| Module and task | Purpose and required inputs |
|---|---|
| Preprocessing | Symmetry & K-Path: analyze crystal symmetry and generate a high-symmetry path from the selected structure. |
| Computation | Build Molecule: convert a project structure into a calculation object and set functional, pseudopotentials, cutoff, and sampling. |
| Computation | SCF: find a self-consistent density and produce ground-state energy, electronic states, and iteration history. |
| Computation | NSCF: prepare options for later band or DOS calculations. This node alone does not produce a new band plot. |
| Computation | Geometry Optimization: update atomic positions through electronic calculations; requires a complete ground-state specification. |
| Computation | TDDFT: prepare a Casida excited-state calculation from ground-state electronic states. See the excited-state section for parameters and solving. |
| Postprocessing | Band Calculation: generate $k$ points along a symmetry path and solve non-self-consistently; needs SCF results, NSCF options, and a symmetry path. |
| Postprocessing | Band Calculation 3D: sample a reciprocal-space plane; needs SCF results and NSCF options. |

| Module and task | Purpose and required inputs (continued) |
|---|---|
| Visualization | Energy Curve: plot SCF energy and error histories. |
| Visualization | Band Structure: plot the path bands produced by Band Calculation. |
| Visualization | Band Structure 3D: plot energy surfaces from Band Calculation 3D. |
| Visualization | DOS: use an NSCF grid to calculate and display total density of states. |
| Visualization | Brillouin Zone 2D: display a 2D Brillouin zone and path from symmetry results. |
| Visualization | Brillouin Zone 3D: display a 3D Brillouin zone and path from symmetry results. |

Task names match the interface. Preprocessing defines geometry and paths, computation prepares electronic structure, postprocessing samples the resulting states, and visualization presents data. DOS also performs grid sampling within its own task, so a module name alone does not tell you whether numerical calculation is required.

## 10.3 Add node ports

Top ports accept preceding connections; bottom ports connect to subsequent nodes. A 0-0 node has no ports, a 0-1 node has only a bottom port, a 1-0 node has only a top port, and a 1-1 node has both. These are canvas connection patterns, not different solvers.

1. Activate a blank workflow and add a 0-1 node from Workflow → Add Node.
2. Configure Computation Module → Build Molecule and select an imported structure.

3. Add two 1-1 nodes for SCF and Energy Curve; the latter is in Visualization Module.
4. Drag from each preceding node's bottom port to the next node's top port to connect the three.
5. If a node has the wrong port pattern, select it and add or remove top/bottom ports through Edit Node. Check connections before removing a port in use.
6. Use Zoom In, Zoom Out, and Fit to View to inspect the complete graph, then save.

Check each node's task and parameters before running SCF. Undo and Redo restore graph edits. Save a `.wf` to import the same graph into another project later.

## 10.4 Understand general node settings

General contains the label, module, and task. Options and Advanced Options contain task parameters. Changing tasks replaces the parameter set; it does not convert the previous task's values into new ones. Select the task before entering its settings.

The editor also shows Inputs, Resources, and Outputs. Resource execution mode, process count, and thread count affect actual execution. Input-access and output-save/forward checkboxes do not filter results, and the resource time-limit field does not stop a task automatically. View results under Results and save the project to retain them.

## 10.5 Allocate node resources

### Node order and parallel work within a node

Nodes run in the dependency order defined by connections. Drawing two branches does not make those branches execute simultaneously. Parallelism

within a node divides parts of a task among computing processes, such as distributing NSCF $k$ points. These are different execution concepts.

Select a node, choose Workflow → Configure Node, and expand Resources to see its supported execution modes. Choices depend on the task and the diagonalization method selected by DiagMethod.

**Table 10.2** Node resource fields.

| Field | Meaning |
|---|---|
| Parallel environment | Choose Serial, Parallel (process pool), GPU (diagonalization), or Use NSCF settings for a supported follow-up task. Available choices depend on the task. |
| Maximum processes | Positive integer, default 2. Limits pool workers; editable only in process-pool mode. |
| Threads/process | Positive integer, default 1. Limits CPU computational threads per process; editable in pool or GPU mode. |
| Time limit | No Limit, 30 minutes, 1 hour, or 2 hours. This field does not trigger automatic stopping. |

A worker is a calculation process. A maximum of 2 processes with 1 thread/process allows at most two workers, each with at most one CPU computational thread. The main application still runs its interface and schedules tasks, so these values do not count every operating-system process or thread. More workers also need more memory and may not speed up a small exercise.

Serial disables the node's pool and GPU calculation paths; it does not force all CPU work onto one core. Numerical libraries may still use their own threads. Threads/process does not select a thread-based parallel pool: node pools use process workers.

### Check task support

GPU options support `lobpcg`, `davidson`, `davidson2`, and `davpcg`. The default SCF `davidson_qe` is not among them, so a newly opened SCF resource panel

**Table 10.3** Resource choices for common tasks.

| Task | Available execution modes |
|---|---|
| SCF | Serial; GPU for supported diagonalizers. No process-pool option. |
| NSCF | Serial; process pool with HT interpolation disabled, and GPU for supported methods. Later band and DOS tasks can inherit these settings. |
| Band Calculation, DOS | Serial, inherited NSCF settings, process pool, and supported GPU methods. Pool and GPU modes are unavailable with HT interpolation. |
| Band Calculation 3D | Serial, inherited NSCF settings, process pool, and supported GPU methods. Direct NSCF sampling is used, without HT interpolation. |
| Other tasks | Currently serial, including structure construction, symmetry, geometry optimization, TDDFT, and general plotting. |

may offer only Serial. GPU acceleration applies to supported diagonalization steps; other work retains its own execution mode. Threads/process in GPU mode limits CPU-side computational threads. The device manages GPU kernel threads.

Band and DOS tasks use the preceding NSCF diagonalization options, so check its DiagMethod and Use HT too. A task or method change can reset an incompatible resource choice to Serial; recheck Resources before running. Use NSCF settings displays inherited process and thread counts in noneditable fields. Change them in NSCF, or choose an independent mode in the later node.

If a workflow contains several NSCF nodes, later sampling uses the most recently executed NSCF configuration. It does not choose by canvas proximity or retain a separate configuration for each branch. Keep separate workflows when comparing resources or solver conditions so that inheritance is unambiguous.

### Exercise: two workers for silicon DOS

Use this chapter's silicon DOS workflow without changing its structure, cutoff, grid, energy range, or step. Complete a serial reference first, then compare the execution mode so that environmental and physical changes remain distinguishable.

1. Open the silicon DOS workflow, select NSCF, and choose Configure Node.
2. Under Options, keep DiagMethod at `lobpcg` and Use HT cleared.
3. Expand Resources, choose Parallel (process pool), and enter 2 maximum processes and 1 thread/process.
4. Select DOS, choose Use NSCF settings in its Resources, and check the displayed counts of 2 and 1.
5. Save. Run the complete workflow in an environment with the required parallel support, inspect $k$-point messages, and check the new DOS result.
6. Record resources and timing. To compare serial execution again, set NSCF to Serial and check that DOS still inherits it.

NSCF mainly prepares solver options; it does not complete band or DOS sampling itself. The subsequent non-self-consistent sampling task actually uses the pool. A supported $k$-point loop is distributed only when there is more than one $k$ point; sampling only one $k$ point cannot provide the same parallel benefit. Serial disables the node's pool path while retaining an existing pool.

Resource choices do not change the physical model. Still compare convergence, sampling, and energy references; execution modes may differ in their last numerical digits. Distinguish pool startup time from solving time as well.

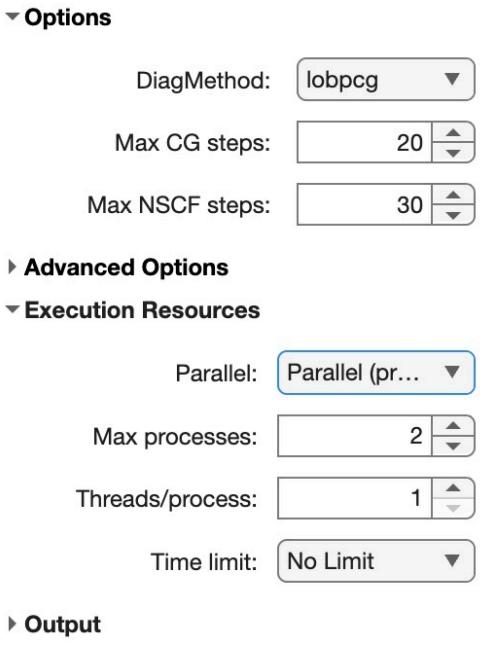



**Figure 10.1** NSCF process-pool settings: at most two workers, with one CPU computational thread per worker.

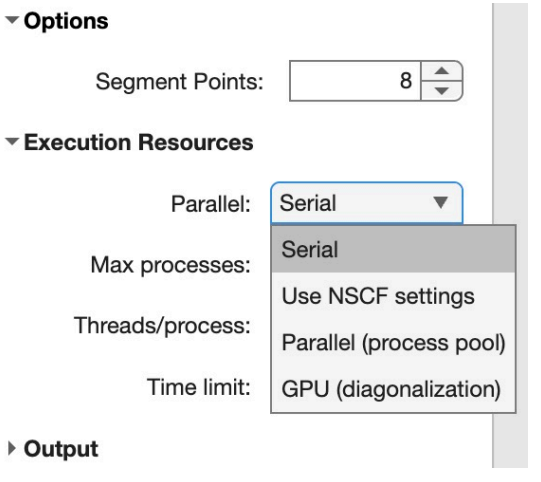



**Figure 10.2** Band Calculation can inherit NSCF settings or use its own execution mode.

### Respond to unavailable resources

A dropdown choice indicates task support, but the execution machine must also support it. Pools need Parallel Computing Toolbox and a process-based pool configuration. GPU execution needs a supported available GPU and its calculation components. An Online browser displays the interface on your computer, but the server provides calculation resources. A disabled Home Parallel button does not make all node parallel options unavailable; check the node and actual server environment.

If no pool exists, the application attempts to create a local process pool up to the node's maximum process count. A request above the local profile's worker limit produces an error rather than silently enlarging that limit. An existing pool is checked for type, worker count, and CPU threads per worker. Thread-based pools, oversized pools, and excessive thread counts cause conflicts. A smaller compatible pool can be reused: the maximum does not guarantee that a new pool of exactly that size is created for every task.

Begin with Serial if parallel execution is not configured, then check the environment separately. Full MATLAB® users can close an incompatible pool they created before rerunning with node settings. The application preserves user-created pool configurations. A temporary pool created by the current node is released when that task finishes or exits.

### Use node settings remotely

Node resources determine how a task uses computing resources; remote settings in Chapter 13 determine where the workflow is submitted. They can be used together, but entering two workers neither submits a cluster job nor changes node order.

Remote submissions carry node resource settings. A job already executing on a remote worker cannot create a nested pool automatically inside a node. Request a suitable pool at submission and ensure remote thread settings are compatible. For a node allowing at most 2 workers with 1 thread

each, an existing 4-worker remote pool exceeds the limit. Check cluster worker counts, PoolSize, and node limits together.

Online CPU and memory status monitors the current session; it is not a resource-request control. Save node settings with the project. For remote work, also retain the cluster and submission conditions to reproduce the same environment later.

## 10.6 Silicon: your first periodic calculation

### Prepare the eight-atom silicon cell

Silicon is a useful introduction to periodic calculations and semiconductor bands. This tutorial uses an eight-atom cubic cell with lattice constant 5.4437023729 Å, stored in `examples/silicon.cif`. Similarly named files elsewhere in the software may use different lattice constants. Use the supplied file for numerical comparisons.

For a new project, follow these steps. If `silicon-tutorial.ks` is open, select its silicon SCF workflow and continue with the settings below.

1. Save and close existing work to return to an empty project.
2. Import `silicon.cif` and check eight independent silicon sites and a cubic cell.
3. Save as `silicon-learning.ks`.
4. Show cell boundaries as needed and rotate the model to inspect the arrangement.
5. Open the SCF template and select this silicon structure as input.

A periodic viewer may show bonds across cell boundaries and neighboring images. Repeated spheres do not necessarily mean duplicate atoms were imported. Use structure information for atom counts and cell boundaries for periodic connectivity.

### Set SCF conditions

Choose Crystal. Unlike hydrogen, this calculation uses periodic sampling and extra empty states. A $2 \times 2 \times 2$ grid builds the density, and four additional bands provide low-energy unoccupied states. The band tutorial uses a separate standardized two-atom primitive cell; DOS retains this eight-atom cell.

**Table 10.4** Silicon tutorial SCF settings.

| Location | Setting |
|---|---|
| Build Molecule / Type | Crystal |
| Structure | Only the supplied silicon structure |
| Functional / Pseudopotential | PBE / default |
| Ecut (Hartree) | 20 |
| Smearing / Temperature | fermi-dirac / 0 K |
| Nspin | 1 |
| Autokpts | `[2 2 2 0 0 0]` |
| Extra bands | 4 |
| SCF / DiagMethod | davidson_qe |
| SCF / Max SCF steps | 100 |

These modest settings keep a complete periodic exercise manageable. Test cutoff and $k$-point convergence as described in Chapter 6 when greater energy, gap, or DOS accuracy is needed.

### Run and check

Activate and run silicon SCF. Match the structure name and loaded silicon pseudopotential to the example, wait for self-consistency, then inspect the energy curve. The reference converges in seven iterations, with total energy approximately $-31.1058309605$ Ha and final relative density error approximately $7.35 \times 10^{-9}$.

This total energy belongs to the eight-atom cell. Normalize consistently before comparing primitive cells, supercells, or per-atom energies. The total energies of eight- and two-atom cells cannot be compared directly to decide which is lower.

## 10.7 Silicon: bands along a high-symmetry path

### What the band workflow does

A band plot shows electronic energies as functions of wavevector along representative high-symmetry directions. This is easier to interpret than a list of eigenvalues at every $k$ point. The full workflow builds the structure, analyzes symmetry, converges the density, prepares NSCF options, samples a path, and plots the result.

Use `examples/silicon_primitive.cif`, or open `examples/silicon-band-primitive.ks`. This standardized two-atom cell has three lattice vectors of equal length, approximately $3.84928$ Å, at mutual angles of $60°$. It represents the same periodic crystal as the eight-atom conventional cell. Recommended path coordinates are expressed in the standardized primitive reciprocal basis, so using that cell keeps path and calculation coordinates consistent. Start with Energy Band. Build Molecule supplies the structure, Symmetry & K-Path the path, SCF the density, NSCF solver options, Band Calculation the path sampling, and Band Structure the plot.

Symmetry and SCF may occupy different branches. Check that both kinds of information are available before band calculation rather than forcing every node into one column. Preserve template dependencies when rearranging the canvas.

### Configure symmetry and the path

Select only `silicon_primitive` in Build Molecule. Keep Crystal, PBE, default, 20 Ha, Nspin = 1, 0 K, the `[2 2 2 0 0 0]` SCF grid, and 4 extra bands. The same integer grid gives different sampling densities in primitive and conventional cells, so it does not establish equal per-atom energy accuracy. In Symmetry & K-Path, keep lattice threshold $10^{-7}$, symmetry precision $10^{-5}$, and angle tolerance $-1$. The reported space group should agree with the actual geometry.

The recommended path depends on the current crystal. Cell choices, small distortions, and tolerances can change it. Accept the recommendation

initially and check the point labels in the result. In later strain or defect studies, path changes may themselves reflect changed symmetry.

### Choose NSCF solver options

NSCF's DiagMethod chooses the electronic-state solver. Max CG steps and Max NSCF steps limit two iteration levels, not path $k$ points. Path resolution belongs to the later Band Calculation node.

**Table 10.5** Basic NSCF options.

| Field | Default and meaning |
|---|---|
| DiagMethod | lobpcg; follow another example's parameter table if it specifies a different method. |
| Max CG steps | 20, the internal LOBPCG iteration limit. |
| Max NSCF steps | 30, the outer NSCF iteration limit. |
| Use GACE | Cleared by default; a specialized exchange-processing option. |
| Use HT | Cleared by default; relevant to Hamiltonian transformation interpolation. |

HT interpolation needs complete inputs, including ground-state wavefunctions and eigenvalues. The PBE path-band and DOS examples keep GACE and HT off and use direct NSCF sampling. Retain those inputs for advanced calculations in full MATLAB®.

### Set NSCF and path sampling

1. Keep SCF at davidson_qe and at most 100 iterations.
2. Retain NSCF defaults: lobpcg, Max CG steps 20, and Max NSCF steps 30.
3. Leave Use GACE and Use HT cleared.
4. Set Segment Points in Band Calculation to 8.
5. Check the final Band Structure node and save.

Eight points per segment, including both endpoints, give a quick overview. Use 20 or more for a smoother curve. More path points mainly increase follow-up solving work; they do not replace SCF-grid convergence testing.

### Run and read the plot

Activate and run this workflow. After SCF, the program samples the path. Wait for the plotting node to finish.

The horizontal axis lists high-symmetry points in path order. The vertical axis is energy relative to the run's Fermi energy in eV. Several occupied bands and some low-energy unoccupied bands should appear. The number of curves is the number of calculated bands, not the atom count.

The reference has six recommended path segments and eight bands. Its primitive-cell SCF converges in seven iterations, at approximately $-7.6905500238$ Ha and $8.19 \times 10^{-10}$ final relative density error. The coarse SCF grid and sparse path sampling make the workflow easy to learn. A band-gap study requires further sampling tests and identification of the band extrema.

### Make a controlled comparison

Save the workflow as `.wf`, then use Import workflow from file... to create a copy. Change only Segment Points from 8 to 20. Check whether curves become smoother while energies at symmetry points agree. This distinguishes finer path plotting from a denser density-building grid.

Next, hold path resolution fixed and increase only Autokpts in Build Molecule for SCF. That comparison tests density and potential against integration accuracy. Record both settings with each plot to explain any differences.

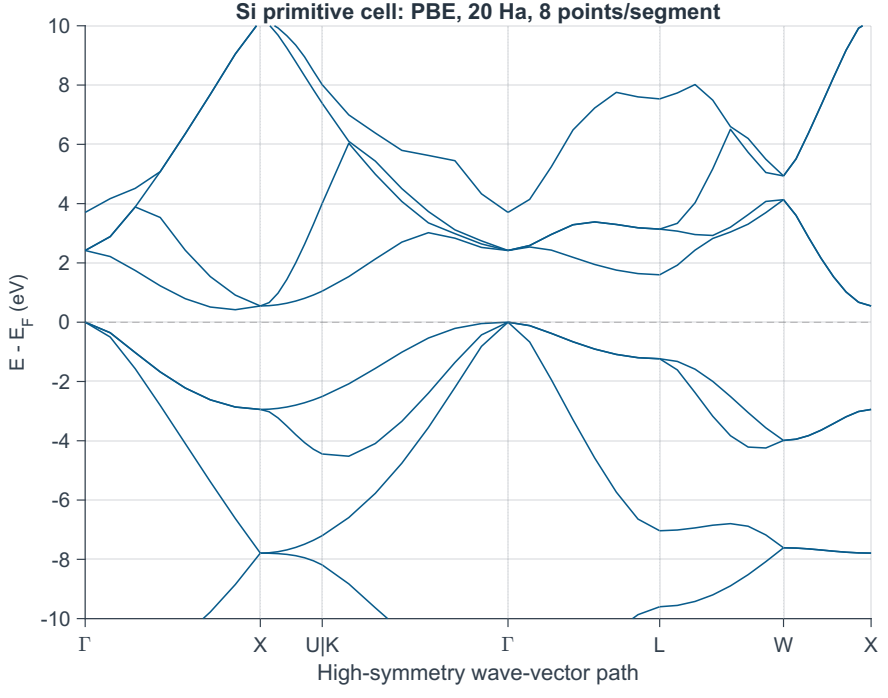



**Figure 10.3** Reference bands for the two-atom silicon primitive cell, with 8 points per segment and the Fermi energy set to zero. Combined labels mark disconnected path segments.

## 10.8 Silicon: calculate total density of states

### Why DOS needs another grid

Bands show how electronic states vary along selected directions. DOS describes how many states are available near an energy and needs sampling throughout the Brillouin zone. It cannot be obtained by simply counting states along the high-symmetry path.

Open the DOS template and inspect Build Molecule, SCF, NSCF, and DOS. Return to the eight-atom `silicon.cif` and the initial periodic SCF settings in this chapter. Primitive-cell bands and conventional-cell DOS share functional and pseudopotentials but differ in cell and sampling. Match models, sampling, and energy references for quantitative comparison. The DOS node performs NSCF sampling on its grid and obtains total DOS by tetrahedron integration.

### Set the grid and energy range

1. Set Build Molecule to Crystal and select this silicon structure. Keep PBE, default, 20 Ha, Nspin = 1, and 4 extra bands.
2. In Build Molecule, set the SCF Autokpts to: `[2 2 2 0 0 0]`.
3. Keep the NSCF solver defaults described above.
4. Set NSCF Grid in DOS to `[4 4 4]`.
5. Set Start Energy to `-10` eV, End Energy to `10` eV, and Energy Step Size to `0.05` eV.
6. Save and run the active DOS workflow.

The energy step determines where DOS is evaluated along the energy axis. Reducing it refines that axis but cannot compensate for too few $k$ points. Vary these two approximations separately.

Tetrahedron DOS requires more than one grid point in each direction; this tutorial uses $4 \times 4 \times 4$. Layered and wire-like models must also meet this requirement, with convergence checked for the specific system.

### Read and check the result

The plot places DOS horizontally and energy vertically, allowing a shared energy direction with a neighboring band plot. Other sources often place energy horizontally. Both layouts are valid; read axis labels first.

The specified range gives 401 energy samples, and DOS values are non-negative. Peaks and fine structure change with the grid. Use a denser grid to check the stability of the principal peak positions and widths.

Figure 12.3 shows this example's DOS.

### Improve the DOS

After the first run, change NSCF Grid to `[6 6 6]` or `[8 8 8]` without changing other conditions. More points substantially increase NSCF work and may stabilize peak shapes. Save the existing result before running so that the curves can be compared.

For higher unoccupied energies, increase Extra bands before expanding the energy limit. Changing End Energy from 10 to 20 eV does not create uncomputed states. The energy range, band count, and $k$ sampling must together cover the region of interest.

## 10.9 Hydrogen: find a lower-energy geometry

### Optimization versus a single-point calculation

Chapter 5 holds hydrogen positions fixed. Here, allow atoms to move to lower the total energy. Bond length is the clearest geometric measure for a diatomic molecule. Start from the same 10 Bohr cube and 1.5 Bohr bond, retaining PBE, default pseudopotentials, and 20 Ha.

Open the hydrogen project and choose Relaxation. Optimization resolves the electronic problem after every position update and costs more than one SCF run. The task manages its own geometry iterations.

### Configure optimization

1. Select the original hydrogen structure in Build Molecule and choose Molecule.
2. Check PBE, default, 20 Ha, Nspin = 1, and 0 K against the original SCF.
3. Select Geometry Optimization.
4. Set RelaxMethod to fminunc.
5. Explicitly enter `1e-4` for RelaxTolerance and `100` for MaxRelaxSteps.
6. Save, activate this workflow, and run it.

Other methods include nlcg2, bfgs, and fire, which update positions differently. This tutorial uses fminunc. In the MATLAB® toolbox edition, it requires Optimization Toolbox. The standalone application includes the corresponding runtime components.

### Follow the two iteration levels

Repeated SCF sequences in the optimization log solve electronic structures at successive geometries. One converged SCF means that geometry's electronic calculation is complete, not that optimization has finished. Locate the final geometry-optimization status and check its last electronic solve.

Function evaluations, electronic iterations, and geometry steps need not have the same count. Some methods test several positions to choose a step length. Do not count every logged energy as an accepted atomic move.

### Reference result

With these settings, fminunc gives a final bond length of approximately 1.43107436 Bohr, or 0.75729194 Å. Total energy is approximately

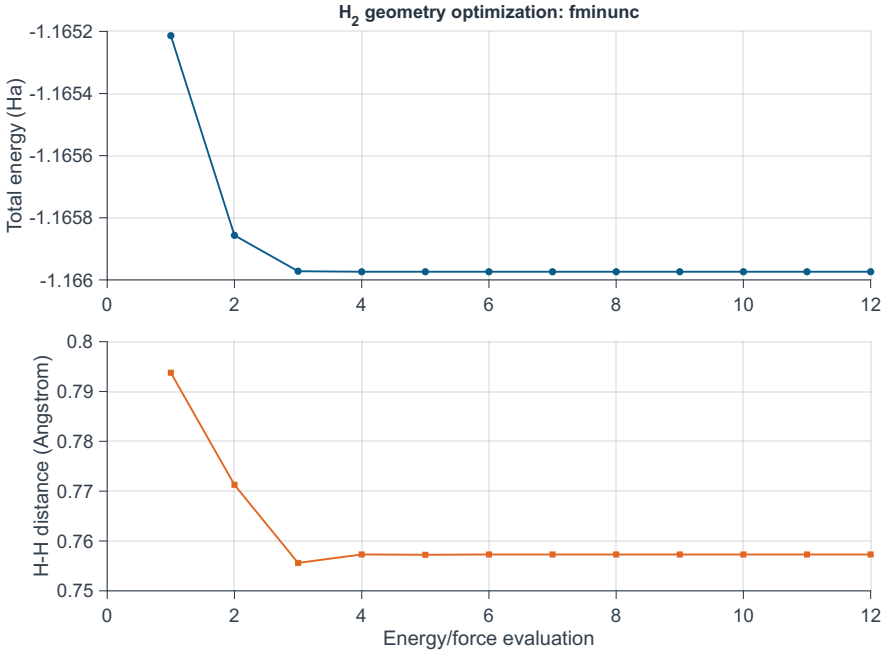



**Figure 10.4** Hydrogen energy and bond length during optimization. The horizontal axis counts energy/force function evaluations, including trial geometries.

$-1.1659737582$ Ha, and the force norm approximately $5.55 \times 10^{-5}$ atomic units. Both optimization and the final SCF report convergence.

The bond is slightly shorter and the energy lower than initially. This result belongs to the specified box, functional, pseudopotentials, and cutoff; changing them can change the value.

Optimization records are stored with workflow results. The original model under Structure remains unchanged. Import `examples/h2_optimized.cif` to measure the final bond in the 3D viewer. The accompanying script also saves an optimized structure for further analysis.

## 10.10 Symmetry as an independent workflow

To inspect symmetry or a recommended path alone, open Symmetry in the two-atom primitive-cell example without performing a complete electronic calculation. This is useful after importing a crystal, building a supercell, or applying deformation.

Symmetry tolerances determine which small numerical differences in positions or lattice parameters are treated as equivalent. An overly strict tolerance can interpret rounding as reduced symmetry; an overly loose one can hide a real distortion. Begin with defaults for ideal tutorial silicon. Interpret experimental or optimized structures in light of coordinate precision.

Symmetry describes geometric relationships. Energetic stability requires electronic calculations and force checks. Read symmetry alongside optimization records when assessing a relaxed structure.

## 10.11 View the Brillouin zone

A Brillouin-zone plot places the one-dimensional band path in reciprocal space. A band plot's horizontal axis gives distance along the path but not its course through the reciprocal-space polyhedron. A 3D zone plot shows boundaries, symmetry points, and paths together. This exercise needs no SCF calculation.

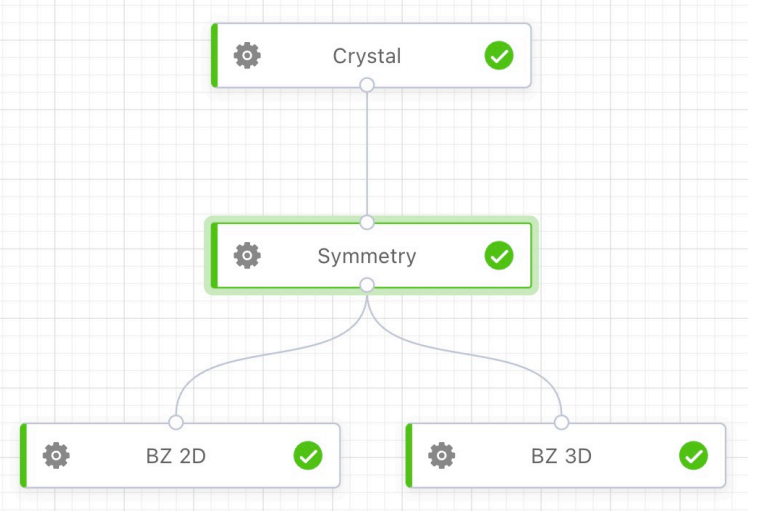



**Figure 10.5** The four-node Symmetry template. Crystal supplies the structure to Symmetry, which passes its analysis to BZ 2D and BZ 3D. All four nodes show green completion markers after a successful run.

1. Import `silicon_primitive.cif` and confirm a two-site primitive cell under Structure. Choose Home → Workflow → Import workflow from template → Symmetry.

2. Keep the template's four nodes: Crystal supplies the structure, Symmetry analyzes it, and branches lead to BZ 2D and BZ 3D. Check the inputs before changing anything.

3. Open Crystal with its gear. Confirm Build Molecule, type Crystal, and only `silicon_primitive` under Choose imported structures. Keep PBE, default pseudopotentials, 20 Ha, 0 K, and Nspin=1. These create a crystal object; this workflow has no SCF task.

4. Open Symmetry and confirm Symmetry & K-Path, SymmetryThreshold `1e-7`, and SymmetryPrecision `1e-5`; retain other defaults. Open BZ 3D and confirm Visualization Module → Brillouin Zone 3D. It has no further task-specific parameters.

5. Keep this workflow active and choose Home → Run. Both plotting branches produce results. Neither a preceding SCF nor deletion of BZ 2D is needed.

6. Explicitly open BrillouinZone3D under Results → Plots. Use this entry if the 2D plot appears first.

7. Rotate the plot to identify zone boundaries, reciprocal axes, and Γ, X, U, W, K, and L. Follow paths between front and rear faces, and compare the labels and segments with the same primitive-cell band tutorial.

8. Save the project and, in Online, download the `.ks`. To use the plot in a report, choose the toolbar's image export action, save the image separately, and retain the project.

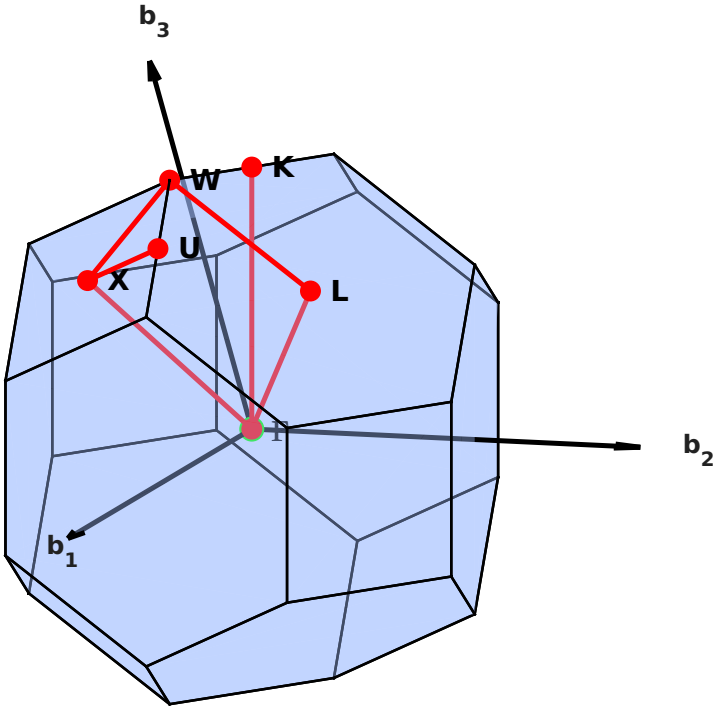



**Figure 10.6** The two-site silicon primitive cell's 3D Brillouin zone. The Symmetry template produces a blue boundary polyhedron, a red high-symmetry path, and black reciprocal-axis arrows. Rotate to inspect Γ, X, U, W, K, L, and their connections. No SCF calculation is required.

Figure 10.6 shows the 3D zone. The supplied `Si-primitive-BZ-Online-050.pdf` is a vector figure retaining path lines, point labels, and reciprocal axes for enlargement and reports.

BZ 2D uses Brillouin Zone 2D, plotting plane information from the first two reciprocal vectors. Use it in a separate workflow for a 2D model with well-defined in-plane vectors. It does not convert an arbitrary oblique section of a 3D crystal into a 2D material. Focus on the 3D view for silicon here, and use a suitably oriented 2D lattice for later 2D exercises. Cell, path, and symmetry choices for both plotting tasks are configured upstream.

Reopen the saved project and check the figure, structure, and path before using its views in a report.

## 10.12 Plot band-energy surfaces

Path bands sample line segments; an energy surface samples a reciprocal-space plane and uses height for electronic energy. This helps reveal directional variation near a band edge. Here, "3D" means two sampling coordinates and one energy coordinate, not a full 3D $k$-point volume. It also differs from the real-space volumetric data in Chapter 11.

Prepare a separate project using `silicon_primitive.cif`. Keep Crystal, PBE, default, 20 Ha, and the primitive-cell SCF grid; set 4 extra bands to include unoccupied states. This exercise samples a small region and uses a non-spin-polarized insulating system so that the highest occupied and lowest unoccupied bands are easy to identify.

1. Save a copy of the SCF project and retain Build Molecule → SCF.
2. Append NSCF, Band Calculation 3D, and Band Structure 3D. The middle task is Postprocessing and the last Visualization.
3. Keep the tutorial NSCF options. Enter the center, plane, and grid below in Band Calculation 3D.
4. Save and run. The workflow first solves the ground state, then samples the plane non-self-consistently, so it can take longer than SCF alone.
5. If calculation completes, open EnergyBandStructure3D under Results. Rotate the two band-edge surfaces, inspect their relationship to the sampling plane, and save.

The five points in each direction span 0.0375 on either side of the center, including both endpoints. Plane Grid Spacing controls the extent in this task. Record the center, plane, point counts, and extent when comparing surfaces quantitatively.

The plotting task selects the highest occupied and lowest unoccupied bands and sets the run's Fermi energy to zero. These surfaces show band-edge dispersion. Axis ticks may be absent, so use sampling data for quantitative analysis and test grid and range convergence.

**Table 10.6** A small-grid band-surface example.

| Option | Input and meaning |
|---|---|
| Center KPoint | `[0 0 0]`, centered near Gamma. |
| Slice Plane | XY; alternatives are XZ and YZ. |
| Slice Plane Grid | `[5 5]`, five points per direction, 25 in total. |
| Plane Grid Spacing | 0.015, controlling this task's sampling extent. This is a reciprocal-space parameter, not an interatomic distance. |

## 10.13 Prepare an excited-state calculation

SCF gives ground-state electronic structure. Excitations in response to a small perturbation require a different calculation. The TDDFT task prepares Casida-method inputs from converged ground-state electronic states with enough unoccupied orbitals. Replacing the last SCF task with TDDFT while retaining occupied orbitals only does not provide the required excitation space.

### Retain unoccupied hydrogen orbitals

Start from Chapter 5's `h2.cif` in a separate project, with Molecule, PBE, default, 20 Ha, and Nspin=1. Set Extra bands to 4 in Build Molecule's advanced options, then run SCF. The neutral hydrogen molecule has one doubly occupied spatial orbital, and four extra bands provide the later unoccupied-orbital selection. They do not change atom count or charge.

Add a Computation TDDFT node after SCF. Select it, choose Configure Node, and inspect Options and Advanced Options.

Explicitly enter nvbands=1 and ncbands=4. The solver requires Nspin=1. Check ground-state extra bands together with any changes in occupied bands, unoccupied bands, or root count. To retain a degenerate group of transitions, the program may increase the final number of roots. Use the actual count in logs and output arrays for analysis.

**Table 10.7** Preparatory hydrogen TDDFT settings.

| Field | Choice and meaning |
|---|---|
| TDDFTMethod | casida, the method provided by this task. |
| DiagMethod | direct, direct diagonalization. |
| nvbands | 1 occupied orbital. |
| ncbands | 4 unoccupied orbitals; cannot exceed those obtained in the ground-state calculation. |
| nroots | 3 requested excited states; no greater than $nvbands \times ncbands$. |
| Use TDA | Selected for the Tamm–Dancoff approximation. |
| Use IPA / Use RPA | Both cleared here; use separately to compare response approximations. |

### How to solve TDDFT

A parameter-passing issue in the graphical TDDFT node prevents it from completing this excited-state solve. Prepare the ground state and excitation space above, then pass explicit name/value arguments in full MATLAB® as shown below to obtain excitation energies and transition vectors. The associated message is `Wrong input for tddft_input`.

### Pass explicit parameters in full MATLAB®

The following code assumes an initialized KSSOLV numerical environment in full MATLAB®. It reruns the ground state with four extra bands, then passes Casida arguments explicitly. Inspect ground-state convergence, excitation energies, and transition vectors afterward.

```
pseudopotential.PpType('default');
molecule = example.structure.H2();
molecule.ecut = 20;
molecule.funct = 'PBE';
molecule.extranbnd = 4;
scfOptions = options.SCF();
scfOptions.enableGPU = false;
scfOptions.enableParallelPool = false;
[molecule, H, X, scfInfo] = scf(molecule, scfOptions);
if ~scfInfo.converge
    error('Ground-state SCF did not converge.');
end
td = tddft_setup(molecule, scfOptions, H, X, ...
    'method', 'casida', 'diag', 'direct', ...
    'nvbands', 1, 'ncbands', 4, 'nroots', 3, ...
    'istda', true, 'isipa', false, 'isrpa', false);
[excitationHa, transitionVectors] = ...
    tddft_casida_direct(td, molecule);
save('h2-tddft.mat', 'excitationHa', ...
    'transitionVectors', 'scfInfo', 'td');
```

Excitation energies are returned in Ha; use the manual's conversion for eV. Transition vectors describe orbital combinations in an excitation. They are not directly an absorption spectrum with intensities and broadening. Interpret excitations with the ground-state conditions, selected bands, response approximation, and original outputs.

## 10.14 Make workflows reproducible and comparable

### Give each workflow a clear purpose

While learning, keep separate "Silicon SCF," "Silicon bands," and "Silicon DOS" workflows. Each then has an obvious purpose and run target. Combine or export personal templates once you are comfortable.

After copying a workflow by saving and reimporting `.wf`, check its structure selection. Saved settings may retain associations with the original structure. Change Build Molecule first for a new system, then inspect its model parameters. A renamed workflow still calculates its selected input.

### Document one change at a time

Short names such as “Silicon DOS, 4×4×4” and “Silicon DOS, 8×8×8” distinguish comparisons. Record the SCF grid, extra bands, energy range, and step in result notes. This keeps figures recognizable without losing reproducibility.

An old plot does not become a result for new settings when a node is edited. Rerun and check which new datasets and figures belong to the latest calculation. Retain useful comparisons and remove redundant or abandoned trials so that the project remains readable.

### Final workflow checks

Reopen the saved project and check the structure, node parameters, and results in turn. Confirm that the calculation object and method match the description and that results can still be read. For bands and DOS, retain the path or grid and the energy reference too.

Chapter 11

# Viewing volumetric data

An atomic structure tells you where the atoms are; volumetric data describes how numerical values vary throughout space. Charge density, magnetization density, and other scalar fields can be viewed as isosurfaces, slices, or direct volume renderings. Using CHGCAR, Cube, and XSF examples, this chapter explains how to select channels, interpret units, compare views, and export figures and data for reports or further analysis.

## 11.1 Understand the data

### What a volumetric dataset represents

A three-dimensional scalar field is a set of values on a spatial grid. Each grid point has a position and a scalar value; the viewer interpolates between neighboring points to produce a continuous display. The grid dimensions specify the number of samples in each direction. The voxel vectors, origin, and cell determine their positions in real space.

A grid need not be orthogonal or aligned with the Cartesian axes. In a skewed cell or a dataset from another program, slices may therefore be tilted. Check the file's source, dimensions, and units before relating the field to the atomic structure.

One file can contain several channels. For example, a spin-polarized CHGCAR contains both total charge density and magnetization information, and a Cube file can contain multiple datasets. Changing the channel changes the scalar field being displayed, rather than merely its colors.

Cube and XSF values are not universally interpreted as electron density; the interface may show `arbitrary` as the unit. This means that the viewer retains the source values without assigning the same physical unit to ev-

**Table 11.1** Common volumetric formats and their interpretation in the viewer.

| File | Contents and interpretation |
|---|---|
| CHGCAR, CHG | VASP scalar grids and atomic structures. Charge density is normalized by the cell volume and displayed in $Å^{-3}$; spin-polarized files provide additional channels. |
| Cube, CUB | An origin, voxel vectors, atoms, and one or more datasets. The scalar's physical meaning depends on the program and calculation that produced it. |
| XSF | Labeled two- or three-dimensional datagrids, optionally with atomic structures. Identify each grid block and channel by its label. |

ery file. Even if a field resembles a charge distribution, interpret its values according to the calculation that produced it.

### Spin channels in CHGCAR

A non-spin-polarized CHGCAR normally uses `total`. For collinear spin data, `diff` is the magnetization density associated with the spin difference. The viewer also provides derived `up` and `down` channels:

$$\rho_{\uparrow} = \frac{\rho_{\text{total}} + \rho_{\text{diff}}}{2}, \qquad \rho_{\downarrow} = \frac{\rho_{\text{total}} - \rho_{\text{diff}}}{2}.$$

Negative `diff` values indicate the sign of the spin difference, not a negative total electron density. Noncollinear data can also contain the three magnetization components and their magnitude. Distinguish signed components from a vector magnitude when interpreting them.

## 11.2 Import volumetric data

1. Open the Project Browser from the sidebar, double-click the top-level Volume category, and select a volumetric file. Start with `CHGCAR.nospin.gz`. If the project tree is hidden, select Project Browser from the sidebar's Open Panels gallery.

2. Wait for the initial view and summary. Check the filename, Grid dimensions, and Units to confirm that the intended dataset is open.
3. Drag with the left mouse button to rotate, scroll to zoom, and drag with the right button to pan. Press Space to center the view and fit the model.
4. Open Viewer settings on the volume view's own toolbar and inspect Channel and Display mode. Identify the channel and mode before changing the appearance.
5. If the file includes an atomic structure, retain the cell and atom overlays to see where the field lies relative to the atoms.

A large dataset may initially appear at reduced display resolution before the full data loads. The status area can report loading, construction, or a fallback mode. Wait for completion before recording the grid or preparing a final figure, so that an initial preview is not mistaken for the complete dataset.

The toolbar also provides automatic rotation, lattice- and reciprocal-axis views, structure information, image export, scene-description export, and full screen. Press I to hide or restore supporting elements such as the summary and distribution plot. Restore them when recording values and units.

Imported volumes appear below Volume; double-click a specific item to reopen it. The top-level category imports a new file, while child items open existing data. Sidebar panels manage the project; settings inside the volume view control the image. Use the latter for atom scale, channels, and slices.

Figure 11.1 shows the total charge density from the supplied non-spin-polarized file. The pale blue isosurface is displayed with the cell and atoms, making its position relative to the structure easier to judge. The next section explains how to check the file, channel, and threshold.

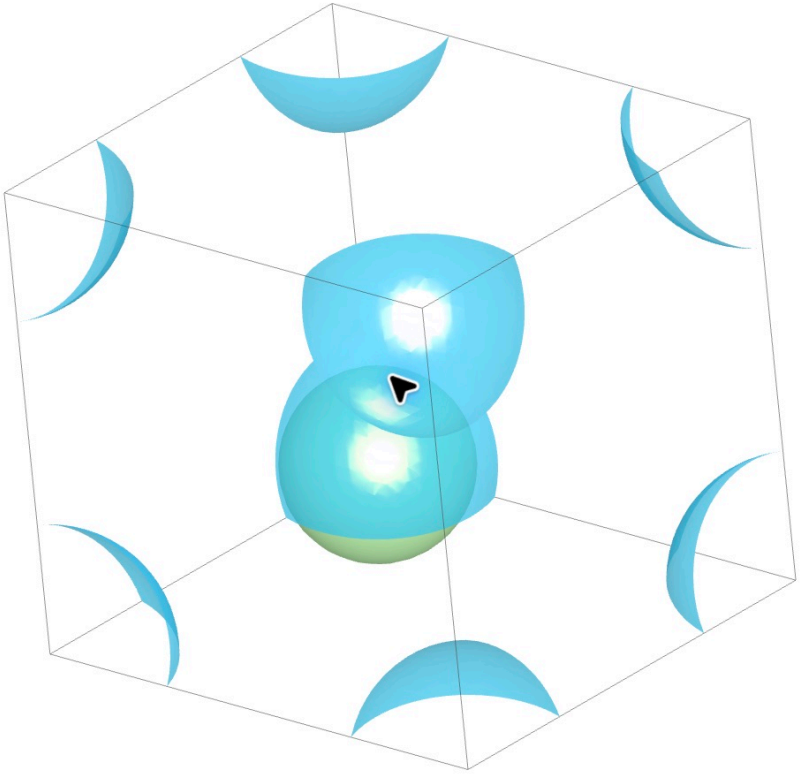


**Figure 11.1** Total charge density from `CHGCAR.nospin.gz`. The grid is $32 \times 32 \times 32$ and the channel is Total charge density. The positive threshold is $0.04000\,\text{Å}^{-3}$, the negative surface is disabled, and opacity is 0.72.

## 11.3 Isosurfaces: shapes at a fixed value

### What the threshold controls

An isosurface connects positions with the same scalar value. Raising a positive threshold generally focuses the display on higher-valued regions; lowering it can enlarge or connect surfaces. If no surface appears, first check whether the threshold lies within the data range and whether the channel contains values of the requested sign.

For a scalar field $s(\mathbf{r})$ and threshold $\tau$, the isosurface is

$$S_\tau = \{\, \mathbf{r} \mid s(\mathbf{r}) = \tau \,\}. \tag{11.1}$$

The value is fixed, while the data determines the surface's position and shape. A slice instead fixes a spatial plane and colors it according to the values of $s$ on that plane. The two views answer different questions, as illustrated in Figure 11.2. Direct volume rendering additionally combines information through a volume along each line of sight.

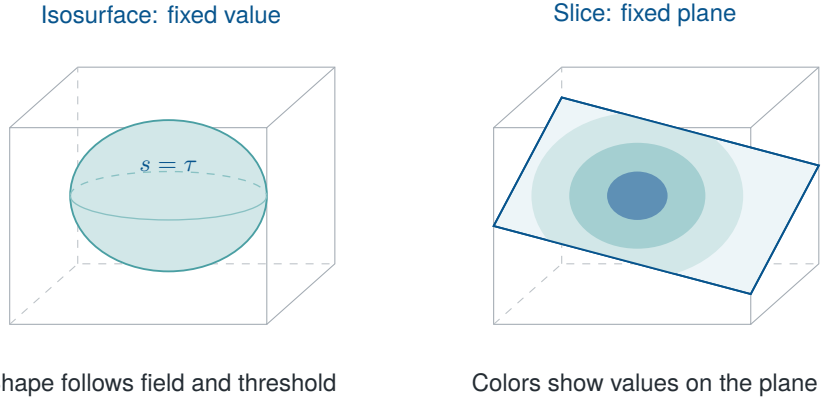



**Figure 11.2** Two views of a scalar field (conceptual schematic). An isosurface connects equal values; a slice colors one plane by its field values. Geometry and colors are illustrative.

Select Isosurfaces under Display mode. Positive and negative surfaces have separate switches and thresholds. Total charge density usually calls for the positive surface; signed fields such as magnetization differences or orbitals may need both. Opacity changes visibility without changing the threshold or source data.

Isovalue units offers three threshold conventions: absolute values, standard deviations, and percentiles. Absolute values suit explicit physical thresholds. Standard deviations locate a value relative to the mean and distribution width; percentiles select a position in the value distribution. Use consistent units and record each threshold when comparing absolute distributions between models.

Absolute value means entering $\tau$ directly in the selected display units; it does not turn negative values positive. With Standard deviations (σ), the input is a dimensionless multiple $z$ relative to the mean, giving

$$\tau = \mu + z\sigma. \tag{11.2}$$

Here $\mu$ and $\sigma$ are the mean and standard deviation of the current channel's grid values. Thus $z = 0$ selects the mean and $z = 1$ selects one standard deviation above it, rather than a field value of 1. If the mean is nonzero, the threshold cannot be shortened to $z\sigma$.

**Table 11.2** Quantities used for thresholds and color ranges.

| Quantity | Meaning in the interface |
|---|---|
| $s$, $\tau$ | Current channel value and isosurface threshold; absolute input uses the channel's display units |
| $\mu$, $\sigma$ | Channel mean and standard deviation, in the same units as the values |
| $z$ | Dimensionless number of standard deviations, used to convert a statistical position to a threshold |
| Color limits | The numerical interval covered by the colormap; these do not rewrite field values or CSV data |

Percentile selects a threshold by ranking the grid samples by value. The 90th percentile, for example, identifies a position in that distribution; it does not mean that the surface encloses 90% of the electrons or cell volume. Changing channels changes the mean, standard deviation, and percentiles. Record the threshold convention as well as its value when comparing figures.

### Example: total charge density

1. Open `CHGCAR.nospin.gz`. In the summary, check Grid $32\times32\times32$, Channel Total charge density, and units $\text{Å}^{-3}$.
2. Select Isosurfaces and set Isovalue units to Absolute value. Enable Positive and enter `0.04000` in its Threshold field.

3. Disable Negative and set Opacity to `0.72`. This keeps the surface outline visible while allowing some interior atoms to show through.

4. Retain Atoms and the cell. Temporarily disable Bonds, Depth Cueing, Boundary image atoms, and One-hop bonded atoms to focus on the field.

5. Rotate the view and compare surfaces near the center and boundaries. Record the file, channel, units, threshold, and opacity before exporting a figure.

Figure 11.3 enlarges the summary and settings separately. Use the left panel to identify the dataset and the right panel to identify its display conditions. Grid size, density threshold, and opacity are different kinds of parameters. The value 0.04000 applies to this example; inspect the range before choosing a threshold for another file.

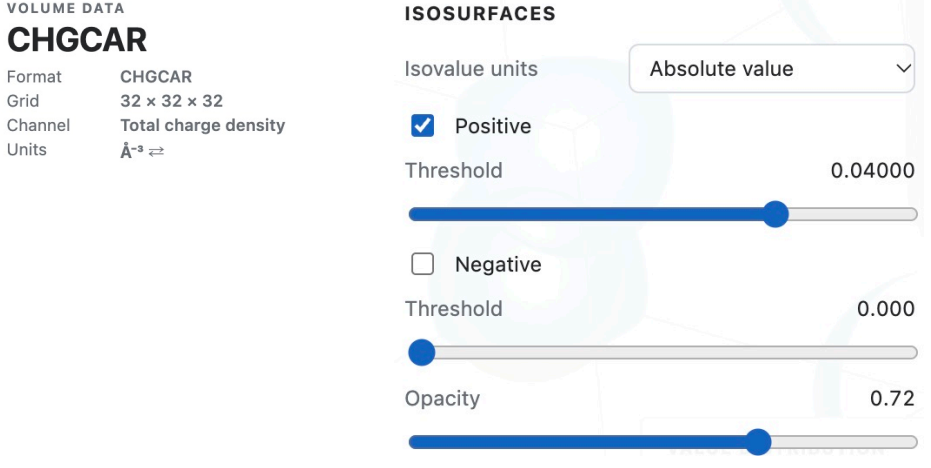



**Figure 11.3** Summary and threshold settings for the same total-density view. The summary lists the grid, channel, and units. The settings use an absolute threshold of 0.04000, no negative surface, and opacity 0.72.

### Example: compare spin channels

Open `CHGCAR.spin.gz`, inspect the total density, then switch to Magnetization density, the `diff` channel. The summary now shows a $48 \times 48 \times 48$

grid. The channel's distribution ranges approximately from $-0.001142$ to $-0.0001356\ \text{Å}^{-3}$, entirely below zero.

1. Select Magnetization density, Isosurfaces, and Absolute value. Check the channel and units in the summary.
2. Enable Negative and enter `-0.0006000` in its Threshold field. This lies within the example's range and should produce a red negative isosurface.
3. Set Opacity to `0.72`. Keep atoms and the cell, and retain the previous settings that hide bonds and surrounding images. Rotate the view to inspect surfaces inside the cell and near its boundaries.
4. You can leave Positive enabled at threshold 0, as in Figure 11.5. This file contains no zero or positive values, so that switch adds no positive surface. The green spheres are the atom overlay, controlled separately from surface signs.
5. Switch to `up` and `down` to inspect the channels derived from total and difference density. Recheck the range and threshold after each change and record the settings for each channel separately.

Negative magnetization density means that the difference is negative under the current spin convention, not that total charge density is negative. This example illustrates how the negative-surface switch and threshold work. Showing both signs requires a dataset with the corresponding value range.

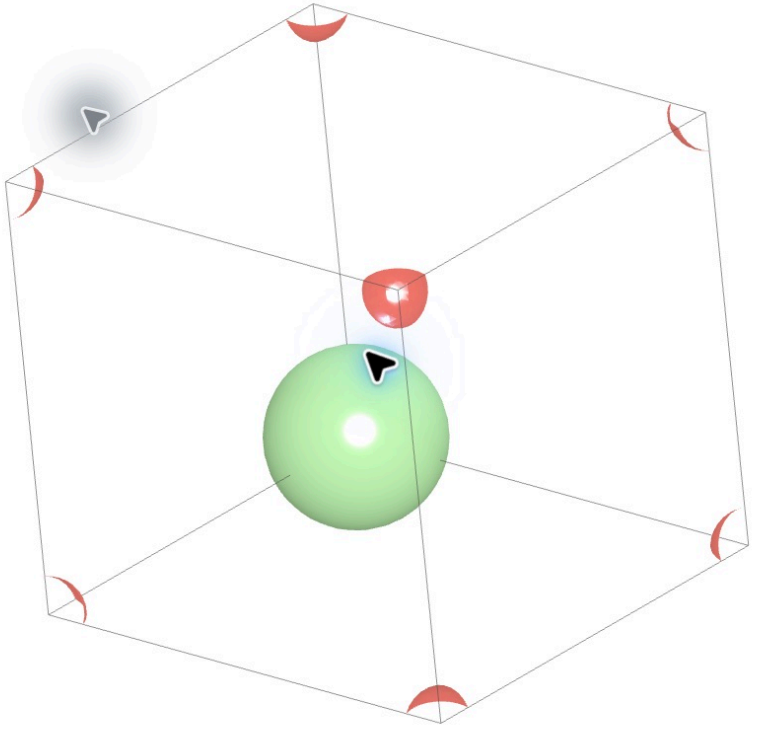


**Figure 11.4** The Magnetization density channel in `CHGCAR.spin.gz`. The grid is $48 \times 48 \times 48$; the red surface has threshold $-0.0006000\,\text{Å}^{-3}$ and opacity 0.72. Green spheres are the overlaid atoms.

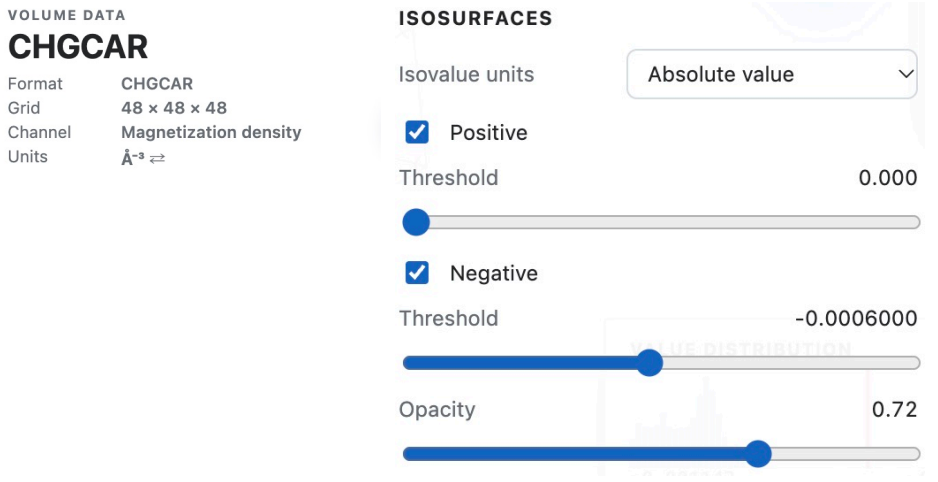



**Figure 11.5** Magnetization-density summary and thresholds. The positive surface remains enabled at 0 and the negative threshold is $-0.0006000\,\text{Å}^{-3}$. Since every value is negative, only the negative surface appears.

### Smoothing and periodic boundaries

Smooth vertex normals improves isosurface lighting by making shading more continuous between triangles. It does not increase the source grid resolution. For periodically sampled data, Wrap periodic boundaries connects surfaces across cell boundaries. Enable it only when appropriate to the file's periodicity.

If a surface appears cut off at a boundary, check the cell limits and periodic settings before attributing it to the grid. A higher-quality export sharpens text and edges but cannot compensate for coarse source sampling.

## 11.4 Miller slices: inspect internal values

### One index triplet defines one plane

Isosurfaces emphasize shapes; slices show continuous variation across a section. After selecting Miller plane slice, I, J, and K form one Miller-index triplet defining a single plane through the center of the volumetric data. They cannot all be zero.

For example, $(0, 0, 1)$ and $(1, 0, 0)$ select different orientations, while $(1, 1, 0)$ gives an oblique section. These parameters orient one plane, rather than three independently movable slices. In a nonorthogonal grid, the actual voxel vectors determine the plane's spatial orientation. Do not interpret I, J, and K as Cartesian $x$, $y$, and $z$ sliders.

For a three-dimensional grid with multiple samples in every direction, let $u, v, w$ be normalized positions from the first to last sample along each grid direction, ranging from 0 to 1. Writing the interface's I, J, and K as $h, k, l$, the central plane satisfies

$$h\left(u - \frac{1}{2}\right) + k\left(v - \frac{1}{2}\right) + l\left(w - \frac{1}{2}\right) = 0. \tag{11.3}$$

Thus $(0, 0, 1)$ gives $w = 1/2$, and $(1, 1, 0)$ gives $u + v = 1$; both pass through the grid center. These normalized positions span the sampled range and are not universally equivalent to fractional coordinates of a complete cell. The viewer maps the plane into real space using the voxel vectors, preserving the orientation of a skewed grid. Changing the integer triplet rotates the central plane rather than translating it along an axis.

Example: the (110) slice of a Cube file

1. Open `elec.cube.gz`. Check grid $23 \times 23 \times 24$, channel Dataset 1, and units `arbitrary` in the summary.
2. Select Miller plane slice and set I, J, and K to 1, 1, and 0.
3. Choose Cool–warm under Colormap and Linear under Interpolation. Leave Range minimum at `0.0034185` and change Range maximum to `0.2`.
4. Retain atoms and the cell, set Atom scale to `0.22`, and hide bonds, coordination polyhedra, depth cueing, and surrounding images so that the tilted plane is clear.
5. Close the settings panel and rotate the model to inspect where the plane crosses the cell and how atoms lie relative to it. To compare sampling, switch to Nearest to see discrete blocks, then return to Linear.

6. Record the indices, colormap, color range, and interpolation, then export PNG or CSV through Slice export. The export section below explains their contents.

Figure 11.6 uses a relatively low upper color limit to emphasize variation in lower-valued regions. Values above 0.2 use the colormap's high-end color but remain unchanged in the source data and CSV. Red and blue denote the ends of the selected range; color alone does not establish a value's sign.

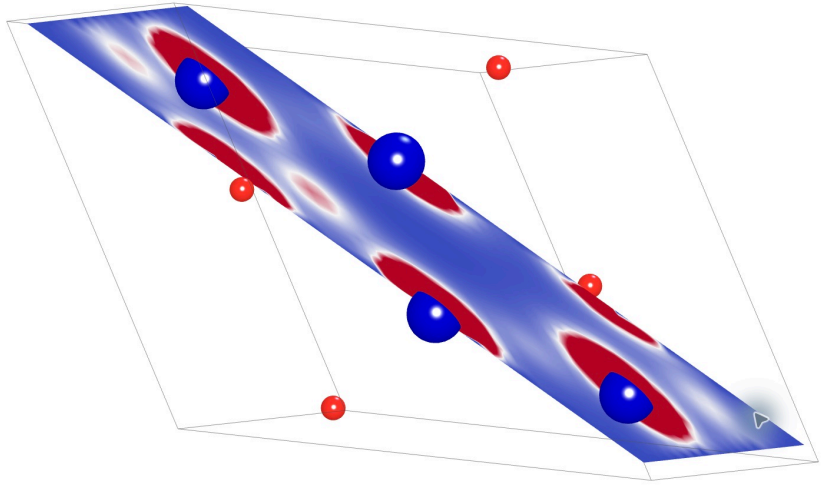


**Figure 11.6** The (110) slice of `elec.cube.gz`. Grid $23 \times 23 \times 24$, channel Dataset 1, units `arbitrary`; Cool–warm, linear interpolation, and a color range of 0.0034185 to 0.2.

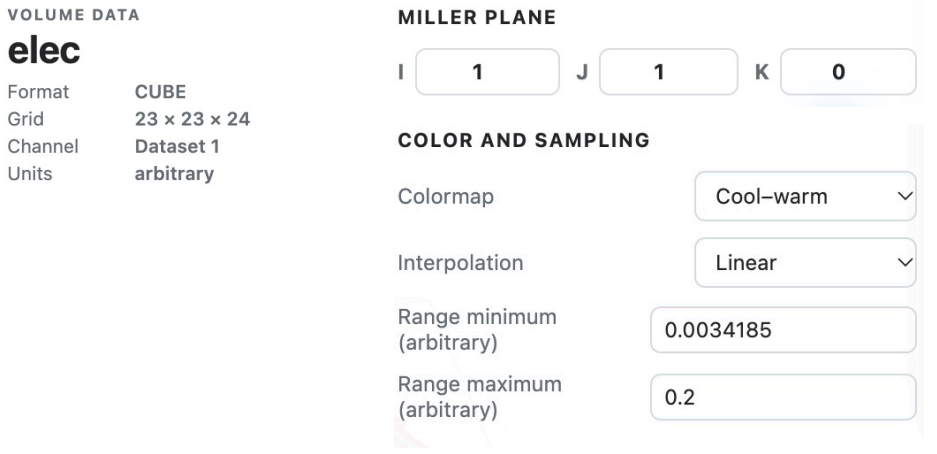



**Figure 11.7** Slice summary and settings. The left panel identifies the file, grid, and units. On the right, the upper panel specifies one Miller-index triplet and the lower panel sets colors, interpolation, and range. The upper color limit of 0.2 affects display only.

You can repeat the exercise with `datagrid_3d.xsf`, identifying channels by their labels first. Compare both the grid orientation and the meaning of its scalar values. The same (110) indices do not imply that two files have identical dimensions or numerical ranges.

### Color ranges and the distribution plot

Range minimum and Range maximum map numerical values to the ends of the colormap. A wide range can hide local variation; a narrower one can emphasize the interval of interest. Out-of-range values still exist but use the endpoint colors.

Value Distribution shows the current channel's value distribution. In isosurface mode, clicking it sets the corresponding positive or negative threshold. In slice or direct-volume mode, clicking adjusts the nearer color limit; drag the markers for finer adjustment. This is a display-range tool, not an energy spectrum or a profile along a spatial path.

Cool–warm often helps distinguish the two signs of a signed field. A sequential colormap is generally easier to read for nonnegative data. Use

consistent colormaps and ranges when comparing models, so that color differences have the same meaning.

## 11.5 Direct volume rendering and structure overlays

Direct volume rendering accumulates scalar-field colors and opacity along each viewing ray, revealing variation through a volume. Choose Direct volume (GPU) under Display mode, then adjust opacity, gradient opacity, and sampling quality. Fast suits interactive exploration, Balanced suits general viewing, and High reveals finer detail when the device supports it.

Clipping limits along I, J, and K are fractions from 0 to 1. They hide part of the box to expose the interior. Reducing one direction from 0–1 to 0.25–0.75, for example, isolates its middle region. Clipping changes the displayed extent without deleting data.

### Example: inspect the central volume along a grid direction

Continue with `elec.cube.gz` to compare slices and volume rendering of the same data. A slice displays values on one plane; volume rendering combines color and opacity along the line of sight. Their brightness need not match point by point.

1. Confirm `elec.cube.gz`, Dataset 1, grid $23 \times 23 \times 24$, and units `arbitrary`. Change Display mode to Direct volume (GPU).
2. Retain Cool–warm, Linear, and the range 0.0034185 to 0.2. Keep colors consistent before comparing the modes.
3. Set Sampling quality to Balanced, Opacity to `0.72`, and Gradient opacity to `0.35`. Gradient opacity controls how local variation contributes to transparency and can be adjusted separately from overall opacity.
4. Set the lower and upper I clipping limits to `0.25` and `0.75`. Leave J clipping and K clipping at 0 and 1. Only the middle half along I is shown; the other directions remain complete.

5. Retain atoms and the cell; hide bonds, polyhedra, and surrounding images. Close settings and press I to hide the summary and distribution plot. Rotate to inspect continuous internal variation, and press I again whenever you need to check the file information.

Figure 11.8 shows the result; Figure 11.9 shows the settings. Here I, J, and K specify clipping fractions along grid directions. This differs from the integer triplet defining a Miller slice and need not align with global Cartesian axes. Restore each clipping interval to 0–1 to display the whole volume. The source grid remains $23 \times 23 \times 24$.

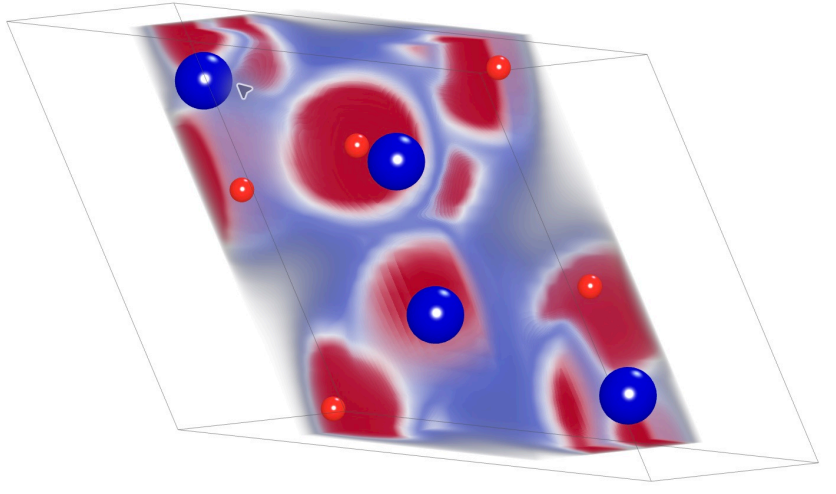


**Figure 11.8** Direct rendering of `elec.cube.gz` using Direct volume (GPU). The Dataset 1 color range is 0.0034185 to 0.2 and I spans 0.25 to 0.75. Atoms and the full cell outline provide spatial references.

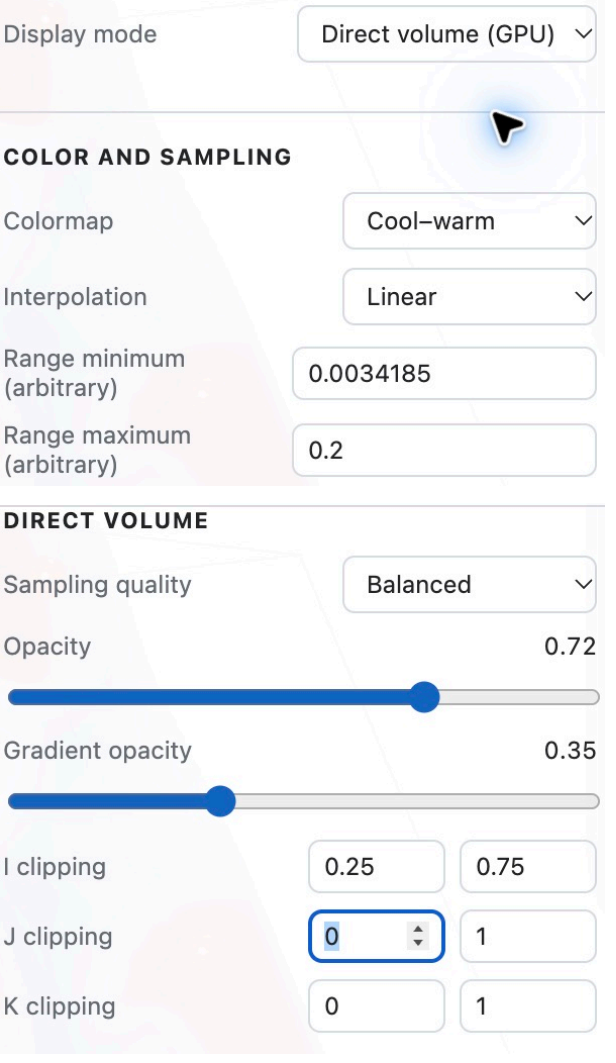



**Figure 11.9** Direct-volume settings. Above: mode, colormap, interpolation, and color range. Below: Balanced sampling, opacity 0.72, gradient opacity 0.35, and clipping fractions along all three directions.

### Locate the field using structure overlays

When the file includes an atomic structure, Visibility controls atoms, bonds, the cell, coordination polyhedra, axes, periodic image atoms, magnetic mo-

ments, and other objects. Begin with a few useful spatial references, then add overlays individually to keep the field clear.

Lighting & surface controls ambient and directional light, metalness, roughness, brightness, and contrast for the atom overlay. These affect appearance, not the scalar values. Keep them consistent across comparison figures, and state the scientifically meaningful channel, threshold, and units in each caption.

## 11.6 Export volumetric results

### Sample values in the view

Double-click the volume to display a sample value and grid position. The point probe uses trilinear interpolation, so its value need not equal an original grid sample. For precise comparisons, record the channel and sampling position and use the numerical value. Nearest-neighbor and linear slice options govern slice display and export separately; they do not change point-probe interpolation.

Selecting an overlaid atom shows its Cartesian and fractional coordinates in the volume view's atom-information area. Copy buttons retain more precision than the shortened panel display. Atomic coordinates locate an atom; scalar sampling reads the field at a position. These are different kinds of information.

For convertible density units, Density units switches between $\text{Å}^{-3}$ and $\text{bohr}^{-3}$. The displayed number changes while the physical density remains the same. Convert to common units before comparing figures or tabulated values.

### Export the current image

The image export menu offers PNG, JPEG, TIFF, SVG, and vector and raster PDF. PNG and TIFF preserve clear raster images; JPEG provides compact files for sharing; SVG and PDF suit document preparation. Stop automatic

rotation, choose the view, and check the atoms, cell, and overlays before exporting.

In the text or caption, record the file source, channel, threshold or color range, units, and relevant orientation. Retain the source data as well.

### Export isosurfaces and slices

In isosurface mode, Isosurface export offers glTF, GLB, PLY, and STL for transferring surface geometry to other 3D tools. The formats preserve different information. Check dimensions and appearance after import elsewhere; coordinates are in Å.

In slice mode, Slice export offers PNG and CSV. PNG saves the colored plane; CSV supports further plotting or numerical analysis. The CSV begins with grid positions `i`, `j`, and `k`, followed by Cartesian positions in Å: `x_angstrom`, `y_angstrom`, and `z_angstrom`. The final `value` column contains source values for the current channel. Samples outside the volume boundary may be invalid; restrict analysis to the valid range.

Reopen the PNG to check orientation, colors, and the region of interest, then inspect the CSV header for coordinate units and the value column. Before plotting matching slices from two models together, ensure that the spatial plane, grid extent, and channel agree. Identical Miller indices specify the same type of direction, but changing the cell can change the plane's real-space size and position.

The example exports `elec.total.slice-1_1_0.csv` and a PNG with the same base name. The CSV contains 1568 sampling positions: 778 have finite values and 790 lie outside the volume boundary, with `value` recorded as `NaN`. NaN is not zero; filter valid values before plotting or calculating statistics.

The finite slice values range approximately from 0.003525 to 2.3995, and 147 exceed the display limit of 0.2. They illustrate the distinction between color settings and source values. The slice PNG is $384 \times 768$ pixels and shows the plane's color distribution; Figure 11.6 additionally shows its orientation inside the cell.

### Display units and CSV units

CSV coordinates are exported in Å, while the value column retains the channel's internal units. For CHGCAR, density values are in $\text{Å}^{-3}$ even when the interface displays $\text{bohr}^{-3}$. Record Cube and XSF scalar units according to the source file documentation.

Export scene JSON saves a scene description, including information about the grid, channel, and structure. It does not include the complete voxel array and cannot replace the original field file. Record the camera position and temporary display settings separately. To reproduce an image, retain the source data and note its threshold, color range, and framing.

### Organize comparable results

Give matching names to data and figures, including the system, channel, and slice where useful. Keep separate images for CHGCAR `total` and `diff`; calling both simply a density plot obscures their different meanings. State positive- and negative-surface colors and thresholds in captions, or the color range for slices. Readers should be able to interpret surfaces and colors without opening the original file.

For a research report, keep a working screenshot showing the full summary and settings as well as a clean exported figure. The former helps you check the result; the latter suits presentation. Choose the view according to the feature you want to explain: isosurfaces emphasize shape and connectivity, slices show local variation, and volume rendering conveys spatial depth.

## 11.7 If the image is not what you expected

For an empty isosurface, check the channel, sign switches, and threshold range. For a nearly uniform slice color, check whether the color range is too broad. If atoms obscure the field, reduce their scale or temporarily hide bonds and polyhedra. Press Space to recenter a model lost during rotation.

Direct volume rendering needs appropriate graphics support. Without WebGL2, the viewer can fall back to CPU slices while retaining slice PNG/CSV export. Insufficient 3D texture capacity can also trigger a slice fallback. Read the stated reason to select a mode supported by the current device. Two-dimensional XSF data is displayed as a slice.

Interpolation, smoothing, and larger exported images improve display only. If your research requires finer spatial information, increase the source grid resolution in the calculation and check convergence again. Only then are the visible details supported by additional data.

Chapter 12

# Interpreting calculation results

After a calculation, use its figures, data, and workflow together to interpret the results. Figures reveal trends, data records establish their origin, and the saved workflow specifies the calculation conditions. This chapter explains how to read these materials and turn them into conclusions you can support in a report or research notebook.

The hydrogen molecule and silicon examples illustrate single-point energies, atomic forces, geometry optimization, band structures, and densities of states. For your own results, ask three questions in order: Did the calculation finish? How is the plotted quantity defined? What conclusion can it support?

## 12.1 Find results in the project

### Where results are stored

Open the Project Browser from the left sidebar and expand Results. Datasets holds datasets; Plots holds figures. A dataset records information from a run, while a figure presents part of it in a readable form. Band, DOS, and convergence plots are all graphical results, but represent different quantities.

Double-click a figure generated by the current version to reopen it. If its document is already open, the application activates it. Figure objects from older projects may not survive a version change; the procedures below redraw them from saved numerical data. Before comparing runs, identify the figure and its associated run record. A result retained in a project does not automatically update when you edit a node.

For example, changing Ecut from 20 to 30 Ha leaves the existing convergence plot associated with the 20 Ha run. Run the workflow again to

generate results for the new cutoff. Give each trial figure a descriptive name promptly rather than accumulating indistinguishable defaults.

### Read the record before the figure

A figure can expose a problem quickly, but can also conceal inconsistent conditions. Check the structure, functional, pseudopotentials, cutoff, and sampling first, especially when a project contains several structures. For optimization, distinguish initial and final geometries.

A report need not reproduce the log line by line, but retain the original record. State key conditions in captions and keep complete inputs in an appendix or accompanying files. This keeps the text readable while allowing important numbers to be checked.

## 12.2 Read a dataset

Select a run under Results → Datasets and check its name, time, and description in the Information Browser. Labels usually begin with Run, followed by the workflow name used at execution. Distinguish similar records by their input conditions, not just their order in the list. The current Information Browser does not expand all numerical values in these calculation datasets; inspect the underlying contents in full MATLAB® or the appropriate command workspace.

The table below identifies common names encountered after exporting a dataset, linking each array to its producing task. The next section explains how to export a selected dataset to the workspace.

Not every run contains all these entries. SCF alone does not produce DOS, and symmetry analysis alone does not produce electronic energies. Electronic states and other data may also be stored as objects whose full details are not expanded in the Information Browser. Use figures and logs for routine inspection, and full MATLAB® or the supplied scripts for further array processing.

**Table 12.1** Common result-data names.

| Name | What to check first |
|---|---|
| info | SCF status, total energy, and energy/error histories; after optimization, it may describe the last ionic step. |
| relaxInfo | Geometry optimization history; distinguish it from the initial structure's single-point info. |
| symmetry | Primitive-cell and high-symmetry-path information for checking the band calculation. |
| bandProcessing | Path $k$ points and band arrays; interpret the horizontal axis using that path. |
| bandProcessing3D | $k$ points, energies, and plane names for reciprocal-space planes. |
| DOS | Energy samples and total density of states, rather than an atom-projected DOS. |
| TDDFT | Excitation energies Et and transition vectors Z; check completion and energy units first. |

Geometry optimization results are stored in the run record, while the Structure branch retains the original model. To inspect the final geometry in 3D, import the optimized structure file as in the hydrogen example and compare it with the optimization record.

### Export a selected project item to the workspace

Home → Variable → Export variable to MATLAB workspace exports the selected Project Browser item to the corresponding workspace. It transfers the project-item object rather than generating a CSV table; save files separately according to their intended use. The variable name is derived from the item's display name, with spaces removed and invalid variable-name characters adjusted.

1. In the toolbox running in full MATLAB®, select the required dataset and give it an easily recognized display name, such as `H2Run`.
2. Choose Home → Variable → Export variable to MATLAB workspace.

3. In MATLAB®'s own Command Window, use `whos` to check the variable, read its `data`, and inspect the available data names.

4. Check units, structure, and dimensions before reading arrays. Save a MAT file or results table as needed, and retain the original project.

Online places the selected object in the application's command workspace, which differs from full MATLAB®'s base workspace. Standalone users can inspect figures directly and retain results by saving projects and exporting images.

The following illustrates reading an already exported SCF dataset in full MATLAB®. Use the variable name actually created by your export.

```
runData = H2Run.data;
keys(runData)
scfInfo = runData('info');
scfInfo.Etot
scfInfo.converge
save('h2-run-data.mat', 'runData');
```

For a figure, structure, or record from another workflow type, inspect its data names before choosing fields.

## 12.3 Check electronic convergence

### The status is more direct than the curve's appearance

The convergence message at the end of an SCF log states whether the solver met its criteria. A flattening energy curve is useful evidence but cannot replace that check. Small unconverged changes can be almost invisible against a large negative total energy.

The hydrogen reference run finishes with a relative density error of approximately $8.85 \times 10^{-9}$; the eight-atom silicon cell finishes at approximately $7.35 \times 10^{-9}$. Both meet their specified electronic criteria. Your intermediate iterations need not match every point, but the final status should meet the intended conditions.

### Read a convergence plot with two vertical axes

The horizontal axis counts electronic iterations. The blue energy curve uses the left Energy (Ha) axis; the orange error curve uses the right Error axis. Their units and scales differ, so their intersections have no physical significance.

The error axis is logarithmic. A decrease from $10^{-3}$ to $10^{-6}$ is three orders of magnitude, not three units. Look first at the overall trend, then at any repeated oscillations, and finally at whether the stopping criterion was met.

In the hydrogen example, the energy approaches its final value before the density finishes converging. Different indicators need not stabilize together. For forces or small energy differences, use sufficiently strict electronic accuracy rather than stopping manually when the energy curve looks smooth.

### Three common convergence patterns

If the error keeps decreasing but the calculation reaches the iteration limit, increasing the limit may allow it to converge. If it oscillates between two or more levels, check the structure, occupations, and numerical settings. Nonfinite values or clearly abnormal energies from the outset call for checking atomic separations, the cell, and pseudopotential loading.

These patterns require different responses. A complete stretch of iteration history is more informative than "the run failed." Comparing with a small successful example also helps distinguish a new-system problem from an environment-wide issue.

## 12.4 Compare total energies

### Identify the system an energy belongs to

Total energy depends on the elements and atom count, geometry, functional, pseudopotentials, and numerical conditions. Absolute energies of different systems generally cannot be ranked directly. The eight-atom silicon cell having a more negative energy than a hydrogen molecule does not establish that

silicon is more stable: the two numbers describe different atomic and electronic systems.

For cells with the same composition but different sizes, normalize per atom or formula unit. Adsorption, reaction, and defect studies require energy differences constructed from the relevant reaction and reference states. Define the comparison before subtracting energies.

### Practice with the hydrogen molecule

With the same box, functional, pseudopotentials, and cutoff, the initial hydrogen geometry has total energy approximately $-1.1652136531$ Ha; the optimized geometry has approximately $-1.1659737582$ Ha. The lower value shows that the optimizer found a lower-energy geometry within this model.

The difference is approximately $-7.60 \times 10^{-4}$ Ha. It is the energy change from this starting geometry to the optimized structure, not the binding energy relative to two isolated hydrogen atoms. Binding energy requires suitable atomic reference states; an optimization's energy decrease is not a reaction energy.

Tabulate the structure name, bond length, total energy, final force, and convergence status. Such a table relates geometry to electronic results and explains more than an energy column alone.

**Table 12.2** Results from the hydrogen molecule reference run.

| Quantity | Reference value |
|---|---|
| Initial bond length | $1.50000000$ Bohr, approximately $0.79376582$ Å |
| Initial total energy | $-1.1652136531$ Ha |
| Optimized bond length | $1.43107436$ Bohr, approximately $0.75729194$ Å |
| Optimized total energy | $-1.1659737582$ Ha |
| Final force norm | Approximately $5.55 \times 10^{-5}$ atomic units |
| Conditions | PBE, built-in default pseudopotentials, 20 Ha, a 10 Bohr cubic box |

Report an appropriate number of digits. Many decimal places do not confer physical accuracy when the model and numerical inputs have not been thoroughly converged. These reference values help check the supplied example.

## 12.5 Check the optimized structure

### Forces and equilibrium positions

Atomic forces describe how energy changes with atomic positions. Moving in an appropriate direction along a force generally lowers the energy. For hydrogen, the bond-direction forces are easiest to inspect, and the symmetric arrangement normally gives corresponding forces on the two atoms.

A lower energy does not automatically mean that every atom is at equilibrium. Check the final forces and the optimizer's stopping reason. Reaching the step limit may leave optimization unfinished; sufficiently small forces provide stronger evidence for an equilibrium geometry within the chosen model.

### Inspect the final geometry

Open the optimized structure and first check that the elements and atom count are unchanged. Then measure the relevant bond lengths, angles, or lattice parameters. Overall translation and rotation do not change a molecule's internal geometry. Compare relative positions rather than whether an individual atom's absolute coordinates changed.

The Geometry Optimization tutorial relaxes atomic positions in a fixed cell. To study a lattice constant, choose an appropriate scheme explicitly, such as comparing energies at several fixed lattice parameters with consistent treatment of internal coordinates.

### What a local minimum means

An optimizer starts from the supplied geometry and searches nearby for lower energy. This is straightforward for a diatomic molecule, but a com-

plex molecule, adsorbate, or defect may have several locally stable configurations. One successful optimization does not exhaust every possible structure.

Optimize from several physically meaningful starting configurations and compare their final geometries and energies. Name each starting point clearly and keep its provenance, so that a selected lower-energy structure remains traceable.

## 12.6 Read a band structure

### Open the supplied project's bands

`silicon-band-primitive.ks` contains a two-atom silicon primitive cell, a band workflow, and reference numerical records. If an older figure reports a deleted object, redraw the saved band and DOS arrays using `show_reference_results.m`.

This procedure uses the toolbox interface in full MATLAB® and does not repeat the DFT calculation. In the standalone or Online edition, run the corresponding workflow from Chapter 10 to generate a current-version figure.

1. Start the toolbox in full MATLAB® and open `silicon-band-primitive.ks` through Home → Open. Check its two-site Si structure under Structure and keep this project active.
2. Set MATLAB®'s current folder to this manual repository's root, then run the following in MATLAB®'s own Command Window:

```
addpath(fullfile(pwd, 'docs', 'manual-zh-CN', 'examples'));
show_reference_results("bands");
```

3. Wait for the EnergyBandStructure document and its new Results → Plots entry. Match it to the Energy Band workflow and confirm the two-atom primitive-cell reference with 8 points per segment.

4. Read the high-symmetry labels and energy units, then use the axes tools to inspect the region near zero. Restore the full view afterward rather than examining only an enlarged fragment.

5. Export the figure and save the project as your own copy. If an old figure becomes unavailable later, redraw the saved numbers instead of rerunning the electronic solver merely to recover an image.

Figures 10.3 and 12.1 use the same primitive-cell reference data, path coordinates, and energy zero. The figure document supports zooming and interactive inspection.

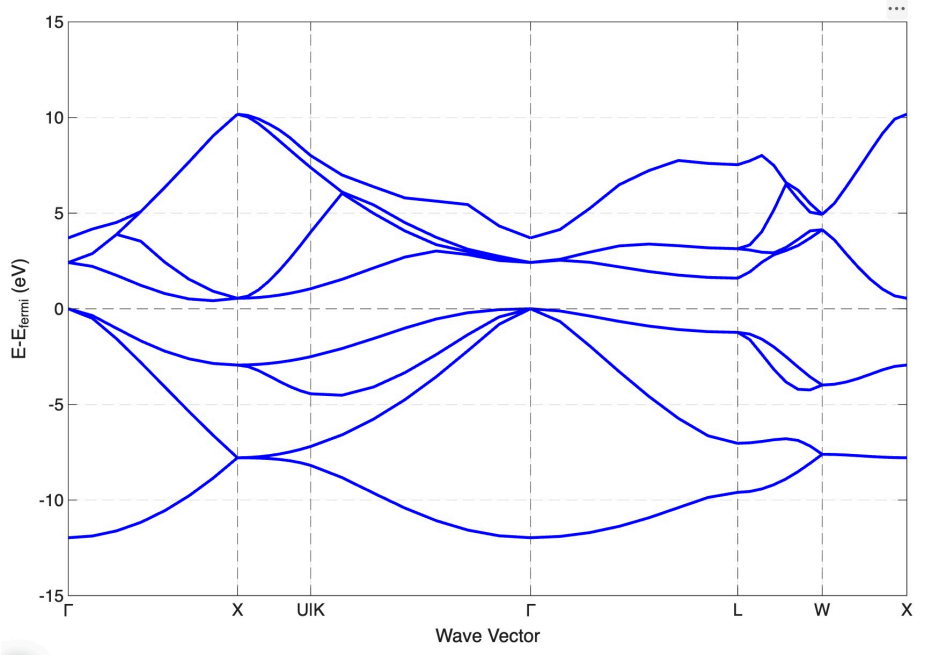



**Figure 12.1** The two-atom silicon primitive-cell band document, with 8 points per path segment and the Fermi energy set to zero.

## Check the axes and energy reference

The horizontal axis follows a path through high-symmetry points; the vertical axis is energy. Here it shows $E - E_{\mathrm{fermi}}$ in eV, so zero is the calculated

Fermi-energy reference. Separators and labels distinguish path segments, which need not all join continuously in reciprocal space.

Horizontal positions follow distance along the path rather than any single $k$ coordinate. Identify the path labels, inspect occupied and unoccupied states near the Fermi energy, and then examine deeper valence or higher unoccupied states.

Each curve is a band branch, not an atomic trajectory. This example stores eight bands, all drawn in blue; the color does not encode an element, orbital, or spin. Reading along one curve follows its energy along the path. Reading vertically at one horizontal position compares different bands at the same $k$ point.

Vertical dashed lines mark segment boundaries. Disconnected segments can have combined labels: U|K means the preceding segment ends at U and the next starts at K, not that there is a new point named U|K. Do not interpret the curves across that boundary as continuing along a single reciprocal-space direction.

### Identify a band gap

In the usual semiconductor picture, the highest occupied bands are valence bands and the lowest unoccupied bands are conduction bands. Their edge-to-edge energy difference is the band gap. A gap is direct when both extrema occur at the same $k$ point and indirect when they occur at different points.

A finite path need not pass through the true band extrema. A smooth plot alone cannot establish the Brillouin-zone minimum merely because no lower conduction state appears on the chosen path. Quantitative gap studies require adequate sampling and band-edge analysis.

The silicon tutorial uses a coarse introductory grid to make the workflow and plotting procedure accessible. State those conditions before discussing how sampling and the functional may affect a quantitative gap.

### Compare band plots

Check the cell, path, energy zero, functional, pseudopotentials, and number of bands. Switching between primitive cells and supercells can fold bands; additional curves need not indicate new physical states. Changing the energy reference can shift an entire plot vertically.

For a strain study, use a consistent calculation procedure, converge each structure self-consistently, and choose comparable paths. Identify the bands and $k$ points where changes occur before refining the sampling. Rule out inconsistent numerical conditions before interpreting every small shift as a physical effect.

## 12.7 Read the total density of states

### Open and check the total DOS

Continue with `silicon-band-primitive.ks`. This two-atom project also stores total DOS values on a $4 \times 4 \times 4$ grid. Redraw them without closing the project or running a new DOS workflow.

1. Confirm that the two-atom primitive-cell project is active. In the MATLAB® Command Window, with the example directory already on the path, run:

```
show_reference_results("dos");
```

2. Wait for Density of States (DOS) and check its new Results → Plots entry. This figure is the current Si2 primitive-cell result, distinct from the separate eight-atom project.

3. Read the vertical Energy (eV) axis first, then the horizontal Density of States (DOS) axis. A curve extending farther right indicates more states near that energy. Do not read energy from left to right as in a conventional energy-on-horizontal-axis plot.

4. Locate zero on the vertical axis and inspect the regions below, near, and above it. Describe populated intervals, low-DOS regions, and main peaks before focusing on small variations.

5. Export the curve using the figure's save tools and retain the project with its original data. Reopen the image to check that both axes are included.

The DOS is sampled from $-10$ to $10$ eV in $0.05$ eV steps, using PBE, default pseudopotentials, and a 20 Ha cutoff. To change the grid or energy range, import the DOS template as in Chapter 10, select the two-atom cell, and run with new parameters. Redrawing existing arrays does not improve their sampling accuracy.

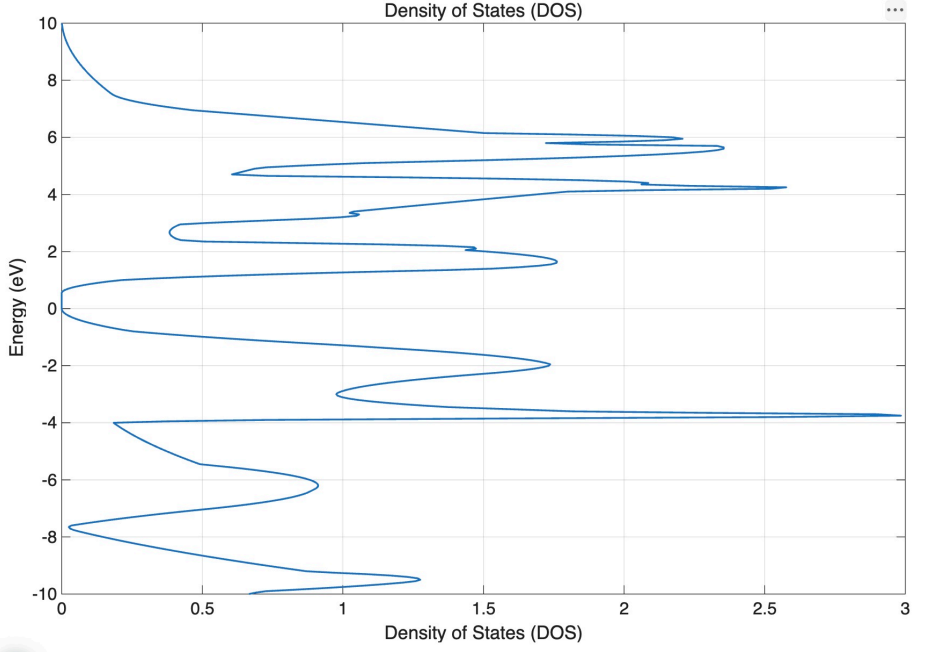



**Figure 12.2** Total DOS of the same two-atom silicon primitive cell. DOS is horizontal; energy relative to the Fermi energy is vertical.

### Peaks and low-density regions

A high DOS means that many electronic states lie near that energy. Peaks often reflect band dispersion, but do not identify which atoms or orbitals contribute. That requires projection or composition analysis.

The DOS node gives the total density of states, combining orbitals and $k$ points. Element- or orbital-resolved assignments require the corresponding projected DOS data.

### Read bands and DOS together

With a common energy reference, locate the Fermi energy and identify intervals with many states or no states. Bands show variation along a path; DOS summarizes sampling over a grid. They describe the electronic structure from complementary perspectives.

`silicon-tutorial.ks` stores DOS for an eight-atom conventional cell. Opening that project and running the same `show_reference_results("dos")` command redraws the Si8 result. Figure 12.3 retains that eight-atom reference for reading axes and peak shapes. It uses a different cell from the primitive-cell figure above. Normalize consistently per cell or per atom and align conditions and energy references before comparing them.

The software plots DOS horizontally and Energy (eV) vertically. Placing DOS to the right of a band plot in a report lets both figures share an upward energy direction. Match their energy ranges and zeros in the final layout.

### Distinguish sampling artifacts from physical features

A coarse $k$ grid can strongly affect DOS peak shapes, particularly in sparse or strongly dispersive regions. A smaller energy step only evaluates the current approximate curve more densely; it adds no Brillouin-zone information. Test whether a small peak survives a denser $k$ grid before treating it as reliable.

The $4 \times 4 \times 4$ silicon DOS is a starting point. With an $8 \times 8 \times 8$ grid, compare the main peak positions, low-DOS region near the Fermi energy,

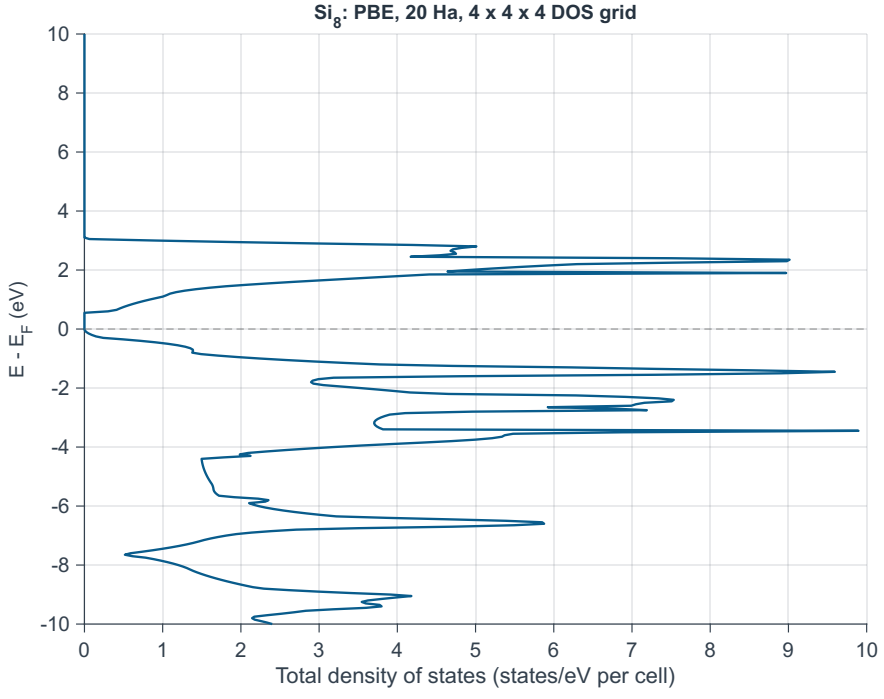



**Figure 12.3** Reference total DOS for the eight-atom silicon cell. The horizontal axis counts states per cell per eV; the vertical axis is energy relative to the Fermi energy. The introductory grid is $4 \times 4 \times 4$.

and overall shape, rather than simply judging which curve looks smoother. Features stable with increasing accuracy warrant further analysis.

## 12.8 Check three-dimensional results

Geometric views show atoms, cells, and local environments. Spatial quantities such as density, potential, or orbitals require the volumetric viewer. Import the actual field file and inspect it using isosurfaces, slices, and colormaps.

The input file and data type determine the meaning. A CUBE containing electron density and one containing orbital amplitudes can both produce

isosurfaces, but their colors and values have different physical interpretations. Identify the variable and units before choosing an isovalue.

Prepare the appropriate field file and import it as in Chapter 11. State the variable, isovalue, and units in the caption.

## 12.9 Prepare atomic structure views

This section uses the viewer inside an atomic structure document. Open the supplied silicon or a molecule with verified connectivity, then choose Viewer settings on its toolbar. These settings control inspection and figure preparation; the functional, cutoff, and electronic sampling remain calculation-node settings.

### Record site and cell readings

Select an atom, check its site index and periodic image in the information card, then copy Cartesian or fractional coordinates using the adjacent buttons. Copied values retain more precision than the abbreviated display. Cartesian coordinates are in Å; fractional coordinates are dimensionless. Record the Image offset too, so that an image in a neighboring cell is not mistaken for an input-site position.

For the full coordinate table, click the toolbar's Structure information button, whose tooltip is Fractional coordinates. The table lists IDs, elements, labels, and fractional coordinates to five decimal places for locating sites. Export the native structure when full precision is needed. Molecular entries without periodic fractional coordinates show dashes.

Read lengths, angles, and volume from Cell parameters in the measurement menu. Copy the required values into your notes and add the filename and cell representation. Close the information panel to resume framing. Copying readings does not save a structure file.

### Compare cell representations and bonding strategies

Scientific scene groups scene-analysis settings. For crystals, Change unit cell offers Input cell, Primitive cell, Conventional cell, and Niggli and LLL reduced cells. These change the scene's representation. To commit a conversion to the project model, use the primitive/conventional-cell modeling commands. Repeat cell controls display repetition and does not build a calculation supercell.

1. In the silicon viewer's Scientific scene, keep Input cell and note the site count and cell.
2. Switch Change unit cell to Primitive cell and wait for reconstruction. Compare the outline and sites, then return to Input cell and confirm the original representation.
3. Compare CrystalNN and Minimum distance under Bonding strategy. Change one setting at a time and wait for analysis before reading coordination or nearest-neighbor information.
4. Set one Repeat cell direction to 2 to inspect periodic continuity, then use Reset to 1 × 1 × 1. Record the representation, bonding strategy, and repeats used for the final image.

**Table 12.3** Bonding strategies in the crystal viewer.

| Option | Purpose and interpretation |
|---|---|
| CrystalNN | Identifies neighbors from local environments, useful for inspecting complete coordination. |
| Custom bonds | Uses VESTA cutoff rules; selects those rules rather than accepting arbitrary bond lengths here. |
| Jmol bonding | Uses Jmol bonding criteria for comparison with that representation. |
| Minimum distance | Identifies neighbors by the minimum distance and the indicated 10% tolerance. |
| O'Keeffe's algorithm | Uses O'Keeffe neighbor criteria. |
| Hoppe's ECoN algorithm | Uses effective-coordination analysis. |
| Brunner's reciprocal | Uses Brunner's reciprocal-distance criterion. |

Molecules have different choices: Source topology · preferred favors source connectivity, Source topology · explicit explicitly uses it, and Open-BabelNN · inferred infers bonds geometrically. For benzene, pyridine, or polymers with explicit bonds, check source connectivity before comparing inference. Changing displayed connectivity does not edit chemical bonds automatically; use add-bond, delete-bond, or bond-order tools to change topology.

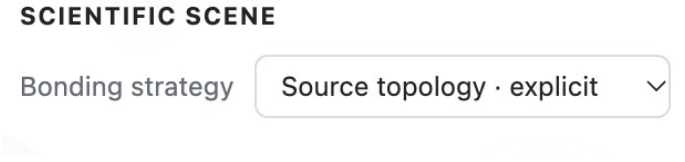



**Figure 12.4** Scientific scene for neutral singlet ethanol. The input contains 9 atoms and 8 explicit bonds; Source topology · explicit displays its original connectivity.

### Control periodic images and overlays

Under Visibility, begin with Atoms, Bonds, and, for crystals, Unit cell. Boundary image atoms controls boundary images; One-hop bonded atoms adds directly bonded neighbors of displayed atoms; Hide incomplete bonds hides connections without complete endpoints. These change the boundary appearance. Read site and image indices rather than simply counting spheres.

Enable Polyhedra for local coordination or Orientation axes for directions. Magnetic moments displays moments already present in the data; it does not calculate them. Molecules also offer Hydrogens and Multiple bond order, which control the display of hydrogen atoms and multiple bonds, respectively. Continuous measurement controls whether you can select another measurement immediately afterward; it does not change coordinates.

Under Style, compare VESTA and Jmol element colors, choose Atomic or Uniform radii for your purpose, and adjust Atom scale and Bond radius. If spheres look too large, reduce their scale before judging a close contact. Lighting & surface adjusts lighting, materials, brightness, and contrast; Reset appearance restores the starting appearance. Use consistent styling for

comparison figures and separately record structural and measurement conditions.

## 12.10 Export atomic structure views

The atomic viewer's Export image saves the current structure image. Export file saves interactive pages, scenes, or structures. These are separate from volumetric isosurface export and calculation-figure save tools. Stop automatic rotation, clear irrelevant selections, and choose the view first.

### Choose from six image formats

**Table 12.4** Atomic-viewer image export options.

| Option | Typical use |
|---|---|
| PNG | Lossless bitmap for technical documents and screen reading. |
| JPEG | Compressed bitmap for sharing; check fine lines and text for compression artifacts. |
| TIFF | Lossless bitmap with a `.tif` extension. |
| SVG | Vector output for further layout or line editing. |
| PDF (Vector) | Vector PDF for document preparation and enlarged viewing. |
| PDF (Raster) | Raster rendering that retains the current scene's appearance. |

1. Open the structure, set its orientation and representation, and check that the cell, atoms, and required annotations are in view.
2. Open Export image, choose PNG, and select a location. In Online, complete the browser download.
3. Export SVG or PDF (Vector) if needed. Vector output can differ from raster lighting; open both actual files before choosing.
4. Check boundaries, text, and local connections, then store the image with the structure, project, and framing notes.

This menu does not use the 72–600 DPI settings described later for calculation figures. An image records the view, not the complete atomic coordinates; export a structure file to exchange a model.

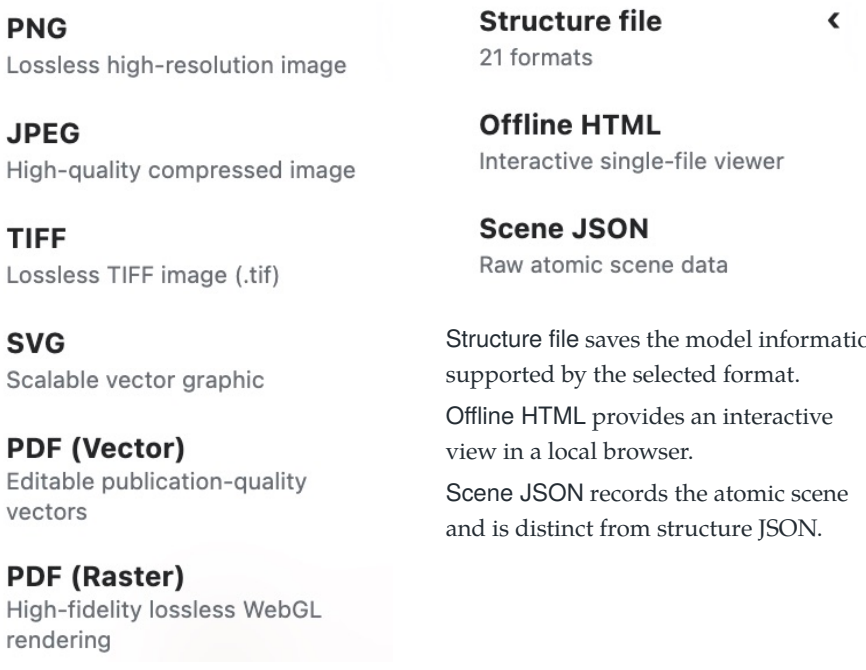



**Figure 12.5** The atomic viewer's export menus. Export image on the left offers six image formats. Export file on the right provides structure files, an offline page, and scene JSON.

### Save an interactive offline page

Offline HTML bundles the current atomic scene, viewer appearance, and camera into one HTML file. It is useful for supplying a rotatable structure with a report. It supports viewing but does not replace a project containing workflows, calculation data, and editing history.

1. Choose the representation, repetitions, and initial view in the atomic viewer.
2. Choose Export file → Offline HTML and save or download the `-offline.html` file.

3. Open it in a local browser. Check the initial structure and appearance, drag to rotate, scroll to zoom, and inspect the required atom information.

4. Identify the structure and purpose when sharing it. Include native structures and projects as well if the recipient needs to continue modeling or calculating.

The offline page retains the scene at export. To reanalyze bonds, convert cells, or commit further model changes in the toolbox, return to the project and export a new page afterward.

### Distinguish scene JSON from structure JSON

Scene JSON in Export file records atomic scene data, including sites, displayed images, connections, and analysis descriptions. It does not save the current camera and all appearance settings as a viewer configuration, and does not replace a project or calculation input.

For continued editing, choose native JSON under Structure file. Reimporting it exposes the lattice, sites, periodicity, and properties; `properties` can retain twist angles, SQS search records, and other metadata. Files named `-scene.json` record scenes, while Offline HTML supports offline viewing. Name files by purpose to distinguish them.

The supplied `ethanol-angle1095.json` describes the neutral singlet ethanol used here: 9 atoms, 8 explicit bonds, and a $109.5^\circ$ angle defined by 1(C)—2(C)—3(O). `ethanol-angle1095.png` is a $4032 \times 1964$-pixel PNG exported from the same model; `ethanol-angle1095-scene.json` records the atomic scene. Import the structure JSON for editing; use the other two for illustration and scene inspection.

### Create a Hero Shot

Hero Shot presents the atomic scene with an emphasis on overall form and depth. Progressive rendering refines a PNG; its lighting and materials do not change atomic positions or electronic structure results.

1. Under Style → Theme, select Gleamoe Noir.
2. Find Hero Shot and click Enter Hero mode. Settings collapse and a dedicated framing interface opens.
3. Rotate and zoom the structure, using crystal-axis buttons if helpful. Hold the view still once composed so that progressive rendering can refine it.
4. Open Export Hero Shot and start with High. High, Ultra, and Poster specify export scales of 2.5, 3, and 4 times the viewport.
5. Choose a location and wait for rendering and writing. Open `-hero.png` to check framing and detail; in Online, also confirm the browser download.
6. Click Exit Hero mode to return to measurement or editing. Save the project and native model through their usual procedures.

Start with a small model and High quality to compare framing and rendering easily. Conventional structure figures suit clear annotations and geometric comparisons; Hero Shot emphasizes form. Scientific descriptions still need model provenance, periodicity, and relevant dimensions.

## 12.11 Export calculation figures

### Choose a suitable format

Figure documents provide axes tools for zooming, panning, and image export. Before exporting, check that labels and units are complete and lines are not clipped. Screenshots document interface operations; direct exports generally suit research data figures better.

Raster formats such as PNG work well in documents and presentations. PDF and SVG can retain crisp lines and text, although complex figures or certain rendering elements may use different output methods. Open the export to check text, lines, and rendering.

Online exports through the browser. Check the browser's actual download location rather than a server-side session path. Desktop editions use the location selected in the file dialog.

### Inspect details interactively

Move over the axes and use the tools available for the current figure to pan, zoom, rotate, or restore the view. Enlarge the last few SCF iterations in a 2D convergence plot; rotate Brillouin zones or band surfaces. Restore the full range before exporting a complete result, rather than inadvertently saving a magnified fragment.

Three-dimensional band surfaces mainly show two band-edge surfaces. Do not interpret their separation as the global band gap without checking the coordinates and sampling: another direction may contain a lower conduction minimum or higher valence maximum. A Brillouin-zone plot shows reciprocal-space geometry, not energies encoded by polyhedron color. Band surfaces, zones, path bands, and DOS represent different quantities; identify the figure type in its caption.

### Export a figure from Online

Online retains native figure toolbars, but save and copy actions obtain files through the browser. Saving an image opens an export dialog with filename, format, and resolution. Project saving and image download are separate operations.

1. Open the result and restore the desired axes range.
2. Choose save from the figure toolbar and enter a descriptive name, such as `H2-SCF.png`.
3. Choose PNG, JPEG, TIFF, PDF, or SVG. Raster resolution can range from 72 to 600 DPI. Start with 150 DPI for screen reading and increase it for print.
4. Click Save and complete the browser download.

5. Open the downloaded file locally and check the axes, legend, and values needed for the caption.

Copy-image and copy-vector actions also become file-retrieval workflows in a browser. Vector copying supplies SVG content rather than guaranteeing transfer to the operating system clipboard. For PDF and SVG, a DPI setting does not replace checking fonts, lines, and actual output. An image does not retain all original arrays; keep the project and required data too.

### Retain the numerical data

Projects retain run data, and accompanying scripts provide MAT or CSV results for further calculation and consistently styled plots. Give CSV columns clear meanings, such as energy relative to the Fermi level followed by DOS. A separate units note is more reliable than a verbal explanation.

Use original numerical records for fitting, integration, or quantitative comparisons. Screenshots show trends, but values read from them depend on axes limits, resolution, and line width.

### Make figures understandable on their own

A caption should explain the figure without requiring the reader to inspect the entire project. For a structure, identify atoms and the cell; for convergence, the iterated quantities and units; for bands, the path and energy zero; for DOS, the sampling and energy reference. Put key conditions in the caption or nearby text rather than crowding a complete log into the image.

Use consistent fonts, sizes, colors, and energy ranges. Reuse a color for the same quantity across figures. Show initial and optimized hydrogen structures from the same angle, or give two DOS plots matching limits. Such choices aid comparison more than decoration.

## 12.12 Write a complete account of the result

A clear description states the question, conditions, observations, and interpretation. For example, begin with optimizing the hydrogen bond length in a fixed 10 Bohr box, specify PBE, pseudopotentials, and cutoff, report the shorter bond and lower energy, then explain that these describe equilibrium within the chosen model.

Avoid only saying that a result is normal or as expected. State which quantity converged, where the main change occurred, and which conditions were held fixed. Explain numerical accuracy in terms of the cutoff, sampling, and stopping criteria.

Before closing the project, save it and check that the project, inputs, and exported figures correspond. A project that can be reopened, interpreted, and run again is the fullest record of the calculation.

Chapter 13

# Configuring the application environment

After the local structure and workflow exercises, you may want to adjust the interface, connect to a materials database, or submit work to a configured remote environment. These features serve different purposes: settings connect the application to personal services, remote computing chooses where a workflow runs, and logs and help explain its state and support problem reporting.

This chapter covers everyday settings and chat services before remote submission from the full MATLAB® toolbox. Section 10.5 covers CPU, process-pool, and GPU settings within a node; the remote sections here choose the execution environment. Standalone and Runtime-based Online users can use node resources as supported by their deployment and refer to the settings, chat, logs, and help sections.

## 13.1 Adjust application settings

### Open Settings and save changes

Click Settings in the Environment section of Home. General contains language and chat settings. Materials Project contains database access settings. Online environments connected to an account service also provide an Account page.

Click OK to save changes. Cancel or closing the window discards unconfirmed edits. Reopen Settings to read the saved values and check what is actually in effect.

### Change the interface language

1. Open Home → Settings → General.

2. Select the interface language in the Language section.
3. Click OK to save it.
4. Save your project and restart the application.
5. Check that ribbon and panel titles use the new language.

The language takes effect at the next start. Close and reopen a desktop application. In Online, Exit KSSOLV Toolbox only returns temporarily to the portal while the instance keeps running. Save first, then use your platform's sign-out or restart procedure to enter a new instance.

Figure 13.1 shows the language section of Online General settings with English selected. The message below the field says that the change takes effect at the next start, matching the save-and-restart sequence above.




**Figure 13.1** The language field in General settings, with English selected.

## 13.2 Connect a chat service

### Choose a service type

The Command Window accepts both ordinary commands and questions for a large language model. OpenAICompatible uses an OpenAI-compatible API with an endpoint, key, and model supplied by the provider. Ollama connects to a running Ollama server and uses models installed there.

Chat can explain terminology, compare the roles of different parameters, and help organize notes. Models supporting tool calls can also read the current workflow and change node labels. Chat is optional: structure import, modeling, and DFT calculations work independently.

Requests originate from the computer running the toolbox. For a desktop application, this is normally your computer. For Online, it is the server or

**Table 13.1** Information needed to connect a chat service.

| Setting | OpenAICompatible | Ollama |
|---|---|---|
| Address | API Base URL, including the provider's required version path, such as `/v1`. | Server URL, such as `http://127.0.0.1:11434` for a local service. |
| Credentials | API Key for this base URL. | No API Key field is provided here. |
| Model | Model Used: select or enter a model identifier available to your account. | Model Used: select or enter a name and tag already installed on the server. |
| Check | Test Connection or Refresh retrieves the model list. | Test Connection or Refresh retrieves installed models. |

container; the browser's computer does not automatically host the model service.

### Configure an OpenAI-compatible service

Obtain the base URL, personal API key, and authorized model identifier from your provider. "OpenAI-compatible" describes the interface; the provider and available models depend on the service you use.

1. Under Home → Settings → General → LLM, set Type to OpenAICompatible.
2. Enter the provider's API Base URL, retaining its required version path.
3. Enter your personal API Key.
4. Click Test Connection and wait for the model-list status.
5. Select a model in Model Used, or enter its exact identifier as instructed by the provider. Check it and click OK.
6. Ask a short conceptual question in the Command Window to check an actual response.

Figure 13.2 shows the configuration fields. Retain the required version path in the base URL and match the model identifier to the service. Use Test Connection to retrieve its list.

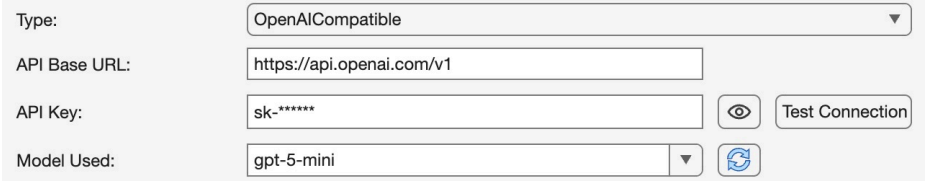



**Figure 13.2** OpenAI-compatible service settings. Base URL, key, and model are separate fields; the key is masked.

Use the provider's API base URL without specific endpoints such as `/chat/completions` or `/models`. Retain a required `/v1` suffix. The application appends model-list and chat paths itself and rejects base URLs already containing those specific endpoint paths.

The key is normally masked. Reveal it briefly with the display control if correction is necessary, then hide it again. To remove a saved OpenAI key, first choose a service that does not use it, clear the field, and click OK. With OpenAICompatible still selected, Settings requires a usable key. Keep credentials hidden in screenshots and screen sharing.

### Configure Ollama

Ollama must already be running at an address reachable by the application. Desktop users may use a local service; Online users need an address reachable from the server. Requests come from the toolbox host.

1. Set Type to Ollama.
2. Enter the service root in Server URL, without `/api/tags` or another endpoint path. The usual local address is `http://127.0.0.1:11434`.
3. Click Test Connection and wait for the server's model list.
4. Select an installed model under Model Used, or enter its complete name and tag, then click OK.

5. Ask a short question in the Command Window and check the status and response.

In Online, `localhost` and `127.0.0.1` refer to the server or container running the toolbox, not to Ollama on your laptop. For a shared service, enter the address supplied by your deployment. Typing a model name does not download it to the server. Install the model and start its service in the hosting environment first.

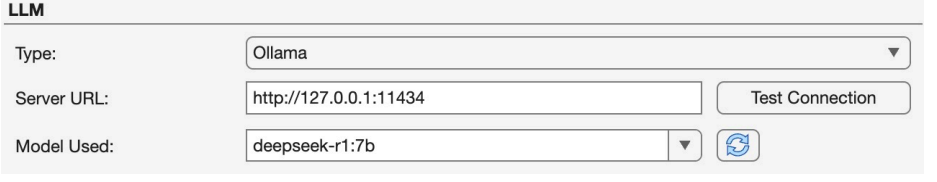



**Figure 13.3** Ollama server address and model settings.

### Refresh the model list

Test Connection and the refresh icon beside the model perform the same check: retrieving the model list from the current address. OpenAI-compatible services use `/models`; Ollama uses `/api/tags`. A nonempty list produces a connection-success message and model count. Check actual text responses and tool support by asking a question.

Refresh after changing an address or key, then inspect Model Used. During testing, the button changes to Stop Test to end the wait. The model field also accepts manual entry. If the service has no enumerable list but its provider gives an exact model identifier, enter and save it, then test a short question. The list check may still report that no models were found.

For authentication errors, check that the address and key belong to the same service. For missing endpoints, check the base URL and version path. For timeouts or secure-connection messages, check connectivity from the toolbox host. If list retrieval succeeds but chat fails, inspect model permissions, the chat endpoint, and tool-call support instead of repeatedly refreshing. A message that the distribution lacks chat components means you need

an environment providing them; the structure and DFT tutorials are unaffected.

### Ask a question in the Command Window

Commands and chat share one input area and are distinguished by their prefix. Enter MATLAB® expressions directly. One dollar sign starts a question using the existing conversation context; two dollar signs make an independent request without that history. The signs are part of the input; no separate chat page is needed.

**Table 13.2** Commands and questions in the shared input area.

| Input | Effect |
|---|---|
| `2 + 3` | Evaluates a MATLAB® expression and displays its result. |
| `$` followed by a question | Asks the model using context from earlier successfully completed turns. |
| `$$` followed by a question | Makes an independent request without reading or appending to the retained conversation context. |

1. Confirm service and model settings, open the Command Window, and wait for input to become available. Enter `2 + 3` and press Enter to see where ordinary output appears.
2. Enter `$ Explain the plane-wave cutoff energy` and press Enter. Observe the waiting status and streamed response.
3. Follow up with `$ Why do I also need to test k-point convergence` to use the same conversation.
4. For a separate question, enter `$$ Explain the three components of fractional coordinates`. This does not clear the earlier context; a later single-dollar question can still use it.
5. Keep useful explanations in your notes, then return to modeling or calculation.

The application creates the selected chat client on the first question. Watch its waiting status and streamed reply; the final status may identify the model actually returned by the service. Wait for one response to finish before asking the next question.

Replies can contain text, code blocks, and mathematical expressions. Displayed code is not executed automatically. Check its variables and inputs before submitting any command separately without a dollar sign. Include the relevant structure, calculation conditions, and numerical values when asking a specific question.

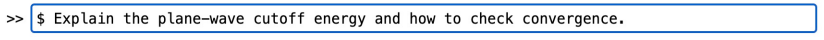



**Figure 13.4** A question entered in the Command Window before sending. The leading dollar sign distinguishes chat from an ordinary command.

### Let the model inspect and organize a workflow

Tool-capable models can request two application operations: read the current graph's nodes and connections through `getCurrentGraph`, and change a specified node label through `changeNodeLabel`. These tools do not create structures, change calculation parameters, run calculations, or read all numerical results in the project.

OpenAI-compatible requests include both tool definitions, so the provider and model must support that tool-call format. Ollama exposes tools according to the model capabilities reported by its service. A model without tools can still answer ordinary text questions. Check both the conversation and canvas to see whether a tool ran.

1. Save the project and activate an existing workflow.
2. Enter `$ Read the current workflow and describe its node labels and connection order`. A tool-capable model can base its answer on the actual graph.

3. Compare its labels with the canvas. If the reply is generic, explicitly request the current workflow. If actual graph information is still absent, check model tool support.
4. Identify an unambiguous node, such as the workflow's only SCF node. Enter `$ Change the SCF node label in the current workflow to "Electronic self-consistency".` With several matching nodes, supply the original label to identify the target first.
5. Check the changed label on the canvas, open the node to confirm that its task remains SCF, and save.

Renaming changes only the canvas label. It does not turn another task into SCF or rerun the solver. Inspect the canvas for the actual change. Continue using node configuration and graph editing for cutoffs, grids, and execution order as described elsewhere.

### Chat after changing settings

Clicking OK in Settings rebuilds an existing chat client, so the next question starts a new context. Retain any explanations you need even if you are merely confirming unchanged settings. Cancel does not apply edits. Keep important explanations, notes, and conclusions in your calculation record alongside project inputs and results.

If a streamed response disconnects, the interface reports an incomplete or failed turn. A failed turn does not replace previously retained complete context; check the connection and ask again. The incomplete reply remains visible. If a tool already changed a label during that turn, inspect the canvas to check the actual state.

## 13.3 Set up Materials Project access

Materials Project is one source of structures. Its import tool needs personal access settings. Importing a local CIF or XYZ does not.

### Save an API key

1. Open Settings → Materials Project.
2. Click View My API Key to open your API page in the browser.
3. Sign in as required by the database website and copy your own key.
4. Return to the application, enter it in API Key, and click OK.
5. Choose Home → Structure → Import structure from Materials Project....

Settings stores service credentials, not a particular structure. Searches and downloads originate from the toolbox host. In Online, check that the server can reach the database.

After saving access settings, use Section 3.7 to search by periodic-table selection, formula, or MP ID. Its silicon example shows the results table, mp-149 selection, and imported 3D structure.

## 13.4 Understand remote computing

### Online and remote clusters

The toolbox in full MATLAB® provides a Remote menu to submit the current workflow to another MATLAB® environment or cluster. The standalone application and Runtime-based Online do not show this menu. The following configuration and submission procedures apply to the full MATLAB® toolbox.

Online already runs on a server and does not require browser users to configure SSH first. The location of the workspace and submission to a cluster are separate matters: one identifies where the application runs, and the other sends a task to a chosen computing environment.

A remote job record contains the workflow, submission target, status, and output. Successful submission means the target accepted the job; it may still be queued or encounter problems during execution. After completion, fetch results into the project and save it.

### Check location and resources separately

The Resources panel in Section 10.5 chooses Serial, a process pool, or GPU on the execution machine. Remote configuration chooses the machine, cluster profile, and retrieval method. Enabling a node pool does not submit a cluster job; selecting a cluster does not make every node parallel.

In Online, the Home Parallel button is disabled, while the node Resources panel still offers options appropriate to each task. Actual capability depends on the server's components, pools, and GPUs. Cluster submission here applies to the Remote menu in full MATLAB®.

### Choose an execution mode

**Table 13.3** Remote execution modes.

| Mode | Environment |
|---|---|
| Standard | Local MATLAB® submits through MATLAB® Parallel Server and Slurm; local and remote calculation components use the same release. |
| Bridge | Remote MATLAB® submits through its own cluster profile; that MATLAB® and its workers use the same release. |
| Mirror | Ordinary MATLAB® on one remote node; MATLAB® Parallel Server is not required. |
| Cloud | An existing, configured, and validated cloud MATLAB® cluster profile. |

Prefer a mode already validated by your institution. Mirror can suit an ordinary single-node MATLAB® environment. For Parallel Server with Slurm, follow administrator guidance on Standard or Bridge. Cloud requires a prepared profile; selecting it does not provision a new cloud cluster.

Remote execution requires the corresponding MATLAB® products and licenses, with Parallel Computing Toolbox locally. Administrators prepare version compatibility and cluster installations. Check that your selected configuration matches the actual environment.

## 13.5 Create a cluster configuration

### Gather remote information

Ask the administrator for the host, authentication method, remote MATLAB® root, and writable job directory. Also obtain a profile name if used, or Slurm partition, account, QoS, and wall-time requirements.

Name configurations by purpose, such as "Materials cluster," rather than "Server 1." Login nodes, compute nodes, and shared storage may differ. Use supplied site information rather than guessing remote paths from your local computer.

### Enter the configuration

1. Open Home → Remote → Configure Remote Clusters⋯.
2. Click Add, enter a name, and choose the execution mode.
3. On Connection, enter host and authentication details as instructed.
4. On Compute, enter the mode's MATLAB® root, profile, job directory, and resources.
5. Choose whether to attach the current toolbox or use a cluster-installed copy.
6. Click Test Connection and inspect its status.
7. Use Probe Resources if needed, check the discovered values, and click Save.

The MATLAB® root is the installation directory, not `bin/matlab`. Use a writable remote job directory visible to the relevant compute nodes. Respect site resources and licenses when choosing workers; do not simply enter the machine's total core count.

Test Connection checks connection and login steps; a profile-based configuration may also run a minimal test job. Probe Resources discovers remote MATLAB® and resource information. They serve different purposes:

connection testing does not replace resource discovery, and discovered values are not saved automatically.

### Match job resources to node limits

Pool workers requests a parallel pool for each remote job. Its value must lie between 0 and the configuration's Maximum workers. A value of 0 requests no job pool. This differs from a node's maximum-process limit: submission prepares the pool, while the node constrains the pool it accepts.

For NSCF and DOS limited to 2 processes with 1 CPU thread each, request 2 pool workers in a supporting remote mode and ensure remote computational threads do not exceed 1. An existing 4-worker pool conflicts with a node limit of 2; adjust the pool or node settings. An existing compatible 1-worker pool also meets the limit, but uses only 1 worker.

A job already running on a remote worker cannot automatically create a nested pool inside a node. Prepare a suitable pool in advance for job-submission modes such as Standard and Bridge. Mirror execution in an ordinary remote MATLAB® process must also respect that machine's local process-pool capabilities. GPU mode requires a supported GPU on the execution machine; your computer displays the interface while the remote GPU performs the numerical work.

The node time-limit field does not stop a calculation. Scheduler wall time is a separate setting governed by site policy. Save the remote configuration and project, and record pool size, node limits, and scheduler conditions together for timing comparisons.

### Choose how to deploy the toolbox

Attach Current Toolbox transfers the current toolbox to the target, useful when a matching version is not already installed. The first transfer of a version may take longer; later jobs can reuse prepared code.

Cluster Installed uses an administrator-maintained installation. Enter its root and confirm version compatibility. Centrally managed environments

usually specify which option to use; do not modify shared application directories yourself.

A post-login script initializes the environment, for example by loading a site's MATLAB® module. Command templates and prompt rules address special routing or additional authentication; configure them only when the administrator requires them. Keep ordinary direct connections simple when they work.

## 13.6 Exercise: submit and retrieve a calculation

Use a completed small silicon workflow with the earlier settings. Focus on submission and result management without changing material and method at the same time.

### Save and choose the target

1. Open the existing project and workflow; check the structure, connections, and main settings.
2. Save and give the workflow a recognizable name.
3. Choose Home → Remote → Use Remote Computing and select a saved configuration such as "Materials cluster."
4. Use Test Current Cluster if another connection check is needed.
5. Click Run, wait for the submission message, and record the job ID.

Successful submission creates a job record. Do Not Use means execution in the current MATLAB® environment. Check target names when switching so that a comparison run is not sent to the wrong environment.

### Track the job

1. Open Home → Remote → Remote Jobs and Results⋯.
2. Select the new job and check its workflow, cluster, and submission time.

3. Click Refresh to inspect queued, running, or completed status.
4. Click View Diary and read calculation output for parameter and convergence messages.
5. If task settings need correction, use Cancel Job first, then edit the workflow and resubmit.

Unknown means the state cannot currently be read reliably, not that calculation necessarily failed. Check the network and scheduler before refreshing. An empty Diary may mean the job has not started; consider submission time and queue state together.

### Fetch and import results

1. Confirm completion and choose Fetch and Import.
2. Wait for transfer and import confirmation.
3. Find the corresponding data or figures in the Project Browser.
4. Inspect convergence, energy, and related results using the earlier chapters' checks.
5. Save the project to retain the fetched results in its `.ks` file.

If the job was already imported, the application reports that the results exist rather than adding duplicates. Import needs the corresponding workflow in the current project. Retain the submitting project and do not delete or replace the whole workflow before retrieval.

Check SCF convergence, energies, and parameters after fetching. If remote and local results differ, compare structures, pseudopotentials, and node settings before attributing differences to the computing environment.

### Organize job records

Clean Local Bundle removes the local transfer bundle without canceling the remote job. Delete Record removes only its local record, not the remote working directory. Use Cancel Job to stop a calculation.

Existing jobs retain the information identifying their submission environment. Editing a configuration later does not move old jobs to another server. Fetch required results and save the project before cleaning up unneeded transfer contents.

## 13.7 Use the remote Command Window

Run Command Window Commands Remotely opens a persistent MATLAB® session using the selected direct SSH configuration. It is independent of workflow submission and suits short environment checks or interactive operations. A profile-only configuration without direct connection details cannot use it.

1. Select a validated direct configuration.
2. Open the Command Window and enable Run Command Window Commands Remotely.
3. Enter a short check such as `version`.
4. Confirm remote execution from the `[remote]` input/output prefix.
5. Disable remote execution afterward to return to ordinary commands.

Commands in the same remote session share a base workspace. Changing targets, disabling remote commands, or closing the Command Window ends that session. Save lasting data to appropriate files. Use workflows for scientific steps you intend to repeat.

## 13.8 Find help

### Check the current state

The Resources section provides Documentation, Community, and Help. About KSSOLV shows product information, and Check for updates reports new versions. Installation follows the distribution's or deployment's update procedure. Online external links open in the browser; return to the workspace to continue editing.

When reporting a problem, identify whether it occurs during import, parameter entry, execution, retrieval, or display. State the step, what you see, and what you expected instead of simply saying that it does not run.

### Collect logs

1. Save the project and note the application version and edition.
2. Choose Open Log from Help.
3. Desktop opens the log in the system's default application. Online shows it within the page, with refresh and full-file download actions.
4. Locate messages around the time of the issue and retain the relevant portion.
5. Prepare a clear screenshot and a small reproducible project.
6. Click Request Support, then compose and send the report in your mail application.

Request Support opens a draft with a subject. Add your explanation and attachments, review them, and send it yourself. If no default mail application is configured, use another contact route provided by the resources. Check screenshots and logs for personal credentials before sharing.

### Read license and update information

Choose Home → Help → License to read the current product license. Follow the release's license and deployment terms for citation, distribution, or use in another environment.

Check for updates reports available versions; About KSSOLV identifies the running version. Save before updating and follow the platform instructions in Chapter 2. Browser deployments are updated by their administrators. A grayed-out Examples or extension entry is unavailable; the supplied examples can still be imported from this manual's examples directory.

### A useful diagnostic sequence

For chat authentication failures, check the base URL, key, and model permissions. For an empty model list, check that the service provides available models. For remote connection failures, inspect authentication and post-login scripts. For submission failures, inspect partition, account, profile, and version requirements. If a completed job cannot be imported, open the original submitting project and read Diary and import messages.

Keep each check focused rather than changing many settings together. Once resolved, rerun a minimal exercise before returning to a larger project. Save the final configuration and parameters with the project and your notes for reproducibility.

Chapter 14

# Troubleshooting

First identify where the problem occurs: opening the application, reading a file, editing a model, running a calculation, or saving results. A small, targeted check is usually more useful than changing many settings at once. This chapter is organized by the symptoms you may see during everyday use; refer back to the relevant tutorial for the full procedure.

## 14.1 The application will not start after installation

Check the package's operating system and processor architecture, then the MATLAB® Runtime release requested by the installer. The application version and Runtime release have separate numbering, and the standalone application needs a matching Runtime.

If Runtime configuration is still in progress, wait for the installer to report completion. For a Windows offline package reporting missing files, keep the EXE and matching volumes together. If a macOS application-only installer does not detect Runtime, select the correct Runtime root directory in the wizard. If startup still fails, keep the startup message and installation location and use the support resources described in Chapter 13.

## 14.2 A command appears to do nothing

Look for an unfinished parameter dialog, confirmation dialog, or system file picker. A modal dialog prevents interaction with the main window. On macOS in particular, a system file panel may belong to a separate runtime window. Switch between application windows or inspect the Dock's windows to find it, then complete or cancel the selection.

If a calculation is running or a large structure is loading, check the Simulation Browser or status area. Updating messages indicate that work is still in progress. Wait for completion before starting the next operation.

## 14.3 Online stays on the connection page

Return to the service entry page and check whether the workspace is still starting. First startup can take longer; wait with one active tab. If the workspace is already open elsewhere, return to that tab. Use the entry page's retry connection action when available rather than repeatedly refreshing several pages.

If all workspace slots are occupied, follow the portal's instructions and try again later. Save your project before restarting a workspace: a restart ends the in-memory session, although saved server-side files remain. Use reconnect for a dropped connection; restart only when the current instance needs to be replaced.

## 14.4 An uploaded file does not appear as a project structure

The Online Files panel and Project Browser manage different things. Uploading places a local file in a server-side directory. Double-click the uploaded file or choose Import to add its contents to the project. Confirm that the upload finished, then check the Project Browser's Structure branch.

If import still produces no structure, check that the contents match the selected format. Renaming an extension does not convert a file. Try the supplied `silicon.cif` or `hydrogen.xyz` first. If the example opens, inspect your own file's cell, elements, coordinates, and format specification.

## 14.5 The displayed atom count is not what you expected

In the 3D view, sites counts independent model sites; atoms can include boundary and periodic images. Use the site count to check supercells and

defects. A $2 \times 2 \times 2$ supercell built from an eight-site cell should contain 64 sites, although more atomic spheres may be displayed.

If the site count itself is wrong, check whether you changed only Repeat cell or applied a supercell operation to a model that was already enlarged. Also check whether the input is a primitive or conventional cell. Structures built from space groups expand asymmetric sites into equivalent positions, so one input row can produce several atoms.

## 14.6 A modeling command is missing or unavailable

Activate the 3D structure document first. Crystal Modeling and Molecule Modeling depend on the active document and may disappear when you switch to a workflow or plot. Next check the object type and selection count: distance needs two atoms, an angle needs three, a dihedral needs four, and moving one atom precisely requires a single site.

If the selection is unclear, press Esc to leave the current tool, clear the selection, and select atoms again in order. With many periodic images visible, first restore display repetitions to $1 \times 1 \times 1$. If a parameter dialog is open, check its site indices before selecting again.

## 14.7 A preview disappears when the dialog closes

A preview lets you inspect a proposed change. Cancel or closing the dialog restores the previous document. Click Apply to accept the change, then save the project to retain it after reopening. Chapter 7 explains these three stages.

An applied change can be undone. Saving establishes the latest saved state as a new baseline and clears the current undo history. Keep a separate project or structure if you need a lasting before-and-after comparison. A stale preview usually means the document changed after it was created. Reopen the command and generate a new preview instead of repeatedly applying the old dialog.

## 14.8 A modeling command reports an error

For a surface, check that the Miller indices are not all zero, thickness and vacuum are positive and appropriate, and the termination index is within the available range. Orthogonalization can fail when no suitable integer supercell fits the current length, angle, and atom-count limits. Adjust the search range deliberately.

A message identifying a component that cannot load, such as "Invalid MEX-file," indicates an application component loading problem. Keep the input and complete message. You can continue learning with a prepared surface structure while reporting the version and platform to support.

For packing failures, check the available space first: reduce the initial density or molecule count, enlarge the permitted region, or try another seed. Resolve any remaining close contacts before subsequent equilibration; increasing calculation steps alone is not a remedy.

## 14.9 A workflow runs but produces no curve

Check that the intended workflow is active and that the plotting node is connected to the appropriate calculation output. Energy Curve plots results; it does not run SCF on its own. A failed structure input or parameter step leaves downstream plotting nodes without data.

Expand Results → Datasets and Results → Plots to see whether results exist but their documents are closed. Read the last messages in the Simulation Browser to identify the last completed node. If necessary, return to a minimal structure-input, SCF, and curve workflow, verify its data flow, and then add other analyses.

## 14.10 SCF reaches its iteration limit without converging

Distinguish an error that is still decreasing from one that oscillates persistently. Increasing Max SCF steps may help the former. For the latter, in-

spect the structure, occupations, and mixing. A higher iteration limit allows more iterations but does not automatically cure oscillations.

1. Check for overlapping atoms, abnormal bond lengths, or an obviously undersized simulation box.
2. Check elements, pseudopotentials, molecule/crystal type, and cutoff units. A value of 20 in Ecut (Hartree) means 20 Ha.
3. For metals or near-degenerate systems, check that occupations and smearing suit the task. Do not copy the isolated hydrogen molecule's zero-temperature settings indiscriminately.
4. Inspect the error and energy histories. Change one major setting at a time and keep the previous output for comparison.

A visually flat energy curve does not establish density convergence. Read the completion message and final error. Geometry optimization additionally requires force and geometry stopping checks; assess electronic and geometric convergence separately.

## 14.11 Results differ from the tutorial

Compare the structure, functional, pseudopotentials, cutoff, and $k$ points before comparing plot references. If bands use the Fermi energy as zero, compare them with curves using the same zero. Different cells can have different high-symmetry paths, so check path coordinates too.

The DOS examples use tetrahedron integration. Grid density and energy sampling affect curve details, while extra bands set the available unoccupied-state range. Optimization also depends on the initial geometry, fixed atoms, and convergence settings. Reproduce the supplied inputs first, then vary parameters individually. Software versions and initial electronic states can change iteration counts or the last few digits. Checking physical quantities and convergence is more informative than demanding line-by-line identical logs.

## 14.12 Volumetric data is blank or has little color contrast

Check the channel and value range. An isovalue outside the data range produces no surface, and a positive-only channel cannot produce a negative isosurface. Return to Value Distribution, choose a suitable threshold, and confirm that construction finishes.

For a low-contrast slice, narrow the color range appropriately and check the Miller indices. I, J, and K together define one central plane and cannot all be zero. If direct volume rendering is unsupported, use slices as indicated by the status message. Interpolation and smoothing improve appearance but cannot add details absent from the source grid.

## 14.13 A saved project cannot be found locally

Desktop editions save to the chosen local directory. Online saves to server-side storage; download the project to obtain a local copy. An existing project continues saving to its original location even if you select a different folder. Use Save As to rename it or change its directory.

Save before downloading from Online, then select the correct `.ks` file in Files. Check the browser's download list to confirm that transfer and local saving have finished. Chapter 4 lists the file and transfer size limits. Before signing out, confirm that important work is saved in a file rather than relying on the in-memory session.

## 14.14 A remote job finishes but its results are not imported

This section applies to the full MATLAB® toolbox. The standalone and Runtime-based Online editions do not provide the same remote menu. Open the project used for submission, refresh the status in Remote Jobs and Results, inspect Diary, then retrieve and import the results. Keep the original workflow in the project so that imported results can be matched to it.

Unknown means that the state could not be read reliably at that moment; it does not mean failure. Deleting a record does not cancel its job. Use the cancel-job action to stop a calculation. After retrieval, check convergence and save the project.

## 14.15 A chat question is interpreted as a command

Check the start of the input. The Command Window interprets ordinary text as a command. Prefix a natural-language question with one dollar sign, or use two dollar signs for an independent question. See Section 13.2 in Chapter 13. If the input format is correct but configuration is missing, open Settings and check the service, endpoint, and model.

## 14.16 A new task does not produce the expected data

Check what the task actually does. NSCF prepares options; a subsequent Band Calculation or DOS task performs the sampling. A Brillouin-zone plot displays reciprocal-space geometry rather than calculating a band gap. TDDFT needs unoccupied ground-state orbitals; Section 10.13 describes the graphical node's input limitations and the solver procedure in full MATLAB®. Use the complete workflows in Table 10.1, Chapter 10, and retain the last log message before an error. When viewing a figure, check which run produced it.

## 14.17 Write a useful problem report

Include the software version, operating system, edition, input files, and shortest steps that reproduce the problem. Describe the expected and actual results. Attach the complete message, relevant log, and a screenshot that includes the pertinent parameters. A small reproducible project is usually more useful than many disconnected screenshots.

Open logs from Help and locate the relevant time. Request Support opens an email draft; review it and send it yourself. Once the issue is re-

solved, save the final settings and run a small example to confirm that you can continue.

Appendix A

# Quick reference

This appendix brings together frequently used formats, conversions, shortcuts, and terms, and identifies the purpose of the accompanying example files. Use it as a reference to the tutorials; when trying a feature for the first time, work through its complete example in the relevant chapter.

## A.1 Supported file formats

### Projects and workflows

A `.ks` project stores structures, workflows, and project results. Use it to move an entire exercise between computers or continue it later. A `.wf` stores workflow organization and settings for reuse. A structure file alone generally cannot reconstruct all nodes, parameters, and results in the original project.

### Atomic structures

**Table A.1** Common structure formats and their uses.

| Format | Information suited to storage and exchange |
|---|---|
| CIF, MCIF | Crystal information such as lattice, fractional positions, elements, and occupancies. Magnetic information depends on the file and reader support. |
| POSCAR, CONTCAR, VASP | Lattice, elements, and periodic positions for exchanging explicit cells and supercells. |
| XYZ | Elements and Cartesian positions for molecular geometry. Ordinary XYZ does not fully preserve cells or explicit bond orders. |

**Table A.1** Common structure formats and their uses (continued).

| Format | Information suited to storage and exchange |
|---|---|
| MOL, SDF, MOL2 | Molecular atoms and connectivity. MOL export does not support aromatic bond order 1.5; use native JSON for aromatic molecules. |
| JSON, MSON | Structured object information. Native molecular JSON retains atoms, explicit bonds, chemical attributes, and coordinates. |
| XSF | Atoms, lattice, and optional grids. Use the appropriate structure or volumetric import entry point. |
| PDB, CML | Molecular exchange formats offered by the selector. Check elements, positions, and bonds after import; extra annotations in complex files are not automatically calculation parameters. |
| YAML, YML | Structured text objects requiring the appropriate model fields, as with native JSON; arbitrary configuration files with these extensions are not structures. |
| CSSR, CONFIG, PWMAT | Program-specific structure text for importing atoms and cells. Configure calculation tasks and controls in the workflow. |

Menus may offer other programs' input or output formats. A format name does not imply that every parameter and result can be converted to the same model. Decide whether you need an atomic structure or a complete calculation task. Reimport important exported models and check element counts, cells, positions, and molecular connectivity to identify format limitations.

### Volumetric data and images

Volumetric import supports CHGCAR/CHG, Cube/CUB, and XSF, including common compressed extensions such as `.gz` and `.bz2`. Retain the underlying format in compressed filenames for identification. The generating calculation determines the meaning of Cube and XSF scalars; their extensions alone do not identify charge density.

Image exports include PNG, JPEG, TIFF, SVG, and PDF. Isosurface geometry exports include glTF, GLB, PLY, and STL; slices also export CSV. Images support presentation, geometry supports further 3D processing, and CSV supports numerical analysis. Their information content differs: an image is not an editable calculation structure.

## A.2 Common units and conversions

Units appear beside fields or axes. Structure modeling commonly uses Å, electronic calculation inputs often use atomic units, and bands and DOS usually use eV. Check each field and axis before processing data.

**Table A.2** Common conversions, shown to useful working precision.

| Quantity | Conversion |
|---|---|
| Length | $1\,\text{Bohr} \approx 0.529177210544\,\text{Å}$; $1\,\text{Å} \approx 1.8897261259\,\text{Bohr}$. |
| Energy | $1\,\text{Ha} = 2\,\text{Ry} \approx 27.211386246\,\text{eV}$. |
| Force | $1\,\text{Ha/Bohr} \approx 51.4221\,\text{eV/Å}$. |
| Temperature and energy | $k_\text{B} \approx 8.617333262 \times 10^{-5}\,\text{eV/K}$; electronic temperature is not atomic temperature. |
| Density | $1\,\text{Å}^{-3} \approx 0.148184711\,\text{Bohr}^{-3}$; convert units and numerical values together. |
| Angle | Modeling rotations, bond angles, and dihedrals use degrees; $180^\circ = \pi$ radians. |
| Mass density | Molecular packing commonly uses $\text{g/cm}^3$, not electron-density units. |

For example, 20 in Ecut (Hartree) is approximately $544.23\,\mathrm{eV}$. Do not enter 400 unchanged in that field when another program specifies $400\,\mathrm{eV}$. Similarly, strain 0.01 means 1%, not a length increase of $0.01$ Å.

CHGCAR density is normalized by cell volume into $\text{Å}^{-3}$. Viewer unit switching changes its displayed representation only. Slice CSV coordinate columns are named `x_angstrom` and similar; values retain the channel's internal units, as explained in Chapter 11. Do not apply density conversions to arbitrary scalars from other sources.

Per-atom energy comparisons also require normalization. An eight-atom cell with energy $E_8$ has per-atom energy $E_8/8$. For a 64-atom supercell of the same structure, compare $E_{64}/64$ with $E_8/8$, rather than the totals. Defects, different elements, or different charge states require reference states appropriate to the question; division by atom count alone may not give a formation energy.

## A.3 Keyboard and mouse reference

These shortcuts mainly apply to the 3D structure view. Give the view focus first; an active text or numeric field may handle the key instead. Cmd/Ctrl means Command on macOS or Ctrl on Windows/Linux.

**Table A.3** Viewing and selection.

| Key or mouse action | Effect |
|---|---|
| Left drag | Rotate the view. |
| Right drag | Pan the view. |
| Scroll or middle drag | Zoom. |
| Space | Center and fit the model. |
| Click | Select an atom or object. |
| Shift/Ctrl+click | Add atoms to or remove them from the selection. |
| Double-click | Select a connected structure in modeling; sample a point in volumetric data. |

**Table A.3** Viewing and selection (continued).

| Key or mouse action | Effect |
|---|---|
| B, then drag | Box-select atoms. |
| L, then drag | Lasso-select atoms. |
| Cmd/Ctrl+A | Select all atoms. |
| I | Toggle the minimal view. |
| ? or / | Open structure-view shortcut help. |

**Table A.4** Modeling and history.

| Key or mouse action | Effect |
|---|---|
| G, then drag | Move the selection. |
| R, then drag | Rotate the selection. |
| X, Y, Z | Constrain a transform to the corresponding axis. |
| Shift+middle drag | Move the selection directly. |
| Shift+right drag | Rotate the selection directly. |
| S | Enter 3D molecular sketching. |
| O | Enter interactive adsorbate placement. |
| Enter | Accept the current interactive placement when available. |
| Delete or Backspace | Delete the selection. |
| Cmd/Ctrl+Z | Undo an applied edit. |
| Shift+Cmd/Ctrl+Z | Redo. |
| Esc | Leave the current tool or clear the selection. |

Adsorbate placement uses different mouse actions: left drag changes orientation, Alt+left drag adjusts the anchor distance, and right drag rotates around the anchoring direction. Check the active tool before interacting.

Save projects with Cmd/Ctrl+S. In Online, download after saving to obtain a local copy.

## A.4 Terminology

**Table A.5** Common workspace, structure, and calculation terms.

| Term | Meaning |
|---|---|
| Project | A container organizing structures, workflows, and results. |
| Workflow / Node | Tasks and their data connections. |
| Dataset / Plot | Numerical records / graphical results. |
| Site / Image atom | A model position / its periodic display copy. |
| Lattice / Unit cell | The geometry describing periodicity. |
| Primitive / Conventional cell | Different representations of the same crystal. |
| Supercell | A periodic model with an integer multiple of the base cell's atoms. |
| Fractional / Cartesian | Coordinates in the lattice basis / orthogonal spatial axes. |
| SCF / NSCF | Self-consistent / non-self-consistent field calculation. |
| Pseudopotential | An effective description of ionic-core interactions with valence electrons. |
| Cutoff energy | The limit defining the plane-wave basis. |
| Smearing / Occupation | Energy smoothing / electronic state populations. |
| Fermi energy | A common energy reference for bands and DOS. |
| Relaxation | Adjustment of geometry using forces or related criteria. |
| DOS / PDOS | Density of states / projected density of states; a projection must identify its target. |

For surfaces, a slab is a thin periodic model; a termination identifies the exposed surface, an adsorbate is the attached species, and an anchor is its ref-

erence attachment point. Ontop, bridge, and hollow name adsorption sites. A hollow adsorption site is distinct from a vacancy created by removing a lattice atom.

For volumetric data, a channel selects a scalar dataset. An isosurface connects equal values, a Miller plane slice samples one plane, and direct volume rendering combines information through a volume. A threshold sets an isosurface value, a color range sets the colormap interval, and opacity controls transparency. These names identify the corresponding viewer settings.

### Terms that are easy to confuse

Preview checks an uncommitted geometric change, Apply accepts it, and Save writes work to a file. Resetting a structure restores the editing baseline; Reset camera changes framing only. Repeat cell expands the display; Build Supercell expands the model. Distinguish these commands by the model or file information they change, rather than whether the screen changes.

Online file operations also have distinct roles: upload copies a local file to server storage, import adds its contents to the current project, save writes the project to a server-side file, and download obtains a local copy. This distinction tells you whether the next action belongs in the Project Browser or Files panel.

## A.5 Workflow task reference

Workflow nodes belong to four modules. For connections and parameters, use Table 10.1 in Chapter 10 and its worked examples. The table below starts from the desired result.

A task with 3D in its name does not necessarily require a three-dimensional bulk model. The two horizontal band-surface coordinates are reciprocal-space coordinates, whereas volumetric figures describe real space. Identify the desired quantity before choosing model dimensionality and numerical sampling.

**Table A.6** Find a task by its purpose.

| Goal | Tasks and tutorial |
|---|---|
| Ground-state total energy | Build Molecule, SCF, Energy Curve; Chapter 5. |
| Bands along a path | Symmetry & K-Path, NSCF, Band Calculation, Band Structure; silicon primitive-cell example in Chapter 10. |
| Total DOS | SCF, NSCF, DOS; DOS example in Chapter 10. |
| Relax atomic positions | Build Molecule, Geometry Optimization; hydrogen optimization in Chapter 10. |
| Reciprocal geometry | Symmetry & K-Path, Brillouin Zone 2D or 3D; Section 10.11. |
| Band-edge surfaces | SCF, NSCF, Band Calculation 3D, Band Structure 3D; Section 10.12. |
| Excited-state inputs | SCF ground state and extra unoccupied orbitals, TDDFT; Section 10.13 explains the solution procedure. |

## A.6 Accompanying example files

Files in `examples` include original inputs and projects for continued work. Copy them into your own study directory to retain the supplied versions as references. The table lists files used in this edition. Save logs, reports, and numerical outputs according to each script's actual output locations.

**Table A.7** Example files and their purposes.

| File | Purpose |
|---|---|
| `h2.cif` | Hydrogen simulation box for the first calculation; initial bond approximately 0.793766 Å. |
| `h2_optimized.cif` | Optimized reference hydrogen geometry; bond approximately 0.757292 Å. |

**Table A.7** Example files and their purposes (continued).

| File | Purpose |
|---|---|
| `hydrogen.xyz` | Coordinate import exercise; bond approximately 0.740848 Å, different from the calculation input above. |
| `water.xyz` | Water geometry and coordinate import exercise. |
| `silicon.cif` | Eight-site conventional silicon cell, side approximately 5.44370 Å, for workspace and calculation tutorials. |
| `Si.poscar` | Eight-site silicon cell, side approximately 5.46873 Å, for supercell and vacancy modeling. |
| `Si-ground-state.ks` | Silicon structure project from Chapter 4; no SCF calculation yet. |
| `Si-introduction.ks` | Silicon import project from Chapter 3, used to continue modeling in Chapter 7. |
| `Si-vacancy-tutorial.ks` | Chapter 7 vacancy project with reference silicon and a 63-site vacancy model. |
| `Si-primitive-BZ-Online-050.pdf` | Vector 3D Brillouin zone and path for two-site silicon, generated by the Symmetry template. |
| `benzene-ring.json` | Six-carbon aromatic ring, side 1.4 Å, before hydrogen addition. |
| `benzene.json` | Initial benzene after hydrogen |
| `benzene-optimized.json` | addition and a reference structure optimized with the generic molecular model. |

**Table A.7** Example files and their purposes (continued).

| File | Purpose |
|---|---|
| `pyridine.json`<br>`pyridine-optimized.json` | Initial neutral singlet pyridine and its reference structure optimized with the generic molecular model, retaining six aromatic bonds. |
| `ethanol-backbone.json`<br>`ethanol-initial.json` | Collinear heavy-atom ethanol backbone and initial structure after hydrogen addition. |
| `ethanol-edited.json`<br>`ethanol-cleaned.json`<br>`ethanol-optimized.json` | Ethanol after geometric editing, rule-based cleanup, and generic molecular-model optimization. |
| `ethanol-angle1095.json` | Neutral singlet ethanol with 9 atoms, 8 explicit bonds, and a C1–C2–O3 angle of $109.5°$. |
| `ethanol-angle1095.png` | The same ethanol exported through the atomic viewer's PNG menu, $4032 \times 1964$ pixels. |
| `ethanol-angle1095-scene.json` | Scene JSON recording displayed sites and connections, separate from the editable structure JSON. |
| `Cu-fcc.cif`<br>`Cu-vacancy.cif` | Four-site copper cell with $a = 3.61$ Å and a three-site vacancy candidate. |
| `Pt-fcc.cif`<br>`Pt111-slab.cif` | Platinum bulk cell with $a = 3.92$ Å and a four-layer $(111)$ slab generated using Chapter 8 settings. |
| `Pt111-normal-c.json`<br>`oh-fragment.mol`<br>`Pt111-OH-outward.cif` | Cell with its third lattice vector aligned to the surface normal, singly bonded O–H input, and six-site adsorption geometry with O–H pointing away from the surface. |

**Table A.7** Example files and their purposes (continued).

| File | Purpose |
|---|---|
| `Pt111-OH-Online-050.json` | Six-site structure after O–H placement and wrapping; O/H heights along the normal 2.00/2.96 Å, O–H length 0.96 Å. |
| `Pt111-OH-candidate-1.json` | Highest-ranked structure among 144 reference geometric candidates, for comparing anchor, height, and rotation. |
| `modeling_examples.m`<br>`verify_modeling_examples.m` | Generation and reimport checks with JSON reports; see `MODELING-CASES.zh-CN.md`. |
| `PEO3-PPO4-PEO3.json`<br>`PE6-PP14-seed91.json` | Initial block and exact-composition random chains with 84 and 164 atoms, respectively. |
| `neb-exchange/initial.json`<br>`neb-exchange/final.json` | Geometric path endpoints exchanging two sites while retaining atom correspondence. |
| `neb-exchange/image-00.json` through `image-06.json` | Complete seven-image IDPP path: two endpoints and five intermediate configurations. |
| `polymer_neb_examples.m` | Chain and path construction, reimport, and geometry checks; see `POLYMER-NEB-CASES.zh-CN.md`. |
| `graphene-layer.json`<br>`graphene-inserted.json`<br>`graphene-stacked.json` | Two-site graphene layer, directly inserted bilayer, and layer-matched bilayer. |
| `hBN-layer.json`<br>`graphene-hBN-stacked.json` | Ideal h-BN monolayer and matched graphene/h-BN, with 1.6% in-plane strain. |

**Table A.7** Example files and their purposes (continued).

| File | Purpose |
|---|---|
| twist-graphene-bottom.json<br>twist-graphene-top.json | Original two-site graphene monolayers for the twist tutorial. |
| graphene-twist-21p786789.json<br>graphene-twist-target22.json<br>graphene-hBN-twist-21p786789.json | Two target-angle cases and a heteromaterial twisted structure, all with 28 sites; matching CIF files support geometry exchange. |
| twist_examples.m<br>verify_twist_examples.m | Twisted structure generation and reimport verification; see TWIST-CASES.zh-CN.md. |
| nano-graphene-60.json<br>graphene-armchair-w3-l2.json<br>graphene-tube-4-0-l2.json | Graphene input with a 60-degree basis, a 24-site ribbon, and a 32-site tube. |
| Cu-wire-001-r3-l2.json<br>Cu-dot-r3.json<br>Cu-void-r2p7.json | Copper wire, finite cluster, and spherical void; parameters in NANO-CASES.zh-CN.md. |
| user-repeat-ethane-input.json<br>user-repeat-CC-dimer.json<br>user-repeat-unit-library.json | Ethane input, 14-atom dimer, and library for the custom two-carbon repeat unit. |
| user_repeat_unit_example.m<br>verify_user_repeat_unit_example.m | Custom-unit generation and independent read-back; see USER-REPEAT-UNIT-CASE.zh-CN.md. |
| nano_examples.m<br>verify_nano_examples.m | Generation, periodicity, coordination, and exchange-file checks for five nanostructure types. |
| sqs-cu-ni-input.json<br>sqs-cu-ni-ordered.json<br>sqs_example.m | Mixed-occupancy Cu–Ni input, four-site ordered SQS result, and generator; see SQS-CASE.zh-CN.md. |

**Table A.7** Example files and their purposes (continued).

| File | Purpose |
|---|---|
| `adsorption_bn_examples.m` | Adsorption candidates, outward orientation, and heteromaterial matching; see `ADSORPTION-BN-CASES.zh-CN.md`. |
| `orient_oh_example.m` | |
| `export_oh_cif.m` | |
| `water-packing.json` | Explicitly bonded water input and a low-density initial packing of 10 water and 10 ethanol molecules. |
| `water10-ethanol10-packed.json` | |
| `interface_packing_examples.m` | Bilayer candidates, mixture packing, and independent geometry checks; see `INTERFACE-PACKING-CASES.zh-CN.md`. |
| `h2-tutorial.ks` | Hydrogen tutorial project for inspecting structures and workflows. |
| `H2-Online-SCF-050.ks` | Hydrogen SCF project with a seven-iteration convergence record. |
| `silicon-tutorial.ks` | Silicon tutorial project and its workflow. |
| `h2_scf_and_relax.m` | Script to rerun hydrogen SCF and geometry optimization in full MATLAB®. |
| `silicon_example.m` | Script to rerun eight-atom silicon SCF and DOS in full MATLAB®. |
| `silicon_primitive_example.m` | Script to rerun the band calculation for a standardized two-atom silicon primitive cell. |
| `silicon-band-primitive.ks` | Two-atom silicon band and symmetry project. |
| `CHGCAR-teaching` | Volumetric import and viewing example. |

**Table A.7** Example files and their purposes (continued).

| File | Purpose |
|---|---|
| `CHGCAR.nospin.gz` | Non-spin-polarized CHGCAR for total density. |
| `CHGCAR.spin.gz` | Spin-polarized CHGCAR for channel and sign comparisons. |
| `elec.cube.gz` | Cube data for slicing and export. |
| `elec.total.slice-1_1_0.csv` | (110) slice with 1568 data rows; coordinates in Å, values in channel-internal units. |
| `elec.total.slice-1_1_0.png` | RGBA image of that slice, $384 \times 768$ pixels, for checking colors and shape. |
| `datagrid_3d.xsf` | XSF scalar grid for format and slice inspection. |

The supplied silicon and hydrogen inputs have different geometries and parameters; select the specified file for numerical comparisons. Scripts require full MATLAB® and the matching toolbox. Configure its path and run from this manual repository's root. Calculation chapters and example notes list required products and commands. All three editions support the applicable graphical tutorials.

### Document your own examples

When changing materials, add a short note recording the source structure, supercell or surface operations, final model name, workflow purpose, and result locations. For random construction, include seed and composition. For figures, include the energy reference, channel, or threshold. These notes reconnect structures, calculations, and presentation when you return later.

The compressed CHGCAR, Cube, and XSF examples come from `pymatgen-core` test data. Source notes accompany the files.

## A.7 References

The following references provide further reading directly related to the fundamentals used in this manual.


1. Liu Yang, Jinlong Yang, and Wei Hu. *KSSOLV Toolbox: A MATLAB® Graphical User Interface for Plane-Wave Density Functional Theory Calculations*. Journal of Chemical Theory and Computation, 2026. doi:10.1021/acs.jctc.6c00523.
2. Shizhe Jiao et al. *KSSOLV 2.0: An efficient MATLAB® toolbox for solving the Kohn–Sham equations with plane-wave basis set*. Computer Physics Communications, 279, 108424, 2022. doi:10.1016/j.cpc.2022.108424.
3. P. Hohenberg and W. Kohn. *Inhomogeneous Electron Gas*. Physical Review, 136, B864–B871, 1964. doi:10.1103/PhysRev.136.B864.
4. W. Kohn and L. J. Sham. *Self-Consistent Equations Including Exchange and Correlation Effects*. Physical Review, 140, A1133–A1138, 1965. doi:10.1103/PhysRev.140.A1133.
5. John P. Perdew, Kieron Burke, and Matthias Ernzerhof. *Generalized Gradient Approximation Made Simple*. Physical Review Letters, 77, 3865–3868, 1996. doi:10.1103/PhysRevLett.77.3865.
6. D. R. Hamann. *Optimized norm-conserving Vanderbilt pseudopotentials*. Physical Review B, 88, 085117, 2013. doi:10.1103/PhysRevB.88.085117.

# Subject index

Entries are arranged alphabetically by subject.


**A**
Adsorption modeling, 125
Aromatic molecules, 105
Atomic forces, 249
Atomic structure images, 260
**B**
Band gaps, 252
Band structures, 206, 250
Band-energy surfaces, 218
Bond angles, 109
Brillouin zone, 214
**C**
Charge density, 223
Chat services, 268
CHGCAR, 224
CIF, 292
Cluster configurations, 277
Command Window, 40
Config Editor, 33
Convergence checks, 246, 287
CSV export, 240
Cutoff energy, 82
**D**
Data export, 265
Defect enumeration, 118
Density functional theory, 75
Density mixing, 90
Density of states (DOS), 210, 253
Dihedral angles, 109
Direct volume rendering, 236
Distance measurements, 109
Downloading projects, 52
**E**
Exporting
    Images and data, 239
Extra bands, 88
**F**
Fermi energy, 87
Fixed atoms, 122
Fragment library, 173
**G**
Geometry optimization, 90, 211
**H**
Hero Shot, 262
Heterojunctions, 130
**I**
Importing structures, 50
Installation and Runtime, 17
Installation problems, 284
Interface language, 267
Isosurfaces, 226
**K**
k-point sampling, 84
Keyboard shortcuts, 295
**L**
Layer matching, 130
Logs and help, 282
**M**
Materials Project, 274
    Structure search, 41
Miller plane slices, 232
Mixture packing, 161
Modeling recipes, 191
Moiré superlattices, 133
Molecular geometry, 109
Molecular modeling, 93
**N**
Nano voids, 156
Nanoribbons, 146
Nanotubes, 146
Nanowires, 146
NEB paths, 144
**O**
Offline HTML, 261
Online connection problems, 285
Online edition, 46
**P**
Parallel computing, 198
Parameter presets, 190

Periodic images and atom counts, 285
Plane waves, 82
Polymers, 157
Project Browser, 32
Project files, 39
Pseudopotentials, 81
**Q**
Quantum dots, 155
**R**
Reconnecting, 56
Remote computing, 267
Remote jobs, 279
**S**
Sampling values, 239
Save As, 52
Saving projects, 37, 50
Selection sets, 164
Self-consistent field (SCF), 78
Server-side files, 48
Sidebars, 33
Slices, 232
Smearing and occupations, 87
Space groups, 115
Spin, 88
SQS models, 180
Strain, 117
Structural relaxation, 211
Supercells, 101
Surface models, 119
Symmetry, 206
**T**
TDDFT, 219
Total energy, 247
Twisted bilayers, 133
**U**
Unit conversions, 294
Uploading files, 49
**V**
Vacancies, 103
Vacuum, 122
Volumetric data, 223
    Channels, 223
**W**
Workflow nodes, 194
Workflows, 194
Workspace, 29
**X**
XYZ, 292




# KSSOLV Toolbox

Structure modeling · Electronic structure · Analysis

Hefei Hanhai Quantum Technology Co., Ltd.